University of the Philippines Manila
College of Arts and Sciences
Department of Physical Sciences and Mathematics

HRP-AI: A WEB APPLICATION FOR HYPERTENSION RISK PREDICTION WITH CALIBRATED EXPLAINABLE AI

A special problem in partial fulfillment
of the requirements for the degree of
Bachelor of Science in Computer Science

Submitted by:

Jonathan Ralph F. Baes
May 2026

Permission is given for the following people to have access to this SP:

Available to the general public	Yes
Available only after consultation with author/SP adviser	No
Available only to those bound by confidentiality agreement	No

APPROVAL SHEET

The special problem entitled "*HRP-AI: A WEB APPLICATION FOR HYPERTENSION RISK PREDICTION WITH CALIBRATED EXPLAINABLE AI*" prepared and submitted by Jonathan Ralph F. Baes in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science is hereby accepted.

Geoffrey A. Solano, Ph.D.
Adviser

Accepted as partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

Marie Josephine M. De Luna, Ph.D.
Chair
Department of Physical Sciences and Mathematics

Maria Constancia O. Carrillo, Ph.D.
Dean
College of Arts and Sciences

Abstract

Hypertension remains a major public health concern in the Philippines, contributing significantly to cardiovascular morbidity and mortality. This study develops HRP-AI, a web-based decision-support application for predicting clinical hypertension risk ($\geq$ 140/90 mmHg) among Filipino adults using calibrated and explainable machine learning.

Using the DOST-FNRI 2015 National Nutrition Survey (NNS), a final dataset of 151,189 adults was constructed, with 22,988 hypertensive cases corresponding to a 15.20% prevalence rate. Eight machine learning algorithms were evaluated across multiple imbalance-handling configurations through a Two-Stage Randomized Search. XGBoost with algorithmic Class Weights was selected as the final predictive engine because it provided the strongest clinical compromise between sensitivity, discrimination, and calibration compatibility. At the standard 0.50 decision threshold, the model achieved 76.87% Accuracy, 76.51% Recall, an F1-Score of 0.5014, and an AUC of 0.8469. Under the selected 0.35 screening threshold, its Recall increased to 87.88%.

To improve probability reliability, Venn-Abers calibration was applied, reducing the Expected Calibration Error (ECE) to 0.0035. The final prototype integrates calibrated risk scores, uncertainty bounds, SHAP and LIME explanations, and Wachter-style counterfactuals optimized through bounded Brent's method. The resulting system demonstrates that non-linear machine learning models can be made more transparent, probability-aware, and suitable for exploratory hypertension screening.

Keywords: Hypertension, Machine Learning, XGBoost, Probability Calibration, Venn-Abers, Explainable AI, SHAP, LIME, Counterfactual Explanations

Contents

List of Figures

List of Tables

1 Introduction

1.1 Background of the Study

Hypertension, often referred to as the ”silent killer” due to its asymptomatic nature, is a leading non-communicable disease and a principal contributor to cardiovascular morbidity and premature mortality worldwide. In the Philippine context, the burden remains substantial. Recent survey cycles, specifically the 2018 National Nutrition Survey (NNS), report a prevalence of 19.2% among Filipino adults aged 20 to 59, escalating dramatically to 35.0% among the elderly. Despite a reported disease awareness rate of 67.8%, blood pressure control remains critically poor; only 75% of those aware are on treatment, and of those treated, only 27% have their blood pressure successfully controlled [1]. This gap contributes significantly to heart disease and stroke, which remain the top causes of mortality in the country.

Advances in artificial intelligence (AI) and machine learning (ML) enable scalable risk stratification and targeted public-health screening by leveraging large, multimodal datasets. These techniques have been applied across settings to predict cardiovascular risk, specifically utilizing the National Nutrition Survey (NNS) provided by the DOST-FNRI [2], and successful deployments as accessible web applications have been demonstrated [3, 4].

Two practical concerns limit the clinical utility of many ML models: (1) model explainability, and (2) probability reliability. Explainable AI (XAI) methods such as SHAP and LIME increase transparency and clinician trust [5, 6]. Independently, probability calibration techniques ensure that a model’s predicted probabilities reflect observed frequencies, a requirement when predictions are used in decision-making workflows. Recent methodological work integrates these concerns through the Calibrated Explanations framework, which produces uncertainty-aware explanations and Wachter-style counterfactuals optimized for minimal clinical perturbation aligned with calibrated probabilities [7, 8].

1.2 Statement of the Problem

Existing ML systems for hypertension risk prediction, such as those developed by Eldem [9] and Naik et al. [3], often prioritize discrimination metrics like accuracy and AUC. However, these systems possibly yield unreliable probability estimates for clinical application since they have not been calibrated nor even been tested with calibration metrics. Current systems aiming for reliable explainability typically focus on post hoc testing of calibration rather than employing formal post hoc calibration methods to actively correct probability distortion, as seen in recent studies using survey data [4, 10].

This omission means that many explainable AI outputs are derived from uncalibrated scores, which misrepresent model confidence and undermine clinical trustworthiness. The current ”standard” approach to XAI is not completely ideal due to its nature; explaining the raw output of a black-box model without ensuring its probabilistic reliability can lead to misleading interpretations [11]. Furthermore, XAI and other ML systems are not without flaws; an explanation that justifies an overconfident but incorrect prediction provides a false sense of security to the clinician [7]. Specifically, a Machine Learning System that utilizes the Philippine NNS to produce discriminative hypertension risk predictions, actively corrects probability unreliability using formal calibration methods (Platt scaling, Isotonic Regression, Venn-Abers), and generates a calibrated risk score, uncertainty interval, and actionable, Wachter-style counterfactuals, has not been systematically implemented and evaluated.

1.3 Research Questions

This study aims to answer the following research questions:

1. How can machine learning models (specifically k-Nearest Neighbors, Naive Bayes, Logistic Regression, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost) be developed and evaluated for predicting hypertension risk using the

Clinical and Health, Anthropometry, and Dietary Survey Components of the DOST-FNRI 2015 National Nutrition Survey (NNS) dataset?

2. How can a Two-Stage Rigorous Optimization strategy be implemented to enhance model performance, and how can the Calibrated Explanations framework be integrated to provide reliable probability estimates using Platt Scaling, Isotonic Regression, and Venn-Abers predictors?

3. How can Explainable Artificial Intelligence (specifically SHAP and LIME) and bounded Brent's optimization be integrated to extract transparent clinical risk drivers and prescribe physiologically realistic, Wachter-style lifestyle counterfactuals?

4. How can the predictive model, along with its interpretable feature importance, uncertainty intervals, and actionable, Wachter-style counterfactuals, be effectively presented in a web application prototype designed as a decision-support tool for potential users like clinicians or health-conscious individuals?

1.4 Objectives of the Study

The general objective of this study is to develop a web application for hypertension prediction that utilizes a machine learning model enhanced with Calibrated Explainable AI to predict hypertension risk.

1.4.1 Research Objectives

Specifically, this study aims to:

1. **Data Acquisition:** Acquire and secure access to the microdata of the 2015 National Nutrition Survey (NNS) from the DOST-FNRI, specifically the Clinical and Health, Anthropometry, and Individual Dietary components.

2. **Data Merging:** Systematically merge the three distinct datasets using unique respondent identifiers (‘hhnum‘ and ‘member_code‘) to create a unified, multimodal feature set for each respondent.

3. **Target Variable Engineering:** Construct the binary ground-truth variable ‘Hypertension‘ by applying the clinical threshold of Systolic Blood Pressure $\geq$ 140 mmHg or Diastolic Blood Pressure $\geq$ 90 mmHg to the averaged blood pressure readings found in the Clinical dataset.

4. **Data Cleaning:** Perform rigorous data cleaning within the NNS framework by filtering out invalid entries, dropping non-relevant anthropometric groups (infants, children, pregnant/lactating women), and removing administrative variables irrelevant to clinical prediction.

5. **Imputing Outliers:** Identify and handle outliers in numerical features using statistical methods to ensure robust model training without discarding valuable clinical signals.

6. **Imputing Missing Values:** Optimize the K-Nearest Neighbors (KNN) imputation strategy by conducting a masking experiment on 1% of the dataset to determine the optimal number of neighbors (k), standardizing numerical variables prior to imputation to ensuring distance metric validity.

7. **Feature Engineering:** Construct new, clinically relevant features such as Body Mass Index (BMI) and Waist-to-Hip Ratio (WHR) from anthropometric measurements, and categorical lifestyle levels (e.g., Smoking Level, Alcohol Level) from raw survey questionnaire responses.

8. **Feature Selection:** Apply a strict collinearity filter, leakage guards, and correlation analysis to select the most predictive variables and reduce dimensionality, thereby improving model efficiency and reducing noise.

9. **Model Development:** Train eight candidate supervised machine learning algorithms, including k-Nearest Neighbors (kNN), Naive Bayes, Logistic Regression,

Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost, to predict binary hypertension status.

10. **Hyperparameter Optimization:** Optimize the performance of each candidate model using a **Two-Stage Cross-Validation (CV) Optimization strategy**, utilizing a broad **Randomized Search** to identify promising regions of the hyperparameter space, followed by **Local Refinement** to pinpoint the configuration yielding the highest F1-score and AUC.

11. **Class Imbalance Handling:** Evaluate and apply imbalance handling strategies, specifically Baseline, Class Weights, SMOTE, SMOTENC, SMOTE + Class Weights, and SMOTENC + Class Weights, to address the disparity between hypertensive and normotensive classes in the dataset.

12. **Probability Calibration:** Apply and evaluate three post-hoc calibration techniques—Platt Scaling, Isotonic Regression, and Venn-Abers predictors—to ensure the predicted risk scores accurately reflect true empirical probabilities.

13. **Explainability Integration:** Implement Explainable Artificial Intelligence (XAI) modules by integrating SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME). This integration will generate both global feature attributions (via SHAP) to identify dataset-wide risk drivers, and local attribution scores (via SHAP and LIME) to quantify the contribution of specific clinical and dietary variables to individual risk predictions.

14. **Counterfactual Generation:** Implement the extraction of Wachter-style counterfactual explanations by minimizing an objective function that balances decision boundary crossing with a variance-normalized proximity penalty. The target probability is set slightly below the selected screening threshold of 0.35, allowing the system to identify the minimal clinically realistic single-feature perturbations required to alter a high-risk prediction.

15. **Evaluation:** Assess the models using discrimination metrics (Accuracy, Recall,

Precision, F1-Score, AUC) and calibration metrics (Expected Calibration Error, Log Loss) to select the optimal final model.

1.4.2 System Objectives

Specifically, this study aims to:

1. Design and build a prototype web application with a tiered explanation interface to serve as a decision-support tool.
2. Implement an interface for a user to input their required health data, specifically clinical variables (e.g., age, sex, smoking history), anthropometric variables (e.g., BMI, Waist-to-Hip Ratio), and dietary variables (e.g., daily intake of nutrients and food groups).
3. Integrate the trained and calibrated model to generate a personalized hypertension risk prediction based on the user's input.
4. Present the calibrated risk score, uncertainty interval (Venn Abers predictors), and both global feature importance (SHAP Global) and local patient-specific (SHAP Local and LIME) breakdowns for users or clinicians seeking a detailed technical analysis.

1.5 Significance of the Study

This study is significant to the following:

1. **For the Patient:** The system provides personalized hypertension risk estimates, calibrated probabilities, and interpretable explanations that may guide preventive action and support self-management of their health.
2. **For Doctors and Healthcare Practitioners:** The calibrated outputs, uncertainty intervals, and explanation interface offer transparent insights that may assist in

clinical assessment, patient counseling, and early intervention planning, ensuring that decisions are backed by reliable probability estimates.

3. **For Government and Policy Makers:** The prototype demonstrates how survey-based predictive tools may support screening workflows and wellness initiatives using national-level data such as the NNS, potentially informing public health policies regarding hypertension screening programs.

4. **For the Research Community:** The study provides a reference implementation for integrating probability calibration with explanation methods, contributing to future work on interpretable and uncertainty-aware predictive models.

1.6 Scope and Limitations

The scope of this study and its limitations are as follows:

1. This study utilizes only the Clinical and Health, Anthropometry, and Dietary components of the 2015 NNS microdata for model training and evaluation.

2. The study focuses on training multiple machine learning algorithms, specifically k-Nearest Neighbors, Naive Bayes, Logistic Regression, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost.

3. The study applies specific imbalance-handling strategies, namely Baseline, Class Weights, Synthetic Minority Over-sampling Technique (SMOTE), SMOTENC, SMOTE + Class Weights, and SMOTENC + Class Weights.

4. The study performs probability calibration using Platt scaling, Isotonic Regression, and Venn–Abers predictors.

5. The system integrates SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) within the Calibrated Explanations

framework to produce uncertainty-aware explanations. Specifically, SHAP is utilized for both global (population-level) and local (individual-level) feature attributions, while LIME provides accessible local explanations. These are implemented alongside Wachter-style counterfactuals optimized via bounded Brent's method.

6. The web application developed is a prototype decision-support tool intended for demonstration and exploratory use; it is not intended to replace professional medical judgment.

7. Model performance may be affected by the limitations of NNS data, such as cross-sectional design, measurement error, and reporting biases, particularly in dietary information, as recall bias is inherent in self-reported 24-hour food recall surveys.

8. Generalizability of the hypertension risk prediction model to populations outside the NNS sampling frame or to clinical environments may require further validation and calibration.

1.7 Assumptions

1. Measurements from the National Nutrition Survey (NNS) are assumed to be of adequate quality for hypertension risk modeling.

2. The selected variables from the clinical, anthropometric, and dietary components are assumed to capture meaningful predictors of hypertension risk.

3. Calibration subsets used for Platt scaling, Isotonic Regression, and Venn–Abers are assumed to contain sufficient samples to avoid severe overfitting and yield stable probability estimates.

4. Users of the web application prototype are assumed to provide inputs within realistic ranges consistent with NNS measurement standards (avoiding outliers), which will be enforced via input validation logic within the application.

5. The average systolic and diastolic blood pressure measurements used in this study are assumed to be resting blood pressure readings, consistent with standard clinical practice guidelines for hypertension diagnosis.

2 Review of Related Literature

This section provides a comprehensive critical review of the existing body of knowledge relevant to the development of an explainable and calibrated machine learning system for hypertension risk prediction. The discussion is thematically organized into five key areas: (1) the epidemiological landscape of hypertension, specifically within the Philippine setting; (2) the evolution and application of machine learning methodologies in cardiovascular risk stratification, including modern optimization strategies; (3) the critical role of Explainable Artificial Intelligence (XAI) in fostering trust in clinical decision support systems, specifically distinguishing between interpretability and explainability; (4) the often-overlooked necessity of probability calibration in risk modeling; and (5) a synthesis of these domains to highlight the specific research gap addressed by this study, emphasizing the transition from arbitrary counterfactuals to mathematically rigorous, Wachter-style prescriptive analytics.

2.1 Hypertension: A Global and National Health Crisis

Hypertension, clinically defined as persistently elevated arterial blood pressure, is universally recognized as a primary driver of cardiovascular morbidity and premature mortality. The World Health Organization (WHO) identifies it as a ”silent killer” due to its asymptomatic progression towards severe complications, including stroke, myocardial infarction, heart failure, and chronic kidney disease. The condition often remains undetected until significant organ damage has occurred, necessitating proactive screening measures.

In the context of the Philippines, the burden of hypertension is both substantial and escalating, posing a severe challenge to the public health infrastructure. Dela Rosa et al. [12] highlight that despite various public health initiatives, the prevalence of uncontrolled hypertension remains alarmingly high. Their review of national surveillance data indicates persistent gaps in disease awareness, treatment adherence, and control

rates among Filipinos. Specifically, lifestyle factors prevalent in the region, such as diets high in sodium and processed foods, coupled with increasing urbanization and sedentary behavior, exacerbate the risk profile of the population. This epidemiological reality underscores the urgent need for scalable, accessible screening tools that can identify high-risk individuals early in the disease trajectory, potentially before clinical symptoms manifest.

2.2 Machine Learning in Cardiovascular Risk Prediction

The limitations of traditional linear scoring systems, such as the Framingham Risk Score, have catalyzed a paradigm shift towards Machine Learning (ML) methodologies. Unlike traditional statistical methods, which often assume linear relationships between risk factors and outcomes, ML algorithms excel at modeling the high-dimensional, non-linear interactions inherent in complex physiological data.

2.2.1 The Shift from Linear to Non-Linear Models and Advanced Optimization

Recent literature demonstrates a clear trend towards the use of ensemble learning techniques for medical diagnosis, moving away from simple probabilistic models like Naive Bayes or standalone logistic regression models that may fail to capture complex biological dependencies. Naik et al. [3] and Pandey et al. [4] discuss how Artificial Intelligence (AI) and Digital Twins are reshaping hypertension prediction. Their findings suggest that algorithms capable of capturing complex feature interactions, specifically Random Forest and Gradient Boosting Machines (GBMs), consistently outperform these simpler baselines in terms of discrimination metrics like the Area Under the Receiver Operating Characteristic (AUC-ROC) curve.

However, maximizing the predictive capability of these complex ensembles requires rigorous hyperparameter tuning. Traditional exhaustive methods, such as Grid Search, often suffer from the ”curse of dimensionality” when applied to algorithms like XGBoost

or CatBoost, which possess vast parameter spaces. Recent methodological literature, specifically the foundational work of Bergstra and Bengio [13], demonstrates that Randomized Search is significantly more effective than Grid Search for high-dimensional optimization. By sampling the parameter space statistically, it can identify optimal hyperparameter regions with a fraction of the computational cost. Building upon this, modern implementations frequently adopt a Two-Stage approach: utilizing a broad randomized search for initial landscape exploration, followed by a concentrated local refinement around the best-performing candidates. This ensures a mathematically robust balance between exploration and exploitation.

2.2.2 Application on Survey Data and Class Imbalance

The utilization of national health surveys for training predictive models is a growing area of interest in health informatics. Elvas et al. [14] emphasize the utility of multimodal datasets that combine clinical history with anthropometric and lifestyle variables. In the Philippines, the National Nutrition Survey (NNS) provides a rich, albeit complex, data source. However, effectively utilizing such survey data requires robust preprocessing to handle challenges inherent to the methodology, such as missing values due to participant non-response, the high dimensionality of continuous dietary data, and severe class imbalance (where normotensive individuals significantly outnumber hypertensive ones).

Addressing this class imbalance is critical; standard algorithms biased towards the majority class produce superficially high accuracy but clinically dangerous false-negative rates. The machine learning literature advocates for two primary interventions to resolve this: algorithmic cost-sensitive learning and synthetic data generation. Applying Class Weights directly penalizes the misclassification of the minority class during optimization. Conversely, data-level interventions like the foundational Synthetic Minority Over-sampling Technique (SMOTE) [15], and its variant for mixed continuous and nominal data (SMOTENC), are frequently employed to expand the decision boundary of the minority class without the severe overfitting associated with naive random

oversampling. Addressing these structural challenges through rigorous engineering interventions—such as optimal K-Nearest Neighbors imputation, strict collinearity filtering, and robust resampling protocols—is central to ensuring model validity before calibration.

2.3 The Explainability Challenge: Explainable AI (XAI)

While the predictive power of ”black-box” models such as Deep Neural Networks and XGBoost is undisputed, their opacity poses a significant barrier to clinical adoption. Mohapatra et al. [16] argue that in high-stakes medical environments, trust is contingent upon transparency. A clinician cannot ethically or practically act on a risk score without understanding the rationale behind the prediction, nor can they explain the risk to a patient to motivate lifestyle changes without intelligible insights. It is crucial to distinguish between interpretability, which refers to the extent to which a cause and effect can be observed natively within a system (often inherent in simple models like Decision Trees), and explainability, which involves using external techniques to elucidate the internal mechanics of complex black-box models. This study specifically seeks to extract explainability from high-performance ensemble models to bridge the gap between algorithmic accuracy and user trust.

To bridge this gap, Altukhi et al. [6] and Sreeja et al. [5] detail the rise of model-agnostic XAI methods. SHapley Additive exPlanations (SHAP), grounded in cooperative game theory, has become the gold standard for quantifying global and local feature importance. It provides a consistent way to attribute a prediction's value to each feature, ensuring that the sum of feature contributions equals the model output. Similarly, Ribeiro et al. [17] introduced Local Interpretable Model-agnostic Explanations (LIME), which provides intuitive, perturbation-based explanations for individual instances by approximating the complex model with a simpler, interpretable one (like a linear model) locally around the prediction.

Beyond static explanations, the demand for actionable guidance has led to the development of counterfactual explanations. Wachter et al. [18] pioneered a framework emphasizing that explanations should not merely describe the current state, but prescribe the minimal perturbation required to alter an undesirable outcome. In medical domains, generating these ”Wachter-style” counterfactuals is vital because it shifts XAI from descriptive analytics to prescriptive intervention, offering patients specific, realistic targets (e.g., reducing BMI by a precise, minimal amount) rather than abstract, unachievable metrics.

2.4 The Reliability Gap: Probability Calibration

A predictive model is considered calibrated if its predicted probabilities align with the true empirical frequency of events. For instance, if a model predicts a 30% risk of hypertension for a specific group of patients, exactly 30% of that group should truly be hypertensive in reality.

Löfström et al. [11, 8] identify a pervasive issue in modern ML: complex models like Deep Learning and Boosting ensembles tend to be poorly calibrated. Because these algorithms focus on minimizing classification error or maximizing the margin, they often output overconfident probability scores (pushing probabilities towards 0 or 1). This results in a high Expected Calibration Error (ECE). In a clinical setting, an overconfident false alarm can lead to over-treatment and patient anxiety, while an underconfident miss can lead to negligence and a lack of early intervention. Therefore, calibration is not merely a statistical nicety but a safety requirement for decision support systems. Techniques such as Platt Scaling (fitting a sigmoid function) and Isotonic Regression (fitting a stepwise non-decreasing function) are standard post-hoc methods to correct this, while recent advancements like Venn-Abers predictors offer rigorous probability intervals that further quantify the empirical uncertainty of the prediction.

2.5 Synthesis and Research Gap

The intersection of these fields reveals a distinct research gap. While numerous studies, such as that by Eldem [9], apply ML to hypertension, and others like Niu et al. [19] apply XAI for transparency, there is a scarcity of systems that explicitly integrate probability calibration into the explanation pipeline, particularly for Philippine health data utilizing the National Nutrition Survey (NNS). Most existing systems explain the raw risk score, which poses a significant clinical danger: if the underlying model is uncalibrated, the XAI tool might convincingly explain a probability that is statistically incorrect. Furthermore, standard counterfactual generators often ignore the feasibility of the suggested changes, leading to mathematically optimal but clinically irrelevant recommendations.

This study addresses these gaps by adopting the Calibrated Explanations framework proposed by Löfström et al. [7], heavily augmented with Wachter-style counterfactual generation. By combining state-of-the-art classifiers—optimized via a rigorous Two-Stage Search—with post-hoc calibration methods and uncertainty-aware explanations, this research aims to produce a tool that is not only discriminatively accurate but also probabilistically reliable and transparent. Crucially, by integrating bounded optimization routines to ensure minimal clinical perturbation, this integration ensures that the explanations and counterfactuals provided to the user are mathematically grounded in corrected risk estimates and physiologically actionable, directly addressing the trust deficit in AI-driven healthcare.

3 Theoretical Framework

This section elucidates the theoretical foundations and operational principles of the core machine learning algorithms, sampling strategies, calibration methodologies, and explainability frameworks utilized in this research. It provides the mathematical and conceptual basis for the system's design.

3.1 Heart Disease and Hypertension Modeling

Hypertension is the pathological state of sustained elevated blood pressure. It is the single most significant risk factor for cardiovascular disease. In the context of predictive modeling, the task is formulated as a binary classification problem: mapping a vector of input features X (encompassing clinical, anthropometric, and dietary variables) to a target label $Y \in \{0, 1\}$, where 1 indicates the presence of hypertension based on the thresholds defined by the 2020 Philippine Clinical Practice Guidelines (SBP $\geq$ 140 mmHg or DBP $\geq$ 90 mmHg) [1], as measured in the NNS clinical component [2].

The DOST-FNRI 2015 National Nutrition Survey (NNS) was selected as the primary data source due to the critical need for cultural and genetic relevance. Existing hypertension models trained on Western populations (e.g., Framingham) or homogeneous datasets often fail to generalize to the Filipino demographic due to distinct dietary patterns, such as high sodium and rice consumption, as well as specific genetic predispositions and environmental factors. The 2015 NNS provides the most comprehensive, nationally representative snapshot of these local determinants, ensuring the model captures the specific risk profile of the Philippine population.

3.2 Data Preprocessing

Raw survey data is rarely suitable for direct algorithmic ingestion. Rigorous preprocessing is required to standardize inputs, mitigate biases inherent in data collection, and

prevent artificial performance inflation.

3.2.1 Target Definition and Leakage Guard

In predictive modeling, data leakage occurs when information from outside the training dataset is used to create the model, or when the model is inadvertently given access to the target variable during training. Since the binary ‘Hypertension‘ target is mathematically derived from the raw Systolic Blood Pressure (SBP) and Diastolic Blood Pressure (DBP) readings, retaining these raw continuous variables in the feature set would result in severe target leakage. To ensure the model learns to predict risk from behavioral, anthropometric, and dietary factors rather than simply ”re-identifying” the blood pressure measurements, a strict Leakage Guard is executed immediately after target creation to drop all direct and indirect BP-related variables.

3.2.2 Dimensionality Reduction via Collinearity Filtering

High dimensionality, particularly with highly correlated features, can destabilize machine learning models (multicollinearity) and dilute the interpretability of feature attributions. To address this, a Collinearity Filter is applied to the feature space. Variables exhibiting a Pearson correlation coefficient of $r > 0.70$ with another feature are flagged, and the redundant feature is purged. This ensures that the model trains only on high-signal, independent predictors, optimizing both computational efficiency and explanation clarity.

3.2.3 Handling Missing Values

Real-world survey data frequently contains missing entries due to non-response or measurement errors. To preserve data integrity and maximize the utilization of available information, this study employs advanced imputation techniques rather than simple row deletion.

The quality of K-Nearest Neighbors (KNN) imputation is highly sensitive to the choice of k (the number of neighbors) and the scale of the features. Since KNN relies on Euclidean distance to find similar respondents, variables with large magnitudes (e.g., Annual Income or Total Energy Intake) can disproportionately influence the neighbor selection process. Therefore, Standardization of all numerical variables is a prerequisite step before imputation. Furthermore, to objectively determine the optimal k, an artificial masking experiment is employed. This involves randomly hiding 1% of the observed (non-missing) values in the dataset to create a "masked" copy. The original values serve as the ground truth. The masked dataset is then imputed testing specific odd values for $k \in \{3, 5, 7, 9, 11\}$, and the imputed values are compared against the ground truth using Root Mean Squared Error (RMSE) to select the optimal k.

- **KNN Imputation:** For continuous numerical features, missing values are estimated using the K-Nearest Neighbors (KNN) algorithm with the optimized k. Unlike mean or median imputation which reduces variance, KNN imputation finds the most similar samples in the multi-dimensional feature space and imputes the missing value based on the median of these neighbors. This method preserves the local structure and correlations within the data.
- **Mode Imputation:** For categorical features, missing values are imputed using the most frequent category (mode) of the nearest neighbors to maintain the probability distribution of discrete variables.

3.2.4 Feature Scaling

Feature scaling normalizes the range of independent variables. This step is applied specifically to all continuous numerical features in the dataset (e.g., Age, BMI, Total Energy Intake). This is critical for distance-based algorithms where features with larger magnitudes would otherwise dominate the objective function.

- Standardization (Z-score Scaling): Rescales features to have a mean (μ) of 0

and standard deviation (σ) of 1. This technique assumes that the data follows a Gaussian distribution and is less affected by outliers than normalization.

$$X_{std} = \frac{X - \mu}{\sigma}$$

3.2.5 Class Imbalance Handling

Medical datasets frequently exhibit class imbalance, where the number of healthy individuals (majority class) significantly exceeds those with the condition (minority class). Standard algorithms often bias towards the majority class to maximize accuracy. This study explores resampling to address this by evaluating six distinct configurations:

- **Baseline:** No resampling or weighting applied, serving as the empirical control.
- **Class Weights (CW):** This technique modifies the algorithm's loss function to penalize the misclassification of the minority class more heavily than the majority class. It was selected as a cost-sensitive learning baseline because it addresses imbalance directly within the model optimization process without altering the original data distribution, preserving the integrity of the clinical features.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Synthesizes new minority instances by interpolating between existing continuous samples in the feature space [15]. This expands the decision boundary of the minority class without overfitting as severely as random oversampling.
- **SMOTENC (SMOTE for Nominal and Continuous):** An extension of SMOTE specifically designed to handle datasets with a mix of continuous and categorical features without requiring independent preprocessing steps.
- **Hybrid Approaches (SMOTE + CW and SMOTENC + CW):** Evaluates whether combining synthetic generation with algorithmic cost-sensitive learning yields superior discrimination boundaries compared to utilizing either method in isolation.

3.3 Machine Learning Algorithms

This study utilizes a diverse suite of supervised learning algorithms, ranging from interpretable baselines to complex ensembles. A comparative approach using eight distinct algorithms was chosen to establish a rigorous performance baseline and identify the optimal inductive bias for this specific dataset.

3.3.1 Linear and Instance-Based Models

Logistic Regression (LR) A foundational statistical model for binary classification, Logistic Regression estimates the probability that an instance belongs to a particular class using the logistic (sigmoid) function. For a feature vector $X = (x_1, \ldots, x_n)$, the predicted probability is calculated as:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

While mathematically a linear model, it serves as an essential, highly interpretable parametric baseline to contrast against the complex non-linear decision boundaries formed by the tree-based ensembles.

k-Nearest Neighbors (kNN) A non-parametric, instance-based learner, kNN classifies a new point based on the majority vote of its k closest neighbors in the feature space. Proximity is determined using a distance metric, predominantly the Euclidean distance for standardized continuous variables, defined between two points p and q as:

$$d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

This algorithm was chosen because it captures local similarities in patient profiles, effectively mimicking clinical case-based reasoning where a doctor might compare a patient to similar cases they have seen before. Figure 1 demonstrates this process: a

new data point (green circle) is classified based on whether the majority of its neighbors within a defined radius (dashed lines) belong to the ”blue square” class or the ”red triangle” class.

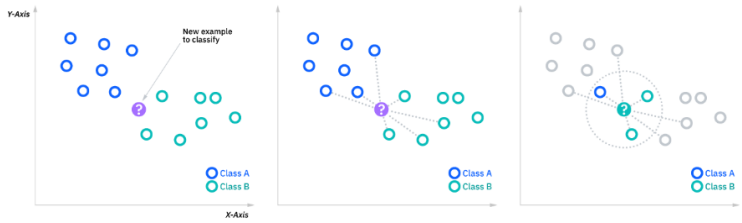

Figure 1: Example of k-Nearest Neighbors (kNN) classification. (Source: [20])

Naive Bayes (NB) A probabilistic classifier based on Bayes' Theorem, Naive Bayes assumes strong (naive) independence between features given the class label. While this assumption rarely holds in complex medical data, NB is computationally efficient. It was selected as a baseline model to justify the use of more complex algorithms; if a simple probabilistic model performs equivalently to an ensemble, the computational cost of the ensemble is unjustified. Furthermore, its probabilistic nature provides a useful reference point for assessing the calibration of more complex models.

3.3.2 Ensemble Methods

Ensemble methods combine multiple base estimators to improve generalizability and robustness over a single estimator.

Random Forest (RF) A bagging ensemble that constructs a multitude of decision trees at training time, Random Forest outputs the class selected by the majority of the trees. It was selected for its inherent resistance to overfitting via bagging and feature randomness. It serves as a strong benchmark for tree-based performance and is generally robust to noise, making it highly suitable for survey data. As depicted in Figure 2, the model trains multiple independent trees (Tree 1, Tree 2, etc.) on random subsets of

data, and their individual predictions are aggregated (via summation or majority vote) to produce the final result.

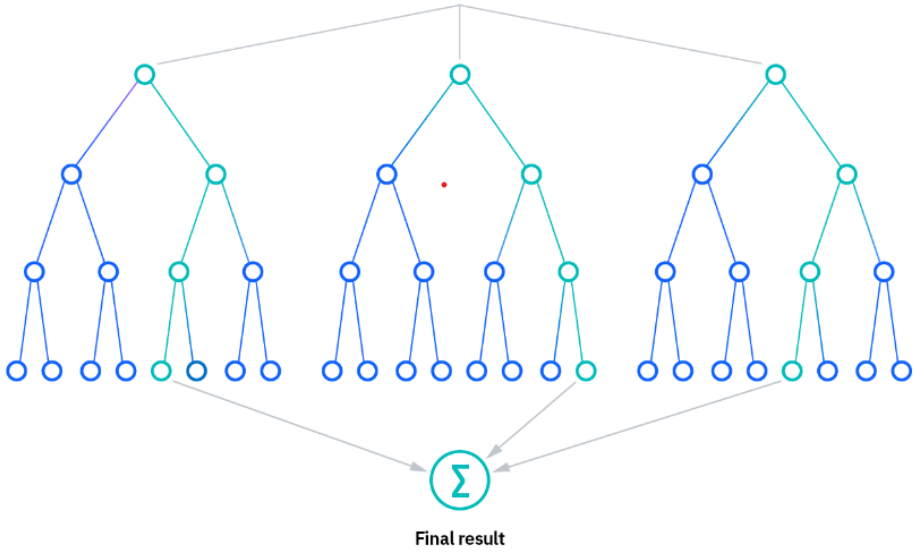

Figure 2: Diagram of a Random Forest (RF) model. (Source: [21])

Adaptive Boosting (AdaBoost) A boosting technique that trains learners sequentially, AdaBoost focuses subsequent learners on the instances that were misclassified by previous ones by increasing their weights. This iterative error correction converts a set of weak learners into a strong classifier. AdaBoost is included to test the efficacy of iterative bias reduction, which can be particularly effective for improving performance on hard-to-classify instances, such as borderline hypertension cases. Figure 3 illustrates this sequential process, where misclassified samples (shown as larger plus/minus signs) are given increasing weight in subsequent iterations to force the model to focus on them.

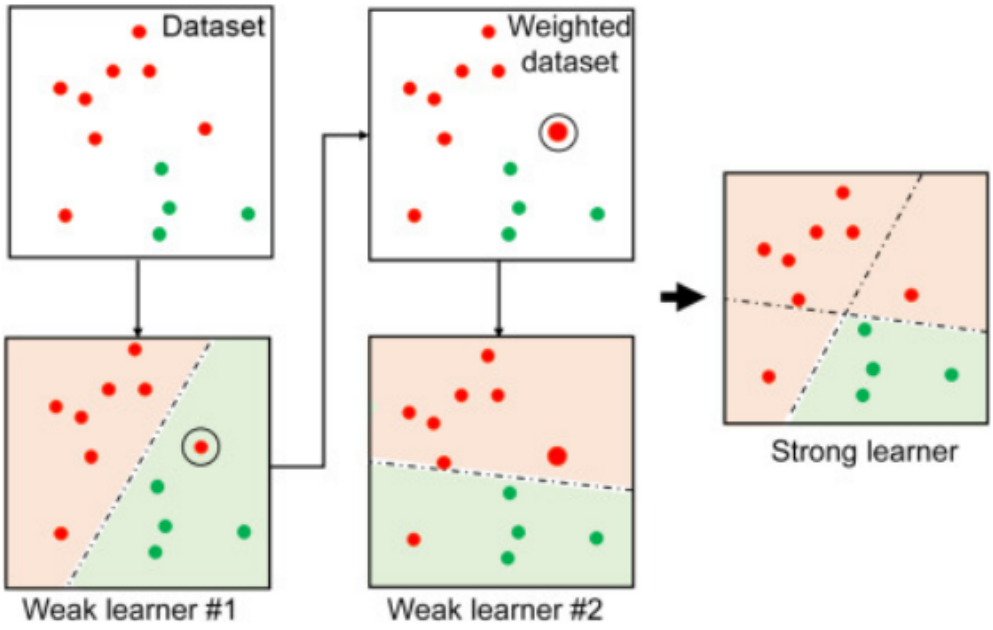

Figure 3: Conceptual model of Adaptive Boosting (AdaBoost). (Source: [22])

XGBoost (Extreme Gradient Boosting) An optimized distributed gradient boosting library, XGBoost builds trees sequentially to minimize a regularized objective function. It is renowned for its execution speed and performance in structured tabular data competitions. XGBoost was included specifically for its regularization capabilities (L1/L2), which are critical for preventing overfitting on high-dimensional medical data like the NNS, representing the current industry standard for tabular data performance. Figure 4 shows the additive nature of the model, where multiple weak learners (trees) are combined, with each new tree attempting to correct the residual errors of the previous ensemble.

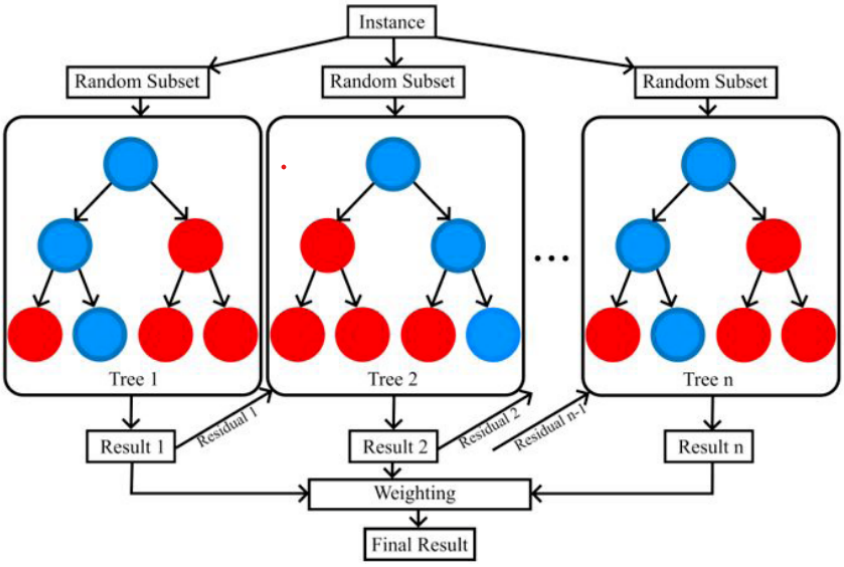

Figure 4: Simplified structure of an XGBoost model. (Source: [23])

LightGBM and CatBoost These are advanced variants of gradient boosting that offer specific advantages. LightGBM uses leaf-wise tree growth and histogram-based algorithms, making it highly efficient for large datasets; it was chosen to ensure the system remains scalable. CatBoost implements ordered boosting and processes categorical features natively. It was specifically selected to handle the numerous categorical variables in the NNS (e.g., region, smoking status) without the information loss or dimensionality explosion often caused by standard One-Hot Encoding [24, 25].

3.4 Two-Stage Rigorous Hyperparameter Optimization

To maximize the predictive performance of the selected algorithms while managing the computational demands of high-dimensional national survey data, the study abandons simplistic exhaustive grid searches. Instead, it employs a Two-Stage Rigorous Optimization workflow informed by the statistical efficiency of randomized parameter exploration [13]. This structured approach searches statistical distributions (e.g., uniform, log-uniform) rather than fixed discrete lists.

- **Stage 1 (Broad Exploration):** A Randomized Search is conducted across the entire allowable hyperparameter space of the chosen models. This stage utilizes 5-fold cross-validation running for 25 proxy epochs. This relatively short epoch allowance acts as an early-stopping mechanism during the search, allowing the system to rapidly evaluate diverse configurations and discard mathematically unpromising hyperparameter regions without wasting computational resources.

- **Stage 2 (Local Refinement):** The top candidate configurations identified in Stage 1 are isolated and subjected to a highly concentrated local search. This phase narrows the boundary distributions around the successful parameters and runs for 80 proxy epochs. This ensures meticulous fine-tuning of delicate weights, learning rates, and tree depths within the most promising ”neighborhoods” of the parameter space.

- **Final Training and Validation Evaluation:** The single best configuration that survives local refinement is advanced to final training. The model is trained on the full training split for up to 300 epochs, utilizing early stopping mechanisms evaluated against the validation holdout. This ensures the model converges optimally while definitively preventing overfitting.

3.5 Model Evaluation Framework

To rigorously assess the performance of the predictive models and the reliability of their probability estimates, a comprehensive suite of evaluation metrics is employed.

3.5.1 Discrimination Metrics

These metrics evaluate the model's ability to correctly classify individuals as hypertensive or normotensive.

- Accuracy: The proportion of true results (both true positives and true negatives)

among the total number of cases examined. While intuitive, it can be misleading in imbalanced datasets.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where TP (True Positive) is the number of hypertensive patients correctly identified, TN (True Negative) is the number of healthy patients correctly identified, FP (False Positive) is the number of healthy patients incorrectly identified as hypertensive, and FN (False Negative) is the number of hypertensive patients incorrectly identified as healthy.

- Recall (Sensitivity): The proportion of actual hypertensive cases that are correctly identified by the model. In medical screening, high recall is critical to minimize false negatives (missed diagnoses).

$$Recall = \frac{TP}{TP + FN}$$

- Precision: The proportion of predicted hypertensive cases that are actually positive. This metric is important for minimizing false alarms.

$$Precision = \frac{TP}{TP + FP}$$

- F1-Score: The harmonic mean of Precision and Recall, providing a single metric that balances the two. It is particularly useful when the class distribution is uneven.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Area Under the ROC Curve (AUC): Measures the ability of the model to distinguish between classes across all classification thresholds. An AUC of 0.5 represents random guessing, while 1.0 represents a perfect classifier. Values between 0.5 and 1.0 indicate that the model has some predictive power, with higher

values representing better discrimination capability.

3.5.2 Calibration Metrics

These metrics evaluate the reliability of the model's predicted probability scores.

- Log Loss (Cross-Entropy Loss): Penalizes confident but wrong predictions. A lower log loss indicates better calibrated probabilities.

$$LogLoss = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

  Where N is the total number of samples, y_i is the true label (0 or 1) of sample i, and p_i is the predicted probability that sample i belongs to the positive class.

- Expected Calibration Error (ECE): Measures the difference between predicted confidence and empirical accuracy. The predictions are grouped into bins, and the weighted average of the absolute difference between accuracy and confidence in each bin is calculated. To calculate ECE, the samples are partitioned into M equally spaced bins (e.g., 10 bins of size 0.1). Let B_m denote the set of indices of samples falling into bin m. The accuracy and confidence of bin m are defined as:

$$\text{acc}(B_m) = \frac{1}{|B_m|}\sum_{i \in B_m} \mathbf{1}(y_i = \hat{y}_i)$$

$$\text{conf}(B_m) = \frac{1}{|B_m|}\sum_{i \in B_m} \hat{p}_i$$

  The Expected Calibration Error is then calculated as:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|$$

  Lower ECE indicates a better-calibrated model.

3.6 Probability Calibration

Calibration addresses the reliability of the prediction. A model is calibrated if for all instances predicted with confidence p, the actual probability of the class is p. The inclusion of post-hoc calibration is the primary technical contribution of this study, addressing the reliability gap identified in the literature.

- Platt Scaling: A parametric method that fits a logistic regression model to the raw scores (logits) of the classifier. It assumes the distortion distribution is sigmoid. It produces a calibrated probability $\hat{P}$ using the formula:

$$\hat{P}(y = 1|f(x)) = \frac{1}{1 + \exp(Af(x) + B)}$$

  Where parameters A (slope) and B (intercept) are scalar parameters learned by minimizing the negative log-likelihood (Log Loss) on the calibration set. This was selected as the standard benchmark for parametric calibration, necessary to correct the tendency of models like SVM and boosting ensembles to produce uncalibrated scores.

- Isotonic Regression: A non-parametric method that fits a piecewise constant non-decreasing function to the raw scores. It learns a function g that transforms uncalibrated scores f_i to calibrated probabilities y_i by solving the following optimization problem:

$$\min_g \sum_{i=1}^{N} (y_i - g(f_i))^2$$

  Subject to the constraint that g must be monotonically non-decreasing ($x_i \leq x_j \implies g(x_i) \leq g(x_j)$). It is more flexible than Platt scaling and can correct non-sigmoid distortions, provided sufficient data is available. It serves as the standard benchmark for non-parametric calibration.

- Venn-Abers Predictors: A rigorous framework based on conformal prediction. Unlike point estimators, Venn-Abers produces a multi-probability prediction interval $[p_0, p_1]$. For a test instance with score f_{test}, it fits two separate Isotonic Regressions (g_0 assuming class 0, and g_1 assuming class 1) on the calibration set combined with the test instance. This yields:

$$p_0 = g_0(f_{test})$$

$$p_1 = g_1(f_{test})$$

A single calibrated probability point estimate p is derived using the minimax optimal combination:

$$p = \frac{p_1}{1 - p_0 + p_1}$$

This produces a probability interval that provides a more informative representation of predictive uncertainty, making it safer for medical decision-support settings than relying on a single uncalibrated point estimate.

3.7 Explainable Artificial Intelligence (XAI)

The Calibrated Explanations framework requires specific XAI methods that can operate on calibrated probabilities to ensure transparency.

3.7.1 SHAP (SHapley Additive exPlanations)

SHAP assigns each feature an importance value for a particular prediction [26]. Based on the concept of Shapley values from cooperative game theory, it calculates the marginal contribution of feature i across all possible feature subsets S (out of the total set of features F):

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{i\}) - f_x(S)]$$

Where $f_x(S)$ is the model's prediction for instance x using only the subset of features S. This mathematical foundation ensures that the feature attribution is fair and consistent. SHAP was selected because it is applied for both **global attribution** (to interpret overarching model behavior and dataset-wide risk drivers) and **local attribution** (to explain exactly why a specific patient was flagged as high risk). This capability is essential for personalized medicine and actionable insights.

3.7.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME explains the predictions of any classifier by learning an interpretable model (such as linear regression) locally around the prediction. It perturbs the input data to probe how the model behaves in the immediate vicinity of the instance x. Mathematically, LIME solves the following optimization problem:

$$\text{explanation}(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

Where $\mathcal{L}$ is a loss function measuring how unfaithful g is to f in the locality defined by the proximity measure π_x, and $\Omega(g)$ is a penalty for the complexity of the explanation model [17]. LIME was chosen for its simplicity and model-agnostic nature. While SHAP is more theoretically consistent globally, LIME provides linear approximations that are often more intuitive for non-technical stakeholders (patients) to understand, serving as an effective first-glance explanation in the web application.

3.7.3 The Calibrated Explanations Framework

This study integrates the above components using the framework proposed by [7]. Instead of explaining the raw output of a black-box model, this framework first calibrates the model and then generates explanations (feature weights and counterfactuals) based on the reliable, calibrated probabilities. This is the core novelty of the study: ensuring that the trust built by the explanation is grounded in a reliable risk estimate, preventing users from being misled by overconfident or underconfident predictions.

3.7.4 Counterfactual Explanations via Wachter-Style Bounded Optimization

The generation of counterfactual explanations in this study adopts the prescriptive principles introduced by Wachter et al. [18]. Instead of simply searching a predefined grid or optimizing for an arbitrary geometric distance, the system formulates counterfactual generation as a rigorous mathematical optimization problem. The goal is to find a counterfactual feature value x' that safely crosses the selected screening boundary while minimizing the perturbation from the patient's original state x.

For each actionable continuous feature, the system uses bounded Brent's method via `scipy.optimize.minimize_scalar` to minimize the following objective function [27]:

$$L(x') = \lambda \cdot (f(x') - y_{target})^2 + \frac{(x' - x)^2}{\sigma^2}$$

Where:

- $f(x')$ is the model's predicted probability at the proposed counterfactual value x'.
- $y_{target} = 0.34$ represents the target probability slightly below the selected 0.35 screening threshold. This acts as an anchor to ensure that the proposed change shifts the prediction below the deployed screening boundary rather than the default

0.50 classification boundary.

- $\lambda = 0.5$ is the hyperparameter balancing the trade-off between reaching the exact target probability and keeping the feature change as minimal as possible.
- The term $\frac{(x'-x)^2}{\sigma^2}$ serves as a variance-normalized proximity penalty. Dividing by the feature's variance or scale width (σ^2) ensures that features with large native scales (e.g., caloric intake) are not unfairly penalized compared to features with small scales (e.g., Waist-to-Hip Ratio).

Unlike naive grid-sweep methods, Brent's optimization efficiently searches for the optimal single-feature change within the physiological bounds of that variable.

4 Design and Implementation

This section details the practical realization of the theoretical framework into a functioning software system. It delineates the system architecture, the specific composition of the dataset, the data engineering processes, the hardware infrastructure required for model development, and the software design patterns employed to build the web application prototype.

4.1 System Framework

The research implements a full-stack Data Science pipeline. The workflow is structured into sequential phases: Data Ingestion, Preprocessing, Model Training, Calibration, and Deployment. This flow ensures that raw survey data is systematically transformed into actionable clinical insights.

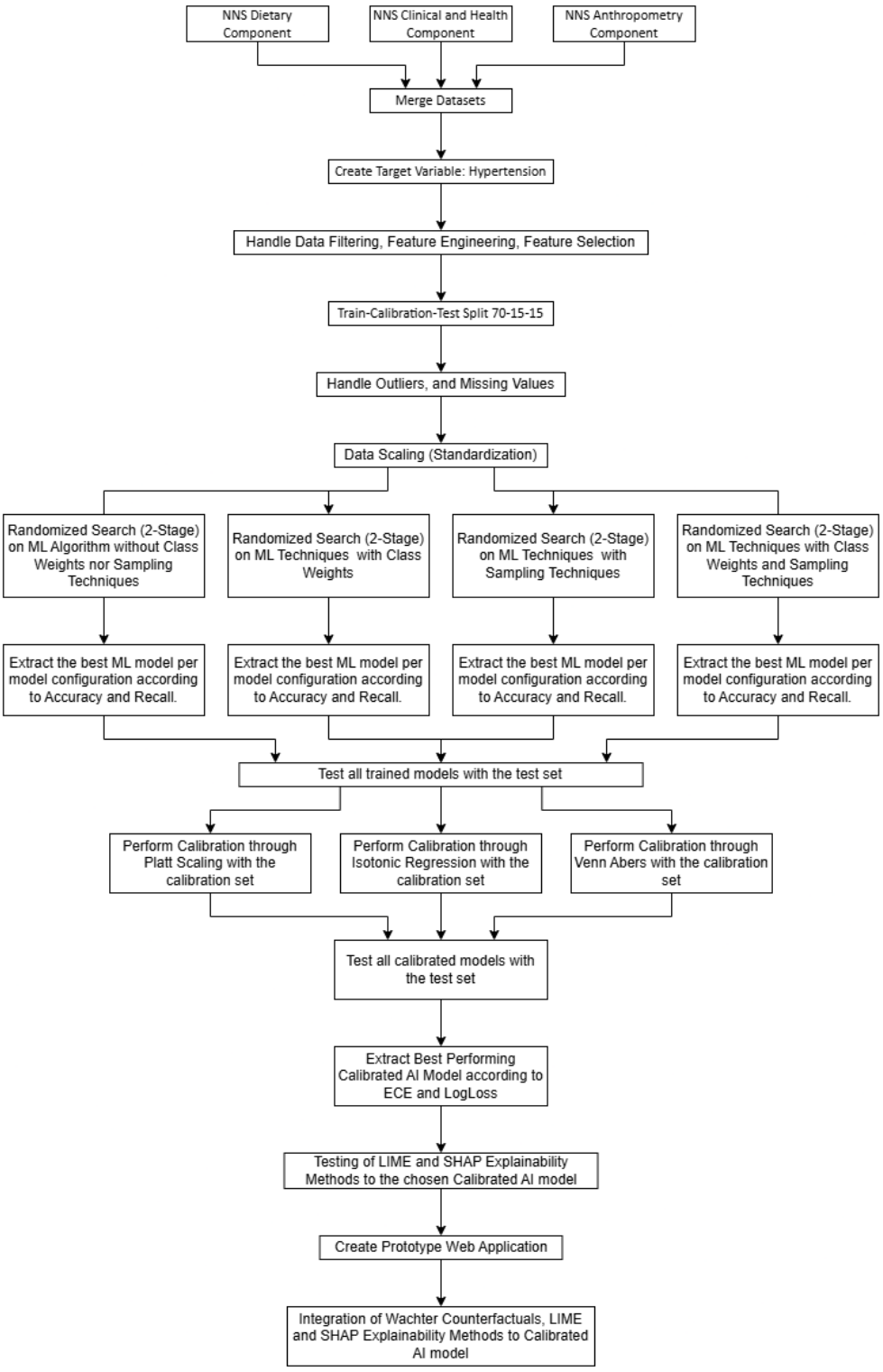

Figure 5: Methodological Workflow Diagram.

4.2 Data Source and Variable Definitions

4.2.1 The DOST-FNRI 2015 NNS Dataset

The study utilizes microdata from the 2015 National Nutrition Survey (NNS) conducted by the Department of Science and Technology - Food and Nutrition Research Institute (DOST-FNRI). To create a comprehensive feature set for hypertension modeling, three distinct survey components were merged using unique respondent identifiers:

1. **Clinical and Health Component:** Provides the ground truth labels (blood pressure measurements) and behavioral risk factors (smoking, alcohol consumption).
2. **Anthropometric Component:** Provides physical body measurements necessary for calculating indices like BMI, Waist, and Hip circumferences.
3. **Individual Dietary Component:** Provides granular data on daily food intake, aggregated into 27 specific food groups and macronutrient totals.

4.2.2 Data Variables

The final dataset consists of 53 variables: 52 predictors and 1 target variable. Table 1 enumerates these variables, their roles in the model, and their descriptions. Note that `fe_alcohol_level` and `fe_smoking_level` are engineered features derived from raw survey questionnaires to create cleaner categorical inputs.

Table 1: Variables Table for Hypertension Prediction Model: Clinical, Anthropometric, and Lifestyle Features

Variable Name	**Role**	**Description**
`Hypertension`	Target	Binary outcome (1: Hypertensive [$\geq 140/90$ mmHg], 0: Normal)
`age`	Feature	Age of Respondent (years)
`sex`	Feature	Biological Sex (1: Male, 2: Female)
`ethnicity`	Feature	Ethnicity Code (0: Non-IP, 1: IP, 2: 2/3 Filipino, 3: 1/2 Foreign)
`pa_met`	Feature	Physical Activity (Metabolic Equivalent)
`fbs`	Feature	Fasting Blood Sugar (mg/dL)
`chol`	Feature	Total Cholesterol (mg/dL)
`tri`	Feature	Triglycerides (mg/dL)
`hdl`	Feature	High-Density Lipoprotein (mg/dL)
`ldl`	Feature	Low-Density Lipoprotein (mg/dL)
`waist`	Feature	Waist circumference (cm)
`hip`	Feature	Hip circumference (cm)
`BMI`	Feature	Body Mass Index (kg/m^2)
`fe_alcohol_level`	Feature	0: Never, 1: Former, 2: Moderate, 3: Binge; derived from alcohol status and binge drinking variables.
`fe_smoking_level`	Feature	0: Never, 1: Former, 2: Occasional, 3: Daily; derived from smoking status and current smoking variables.

Table 2: Variables Table for Hypertension Prediction Model: Dietary Food Groups

Variable Name	**Role**	**Description**
epwt_fg1	Feature	Cereals and cereal products
epwt_fg2	Feature	Rice and rice products
epwt_fg3	Feature	Corn and corn products
epwt_fg4	Feature	Other cereal products
epwt_fg5	Feature	Starchy roots and tubers
epwt_fg6	Feature	Sugar and syrups
epwt_fg7	Feature	Dried beans, nuts, and seeds
epwt_fg8	Feature	Vegetables
epwt_fg9	Feature	Green leafy and yellow vegetables
epwt_fg10	Feature	Other vegetables
epwt_fg11	Feature	Fruits
epwt_fg12	Feature	Vitamin C-rich fruits
epwt_fg13	Feature	Other fruits
epwt_fg14	Feature	Fish, meat, and poultry
epwt_fg15	Feature	Fish and fish products
epwt_fg16	Feature	Meat and meat products
epwt_fg17	Feature	Poultry
epwt_fg18	Feature	Eggs
epwt_fg19	Feature	Milk and milk products
epwt_fg20	Feature	Whole milk
epwt_fg21	Feature	Other milk products
epwt_fg23	Feature	Fats and oils
epwt_fg24	Feature	Miscellaneous
epwt_fg25	Feature	Beverages
epwt_fg26	Feature	Condiments and spices
epwt_fg27	Feature	Other miscellaneous items

Table 3: Variables Table for Hypertension Prediction Model: Nutrient Totals

Variable Name	Role	Description
Total_Food_epwt	Feature	Total food weight (g)
Total_Ener	Feature	Total Energy (kcal)
Total_Prot	Feature	Total Protein (g)
Total_Calc	Feature	Total Calcium (mg)
Total_Iron	Feature	Total Iron (mg)
Total_VitA	Feature	Total Vitamin A (RE)
Total_VitC	Feature	Total Vitamin C (mg)
Total_Thia	Feature	Total Thiamin (mg)
Total_Ribo	Feature	Total Riboflavin (mg)
Total_Nia	Feature	Total Niacin (mg)
Total_CHO	Feature	Total Carbohydrate (g)
Total_Fat	Feature	Total Fat (g)

4.3 Data Preprocessing and Feature Engineering

This section details the rigorous data transformation pipeline applied to the raw NNS microdata to ensure data quality, relevance, and model readiness.

4.3.1 Data Merging Strategy

The study integrates three distinct datasets—Clinical, Anthropometric, and Dietary—to create a unified respondent profile. The merge was performed using a **left join** operation on the primary key formed by the unique household number (hhnum) and member code (member_code). The Clinical dataset served as the left (base) table to ensure that only respondents with recorded physiological data were retained.

4.3.2 Data Cleaning and Filtering

To align the dataset with the study's scope and clinical guidelines, specific filtering steps were applied:

- **Dropping Invalid Target Entries:** Any record missing values for both Systolic Blood Pressure (SBP) and Diastolic Blood Pressure (DBP) was immediately dropped, as ground-truth hypertension status could not be established for these individuals.
- **Exclusion of Non-Relevant Anthropometric Groups:** The raw dataset included specific "anthrogroups" that fall outside the scope of standard adult hypertension guidelines. The following groups were dropped:
  - **Infants and Children (0–120 months):** 43,459 infants (0-59 mos) and 59,383 children (60-120 mos) were excluded as pediatric hypertension requires distinct percentile-based guidelines not covered by the 140/90 mmHg adult threshold.
  - **Adolescents (121–228 months):** 90,682 entries were initially flagged in the clinical component. The 2020 Philippine Clinical Practice Guidelines mandate a switch from percentile-based pediatric diagnosis to fixed adult cutoffs ($\geq 140/90$ mmHg) starting at age 13 [1]. While it is technically possible to isolate individuals aged 13 and above by cross-referencing their exact chronological age from the anthropometry dataset post-merge, integrating this cohort would necessitate the development of two distinct machine learning pipelines—one utilizing percentile-based targets for early adolescents and another utilizing fixed thresholds for older adolescents and adults. To prevent excessive architectural complexity and maintain a highly optimized, singular focus on the adult population, this demographic was excluded entirely.
  - **Pregnant and Lactating Women:** 3,625 pregnant women and 10,904 lac-

tating mothers were excluded due to confounding physiological factors (e.g., gestational hypertension) that require specialized medical management distinct from the general adult population.

- The final dataset retained only the Adults (19.08 years and above) group, resulting in a consolidated analytical dataset of exactly 151,189 rows for analysis.

4.3.3 Feature Engineering

New variables were constructed to capture clinically relevant risk factors:

- **Target Variable (Hypertension):** A binary target variable was engineered based on the 2020 Philippine Clinical Practice Guidelines. A respondent is labeled as Hypertensive (1) if their average SBP $\geq$ 140 mmHg OR average DBP $\geq$ 90 mmHg; otherwise, they are labeled Normal (0).
- **Body Mass Index (BMI):** Calculated from weight and height measurements using the standard formula kg/m^2.
- **Lifestyle Levels & Cascading Fallbacks:** Raw questionnaire responses were aggregated into ordinal levels for Smoking (0-3) and Alcohol (0-3). To maximize data retention, the backend implementation utilizes a cascading fallback logic: if a direct aggregate code (e.g., `smoke_status`) is missing or recorded as "Not Applicable" (codes like 99, 888888), the system systematically infers the behavioral level from supporting raw columns (e.g., falling back to `current_smoking`, then to `ever_smk`) to construct a robust, unified categorical feature.

4.3.4 Handling Missing Values and Outliers

- **Outlier Detection:** Numerical features were screened for biologically implausible values (e.g., negative age, extreme caloric intake) which were treated as missing values to prevent model distortion.

- **Standardization for Imputation:** Prior to imputation, all numerical variables are standardized using a 'StandardScaler'. This step is essential because KNN relies on Euclidean distance, which is sensitive to scale; standardizing ensures that variables with large magnitudes (e.g., Annual Income) do not dominate the neighbor selection process.

- **Optimization of Imputation Parameters:** To objectively determine the optimal number of neighbors (k) for imputation, an artificial masking experiment is conducted. This involves randomly hiding 1% of the observed (non-missing) values in the dataset to create a "masked" copy. The original values serve as the ground truth. The masked dataset is then imputed testing specific odd values for $k \in \{3, 5, 7, 9, 11\}$, and the imputed values are compared against the ground truth using RMSE (for numerical variables) to select the k that yields the lowest error.

- **KNN Imputation Application:** Using the optimal k identified from the masking experiment, missing values are imputed. For numerical features, missing entries are filled with the median of the neighbors. For categorical features, the mode (most frequent value) of the neighbors is used.

4.3.5 Feature Selection

To reduce dimensionality and noise, redundant variables were removed:

- **Identifiers:** Administrative columns like province, region, and survey weights were dropped as they are not clinical predictors.

- **Redundant Dietary Weights:** Raw food weight variables (fg#) were dropped in favor of their edible portion weight equivalents (epwt_fg#), which more accurately represent actual consumption.

- **Origin Variables:** Intermediate variables used solely for engineering the lifestyle levels (e.g., raw 'smoke_status' strings) were removed to prevent information

leakage and multicollinearity.

4.4 Model Development and Optimization Strategy

The study employs a rigorous, multi-stage selection process to identify the optimal model.

4.4.1 Stage 1: Randomized Search and Hyperparameter Spaces

A Randomized Search with Stratified K-fold Cross-Validation was performed across the eight algorithms (XGBoost, CatBoost, LightGBM, Random Forest, AdaBoost, Logistic Regression, k-Nearest Neighbors, and Naive Bayes). Table 4 details the exact statistical distributions and parameter boundaries established for the optimization grid.

Table 4: Hyperparameter Search Space Distributions for Two-Stage Optimization

Algorithm	Hyperparameter	Search Space Boundary / Distribution
Logistic Regression	C (Inverse Regularization)	Log-Uniform [0.0001, 100.0]
	solver	Categorical ['lbfgs', 'liblinear']
k-Nearest Neighbors	n_neighbors	Integer [3, 30]
	weights	Categorical ['uniform', 'distance']
	metric	Categorical ['euclidean', 'manhattan']
Naive Bayes	var_smoothing	Log-Uniform [1e-12, 1e-8]
AdaBoost	n_estimators	Integer [50, 300]
	learning_rate	Uniform [0.01, 0.50]
	base_depth	Integer [1, 4]
Random Forest	max_features	Uniform [0.2, 1.0]
	max_depth	Categorical [None, 10, 20, 30]
	min_samples_split	Integer [2, 19]
	min_samples_leaf	Integer [1, 9]
XGBoost	learning_rate	Log-Uniform [0.01, 0.30]
	max_depth	Integer [3, 8]
	subsample / colsample_bytree	Uniform [0.6, 1.0]
	reg_lambda (L2)	Log-Uniform [0.01, 10.0]
LightGBM	learning_rate	Log-Uniform [0.01, 0.30]
	max_depth	Integer [3, 9]
	num_leaves	Integer [15, 149]
	subsample / colsample_bytree	Uniform [0.6, 1.0]
CatBoost	learning_rate	Log-Uniform [0.01, 0.30]
	depth	Integer [4, 9]
	l2_leaf_reg	Uniform [1.0, 10.0]
	random_strength	Uniform [0.1, 2.0]

- **Configurations Tested:** Each algorithm was strictly evaluated across six handling methodologies: (1) Baseline (No sampling), (2) Class Weights, (3) SMOTE, (4) SMOTENC, (5) SMOTE + Class Weights, and (6) SMOTENC + Class Weights.
- **Selection Criteria:** During Stage 1, models were allocated a limited epoch budget to act as an early-stopping mechanism. For every algorithm configuration, the top

models were extracted based on their F1-Score (to balance precision/recall) and AUC (to assess discrimination capability).

4.4.2 Stage 2: Secondary Selection for Calibration

From the pool of top models extracted in Stage 1, a concentrated local search and second filter was applied to select the candidates for calibration.

- **Selection Criteria:** The top performing models per algorithm type were selected based on Recall (sensitivity to hypertension cases) and Accuracy.

4.4.3 Stage 3: Probability Calibration and Final Selection

The selected models underwent post-hoc calibration using Platt Scaling, Isotonic Regression, and Venn-Abers Predictors.

- **Final Selection Criteria:** The calibrated models were evaluated using a clinically oriented hierarchy rather than a single metric alone. Recall was prioritized to preserve hypertension-positive case detection, AUC was used to verify discriminative ability, and ECE and Log Loss were used to assess probability reliability after calibration. The final model was selected based on its overall suitability for screening deployment, balancing sensitivity, discrimination, calibration quality, and compatibility with the explainability pipeline.

4.4.4 Stage 4: Explainability and Counterfactual Generation

Following the selection of the final calibrated model, the explainability module is instantiated to support the user interface outputs.

- **Feature Masking for Actionability:** To ensure that the generated counterfactuals are clinically realistic, a feature mask is defined. Variables such as Age, Sex, Eth-

nicity, and Height are marked as **Immutable** (cannot be changed). Variables such as BMI, Dietary Intake, and Smoking/Alcohol levels are marked as **Actionable**.

- **Counterfactual Search via Bounded Optimization:** To ensure recommendations are both optimal and physiologically possible, the system utilizes bounded Brent's method (`scipy.optimize.minimize_scalar`) to find the minimal perturbation of a given actionable feature that reduces the predicted risk below the selected 0.35 screening threshold. A variance-normalized penalty is applied to prioritize recommendations requiring the smallest absolute lifestyle change.

4.5 System Design

4.5.1 Context Diagram

The system operates as a web-based application where the user provides health parameters, and the system returns a comprehensive risk assessment. The core process involves data standardization, model inference, probability calibration, and explanation generation. Specifically, the system receives Demographic Data, Clinical Data, and Dietary Intake directly from the user. It accesses an external Model Store containing the serialized ensemble model and Venn-Abers calibrator (persisted via a `joblib` pipeline). Finally, it outputs the Calibrated Risk Score (with its Uncertainty Interval), Tiered Explanations (LIME summary and SHAP waterfall), and prescriptive Actionable Counterfactuals back to the user.

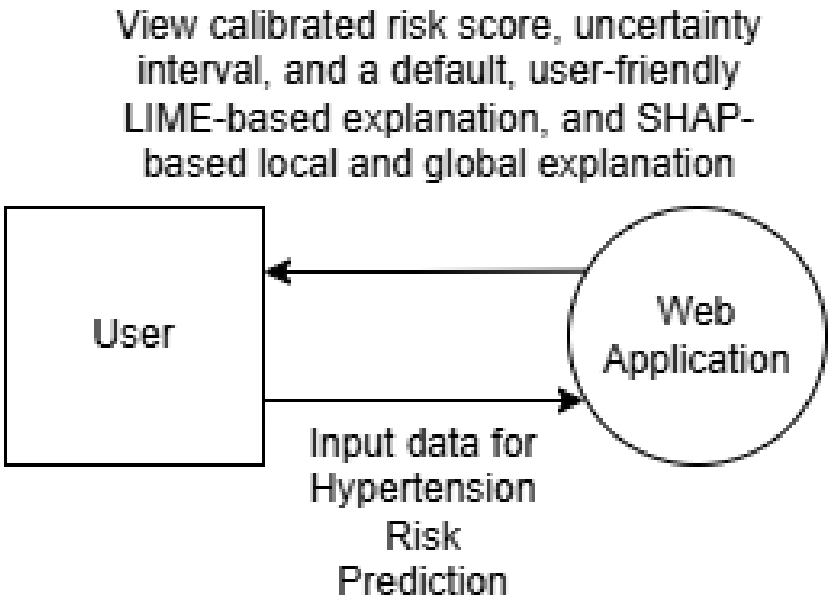

Figure 6: System Context Diagram.

4.5.2 Use-Case Diagram

Figure 7 illustrates the functional requirements of the system. The primary actor, the User (representing either a Patient or Clinician), interacts with the system to execute the following core functions:

- **Input Health Data:** Enter demographic, clinical, and dietary information. This process features an <<include>> relationship to *Calculate Dietary Totals*, representing the dynamic auto-computation engine that reduces user friction.
- **View Risk Assessment:** Includes viewing the calibrated probability score alongside its rigorous mathematical uncertainty interval.
- **View Explanations:** The user is provided a default, accessible LIME explanation. This use case contains an <<extend>> relationship to *Request SHAP Explanation (Advanced)*, indicating that deeper global and local SHAP attributions are an optional, user-triggered flow.

- **View Actionable Counterfactuals:** Review the prescribed lifestyle and dietary modifications required to safely lower the predicted risk class.

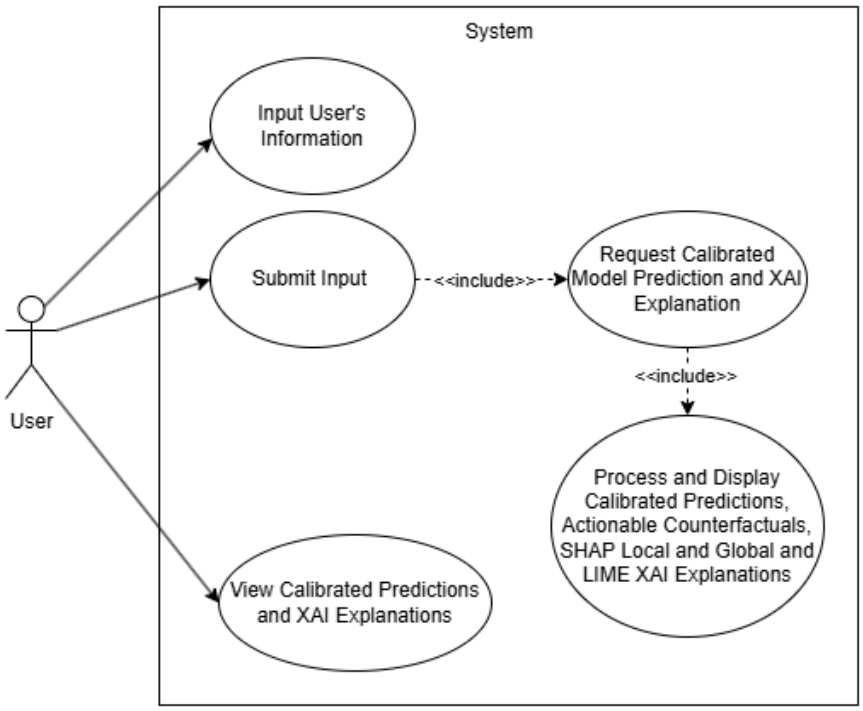

Figure 7: System Use Case Diagram.

4.5.3 Activity Diagram

The operational flow is depicted in Figure 8, detailing both sequential processing and parallel explanation generation:

1. The user initiates the process by inputting their health data.
2. The system preprocesses the input through strict scaling and categorical encoding.
3. The backend ensemble model evaluates the data to generate an uncalibrated *Raw Score*.
4. The calibration layer applies the Venn-Abers transformation to the raw score to calculate the true probability estimate and uncertainty interval.

5. **Parallel Fork:** The system simultaneously (1) generates a local LIME explanation and (2) executes bounded Brent's optimization to generate the Wachter-style counterfactuals.

6. **Join:** The parallel explanation processes consolidate, and the frontend renders the default Results Dashboard for the user.

7. **Decision Node:** The system checks if the user has requested the advanced view. If yes, it computes and displays the SHAP waterfall plot. If no, the process gracefully terminates.

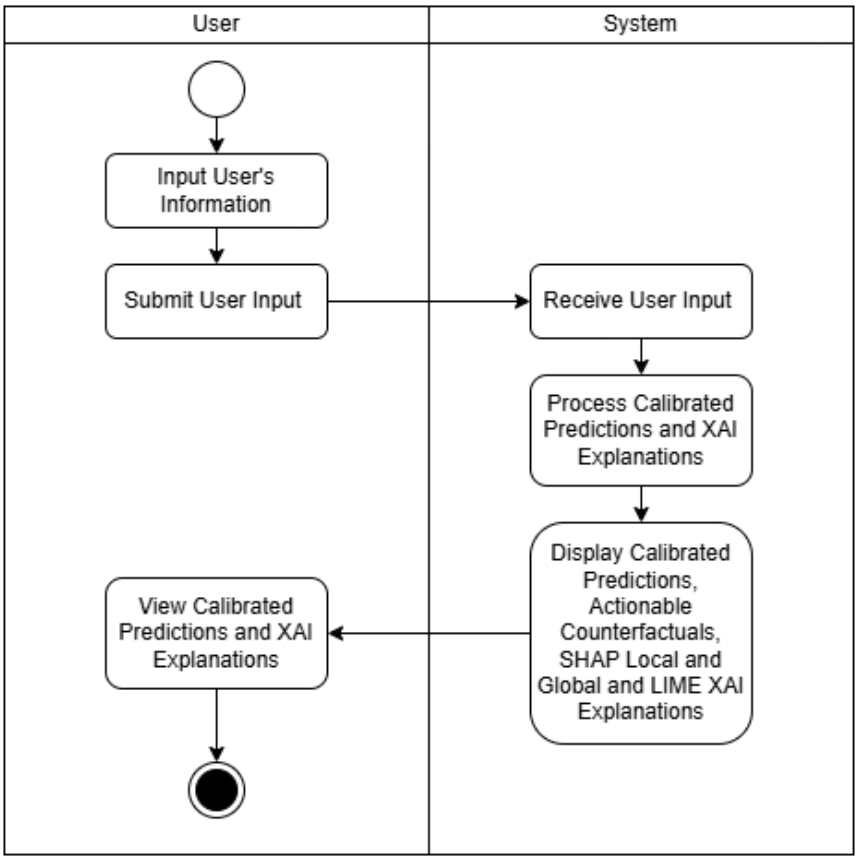

Figure 8: System Activity Diagram.

4.6 Technical Architecture

The system is built on a robust technical stack designed to support both the heavy computational load of model training and the lightweight requirements of the end-user web application.

4.6.1 System Architecture

The software architecture follows a modular design to ensure scalability and maintainability:

- **Backend:** Developed in Python 3.12.3, leveraging libraries such as scikit-learn, PyTorch, and calibrated-explanations for the core logic.
- **Frontend:** Built using Streamlit, allowing for rapid prototyping of interactive data visualizations.
- **Model Persistence:** The trained pipelines (preprocessing + model + calibrator) are serialized using joblib for efficient loading during runtime.

4.6.2 Hardware Requirements for Model Training

The training phase involves computationally intensive tasks such as randomized search for hyperparameter optimization and the calibration of ensemble models. The recommended specifications for the development environment are:

- **Processor:** Multi-core CPU with at least 8 cores to handle parallel processing tasks.
- **Memory (RAM):** Minimum of 16 GB to accommodate the in-memory processing of the large merged NNS dataset.

- **GPU:** Dedicated GPU with CUDA support (e.g., NVIDIA RTX 3050 or higher) to accelerate the training of gradient boosting models (XGBoost, LightGBM, CatBoost) and parallelized ensembles utilizing cuML (Random Forest).
- **Storage:** SSD storage for fast I/O operations during data loading and model serialization, with at least 20 GB of free storage for imports and the dataset.

4.6.3 Requirements for Web Application Usage

The deployment environment is designed to be lightweight and accessible to users with standard consumer-grade hardware.

- **Device:** Any modern computer, tablet, or smartphone.
- **Operating System:** Platform-independent (Windows, macOS, Linux, Android, iOS).
- **Web Browser:** A modern web browser with JavaScript enabled (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).
- **Internet Connection:** A stable internet connection is required to access the Streamlit application hosted on the server.

4.7 Project Development Timeline

The development of the hypertension prediction system is structured over a 24-week period, divided into five distinct phases ranging from data acquisition to final system evaluation.

Table 5: Proposed Development Timeline

Phase	**Activity**	**Duration**	**Description**
Phase 1	Data Acquisition & Preprocessing	Weeks 1-4	Requesting NNS microdata, merging datasets, cleaning data (imputation, outlier removal), and engineering features.
Phase 2	Model Development & Optimization	Weeks 5-10	Implementing 8 algorithms, Randomized Search optimization, and evaluating imbalance handling configurations (Baseline, CW, SMOTE, SMOTENC, SMOTE+CW, SMOTENC+CW).
Phase 3	Calibration & XAI Integration	Weeks 11-15	Implementing Venn-Abers and Isotonic calibration, generating SHAP/LIME explanations, and validating ECE/Log Loss.
Phase 4	System Development (Web App)	Weeks 16-20	Developing the Streamlit interface, integrating the backend model pipeline, and designing the tiered explanation UI.
Phase 5	Testing, Evaluation & Writing	Weeks 21-24	Conducting system testing, analyzing final results, and writing the manuscript.

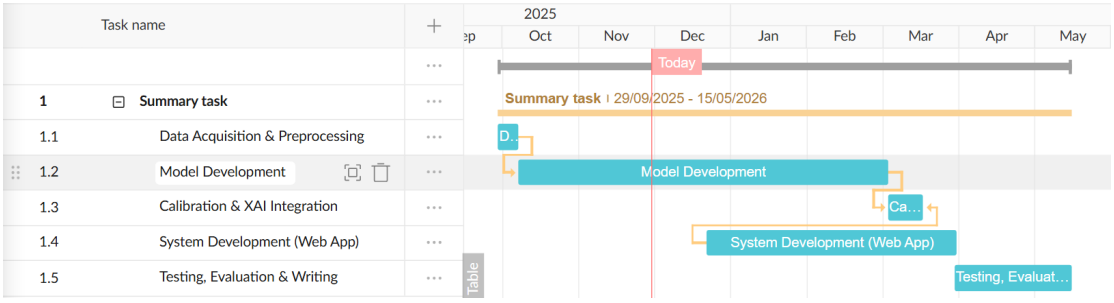

Figure 9: Gantt Chart of Project Development Timeline.

4.8 User Interface Design

This section presents the graphical user interface (GUI) of the web application, architected specifically to maximize clinical usability, reduce cognitive load during data entry, and support mathematical transparency in its predictions

4.8.1 Landing Page

The Landing Page (Figure 10) serves as the entry point, establishing immediate trust and guiding the user to the assessment tool. It features a clean, medical-grade aesthetic with a prominent ”Start Risk Assessment” call-to-action. To contextualize the clinical importance of the tool, the application dynamically displays key statistics regarding the prevalence of hypertension (”The Silent Killer”) in the Philippines. The deployed prototype utilizes the empirical baseline prevalence of **15.20%**, derived directly from the consolidated 2015 NNS adult dataset under the strict $\geq$ 140/90 mmHg clinical guidelines, anchoring the user's expectations to true epidemiological data.

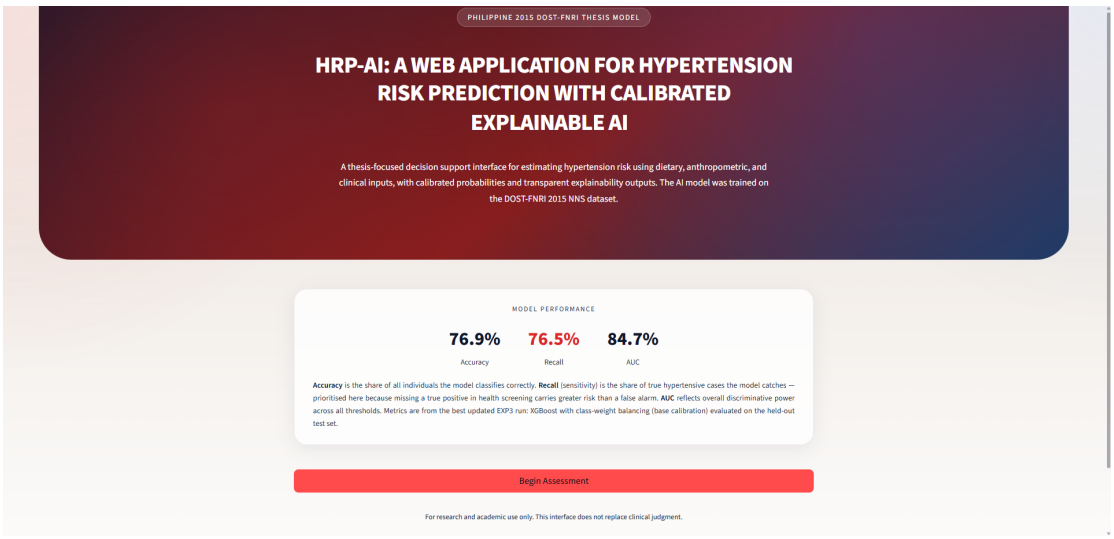

Figure 10: Web Application Landing Page Establishing Epidemiological Context.

4.8.2 Progressive Input Interface and Dynamic Computations

To prevent user fatigue and ensure strict data integrity, the Input Interface is structured as a progressive, four-step form (Figures 11 through 14). The interface guides the user through logical clinical groupings: Clinical Profile, Anthropometrics, Lifestyle/Vices, and Dietary Intake.

Crucially, the Streamlit frontend utilizes a **Dynamic Auto-Computation Engine** to eliminate manual calculation errors. In the Anthropometric step (Figure 12), the system dynamically calculates the patient's Body Mass Index (BMI) in real-time as height and weight are entered. Similarly, in the Dietary Intake module, the system auto-sums total caloric energy utilizing standard Atwater conversion factors (e.g., 4 kcal/g for carbohydrates and proteins, and 9 kcal/g for fats). This real-time mathematical synchronization ensures that the backend predictive engine receives a complete, internally consistent feature array without overwhelming the end-user.

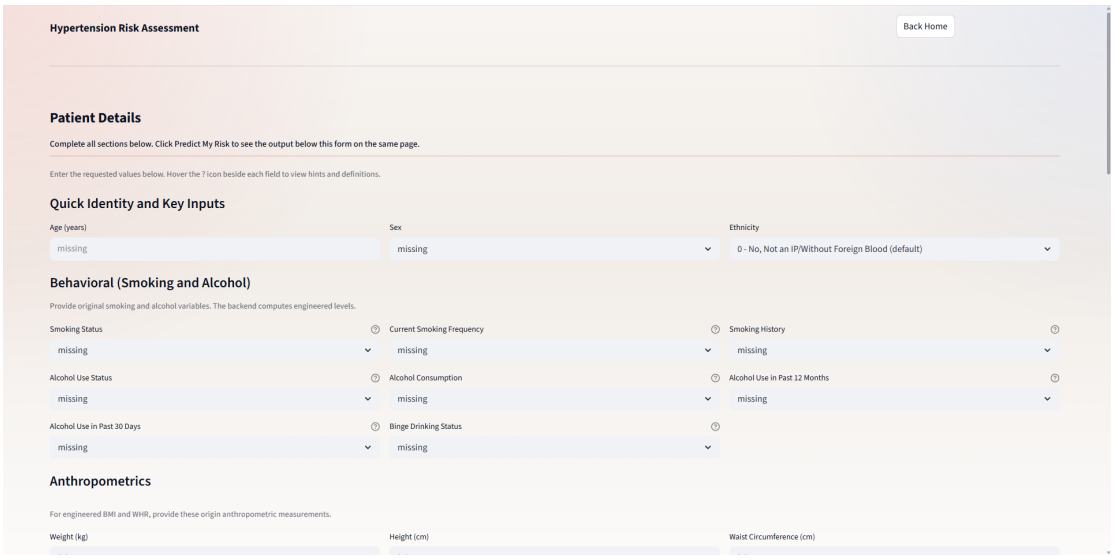

Figure 11: Input Step 1: Foundational Clinical and Biological Profile.

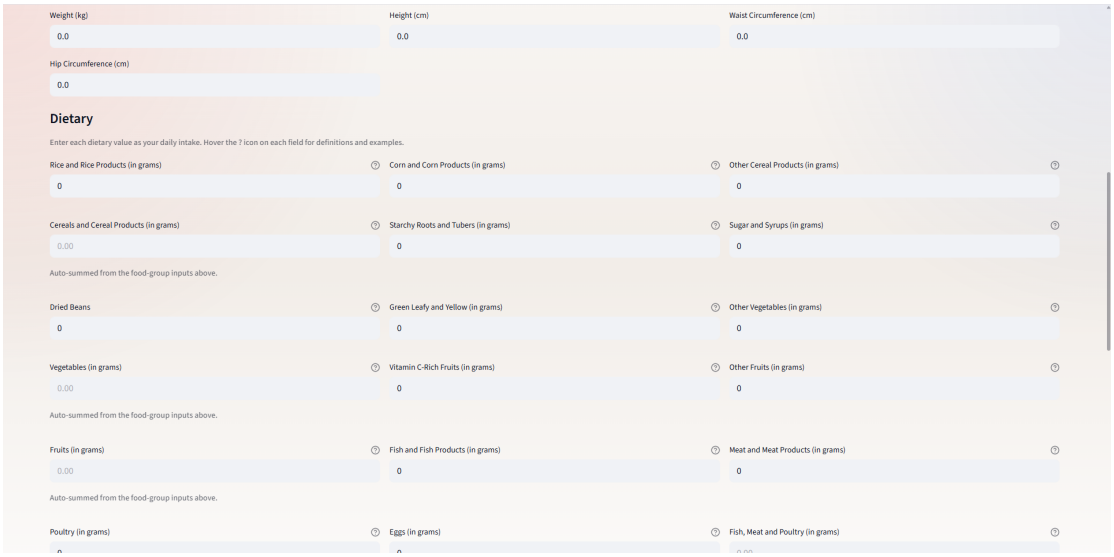

Figure 12: Input Step 2: Anthropometric Measurements with Dynamic BMI Calculation.

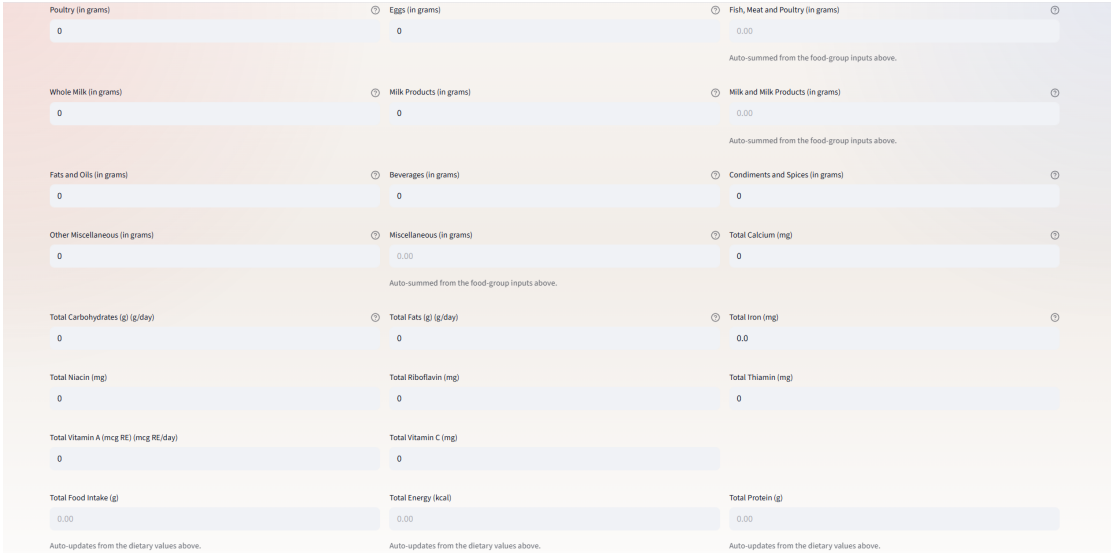

Figure 13: Input Step 3: Behavioral Risk Factors and Lifestyle Profiling.

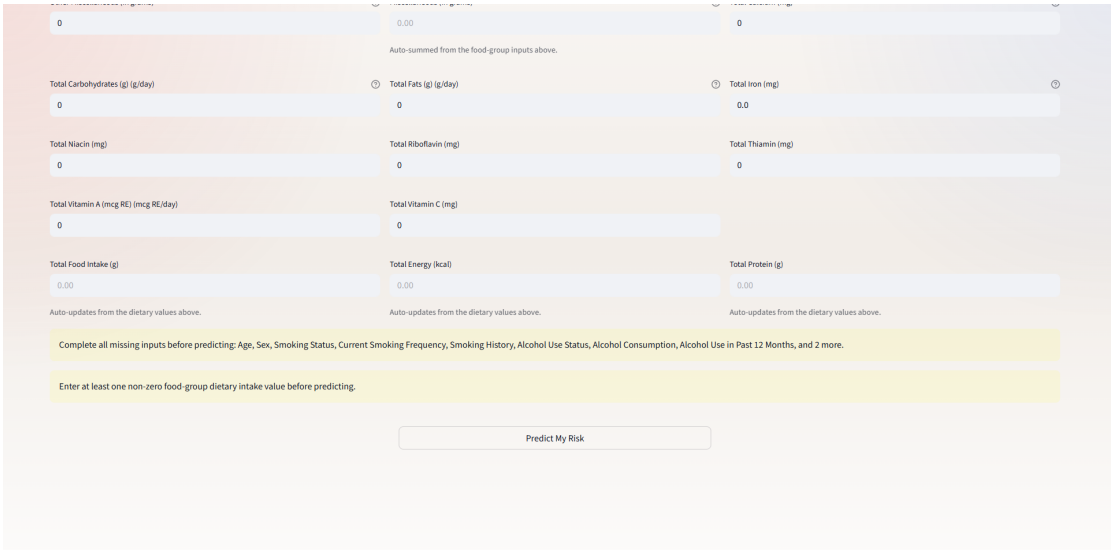

Figure 14: Input Step 4: Dietary Intake and Auto-Computed Macronutrient Totals.

4.8.3 Results Dashboard and Explainability Modules

Upon form submission, the system evaluates the patient data through the calibrated XGBoost pipeline and generates the Results Dashboard. This dashboard translates complex multidimensional predictions into a unified, highly interactive clinical interface, sequentially partitioned into descriptive, prescriptive, and explanatory modules.

The risk assessment output begins with the Calibrated Risk Score (Figure 15), displayed on a dynamic gauge anchored to the empirically selected 0.35 screening threshold. This is immediately accompanied by the Venn-Abers Probability Interval, providing uncertainty bounds around the calibrated prediction. Following the diagnosis, the application renders actionable Wachter-style Counterfactuals (Figure 16). Driven by bounded Brent's optimization, this module estimates the minimal feasible lifestyle modifications required to reduce the patient's modeled risk below the selected 0.35 screening threshold.

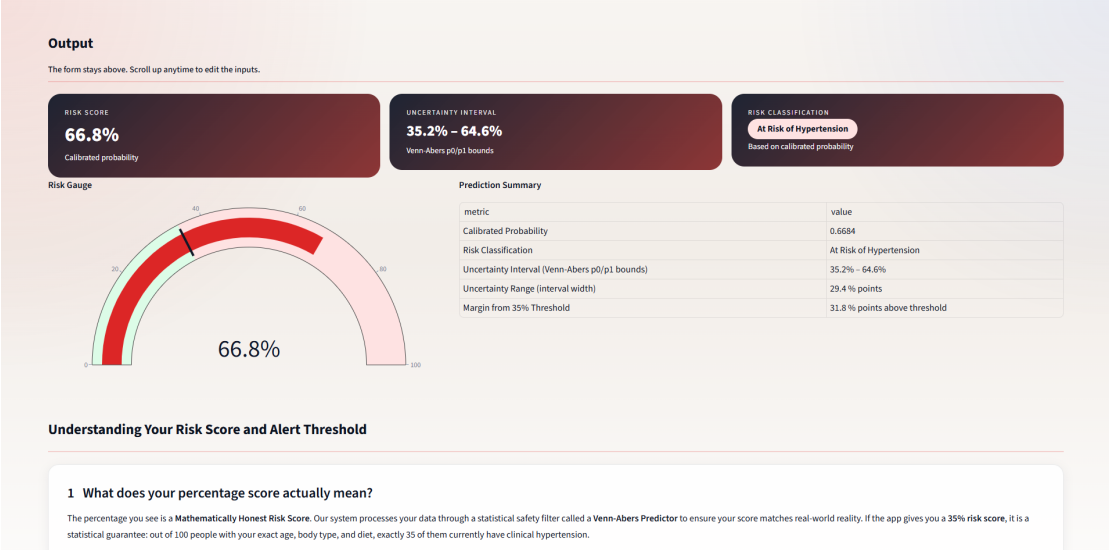

Figure 15: Prediction Results: Calibrated Risk Gauge and Venn-Abers Uncertainty Bounds.

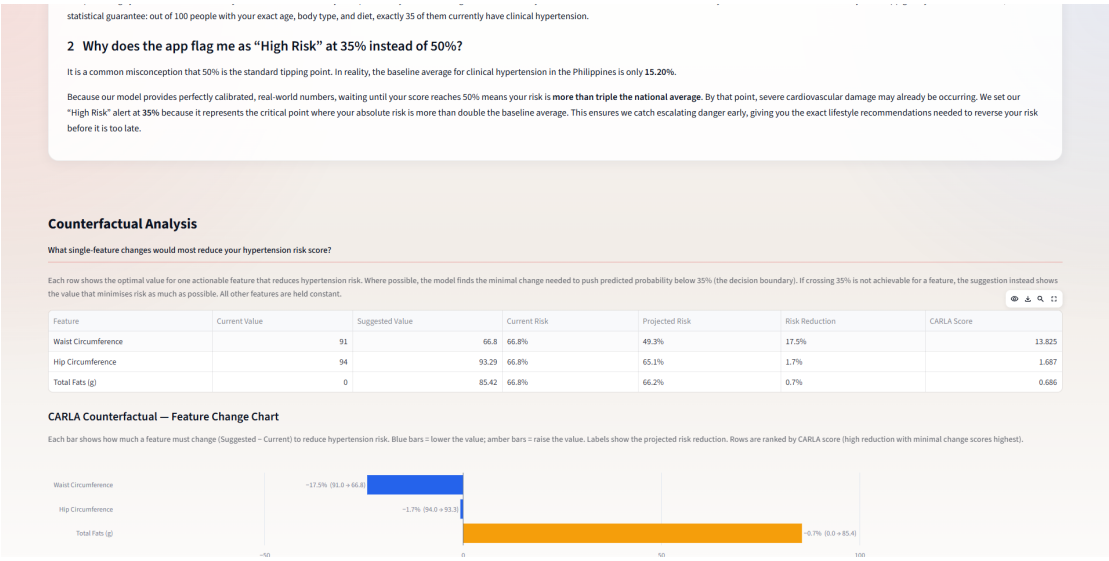

Figure 16: Prediction Results: Prescriptive Lifestyle Counterfactuals via Optimization.

Finally, to establish clinical trust, the dashboard deploys a Tri-Tiered Explainability framework. It first renders a mathematically exact Local SHAP Waterfall Plot (Figure 17) to explicitly quantify feature attributions. To aid in rapid visual interpretation, this is immediately paired with a Local LIME Divergent Bar Chart (Figure 18), allowing

the clinician to establish visual ”explanation consensus.” The dashboard concludes with the Global SHAP Beeswarm Plot (Figure 19), contextualizing the patient’s unique physiological drivers against dataset-wide trends.

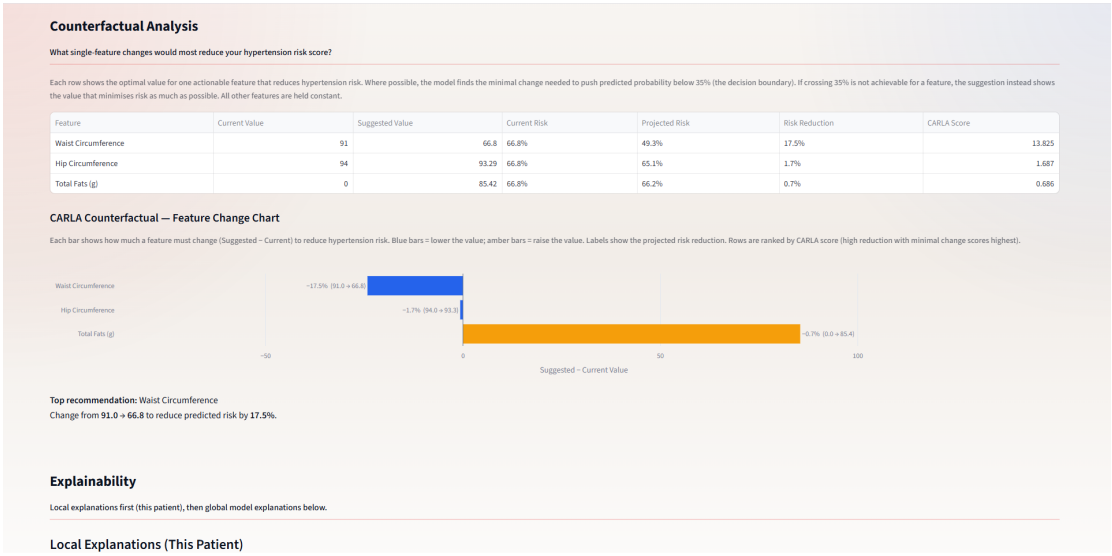

Figure 17: Tri-Tiered Explainability (Part A): Local SHAP Waterfall Plot.

Figure 18: Tri-Tiered Explainability (Part B): Local LIME Bar Chart for Rapid Interpretation.

Figure 19: Tri-Tiered Explainability (Part C): Global SHAP Plot for Population Context.

5 Results

This section presents the empirical findings of the study. It details the demographic distribution of the dataset, the performance metrics of the machine learning models before and after calibration, the secondary empirical experiments used to justify sampling and threshold decisions, the visual outputs from the Explainable Artificial Intelligence (XAI) techniques, and the final web application prototype.

5.1 Dataset Demographics and Class Distribution

Following the strict adherence to the adult diagnostic threshold of $\geq 140/90$ mmHg, the final processed NNS dataset comprised 151,189 samples. The target variable exhibited a significant natural class imbalance, with 128,201 patients (84.80%) classified as normal or elevated (Class 0) and 22,988 patients (15.20%) classified as clinically hypertensive (Class 1).

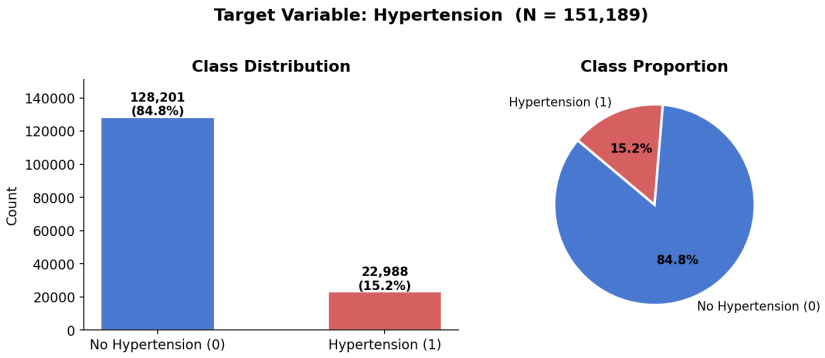

Figure 20: Distribution of the Hypertension Target Variable ($\geq 140/90$ mmHg)

Furthermore, analyzing the continuous variables across these two classes reveals distinct clinical patterns. As illustrated in the feature distribution analysis, hypertensive individuals consistently exhibit higher medians across key anthropometric markers such as Body Mass Index (BMI) and Waist Circumference.

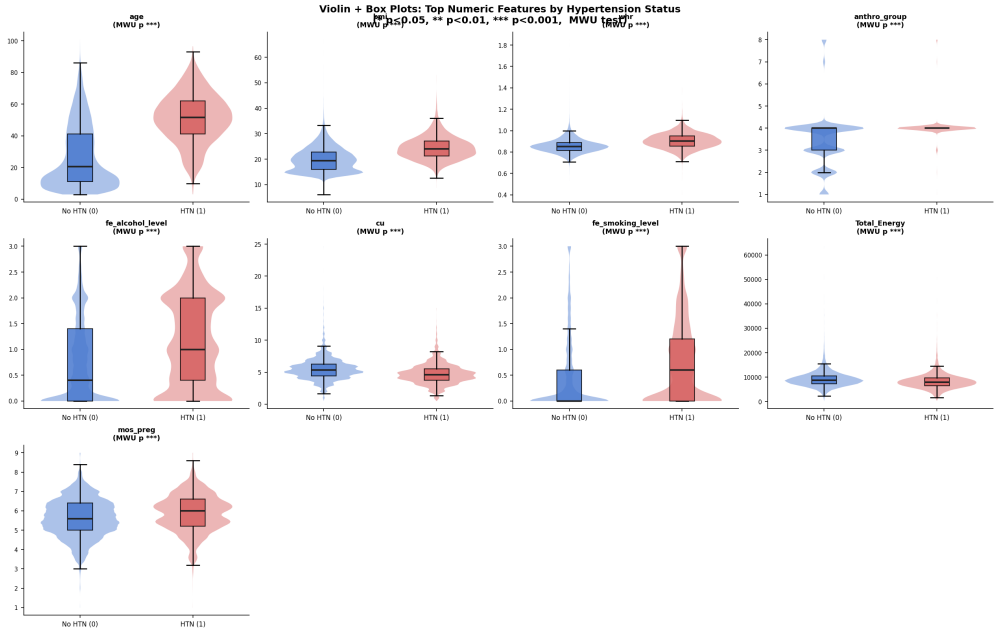

Figure 21: Violin Plots Illustrating Feature Distributions Across Classes

5.2 Performance of Pre-Calibrated Machine Learning Models

Following the execution of the Two-Stage Rigorous Optimization strategy, the first phase of evaluation focused on identifying the strongest-performing configuration for each algorithm based on discriminative performance. Table 6 presents the highest-performing configurations observed across the evaluated algorithms, with XGBoost Class Weights included as an additional clinically important candidate because of its strong sensitivity under the screening-oriented threshold.

Table 6: Top Performing Pre-Calibrated Models

Algorithm	Configuration	Accuracy	Recall	F1-Score	AUC
LightGBM	Class Weights (cw)	0.7428	0.8351	**0.4968**	**0.8522**
XGBoost	Base	0.8262	0.5125	0.4728	0.8519
Random Forest	Class Weights (cw)	0.7307	0.8553	0.4913	0.8487
CatBoost	Class Weights (cw)	0.7300	0.8479	0.4885	0.8486
Logistic Regression	Base	0.8335	0.4449	0.4483	0.8486
XGBoost	Class Weights (cw)	0.7098	**0.8788**	0.4794	0.8469
AdaBoost	Base	0.7163	0.8671	0.4816	0.8456
k-Nearest Neighbors	Base	0.8247	0.4027	0.4112	0.8232
Naive Bayes	Base	0.7152	0.7949	0.4590	0.8094

The results demonstrate that LightGBM configured with Class Weights achieved the strongest overall discriminative performance, producing the highest AUC (0.8522) and the highest F1-Score (0.4968) among the evaluated models. Random Forest and CatBoost also demonstrated competitive sensitivity, both achieving Recall values above 84% while maintaining AUC values greater than 0.84.

However, the final model selection was not based on AUC alone. Since the system is intended for hypertension screening, the ability to identify positive hypertension cases was treated as a primary clinical requirement. A model with a marginally higher AUC but weaker deployment suitability would not automatically be preferred if it failed to provide a stable and clinically usable balance between discrimination, Recall, and calibration compatibility.

Within this context, XGBoost with Class Weights emerged as a strong clinical candidate. Although its AUC of 0.8469 was slightly lower than the highest-AUC models, it achieved very high Recall (0.8788) under the screening-oriented threshold used in the pre-calibration evaluation. This made it particularly relevant for the screening objective of the study, where false negatives are more clinically concerning than false positives.

Therefore, XGBoost CW was retained as the primary candidate for post-calibration evaluation and final deployment consideration.

It is important to note that the pre-calibration classification metrics in Table 6 were computed using the decision threshold selected during the calibration-set threshold sweep. Therefore, the reported Accuracy, Recall, and F1-Score reflect the model's threshold-optimized screening behavior rather than the default 0.50 classification boundary. For XGBoost CW, this lower operating threshold increased sensitivity, producing a Recall of 0.8788 at the cost of lower Accuracy.

5.3 Performance of Calibrated Machine Learning Models

Post-hoc probability calibration using Platt Scaling, Isotonic Regression, and Venn-Abers Predictors was subsequently applied to the machine learning models. The objective of this phase was to evaluate whether probability correction could improve probabilistic reliability while preserving clinically acceptable classification performance.

Table 7 presents the five best-calibrated models ranked according to Expected Calibration Error (ECE).

Table 7: Top 5 Best Calibrated Models Overall

Algorithm	**Configuration**	**Calibration**	**Accuracy**	**Recall**	**AUC**	**ECE**	**Log Loss**
AdaBoost	Class Weights (cw)	Venn-Abers	0.8480	0.0000	0.8226	**0.0006**	0.3236
AdaBoost	Class Weights (cw)	Isotonic Reg.	0.8480	0.0000	0.8226	0.0007	0.3236
Naive Bayes	Base	Isotonic Reg.	0.8480	0.0000	0.8091	0.0024	0.3411
XGBoost	Base	Base	0.8502	0.1190	0.8519	0.0027	**0.3098**
XGBoost	Class Weights (cw)	Venn-Abers	0.8484	0.0979	0.8469	0.0035	0.3140

The calibration phase revealed a significant trade-off between probability calibration and clinical sensitivity. Models that achieved extremely low ECE values frequently suffered from severe reductions in Recall. Several of the best-calibrated models effectively

collapsed into majority-class prediction behavior, achieving near-perfect calibration while failing to identify positive hypertension cases.

This phenomenon demonstrates that minimizing calibration error alone is insufficient for selecting a clinically deployable hypertension screening model. A classifier that produces well-calibrated probabilities but fails to identify at-risk patients cannot function safely in a real-world medical setting. Consequently, model selection required balancing calibration quality with discrimination and minority-class sensitivity.

To identify the strongest clinically viable candidates, the best-performing uncalibrated baseline models were re-evaluated with emphasis on Recall, AUC, and calibration stability. Table 8 summarizes the strongest baseline candidates.

Table 8: Top Uncalibrated Candidate Models

Algorithm	Recall	Accuracy	AUC	Base ECE	Base Log Loss
LightGBM (cw)	0.8351	0.7428	**0.8522**	0.1999	0.4691
Random Forest (cw)	**0.8553**	0.7307	0.8487	0.1171	0.3691
CatBoost (cw)	0.8479	0.7300	0.8486	0.1877	0.4560
XGBoost (base)	0.5125	**0.8262**	0.8519	**0.0027**	**0.3098**
XGBoost (cw)	0.7651	0.7687	0.8469	0.1493	0.4118

The baseline analysis revealed two distinct model behaviors. LightGBM, Random Forest, CatBoost, and XGBoost CW prioritized minority-class sensitivity, producing substantially higher Recall values than their corresponding base configurations. However, these gains were accompanied by elevated calibration errors and increased Log Loss values.

Conversely, the base XGBoost model demonstrated exceptional probabilistic stability, achieving the lowest ECE (0.0027) and lowest Log Loss (0.3098) among all baseline candidates while still maintaining highly competitive discriminative power (AUC = 0.8519). Nevertheless, its Recall of 51.25% remained substantially lower than the

clinically stronger class-weighted alternatives.

Among the clinically optimized models, XGBoost CW emerged as the strongest compromise between discrimination, sensitivity, and calibration stability. Although its ECE (0.1493) was higher than the naturally stable XGBoost base model, it remained substantially lower than the heavily overconfident LightGBM and CatBoost configurations while preserving a clinically meaningful Recall of 76.51% at the default 0.50 boundary.

A distinction must be made between probability calibration and classification thresholding. The row labeled *Base* in the post-calibration results does not apply a probability transformation; it preserves the original model probabilities. However, the post-calibration evaluation computes threshold-dependent metrics using the default 0.50 decision boundary, whereas the pre-calibration evaluation reports metrics using the optimized threshold selected from the calibration split. This explains why XGBoost CW has identical AUC, Log Loss, and ECE across the base probability outputs, but different Accuracy, Recall, Precision, and F1-Score values. AUC, Log Loss, and ECE evaluate the probability scores themselves, while Accuracy, Recall, Precision, and F1-Score depend on the chosen classification threshold.

Table 9: Effect of Decision Threshold on XGBoost CW Base Probability Outputs

Evaluation Setting	Threshold	Accuracy	Recall	Precision	F1-Score	AUC	Log Loss	ECE
Threshold-Optimized Pre-Calibration Evaluation	0.35	0.7098	**0.8788**	0.3296	0.4794	0.8469	0.4118	0.1493
Default Boundary Base Evaluation	0.50	**0.7687**	0.7651	**0.3729**	**0.5014**	0.8469	0.4118	0.1493

Thus, the apparent discrepancy between the pre-calibrated and post-calibrated XGBoost CW results is not due to retraining or a change in probability outputs. It is caused by the use of different classification thresholds. The threshold-optimized evaluation emphasizes screening sensitivity, while the default-boundary evaluation provides a standard reference point at 0.50. For final deployment, the study treats XGBoost CW as the predictive engine, Venn-Abers as the probability calibration layer, and the empirically selected 0.35 threshold as the screening operating point.

This trade-off established the core tension of the study: models optimized for sensitivity tended to produce overconfident and poorly calibrated probabilities, whereas models with strong probabilistic reliability often sacrificed clinically important recall performance.

To further evaluate the balance between calibration quality and classification utility, the strongest candidate models were subjected to additional post-hoc calibration analysis. Table 10 summarizes the best-performing calibrator for each selected candidate model under the standard post-calibration evaluation setting.

Table 10: Final Calibration Shootout: Best Post-Calibrated Candidate Models

Algorithm	Configuration	Best Calibrator	Accuracy	Recall	Calib. ECE	Calib. Log Loss	Combined Metric
LightGBM	Class Weights	Venn-Abers	0.8506	0.1083	0.0036	0.3114	0.6610
CatBoost	Class Weights	Isotonic Reg.	0.8491	0.1173	0.0034	0.3127	0.6626
XGBoost	**Class Weights**	**Venn-Abers**	**0.8484**	**0.0979**	**0.0035**	**0.3140**	**0.6572**

The final calibration analysis demonstrated that post-hoc calibration substantially reduced calibration error across the candidate models. ECE values were compressed to near-zero levels, and Log Loss values converged near 0.31. However, these improvements in probability reliability also produced measurable reductions in Recall when evaluated at the default 0.50 boundary, demonstrating the practical trade-off between probability calibration and sensitivity preservation.

For this reason, the final architecture was not selected solely by the highest isolated post-calibration score. Instead, the final decision followed a clinically oriented hierarchy: (1) the model must preserve strong hypertension-positive detection before calibration, (2) it must retain competitive discriminative ability, (3) it must achieve acceptable post-calibration probability reliability, and (4) it must remain suitable for integration with the calibrated explainability pipeline.

Under this selection framework, XGBoost with Class Weights calibrated using the Venn-Abers Predictor was selected as the final deployment architecture. XGBoost CW

provided a strong pre-calibration clinical foundation by achieving high Recall while maintaining competitive AUC. After Venn-Abers calibration, it achieved near-zero calibration error (ECE = 0.0035), indicating that its probability estimates were substantially corrected while preserving its role as a sensitivity-oriented screening model.

A critical distinction also emerged between the XGBoost Base and XGBoost CW configurations. The base XGBoost model demonstrated stronger native probability stability, achieving very low baseline ECE and Log Loss values. However, its lower Recall made it less appropriate as the final screening model. In contrast, the class-weighted XGBoost configuration provided stronger minority-class detection, which better aligned with the clinical objective of identifying at-risk individuals.

Consequently, the study selected XGBoost CW combined with the Venn-Abers Predictor as the final model architecture. This choice reflects a clinically grounded compromise between sensitivity, discrimination, and calibrated probability reliability. The findings reinforce that model selection for medical screening should not rely on a single aggregate metric, but should instead jointly evaluate discrimination, calibration, and the clinical consequences of missed positive cases.

5.4 Empirical Justification for Hyperparameter and Data Decisions

To ensure the integrity and clinical safety of the final pipeline, secondary empirical experiments were conducted to justify the selection of sampling techniques and decision thresholds.

5.4.1 Experiment G: Sampling Integrity and the Failure of Synthetic Oversampling

Standard literature often relies on the Synthetic Minority Over-sampling Technique (SMOTE) and its categorical variant, SMOTENC, to handle class imbalance. A dedicated sampling experiment was conducted to compare the reliability of the baseline

model, algorithmic Class Weights (cw), SMOTE, SMOTE with Class Weights, SMOTENC, and SMOTENC with Class Weights. This experiment evaluated whether synthetic oversampling improved predictive performance and whether the generated synthetic records remained clinically plausible.

It must be noted that this experiment was conducted using a LightGBM classifier as the sampling test engine. LightGBM was selected for this sampling-integrity experiment because it was the fastest model to train among the high-performing gradient-boosting candidates, making it computationally practical for repeatedly evaluating multiple sampling configurations. Therefore, the results in Table 11 should be interpreted as a controlled sampling-integrity experiment rather than as the final XGBoost model-selection table.

Table 11: Experiment G: Predictive Performance Across Sampling Techniques (LightGBM)

Sampling Technique	**Synthetic Rows**	**Impossible Synthetic %**	**Accuracy**	**Recall**	**F1-Score**	**AUC**
Baseline (No Sampling)	0	–	**0.8516**	0.1257	0.2049	**0.8536**
Class Weights (cw)	0	–	0.7295	**0.8636**	**0.4926**	0.8532
SMOTE	63,127	0.5497%	0.8418	0.3180	0.3793	0.8499
SMOTE + Class Weights	63,127	0.5497%	0.8418	0.3180	0.3793	0.8499
SMOTENC	63,127	0.5576%	0.8404	0.3263	0.3833	0.8492
SMOTENC + Class Weights	63,127	0.5576%	0.8404	0.3263	0.3833	0.8492

The results show that the baseline model achieved the highest AUC (0.8536) and highest Accuracy (0.8516), but its Recall was only 0.1257. This indicates that the baseline model was overly conservative and failed to identify most hypertension-positive cases. In contrast, the Class Weights configuration produced the highest Recall (0.8636) and highest F1-Score (0.4926), showing that cost-sensitive learning was the most effective strategy for recovering minority-class sensitivity without generating artificial patient records.

SMOTE and SMOTENC achieved competitive AUC values near 0.85, but their Recall values remained substantially lower than the Class Weights configuration. SMOTE

achieved a Recall of only 0.3180, while SMOTENC achieved a Recall of 0.3263. Therefore, synthetic oversampling did not provide a clinically meaningful improvement in sensitivity compared with algorithmic Class Weights.

Beyond predictive metrics, the synthetic-data integrity check revealed that SMOTE and SMOTENC generated out-of-scope synthetic records. Both SMOTE-based approaches generated 63,127 synthetic rows. Among these, SMOTE and SMOTE + Class Weights generated 347 impossible synthetic records, equivalent to 0.5497% of their synthetic samples. SMOTENC and SMOTENC + Class Weights generated 352 impossible synthetic records, equivalent to 0.5576% of their synthetic samples.

Table 12: Examples of Out-of-Scope Synthetic Patients Generated by SMOTE

Synthetic Patient ID	Age	BMI	Total Energy	Clinical Viability Issue
Patient 91036	17.18	17.62	10445.09	Below adult cutoff / out-of-scope synthetic age
Patient 91279	17.06	17.16	10602.93	Below adult cutoff / out-of-scope synthetic age
Patient 91303	16.22	31.01	11074.60	Below adult cutoff / out-of-scope synthetic age
Patient 91412	14.70	32.52	9914.50	Below adult cutoff / out-of-scope synthetic age
Patient 91768	12.64	14.04	14906.39	Below adult cutoff / out-of-scope synthetic age

These examples show that ordinary SMOTE can interpolate across patient records in ways that produce structurally invalid synthetic observations. In particular, the generated records included ages below the intended adult modeling scope of the study. Since the target variable and preprocessing pipeline were designed around the adult hypertension threshold, the creation of below-cutoff synthetic observations weakens the clinical validity of the resampled training distribution. Although SMOTENC is designed to better preserve categorical feature structure, it still produced a comparable proportion of out-of-scope synthetic records in this experiment.

Consequently, synthetic oversampling was disqualified from the final pipeline. The experiment supports the use of algorithmic Class Weights because this approach improved minority-class sensitivity without fabricating new patient records. Instead of altering the empirical feature space, Class Weights penalize minority-class misclassifi-

cation during model optimization, allowing the model to learn from authentic observed cases while avoiding synthetic artifacts. Since LightGBM was used primarily as the fastest high-performing test engine for evaluating sampling behavior, the conclusion drawn from Experiment G concerns the sampling strategy rather than the final classifier architecture.

5.4.2 Experiment H: Empirical Threshold Sweep

As previously demonstrated, relying on the default 0.50 decision threshold can produce insufficient sensitivity in imbalanced hypertension screening. To systematically examine the relationship between diagnostic sensitivity and false-positive control, an empirical threshold sweep was conducted across decision thresholds from 0.50 to 0.10.

This experiment was conducted using LightGBM as the threshold-sweep test engine because it was the fastest high-performing model to train and evaluate repeatedly. Therefore, the results in Table 13 should be interpreted as an empirical threshold-selection experiment rather than as the final calibrated XGBoost deployment evaluation.

Table 13: Experiment H: Empirical Threshold Sweep Using LightGBM

Decision Threshold	Accuracy	Recall (Sensitivity)	Precision	F1-Score	Specificity
0.50 (Default)	0.8519	0.1379	0.5523	0.2207	**0.9800**
0.45	0.8493	0.2428	0.5094	0.3288	0.9581
0.40	0.8430	0.3863	0.4796	0.4280	0.9248
0.35 (Selected)	**0.8280**	**0.5164**	**0.4437**	**0.4773**	**0.8839**
0.30	0.8090	0.6343	0.4160	0.5025	0.8403
0.25	0.7860	0.7309	0.3910	**0.5094**	0.7959
0.20	0.7569	0.8101	0.3650	0.5032	0.7473
0.15	0.7197	0.8740	0.3372	0.4867	0.6920
0.10	0.6694	**0.9343**	0.3070	0.4622	0.6219

The threshold sweep demonstrates the expected trade-off between sensitivity and

false-positive control. At the default 0.50 threshold, the model achieved the highest Accuracy (0.8519) and Specificity (0.9800), but its Recall was only 0.1379. This means that the model was highly conservative and failed to detect most hypertension-positive cases.

Lowering the threshold progressively increased Recall. At the most aggressive threshold of 0.10, Recall increased to 0.9343, meaning that the model detected the vast majority of positive cases. However, this came at the cost of substantially reduced Accuracy (0.6694), Precision (0.3070), and Specificity (0.6219), indicating that many normotensive individuals would be incorrectly flagged as high risk.

The threshold of 0.35 was selected as the operating point because it provided a more balanced clinical compromise. At this threshold, Recall increased to 0.5164 while Specificity remained relatively high at 0.8839. Although thresholds such as 0.30, 0.25, and 0.20 achieved higher Recall, they produced progressively larger reductions in Specificity and Precision. Thus, 0.35 was selected to improve sensitivity beyond the default threshold while avoiding an excessive false-positive burden.

Since LightGBM was used as the computationally efficient test engine for this experiment, the threshold result should be interpreted as empirical evidence supporting the use of a lower operating threshold in imbalanced hypertension screening. The final deployed XGBoost CW + Venn-Abers model follows the same clinical principle by using the selected 0.35 value as a screening threshold rather than as a diagnostic threshold.

5.5 Explainable Artificial Intelligence

With XGBoost CW + Venn-Abers established as the final calibrated model architecture, XAI techniques were applied to map both dataset-wide behaviors and individual patient-level risk drivers.

5.5.1 SHAP (SHapley Additive exPlanations)

Global feature attributions were extracted using SHAP. Figure 22 presents the mean absolute SHAP values, ranking the features by their average magnitude of impact on the model's output across the entire dataset. Age, Waist Circumference, and Body Mass Index (BMI) emerged as the highest-signal predictors for Hypertension (140/90).

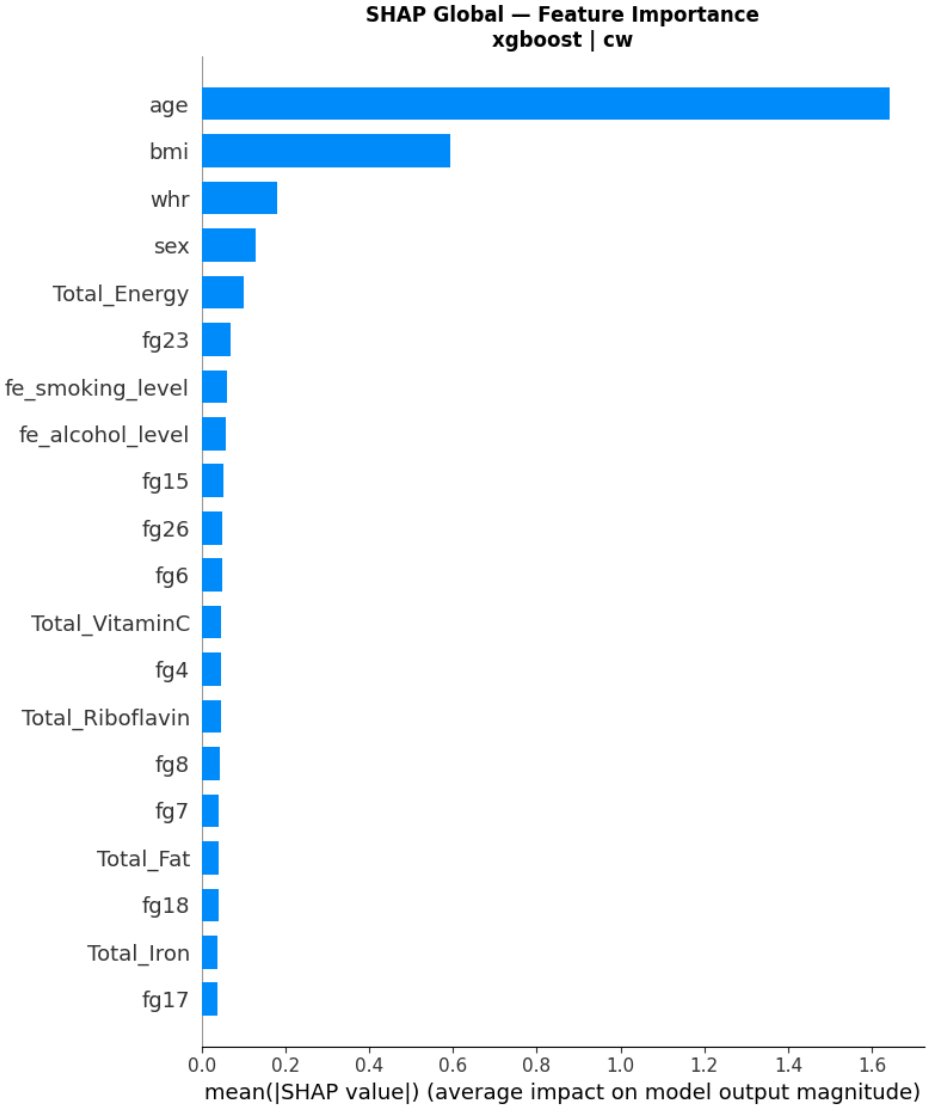

Figure 22: Global SHAP Feature Importance (Mean Absolute SHAP Values)

To further illustrate the directional impact of these features, a SHAP Beeswarm plot (Figure 23) was generated. This plot reveals how higher values of specific features, such as Age and BMI, consistently push the model's prediction toward a positive hypertension

classification.

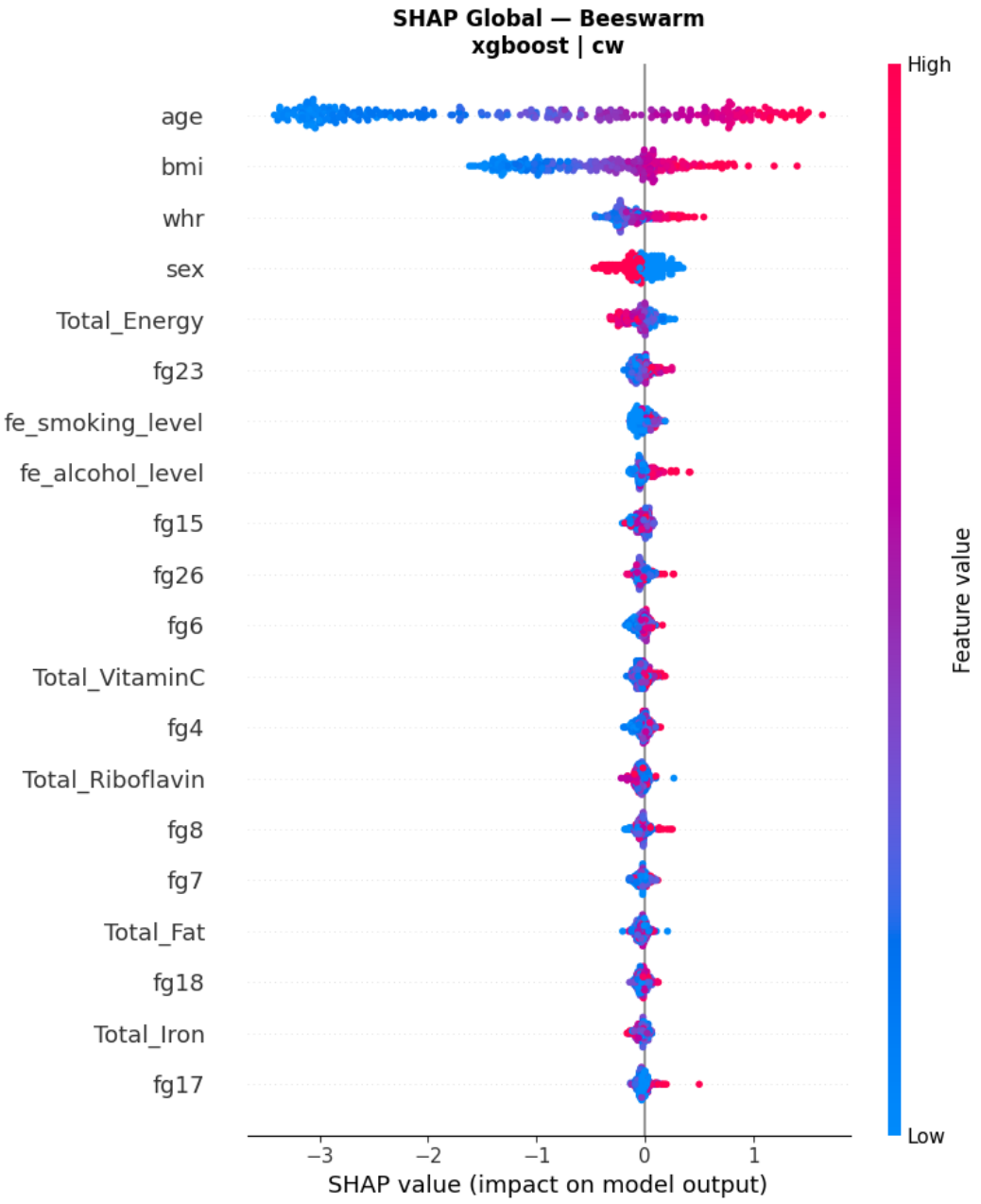

Figure 23: SHAP Beeswarm Plot Demonstrating Feature Directionality and Density

Beyond global trends, SHAP was also utilized for local explanations. Figure 24 demonstrates a waterfall plot for an individual prediction, detailing exactly how the baseline expected probability was mathematically increased or decreased by the patient's specific physiological and dietary values to arrive at their final personalized risk score.

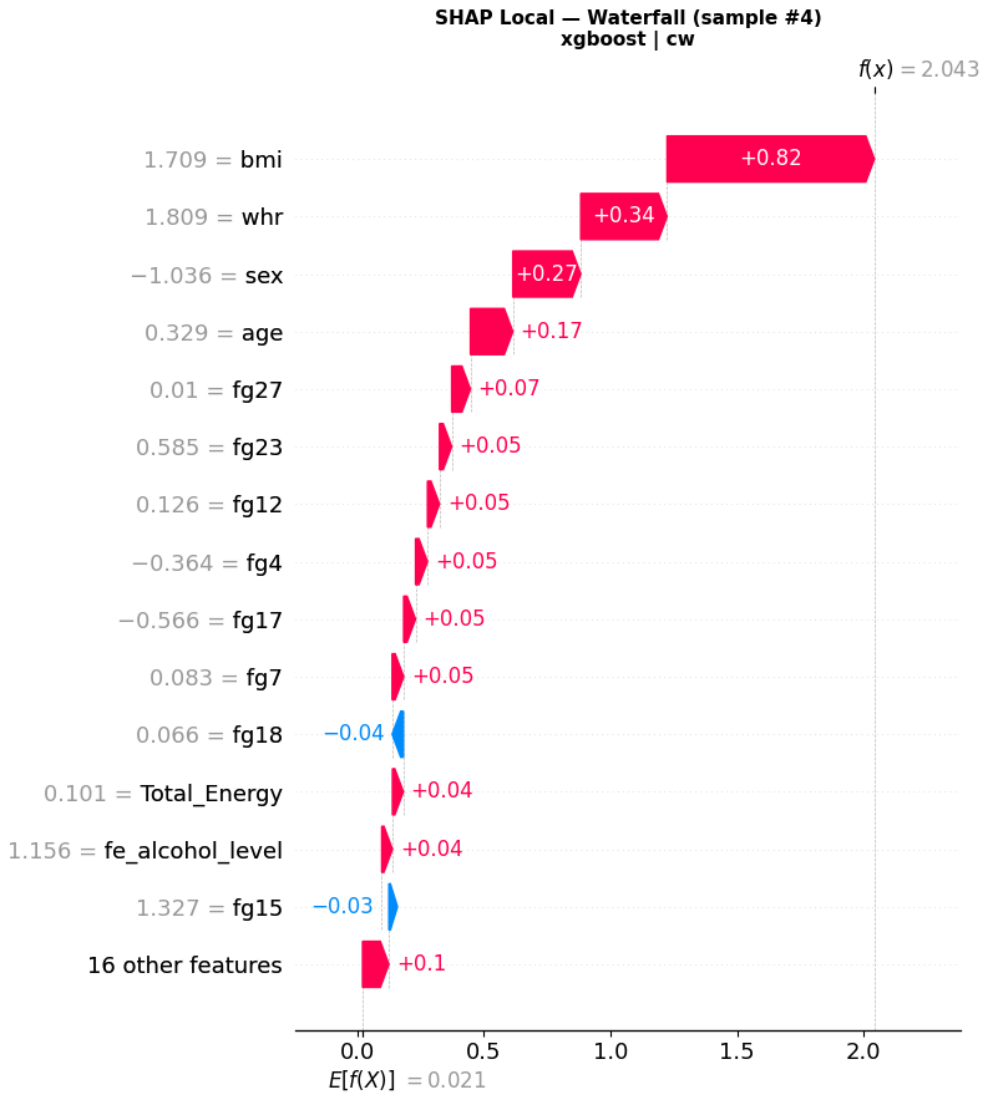

Figure 24: Local SHAP Waterfall Plot for a Single Patient Prediction

While default SHAP waterfall plots provide rigorous mathematical breakdowns, their dense visualization can induce cognitive overload for non-expert users. To bridge this gap within the web application, the system extracts the local SHAP values and dynamically renders them as an accessible, divergent bar chart. This allows the user to immediately identify their top physiological risk contributors versus their protective factors, maintaining game-theoretic mathematical rigor while optimizing human readability.

5.5.2 LIME (Local Interpretable Model-agnostic Explanations)

For a more accessible visualization of individual predictions within the web application, LIME was utilized. Figure 25 provides a straightforward, human-readable bar chart representing the LIME output, highlighting the top features supporting or contradicting the model's local decision boundary for a specific test instance.

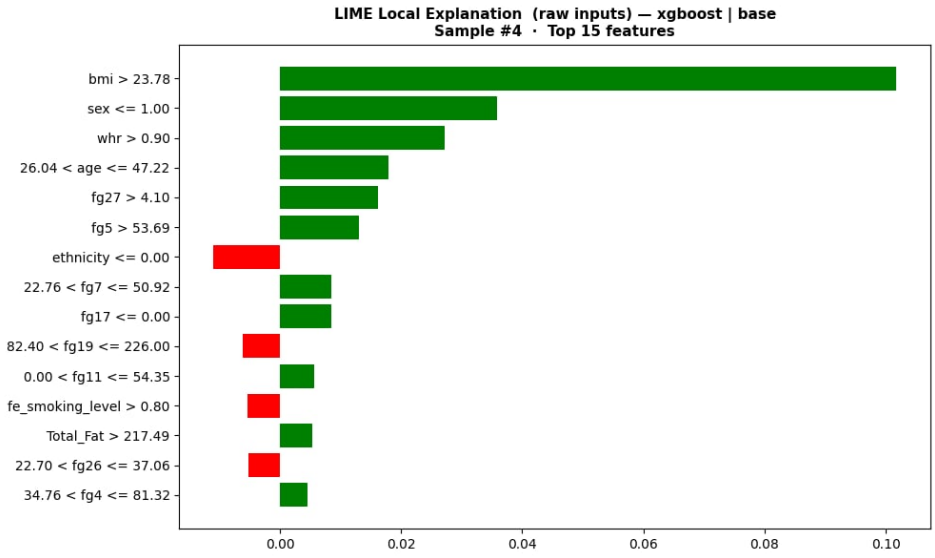

Figure 25: Local LIME Explanation for a Single Patient Prediction

5.6 Web Application Prototype and Clinical Dashboard

The web application serves as the deployment vehicle for the trained XGBoost CW + Venn-Abers model, transforming calibrated risk estimates and explanation outputs into an interactive, highly visual decision-support interface.

5.6.1 Landing Page and Progressive Intake

The Landing Page serves as the entry point, establishing immediate clinical context. It features a clean, medical-grade aesthetic and dynamically displays the empirical baseline prevalence of 15.20% derived from the NNS dataset to anchor user expectations.

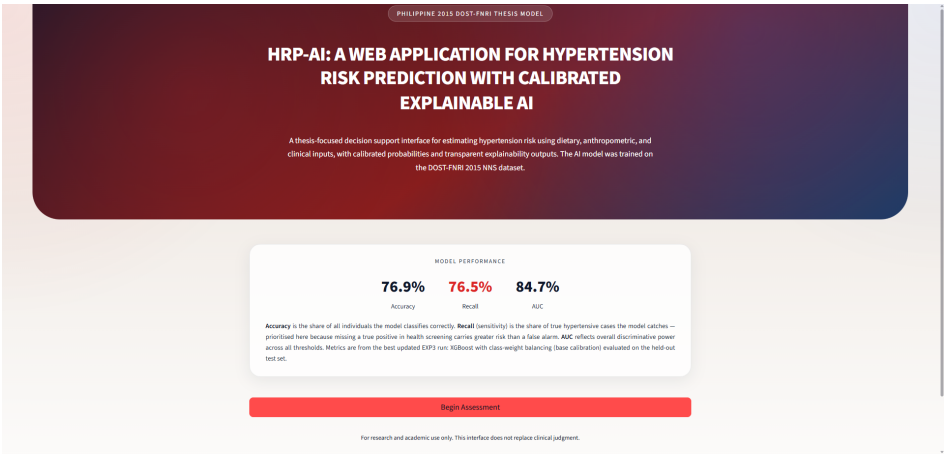

Figure 26: HRP-AI Web Application Landing Page

Data entry is handled through a progressive, four-step intake form (Clinical Profile, Anthropometrics, Lifestyle, and Diet). This multi-step design prevents cognitive fatigue and includes dynamic computational safeguards, such as the automatic calculation of BMI and Total Caloric Intake, ensuring the pipeline receives mathematically valid feature arrays.

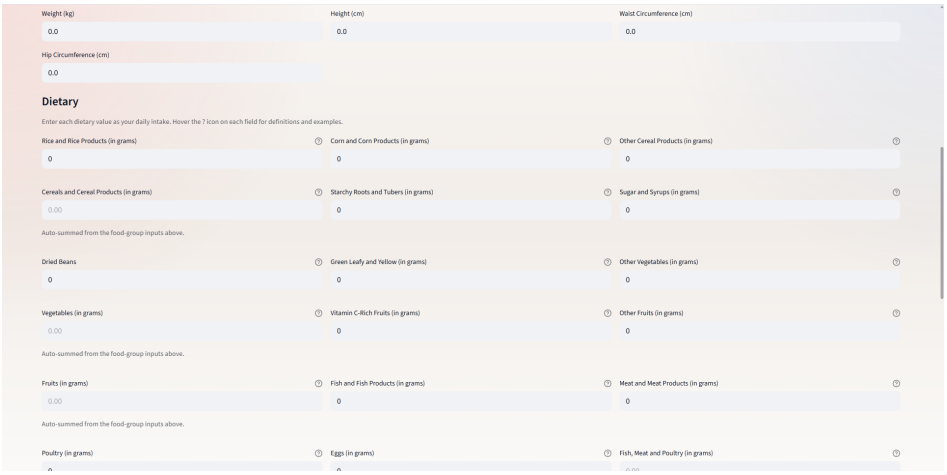

Figure 27: Step 2 of the Progressive Intake Form featuring Dynamic BMI Calculation

5.6.2 Prediction and Results Dashboard

Upon submitting the clinical profile, the user is directed to the interactive Results Dashboard. This dashboard is strictly partitioned into three distinct analytical modules

to maximize explainability and clinical trust.

Module 1: Calibrated Risk Assessment. As shown in Figure 28, the system immediately displays the patient's exact calibrated risk score. Crucially, the interface visually enforces the empirically selected 35.00% screening threshold on a dynamic gauge. Furthermore, the dashboard actively renders the Lower and Upper Uncertainty Bounds generated by the Venn-Abers multipredictor, providing the clinician with a transparent interval of the prediction's reliability.

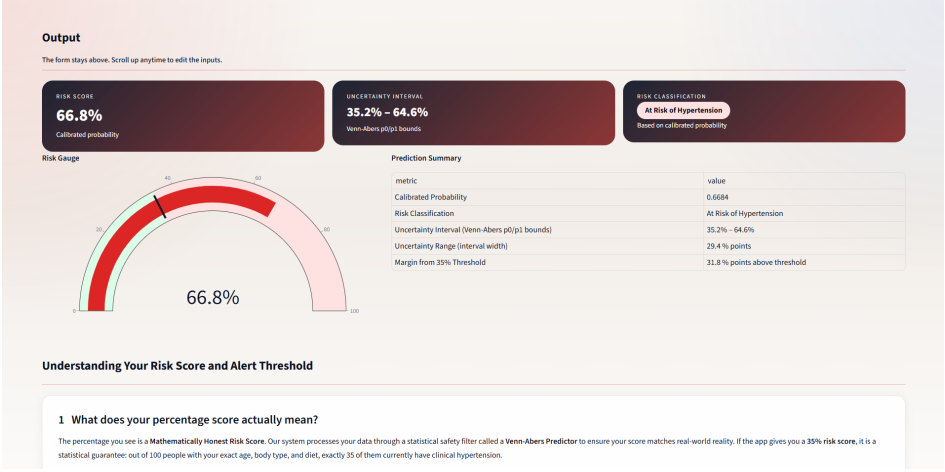

Figure 28: Module 1: Calibrated Risk Gauge and Venn-Abers Uncertainty Bounds

Module 2: Prescriptive Counterfactual Analysis. Transitioning from descriptive to prescriptive AI, the second module leverages bounded Brent's optimization to calculate counterfactual interventions. The system explicitly anchors these calculations to the 0.35 screening boundary. For a patient designated as Low Risk, as depicted in Figure 29, the system correctly identifies that no immediate physiological perturbations are mathematically required to alter the classification, thereby confirming their healthy trajectory and preventing unnecessary medical anxiety.

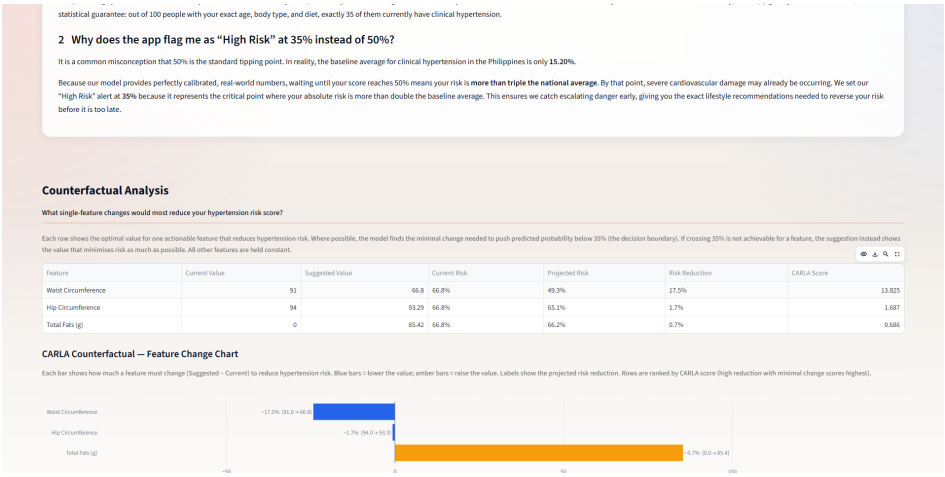

Figure 29: Module 2: Prescriptive Analytics Confirming No Immediate Intervention Required

Module 3: Tri-Tiered Explainability. To reduce the black-box opacity of the prediction, the dashboard integrates three distinct explainability frameworks in a vertical flow. First, the application utilizes Local SHAP (Figure 30) to provide a game-theoretic mathematical breakdown of how the baseline probability was altered by the patient's specific traits.

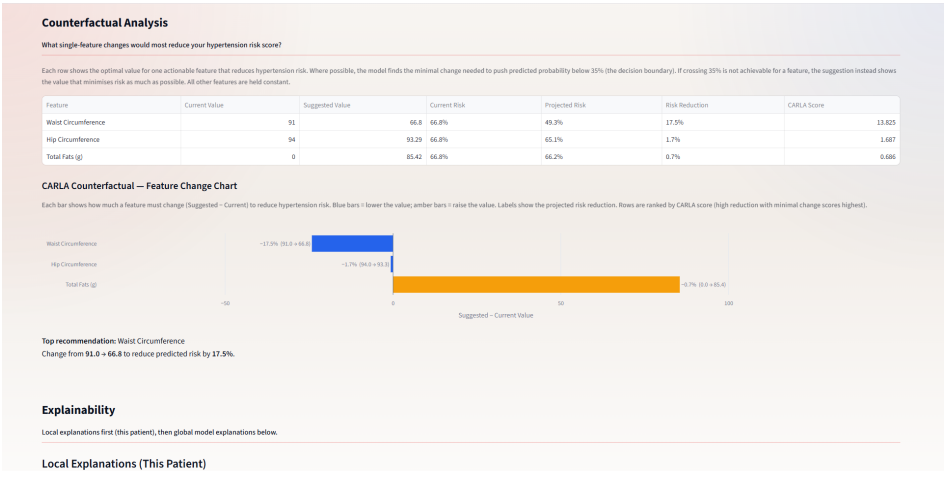

Figure 30: Module 3 (Part A): Local SHAP Waterfall Plot for Exact Feature Attribution

Because dense waterfall plots can be difficult to interpret rapidly, the system immediately follows this with a Local LIME analysis (Figure 31), rendered as an accessible, divergent bar chart. By providing two independent local explainers sequentially, the sys-

tem allows clinicians to establish explanation consistency by visually verifying whether major features act in similar directions across explanation methods.

Figure 31: Module 3 (Part B): Local LIME Divergent Bar Chart for Rapid Visual Interpretation

Finally, the dashboard concludes with the Global SHAP Feature Importance beeswarm plot (Figure 32). This tri-tiered design ensures the clinician can cross-verify the patient's unique risk drivers while contextually anchoring those findings against the broader population-level baseline of the NNS dataset.

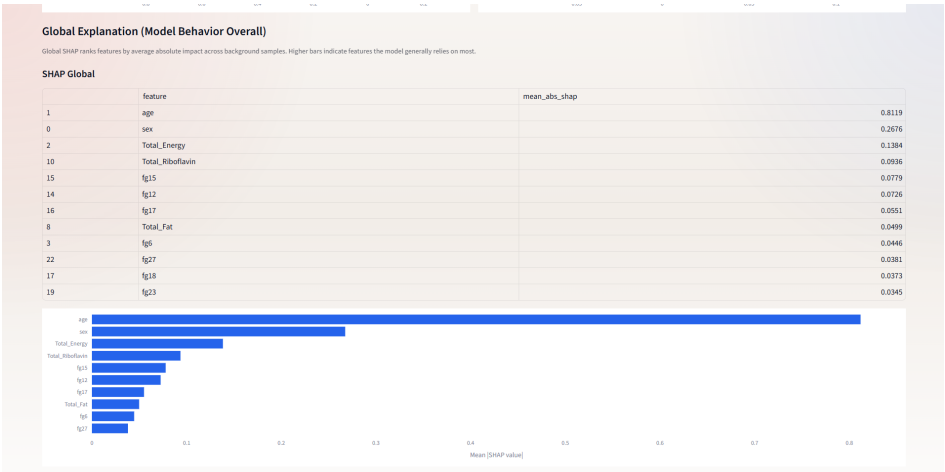

Figure 32: Module 3 (Part C): Global SHAP Beeswarm Plot for Population-Level Context

6 Discussions

The development of the HRP-AI web application bridges the gap between computational risk stratification and interpretable clinical decision support. By integrating XGBoost with algorithmic Class Weights, Venn-Abers calibration, and Explainable Artificial Intelligence (XAI), this study provides a structured approach to hypertension screening using national-level Filipino health data. The empirical findings demonstrate that model selection in medical screening cannot rely on a single metric alone. Instead, predictive performance, minority-class sensitivity, probability reliability, threshold behavior, and explanation usability must be evaluated together.

The empirical results show that XGBoost with Class Weights provided the strongest overall clinical compromise for the final predictive engine. While other models achieved slightly higher isolated AUC or calibration scores, XGBoost CW offered a more appropriate balance for screening because it preserved clinically meaningful hypertension-positive detection while maintaining competitive discrimination. The final selection of XGBoost CW combined with Venn-Abers calibration therefore reflects an end-to-end deployment decision rather than a simple ranking by one metric.

6.1 Clinical Hierarchy of Evaluation Metrics

When deploying machine learning within a healthcare ecosystem, the operational viability of a predictive model cannot be judged purely on standard mathematical aggregates such as global accuracy. Instead, model selection must be dictated by the real-world clinical consequences of misclassification. Because this system is designed as a first-line screening tool for a chronic and often asymptomatic condition, the evaluation framework followed a clinically oriented hierarchy of metrics.

1. **Primary Priority: Diagnostic Sensitivity (Recall).** In medical screening, a false negative is the most dangerous error because it fails to identify a potentially

hypertensive individual. Untreated hypertension may progress toward severe cardiovascular events, including myocardial infarction and stroke. Conversely, the clinical burden of a false positive is more manageable because it can be resolved through follow-up blood pressure measurement. Therefore, Recall was treated as a primary safety metric.

2. **Secondary Priority: Discriminative Power (AUC).** Although high Recall is essential, sensitivity alone is insufficient because a model could trivially classify most individuals as high risk. AUC was therefore used to ensure that the model genuinely learned risk-ranking patterns, assigning higher risk scores to true hypertensive cases than to normotensive cases across thresholds.

3. **Tertiary Priority: Probabilistic Reliability (ECE and Log Loss).** Calibration metrics were used to evaluate whether the predicted probabilities aligned with empirical risk. However, the results showed that aggressively minimizing ECE at the default 0.50 threshold often reduced Recall substantially. Therefore, calibration was treated as a reliability requirement that must be interpreted alongside screening sensitivity, rather than as an isolated selection criterion.

By following this hierarchy, the study selected a final model based on clinical screening suitability rather than mathematical performance alone. At the default 0.50 boundary, XGBoost CW achieved Accuracy = 76.87%, Recall = 76.51%, F1-Score = 0.5014, and AUC = 0.8469 before post-hoc probability correction. Under the threshold-optimized screening setting, the same base probability outputs achieved Recall = 87.88%, demonstrating that the operating threshold materially affects the model's clinical sensitivity.

6.2 The Algorithmic Superiority of Cost-Sensitive Learning over Synthetic Interpolation

A fundamental challenge in medical datasets, particularly when using a strict diagnostic threshold such as $\geq 140/90$ mmHg, is severe class imbalance. In this study, only 15.20% of the final dataset belonged to the hypertension-positive class. Without corrective strategies, machine learning models tend to favor the majority class, producing high apparent accuracy but poor detection of positive cases.

Experiment G empirically compared synthetic oversampling methods with algorithmic Class Weights. The results showed that Class Weights produced the strongest sensitivity improvement without creating artificial patient records. Using LightGBM as a fast sampling-integrity test engine, Class Weights achieved the highest Recall and F1-Score among the tested sampling strategies. In contrast, SMOTE and SMOTENC generated synthetic samples that included out-of-scope ages below the intended adult modeling cutoff, weakening the clinical validity of the resampled training distribution.

This result supports the use of cost-sensitive learning in the final pipeline. Instead of altering the empirical feature space through synthetic interpolation, Class Weights modify the loss function so that minority-class errors are penalized more heavily during training. For XGBoost, this encourages the gradient boosting process to learn from authentic hypertension-positive cases while preserving the integrity of the original NNS data distribution. Therefore, the use of XGBoost CW is justified not only by performance, but also by data validity and clinical plausibility.

6.3 The Recall Collapse Phenomenon and the Calibration Trade-Off

A critical finding of this study is the tension between probability calibration and classification sensitivity in highly imbalanced medical datasets. When post-hoc calibration

methods such as Isotonic Regression and Venn-Abers predictors were applied, several models achieved very low ECE values. However, many of these models also produced very low Recall at the default 0.50 threshold. This occurred because calibrated probabilities were compressed toward the natural prevalence of the dataset, causing many positive cases to fall below the standard classification boundary.

This phenomenon, referred to in this study as Recall Collapse, demonstrates that calibration quality cannot be evaluated independently from clinical utility. A model that produces well-calibrated probabilities but fails to identify hypertension-positive individuals is not suitable for screening. For this reason, the final selection did not simply choose the model with the lowest ECE or highest post-calibration aggregate score.

XGBoost CW provided the most pragmatic compromise. At the default 0.50 boundary, it achieved Accuracy = 76.87%, Recall = 76.51%, F1-Score = 0.5014, and AUC = 0.8469. Its baseline ECE of 0.1493 was not the lowest among all models, but it was acceptable given its much stronger screening sensitivity compared with naturally calibrated but less sensitive alternatives. At the threshold-optimized screening boundary of 0.35, the same base probability outputs achieved Recall = 87.88%, showing that the model could be operated in a more sensitive screening mode.

After Venn-Abers calibration, the model achieved an ECE of 0.0035, showing that its probability estimates could be substantially corrected while still preserving XGBoost CW as the sensitivity-oriented predictive foundation. Thus, the final model architecture separates three important components: XGBoost CW as the predictive engine, Venn-Abers as the calibration layer, and the 0.35 threshold as the screening operating point.

6.4 Deployment Threshold and High-Confidence Alerting

A default decision threshold of 0.50 is not always appropriate in imbalanced medical screening. In this study, the hypertension-positive prevalence was only 15.20%. After

calibration, predicted probabilities were adjusted to better reflect empirical risk, meaning that a calibrated probability of 50% represents a substantially elevated level of risk rather than an ordinary classification boundary.

The results also demonstrate that threshold-dependent metrics such as Accuracy, Recall, Precision, and F1-Score can change even when the underlying probability outputs remain the same. This explains why XGBoost CW produced different classification metrics in the pre-calibration and post-calibration base evaluations. The pre-calibration evaluation used the threshold selected from the calibration-set threshold sweep, while the post-calibration base evaluation used the default 0.50 boundary. In contrast, AUC, Log Loss, and ECE remained the same across these base probability outputs because they evaluate probability scores rather than hard class assignments.

Experiment H therefore conducted an empirical threshold sweep to examine the trade-off between Recall, Precision, Accuracy, and Specificity. LightGBM was used as the threshold-sweep test engine because it was the fastest high-performing model to train and evaluate repeatedly. The results supported the use of a lower screening threshold, with 0.35 selected as a balanced operating point. At this threshold, sensitivity improved beyond the default threshold while maintaining relatively high specificity.

In the deployed HRP-AI interface, the 0.35 value is interpreted as a screening threshold rather than a diagnostic threshold. This distinction is important. The diagnostic definition of hypertension remains $\geq 140/90$ mmHg, while the 35% threshold is used only to classify model-estimated risk within the web application. Since 35% is more than double the observed dataset prevalence of 15.20%, it provides an intuitive escalation boundary for identifying individuals whose modeled risk is substantially above the population baseline.

6.5 Closing the Reliability Gap via Venn-Abers Calibration

A key limitation of many applied health prediction systems is that they report risk probabilities without verifying whether those probabilities are reliable. Gradient boosting models such as XGBoost can achieve strong discrimination but may produce probability estimates that are not well aligned with empirical event frequencies. In clinical decision support, this probability distortion can mislead users by making risk estimates appear more precise or trustworthy than they actually are.

This study addressed the reliability gap by evaluating post-hoc calibration methods, including Platt Scaling, Isotonic Regression, and Venn-Abers predictors. Venn-Abers calibration was selected for the final system because it produced strong probability correction while also providing uncertainty bounds. For the final XGBoost CW model, Venn-Abers calibration reduced the ECE to 0.0035, indicating substantially improved agreement between predicted risk and observed frequency.

However, this result should not be interpreted as guaranteeing perfect prediction for every individual patient. Rather, it means that the model's probability estimates became more statistically reliable at the population level. In the HRP-AI dashboard, this reliability improvement is operationalized through calibrated risk scores and Venn-Abers uncertainty bounds, allowing users to interpret predictions with a clearer sense of uncertainty.

6.6 Game-Theoretic Transparency and Tri-Tiered Explainability

To mitigate the black-box opacity of advanced gradient boosting ensembles, this study implemented a Tri-Tiered Explainability framework using Global SHAP, Local SHAP, and LIME. These explanation tools were designed to provide both population-level and individual-level interpretability.

The global SHAP analysis validated that the model relied on clinically meaningful

predictors, including Age, Waist Circumference, and Body Mass Index (BMI). This supports the clinical plausibility of the model because these variables are consistent with established hypertension risk factors. Local SHAP was then used to explain how individual feature values shifted a specific patient's prediction away from the baseline expectation. LIME provided a complementary local explanation in a more accessible visual format.

The dashboard presents these explanation methods together to support explanation consistency. Local SHAP provides a mathematically rigorous feature attribution, while LIME provides a simpler approximation that is easier for non-expert users to interpret. The global SHAP plot then anchors the individual explanation within the broader population-level behavior of the model. This tri-tiered design improves transparency by allowing users to inspect both why a specific prediction occurred and whether the important features are consistent with the model's overall learned patterns.

It is also important to distinguish prediction calibration from explanation extraction. Venn-Abers calibration was applied to the model's probability outputs, while SHAP and LIME explanations were extracted from the underlying XGBoost model. This preserves faithfulness to the model's internal decision-making structure while allowing the final risk scores shown to users to be calibrated.

6.7 Prescriptive Analytics via Bounded Counterfactual Optimization

The final component of the HRP-AI system is the transition from descriptive prediction to prescriptive guidance. Traditional predictive models provide only a static risk score. In contrast, HRP-AI implements Wachter-style counterfactual explanations to estimate what minimal feature changes would be needed to move a prediction below the selected screening boundary.

To avoid clinically unrealistic recommendations, the system applies bounded Brent's

optimization using `scipy.optimize.minimize_scalar`. The optimization is constrained to actionable or clinically interpretable variables and searches for the smallest feasible perturbation that would reduce the predicted risk below the empirically selected 0.35 screening threshold. This aligns the counterfactual module with the deployed threshold used in the dashboard.

Through this approach, HRP-AI moves beyond passive risk reporting. It provides users with calibrated risk estimates, uncertainty bounds, interpretable risk drivers, and actionable counterfactual guidance. This completes the system's intended workflow from risk prediction to explanation and possible lifestyle intervention.

7 Conclusions

This study successfully engineered, evaluated, and deployed HRP-AI, a calibrated and explainable web application for predicting hypertension risk among Filipino adults using the clinical threshold of $\geq 140/90$ mmHg. By synthesizing data from the 2015 DOST-FNRI National Nutrition Survey (NNS), the study demonstrated that advanced machine learning ensembles can be adapted for cardiovascular screening when discrimination, sensitivity, calibration, and explainability are jointly considered.

Based on the execution of the methodology and the empirical findings, the study addresses the formulated research questions as follows:

1. **Data and Modeling:** Addressing RQ1, the integration of clinical, anthropometric, and dietary components of the 2015 NNS produced a robust multimodal feature space for hypertension risk prediction. The final processed dataset contained 151,189 adult respondents, with 22,988 hypertension-positive cases corresponding to a 15.20% prevalence rate. Eight supervised machine learning algorithms were trained and evaluated, including k-Nearest Neighbors, Naive Bayes, Logistic Regression, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost.

2. **Optimization and Calibration:** Addressing RQ2, the Two-Stage Rigorous Optimization strategy enabled systematic evaluation of candidate algorithms and imbalance-handling configurations. The severe class imbalance was most appropriately handled using algorithmic Class Weights rather than synthetic oversampling. XGBoost with Class Weights was selected as the final predictive engine because it provided the strongest clinically grounded balance between sensitivity, discrimination, and calibration compatibility. The selected model achieved Accuracy = 76.87%, Recall = 76.51%, F1-Score = 0.5014, and AUC = 0.8469. Venn-Abers calibration was then applied to improve probability reliability, reducing the Expected Calibration Error to 0.0035.

3. **Explainability and Actionability:** Addressing RQ3, the black-box nature of the

XGBoost engine was addressed using a tiered XAI framework. Global SHAP identified population-level risk drivers, while Local SHAP and LIME provided patient-specific explanations. The model's most influential features included clinically meaningful variables such as Age, Waist Circumference, and BMI. In addition, bounded Brent's optimization was used to generate Wachter-style counterfactuals by identifying minimal feasible changes needed to reduce modeled risk below the selected 0.35 screening threshold.

4. **System Deployment:** Addressing RQ4, these computational components were integrated into an interactive web application prototype. The HRP-AI dashboard presents calibrated risk scores, Venn-Abers uncertainty bounds, SHAP and LIME explanations, and counterfactual recommendations in a user-facing decision-support interface. The system also uses an empirically selected 35% screening threshold to support risk escalation while distinguishing this model threshold from the clinical diagnostic definition of hypertension.

Overall, HRP-AI demonstrates that a calibrated and explainable machine learning pipeline can support hypertension risk screening using Philippine national survey data. The final system does not replace professional medical diagnosis; rather, it provides a decision-support tool that can flag elevated modeled risk, explain the contributing factors, and suggest actionable directions for follow-up or lifestyle modification. The study reinforces that clinically deployable AI should not be selected by accuracy, AUC, calibration, or explainability alone, but by the combined ability to detect at-risk individuals, provide reliable probabilities, and communicate predictions transparently.

8 Recommendations

While this study successfully establishes a robust, mathematically sound framework for hypertension risk prediction using the 2015 NNS dataset, several avenues for future research and system enhancement remain. Based on the empirical findings and the architectural limitations of the current study, the following recommendations are proposed for future iterations of the HRP-AI system:

- **Data Foundation Update:** This study establishes a highly robust, calibrated baseline architecture using the 2015 DOST-FNRI dataset. It is highly recommended that future iterations retrain this exact XGBoost and Venn-Abers pipeline utilizing the latest publicly available Expanded National Nutrition Survey (ENNS) microdata. Updating the training foundation will ensure the AI captures the most recent post-pandemic demographic and dietary shifts in the Philippine population.

- **Development of a Dedicated Pediatric and Adolescent Pipeline:** Because this study focused strictly on the adult population to avoid the complexities of mixed diagnostic criteria, a significant portion of the dataset (adolescents aged 10-19) was excluded. The 2020 Philippine Clinical Practice Guidelines mandate complex, percentile-based pediatric screening for younger teens before transitioning to the fixed adult cutoffs ($\geq 140/90$ mmHg). Future work should develop a separate, dual-pipeline machine learning architecture capable of dynamically routing patients based on age. This would effectively extend the web application's utility into a holistic, lifespan-spanning predictive tool.

- **Exploration of Deep Learning Architectures for Tabular Data:** While tree-based ensemble methods (specifically XGBoost, CatBoost, and LightGBM) demonstrated optimal performance in this study, the field of tabular deep learning is rapidly advancing. Future research should evaluate architectures such as TabNet, which uses sequential attention to choose which features to reason from at each decision step. Coupling TabNet with conformal prediction and Venn-

Abers calibration could potentially map even more complex, high-dimensional interactions between anthropometric and dietary variables.

- **Clinical Field Testing and Usability Evaluation:** The current iteration of HRP-AI is a functional prototype validated through rigorous offline statistical metrics. The next necessary phase is clinical deployment. It is recommended to deploy the web application in a controlled field test within local Rural Health Units (RHUs) or barangay health centers. This would allow researchers to gather qualitative feedback from healthcare professionals regarding the UI/UX design, the interpretability of the LIME summaries, and the practical, real-world viability of the Wachter-style lifestyle recommendations.

- **Longitudinal Tracking and Wearable Device Integration:** To transition the application from a cross-sectional screening tool to a continuous health management platform, future updates should implement user accounts for longitudinal tracking. By integrating application programming interfaces (APIs) for commercial wearable fitness trackers, the system could automatically ingest real-time physiological markers (e.g., daily step counts, resting heart rate, sleep duration). This continuous data feed would allow the system to dynamically adjust the calibrated risk scores and prescribe dynamically updating counterfactuals as the user's health metrics change over time.

9 Bibliography

[1] D. I. D. Ona, C. A. Jimeno, G. V. Jasul Jr, L. E. Gonzalez-Santos, A. G. Leus, D. D. Bonzon, M. L. E. Bunyi, R. Oliva, L. B. Mercado-Asis, V. A. Luz *et al.*, "Executive summary of the 2020 clinical practice guidelines for the management of hypertension in the philippines," *The Journal of Clinical Hypertension*, vol. 23, no. 8, pp. 1637–1650, 2021.

[2] Department of Science and Technology - Food and Nutrition Research Institute (DOST-FNRI), "2015 national nutrition survey (nns) metadata documentation: Clinical and health," DOST-FNRI, Tech. Rep., 2015. [Online]. Available: https://enutrition.fnri.dost.gov.ph/puf-preview.php?xx=2015477

[3] A. Naik, J. Nalepa, A. M. Wijata, J. Mahon, D. Mistry, A. T. Knowles, E. A. Dawson, G. Y. Lip, I. Olier, and S. Ortega-Martorell, "Artificial intelligence and digital twins for the personalised prediction of hypertension risk," *Computers in Biology and Medicine*, vol. 196, p. 110718, 2025.

[4] S. Pandey, A. Pandey, A. Neupane, D. Subedi, and A. Guragain, "Development of a hypertension risk prediction model using nationally representative survey data: A machine learning approach and web application deployment," *medRxiv*, 2025.

[5] M. Sreeja, A. O. Philip, and M. Supriya, "Towards explainability in artificial intelligence frameworks for heartcare: A comprehensive survey," *Journal of King Saud University-Computer and Information Sciences*, vol. 36, p. 102096, 2024.

[6] Z. M. Altukhi, S. Pradhan, and N. Aljohani, "A systematic literature review of the latest advancements in xai," *Technologies*, vol. 13, no. 3, p. 93, 2025.

[7] H. Löfström, T. Löfström, and U. Johansson, "Calibrated explanations–a primer," *arXiv preprint arXiv:2305.04344*, 2023.

[8] H. Löfström, T. Löfström, U. Johansson, and C. Sönströd, "Calibrated explanations: With uncertainty information and counterfactuals," *Expert Systems with Applications*, vol. 246, p. 123154, 2024.

[9] A. Eldem, "An integrated machine learning framework for the early diagnosis of hypertension disease," *Engineering Applications of Artificial Intelligence*, vol. 162, p. 112564, 2025.

[10] P. Amirian and M. Zarpoosh, "Comprehensive, transparent, and fair machine learning models for hypertension risk prediction: Benchmarking with framingham, external validation, individual-level analysis, and equitable clinical utility," *medRxiv*, pp. 2025–09, 2025.

[11] H. Löfström, T. Löfström, U. Johansson, and C. Sönströd, "Investigating the impact of calibration on the quality of explanations," *Annals of Mathematics and Artificial Intelligence*, pp. 1–26, 2023.

[12] J. G. L. Dela Rosa, C. D. M. Catral, N. A. Reyes, D. M. S. Opiso, E. P. Ong, E. D. B. Ornos, J. R. Santos, E. P. B. Quebral, M. L. J. Callanta, R. V. Oliva *et al.*, "Current status of hypertension care and management in the philippines," *Diabetes & Metabolic Syndrome: Clinical Research & Reviews*, vol. 18, p. 103008, 2024.

[13] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of machine learning research*, vol. 13, no. 2, 2012.

[14] L. B. Elvas, A. Almeida, and J. C. Ferreira, "The role of ai in cardiovascular event monitoring and early detection: Scoping literature review," *JMIR Medical Informatics*, vol. 13, p. e64349, 2025.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[16] R. K. Mohapatra, L. Jolly, and S. P. Dakua, "Advancing explainable ai in healthcare: Necessity, progress, and future directions," *Computational Biology and Chemistry*, vol. 119, p. 108599, 2025.

[17] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[18] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the gdpr," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2017.

[19] S. Niu, Q. Yin, J. Ma, Y. Song, Y. Xu, L. Bai, W. Pan, and X. Yang, "Enhancing healthcare decision support through explainable ai models for risk prediction," *Decision Support Systems*, vol. 181, p. 114228, 2024.

[20] IBM, "What is the k-nearest neighbors algorithm?" https://www.ibm.com/think/topics/knn, 2024, accessed: November 3, 2025.

[21] IBM, "What is a random forest?" https://www.ibm.com/think/topics/random-forest, 2024, accessed: November 3, 2025.

[22] ScienceDirect, "Adaboost - an overview," https://www.sciencedirect.com/topics/engineering/adaboost, 2024, accessed: November 3, 2025.

[23] B. Öztornaci, B. Ata, and S. Kartal, "Analysing household food consumption in turkey using machine learning techniques," *Agris on-line Papers in Economics and Informatics*, vol. 16, no. 2, pp. 97–105, 2024.

[24] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*. Curran Associates, Inc., 2018, pp. 6638–6648. [Online]. Available: http://papers.nips.cc/paper/7892-catboost-unbiased-boosting-with-categorical-features.pdf

[25] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017, pp. 3146–3154. [Online]. Available: http://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf

[26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017, pp. 4765–4774. [Online]. Available: http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf

[27] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, "Scipy 1.0: fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.

10 Appendix

10.1 Source Code

This section contains the core source code utilized for data preprocessing, machine learning model training and rigorous optimization, and the Streamlit web application.

10.1.1 EXP unified pipeline.py

```
from __future__ import annotations

import argparse
import json
import random
import warnings
from copy import deepcopy
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from imblearn.over_sampling import SMOTE, SMOTENC
from scipy.stats import chi2_contingency, loguniform, pointbiserialr, randint, uniform
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.isotonic import IsotonicRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, log_loss, precision_score, recall_score, roc_auc_score
from sklearn.model_selection import ParameterSampler, StratifiedKFold, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, OrdinalEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 300)


try:
    from xgboost import XGBClassifier

    HAS_XGBOOST = True
except Exception:
    XGBClassifier = None
    HAS_XGBOOST = False

try:
    from catboost import CatBoostClassifier

    HAS_CATBOOST = True
except Exception:
    CatBoostClassifier = None
    HAS_CATBOOST = False

try:
    from lightgbm import LGBMClassifier

    HAS_LIGHTGBM = True
except Exception:
    LGBMClassifier = None
    HAS_LIGHTGBM = False

try:
    from venn_abers import VennAbers

    HAS_VENN_ABERS = True
except Exception:
    VennAbers = None
    HAS_VENN_ABERS = False

try:
    import shap

    HAS_SHAP = True
except Exception:
    shap = None
    HAS_SHAP = False

try:
    import lime.lime_tabular

    HAS_LIME = True
except Exception:
    HAS_LIME = False
```

```
try:
    import torch

    HAS_TORCH = True
    TORCH_CUDA_AVAILABLE = bool(torch.cuda.is_available())
except Exception:
    HAS_TORCH = False
    TORCH_CUDA_AVAILABLE = False


SAMPLING_METHODS = ["base", "smote", "smotenc", "cw", "smotecw", "smotencw"]
CW_SAMPLINGS = {"cw", "smotecw", "smotencw"}
CAL_METHODS = ["base", "platt", "isotonic", "venn_abers"]
TARGET_CANDIDATES = ["hypertension", "htn", "target", "label", "outcome"]

MODEL_EXPERIMENT = {
    "knn": "A",
    "randomforest": "A",
    "xgboost": "B",
    "adaboost": "B",
    "logreg": "C",
    "catboost": "C",
    "lightgbm": "C",
    "naive_bayes": "D",
}

# Notebook-faithful per-model stage budgets.
MODEL_STAGE_DEFAULTS: dict[str, dict[str, int]] = {
    # EXP A
    "knn": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    "randomforest": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    # EXP B
    "xgboost": {
        "s1_trials": 25,
        "s1_epochs": 80,
        "s1_folds": 3,
        "top_k_s1": 5,
        "s2_refine": 4,
        "s2_epochs": 200,
        "s2_folds": 4,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    "adaboost": {
        "s1_trials": 25,
        "s1_epochs": 80,
        "s1_folds": 3,
        "top_k_s1": 5,
        "s2_refine": 4,
        "s2_epochs": 200,
        "s2_folds": 4,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    # EXP C
    "logreg": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    "catboost": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    "lightgbm": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
```

```
        "top_k_s2": 2,
        "final_epochs": 400,
    },
    # EXP D
    "naive_bayes": {
        "s1_trials": 15,
        "s1_epochs": 80,
        "s1_folds": 2,
        "top_k_s1": 5,
        "s2_refine": 2,
        "s2_epochs": 200,
        "s2_folds": 3,
        "top_k_s2": 2,
        "final_epochs": 400,
    },
}

DEFAULT_DROP_COLUMNS = [
    "mos_lactation",
    "cu",
    "strrec_anthro",
    "psurec_anthro",
    "provcode_anthro",
    "mos_preg",
    "anthro_group",
]


@dataclass
class PreprocessArtifacts:
    X_train_imp: pd.DataFrame
    X_cal_imp: pd.DataFrame
    X_test_imp: pd.DataFrame
    X_train_proc: np.ndarray
    X_cal_proc: np.ndarray
    X_test_proc: np.ndarray
    y_train: np.ndarray
    y_cal: np.ndarray
    y_test: np.ndarray
    knn_imputer: KNNImputer
    cat_imputer: SimpleImputer | None
    scaler: StandardScaler
    ohe: OneHotEncoder | None
    num_cols_full: list[str]
    num_cols_reduced: list[str]
    cat_cols: list[str]
    feature_names: list[str]


def resolve_training_root(cli_root: str | None) -> Path:
    if cli_root:
        root = Path(cli_root).resolve()
        if not root.exists():
            raise FileNotFoundError(f"Provided root does not exist: {root}")
        return root

    script_dir = Path(__file__).resolve().parent
    candidates = [
        script_dir,
        Path.cwd(),
        Path.cwd() / "src" / "training",
        Path.cwd().parent / "training",
    ]

    for candidate in candidates:
        if (candidate / "Datasets2015").exists():
            return candidate.resolve()

    raise FileNotFoundError(
        "Could not resolve training root. Provide --root pointing to the folder containing Datasets2015."
    )


def _find_first_dataset_csv(folder: Path) -> Path:
    preferred = sorted(folder.glob("*data-set*.csv"))
    if preferred:
        return preferred[0]
    csvs = sorted([p for p in folder.glob("*.csv") if "dictionary" not in p.name.lower()])
    if csvs:
        return csvs[0]
    raise FileNotFoundError(f"No dataset CSV found in {folder}")


def merge_datasets(training_root: Path) -> Path:
    datasets_root = training_root / "Datasets2015"
    clinical_path = _find_first_dataset_csv(datasets_root / "Clinical")
    dietary_path = _find_first_dataset_csv(datasets_root / "Dietary")
    anthropometric_path = _find_first_dataset_csv(datasets_root / "Anthropometric")

    clinical = pd.read_csv(clinical_path, low_memory=False)
    dietary = pd.read_csv(dietary_path, low_memory=False)
    anthropometric = pd.read_csv(anthropometric_path, low_memory=False)

    requested_keys = ["hhnum", "member_code"]

    if "hhnum" not in clinical.columns:
        raise KeyError("Clinical dataset must contain 'hhnum'.")

    anthropometric_join_keys = [
        k for k in requested_keys if k in anthropometric.columns and k in clinical.columns
    ]
    if len(anthropometric_join_keys) < 2:
        raise KeyError("Anthropometric dataset must contain both 'hhnum' and 'member_code'.")

    dietary_join_keys = [k for k in requested_keys if k in dietary.columns and k in clinical.columns]
    if not dietary_join_keys:
```

```
        raise KeyError("Dietary dataset must contain at least 'hhnum' for joining.")

    dietary = dietary.drop_duplicates(subset=dietary_join_keys, keep="first")
    anthropometric = anthropometric.drop_duplicates(subset=anthropometric_join_keys, keep="first")

    dietary_overlap = [
        c for c in dietary.columns if c in clinical.columns and c not in dietary_join_keys
    ]
    if dietary_overlap:
        dietary = dietary.rename(columns={c: f"{c}_dietary" for c in dietary_overlap})

    merged = clinical.merge(dietary, on=dietary_join_keys, how="left")

    anthropometric_overlap = [
        c for c in anthropometric.columns if c in merged.columns and c not in anthropometric_join_keys
    ]
    if anthropometric_overlap:
        anthropometric = anthropometric.rename(columns={c: f"{c}_anthro" for c in anthropometric_overlap})

    merged = merged.merge(anthropometric, on=anthropometric_join_keys, how="left")

    output_path = training_root / "merged_clinical_leftjoin.csv"
    merged.to_csv(output_path, index=False)

    print("Merged dataset created")
    print(f"  Clinical source      : {clinical_path.name}")
    print(f"  Dietary source       : {dietary_path.name}")
    print(f"  Anthropometric source: {anthropometric_path.name}")
    print(f"  Output               : {output_path}")
    print(f"  Rows                 : {len(merged):,}")
    print(f"  Columns              : {merged.shape[1]:,}")

    return output_path


def infer_target_column(df: pd.DataFrame, candidates: list[str]) -> str | None:
    lower_map = {c.lower(): c for c in df.columns}
    for candidate in candidates:
        if candidate.lower() in lower_map:
            return lower_map[candidate.lower()]
    for column in df.columns:
        col_lc = column.lower()
        if any(candidate.lower() in col_lc for candidate in candidates):
            return column
    return None


def infer_bp_columns(df: pd.DataFrame) -> tuple[str | None, str | None]:
    sbp_aliases = ["ave_sbp"]
    dbp_aliases = ["ave_dbp"]
    lower_map = {c.lower(): c for c in df.columns}

    sbp_col = None
    for alias in sbp_aliases:
        if alias in lower_map:
            sbp_col = lower_map[alias]
            break
    if sbp_col is None:
        for column in df.columns:
            if any(alias in column.lower() for alias in sbp_aliases):
                sbp_col = column
                break

    dbp_col = None
    for alias in dbp_aliases:
        if alias in lower_map:
            dbp_col = lower_map[alias]
            break
    if dbp_col is None:
        for column in df.columns:
            if any(alias in column.lower() for alias in dbp_aliases):
                dbp_col = column
                break

    return sbp_col, dbp_col


def find_first_column_case_insensitive(columns: list[str], candidates: list[str]) -> str | None:
    lower_map = {c.lower(): c for c in columns}
    for candidate in candidates:
        if candidate.lower() in lower_map:
            return lower_map[candidate.lower()]
    for column in columns:
        col_lc = column.lower()
        if any(candidate.lower() in col_lc for candidate in candidates):
            return column
    return None


def to_numeric_clean(series: pd.Series) -> pd.Series:
    values = pd.to_numeric(series, errors="coerce")
    return values.where(~values.isin([9, 99, 888888, 999999]), np.nan)


def build_smoking_level_feature(df_in: pd.DataFrame) -> tuple[pd.Series | None, list[str]]:
    used_cols: list[str] = []
    smoking_level_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["smoking_level"])
    smoke_status_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["smoke_status"])
    current_smoking_col = find_first_column_case_insensitive(
        df_in.columns.tolist(), ["current_smoking", "currentsmoking"]
    )
    ever_smoke_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["ever_smk"])

    if smoking_level_col is not None:
        smoking = to_numeric_clean(df_in[smoking_level_col]).clip(lower=0, upper=3)
        used_cols.append(smoking_level_col)
```

```
        return smoking.astype(float), sorted(set(used_cols))

    idx = df_in.index
    smoking = pd.Series(np.nan, index=idx, dtype=float)

    if smoke_status_col is not None:
        status = to_numeric_clean(df_in[smoke_status_col])
        used_cols.append(smoke_status_col)
        smoking.loc[status == 0] = 0
        smoking.loc[status == 2] = 1
        smoking.loc[status == 1] = 2
        if current_smoking_col is not None:
            current = to_numeric_clean(df_in[current_smoking_col])
            used_cols.append(current_smoking_col)
            smoking.loc[(status == 1) & (current == 3)] = 3
        return smoking.astype(float), sorted(set(used_cols))

    if current_smoking_col is not None:
        current = to_numeric_clean(df_in[current_smoking_col])
        used_cols.append(current_smoking_col)
        smoking.loc[current == 0] = 0
        smoking.loc[current.isin([1, 2])] = 2
        smoking.loc[current == 3] = 3
        if ever_smoke_col is not None:
            ever = to_numeric_clean(df_in[ever_smoke_col])
            used_cols.append(ever_smoke_col)
            smoking.loc[(current == 0) & (ever > 0)] = 1
        return smoking.astype(float), sorted(set(used_cols))

    if ever_smoke_col is not None:
        ever = to_numeric_clean(df_in[ever_smoke_col])
        used_cols.append(ever_smoke_col)
        smoking.loc[ever == 0] = 0
        smoking.loc[ever > 0] = 1
        return smoking.astype(float), sorted(set(used_cols))

    return None, []


def build_alcohol_level_feature(df_in: pd.DataFrame) -> tuple[pd.Series | None, list[str]]:
    used_cols: list[str] = []
    alcohol_level_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["alcohol_level"])
    alcohol_status_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["alcohol_status"])
    alcohol_ever_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["alcohol"])
    current_alcohol_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["con_alcohol"])
    drink30_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["drnk_30days"])
    binge_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["binge_drink", "binge_drinking"])

    if alcohol_level_col is not None:
        alcohol = to_numeric_clean(df_in[alcohol_level_col]).clip(lower=0, upper=3)
        used_cols.append(alcohol_level_col)
        return alcohol.astype(float), sorted(set(used_cols))

    idx = df_in.index
    alcohol = pd.Series(np.nan, index=idx, dtype=float)

    if alcohol_status_col is not None:
        status = to_numeric_clean(df_in[alcohol_status_col])
        used_cols.append(alcohol_status_col)
        alcohol.loc[status == 0] = 0
        alcohol.loc[status == 2] = 1
        alcohol.loc[status == 1] = 2
        if binge_col is not None:
            binge = to_numeric_clean(df_in[binge_col])
            used_cols.append(binge_col)
            alcohol.loc[(status == 1) & (binge == 1)] = 3
        return alcohol.astype(float), sorted(set(used_cols))

    alcohol.loc[:] = 0
    if alcohol_ever_col is not None:
        ever = to_numeric_clean(df_in[alcohol_ever_col])
        used_cols.append(alcohol_ever_col)
        alcohol.loc[ever > 0] = 1
    if current_alcohol_col is not None:
        current = to_numeric_clean(df_in[current_alcohol_col])
        used_cols.append(current_alcohol_col)
        alcohol.loc[current == 1] = np.maximum(alcohol.loc[current == 1], 2)
    if drink30_col is not None:
        drink30 = to_numeric_clean(df_in[drink30_col])
        used_cols.append(drink30_col)
        alcohol.loc[drink30 == 1] = np.maximum(alcohol.loc[drink30 == 1], 2)
    if binge_col is not None:
        binge = to_numeric_clean(df_in[binge_col])
        used_cols.append(binge_col)
        alcohol.loc[binge == 1] = 3

    if used_cols:
        return alcohol.astype(float), sorted(set(used_cols))
    return None, []


def build_bmi_feature(df_in: pd.DataFrame) -> tuple[pd.Series | None, list[str]]:
    weight_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["weight"])
    height_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["height"])
    if weight_col is None or height_col is None:
        return None, []

    weight = pd.to_numeric(df_in[weight_col], errors="coerce")
    height = pd.to_numeric(df_in[height_col], errors="coerce")
    height_m = height.copy()
    if pd.notna(height_m.median(skipna=True)) and float(height_m.median(skipna=True)) > 3.0:
        height_m = height_m / 100.0

    bmi = (weight / (height_m**2)).replace([np.inf, -np.inf], np.nan)
    return bmi.astype(float), [weight_col, height_col]
```

```
def build_whr_feature(df_in: pd.DataFrame) -> tuple[pd.Series | None, list[str]]:
    waist_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["waist"])
    hip_col = find_first_column_case_insensitive(df_in.columns.tolist(), ["hip"])
    if waist_col is None or hip_col is None:
        return None, []

    waist = pd.to_numeric(df_in[waist_col], errors="coerce")
    hip = pd.to_numeric(df_in[hip_col], errors="coerce").replace(0, np.nan)
    whr = (waist / hip).replace([np.inf, -np.inf], np.nan)
    return whr.astype(float), [waist_col, hip_col]


def load_and_prepare_features(data_path: Path) -> tuple[pd.DataFrame, pd.Series, dict[str, Any]]:
    df = pd.read_csv(data_path)
    target_col = infer_target_column(df, TARGET_CANDIDATES)
    target_defined_from_bp = False

    if target_col is None:
        sbp_col, dbp_col = infer_bp_columns(df)
        if sbp_col is not None and dbp_col is not None:
            sbp = pd.to_numeric(df[sbp_col], errors="coerce")
            dbp = pd.to_numeric(df[dbp_col], errors="coerce")
            df["Hypertension"] = (((sbp >= 140) | (dbp >= 90)).fillna(False)).astype(int)
            target_col = "Hypertension"
            target_defined_from_bp = True
        else:
            raise ValueError(
                "Could not infer target and could not derive Hypertension from SBP/DBP (140/90 OR rule)."
            )

    df = df.dropna(subset=[target_col]).copy()
    y_raw = df[target_col]
    if y_raw.nunique() != 2:
        raise ValueError(f"Target must be binary. Found {y_raw.nunique()} classes.")

    if y_raw.dtype == "O":
        y = pd.Series(LabelEncoder().fit_transform(y_raw.astype(str)), index=y_raw.index, name=target_col)
    else:
        y = pd.Series(y_raw.astype(int), index=y_raw.index, name=target_col)

    X = df.drop(columns=[target_col]).copy()

    smoking_feature, smoking_sources = build_smoking_level_feature(X)
    if smoking_feature is not None:
        X["fe_smoking_level"] = smoking_feature

    alcohol_feature, alcohol_sources = build_alcohol_level_feature(X)
    if alcohol_feature is not None:
        X["fe_alcohol_level"] = alcohol_feature

    bmi_feature, bmi_sources = build_bmi_feature(X)
    if bmi_feature is not None:
        X["bmi"] = bmi_feature

    whr_feature, whr_sources = build_whr_feature(X)
    if whr_feature is not None:
        X["whr"] = whr_feature

    behavior_raw_candidates = [
        "current_smoking",
        "currentsmoking",
        "ever_smk",
        "smoke_status",
        "smoking_level",
        "alcohol",
        "con_alcohol",
        "drnk_30days",
        "drnk_30d_num",
        "alcohol_status",
        "binge_drink",
        "binge_drinking",
        "alcohol_level",
    ]
    x_lc = {c.lower(): c for c in X.columns}
    behavior_drop = sorted({x_lc[c.lower()] for c in behavior_raw_candidates if c.lower() in x_lc})
    if behavior_drop:
        X = X.drop(columns=behavior_drop, errors="ignore")

    anthropometric_source_drop = sorted(
        {
            c
            for c in set((bmi_sources or []) + (whr_sources or []))
            if c in X.columns and c.lower() not in {"bmi", "whr"}
        }
    )
    if anthropometric_source_drop:
        X = X.drop(columns=anthropometric_source_drop, errors="ignore")

    non_removable_base_aliases = ["age", "sex"]
    manual_non_predictive = [
        "regcode",
        "provcode",
        "provhuc",
        "psc",
        "csc",
        "rhc",
        "psurec",
        "strrec",
        "wgts",
        "fwgt",
        "finalwgt",
        "finalwgt1",
        "finalwgt4",
        "fwgth_natl_var",
        "fwgth_prov",
```

```
        "fwgth_natl2_var",
        "fwgti_natl_var",
        "fwgti_prov",
        "fwgti_natl2_var",
        "fwgti_prov2",
        "rep_natl",
        "rep_prov",
        "ms_psucode",
        "enns_year",
        "wrkplace",
        "interview_status",
        "intdate",
        "enumcode",
        "hhnum",
        "member_code",
        "ave_sbp",
        "ave_dbp",
        "sbp",
        "dbp",
        "systolic",
        "diastolic",
        "sysbp",
        "diabp",
        "blood_pressure",
        "height",
        "weight",
        "waist",
        "hip",
    ]

    x_lc = {c.lower(): c for c in X.columns}
    manual_drop = sorted({x_lc[c.lower()] for c in manual_non_predictive if c.lower() in x_lc})

    protected_base_cols: list[str] = []
    for column in X.columns:
        col_lc = column.lower()
        if any(alias in col_lc for alias in non_removable_base_aliases):
            protected_base_cols.append(column)
    protected_base_cols = sorted(set(protected_base_cols))

    if manual_drop:
        manual_drop = [c for c in manual_drop if c not in protected_base_cols]
        X = X.drop(columns=manual_drop, errors="ignore")

    age_columns = sorted([c for c in X.columns if "age" in c.lower()])
    if len(age_columns) > 1:
        canonical_age = (
            next((c for c in age_columns if c.lower() == "age"), None)
            or next((c for c in age_columns if c.lower() == "agemos"), None)
            or age_columns[0]
        )
        age_drop_duplicates = [c for c in age_columns if c != canonical_age]
        X = X.drop(columns=age_drop_duplicates, errors="ignore")

    sex_columns = sorted([c for c in X.columns if "sex" in c.lower()])
    if len(sex_columns) > 1:
        canonical_sex = next((c for c in sex_columns if c.lower() == "sex"), None) or sex_columns[0]
        sex_drop_duplicates = [c for c in sex_columns if c != canonical_sex]
        X = X.drop(columns=sex_drop_duplicates, errors="ignore")

    for candidate in DEFAULT_DROP_COLUMNS:
        if candidate in X.columns:
            X = X.drop(columns=[candidate], errors="ignore")

    metadata = {
        "target_col": target_col,
        "target_defined_from_bp": target_defined_from_bp,
        "manual_drop_count": len(manual_drop),
        "behavior_drop_count": len(behavior_drop),
        "anthropometric_source_drop_count": len(anthropometric_source_drop),
        "protected_base_cols": protected_base_cols,
        "smoking_sources": smoking_sources,
        "alcohol_sources": alcohol_sources,
    }

    return X, y, metadata


def build_available_models(requested: list[str] | None) -> list[str]:
    base = ["knn", "randomforest", "adaboost", "logreg", "naive_bayes"]
    if HAS_XGBOOST:
        base.append("xgboost")
    if HAS_CATBOOST:
        base.append("catboost")
    if HAS_LIGHTGBM:
        base.append("lightgbm")

    if requested:
        requested_set = set(requested)
        base = [m for m in base if m in requested_set]

    if not base:
        raise RuntimeError("No trainable models available with the current environment and --models filter.")

    return base


def build_model_spaces(models: list[str]) -> dict[str, dict[str, Any]]:
    spaces: dict[str, dict[str, Any]] = {}

    if "knn" in models:
        spaces["knn"] = {
            "n_neighbors": randint(3, 51),
            "weights": ["uniform", "distance"],
            "metric": ["euclidean", "manhattan", "chebyshev"],
        }
```

```
    if "randomforest" in models:
        spaces["randomforest"] = {
            "max_features": uniform(0.1, 0.9),
            "min_samples_split": randint(2, 30),
            "min_samples_leaf": randint(1, 15),
            "max_depth": [None, 10, 20, 30, 50],
        }

    if "xgboost" in models:
        spaces["xgboost"] = {
            "learning_rate": loguniform(0.005, 0.40),
            "max_depth": randint(3, 12),
            "subsample": uniform(0.5, 0.5),
            "colsample_bytree": uniform(0.5, 0.5),
            "min_child_weight": randint(1, 10),
            "gamma": uniform(0.0, 1.0),
            "reg_lambda": loguniform(1e-2, 100),
            "reg_alpha": loguniform(1e-3, 10),
        }

    if "adaboost" in models:
        spaces["adaboost"] = {
            "n_estimators": randint(20, 80),
            "learning_rate": loguniform(0.01, 1.0),
            "base_depth": randint(1, 5),
        }

    if "logreg" in models:
        spaces["logreg"] = {
            "C": loguniform(1e-5, 1e3),
            "solver": ["lbfgs", "liblinear", "newton-cg", "sag", "saga"],
            "penalty": ["l2", "l1", "elasticnet"],
            "max_iter": randint(500, 5000),
        }

    if "catboost" in models:
        spaces["catboost"] = {
            "learning_rate": loguniform(0.001, 0.5),
            "depth": randint(4, 12),
            "l2_leaf_reg": loguniform(1e-2, 100),
            "random_strength": uniform(0.0, 2.5),
            "bagging_temperature": uniform(0.0, 2.0),
        }

    if "lightgbm" in models:
        spaces["lightgbm"] = {
            "learning_rate": loguniform(0.005, 0.40),
            "max_depth": randint(3, 12),
            "num_leaves": randint(20, 200),
            "min_child_samples": randint(5, 50),
            "subsample": uniform(0.5, 0.5),
            "colsample_bytree": uniform(0.5, 0.5),
            "reg_lambda": loguniform(1e-3, 100),
            "reg_alpha": loguniform(1e-3, 10),
        }

    if "naive_bayes" in models:
        spaces["naive_bayes"] = {
            "var_smoothing": loguniform(1e-13, 5e-7),
            "positive_prior": [None, 0.20, 0.24, 0.28, 0.32, 0.36],
            "probability_floor": [0.0, 0.0005, 0.001, 0.0025, 0.005],
            "cw_pos_scale": uniform(0.7, 1.5),
            "smote_sampling_strategy": uniform(0.60, 0.35),
            "smote_k_neighbors": randint(2, 8),
        }

    return spaces


def _ohe_factory() -> OneHotEncoder:
    try:
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown="ignore", sparse=False)


def fit_imputers(X_train: pd.DataFrame, num_cols: list[str], cat_cols: list[str]) -> tuple[KNNImputer, SimpleImputer | None]:
    knn = KNNImputer(n_neighbors=5)
    if num_cols:
        knn.fit(X_train[num_cols])
    else:
        knn.fit(pd.DataFrame(index=X_train.index))

    cat_imputer = None
    if cat_cols:
        cat_imputer = SimpleImputer(strategy="most_frequent")
        cat_imputer.fit(X_train[cat_cols])

    return knn, cat_imputer


def impute_frame(
    knn_imputer: KNNImputer,
    cat_imputer: SimpleImputer | None,
    X: pd.DataFrame,
    num_cols: list[str],
    cat_cols: list[str],
) -> pd.DataFrame:
    frames: list[pd.DataFrame] = []
    if num_cols:
        num_data = pd.DataFrame(
            knn_imputer.transform(X[num_cols]),
            columns=num_cols,
            index=X.index,
        )
```

```
        frames.append(num_data)
    if cat_cols and cat_imputer is not None:
        cat_data = pd.DataFrame(
            cat_imputer.transform(X[cat_cols]),
            columns=cat_cols,
            index=X.index,
        )
        frames.append(cat_data)

    return pd.concat(frames, axis=1) if frames else pd.DataFrame(index=X.index)


def fit_encoder_scaler(
    X_train_imp: pd.DataFrame,
    num_cols: list[str],
    cat_cols: list[str],
) -> tuple[StandardScaler, OneHotEncoder | None, list[str]]:
    scaler = StandardScaler()
    ohe = None

    feature_names = list(num_cols)

    if num_cols:
        scaler.fit(X_train_imp[num_cols])

    if cat_cols:
        ohe = _ohe_factory()
        ohe.fit(X_train_imp[cat_cols].astype(str))
        feature_names += ohe.get_feature_names_out(cat_cols).tolist()

    return scaler, ohe, feature_names


def encode_scale(
    scaler: StandardScaler,
    ohe: OneHotEncoder | None,
    X_imp: pd.DataFrame,
    num_cols: list[str],
    cat_cols: list[str],
) -> np.ndarray:
    parts: list[np.ndarray] = []

    if num_cols:
        parts.append(scaler.transform(X_imp[num_cols]))

    if cat_cols and ohe is not None:
        parts.append(ohe.transform(X_imp[cat_cols].astype(str)))

    return np.hstack(parts) if parts else np.empty((len(X_imp), 0), dtype=float)


def get_sampled_train(
    X_train_imp: pd.DataFrame,
    y_train: np.ndarray,
    sampling: str,
    scaler: StandardScaler,
    ohe: OneHotEncoder | None,
    num_cols: list[str],
    cat_cols: list[str],
    seed: int,
    sampling_params: dict[str, Any] | None = None,
) -> tuple[np.ndarray, np.ndarray]:
    y_arr = np.asarray(y_train, dtype=int)
    class_counts = np.bincount(y_arr)

    # SMOTE/SMOTENC requires at least 2 minority samples.
    if len(class_counts) < 2 or int(class_counts.min()) < 2:
        return encode_scale(scaler, ohe, X_train_imp, num_cols, cat_cols), y_arr.copy()

    minority_count = int(class_counts.min()) if len(class_counts) > 1 else 0
    k_neighbors = max(1, min(5, minority_count - 1)) if minority_count > 1 else 1
    sampling_strategy: float | str = "auto"

    if sampling_params is not None:
        if "smote_k_neighbors" in sampling_params:
            requested_k = max(1, int(round(float(sampling_params["smote_k_neighbors"]))))
            k_neighbors = max(1, min(k_neighbors, requested_k))

        if "smote_sampling_strategy" in sampling_params:
            sampling_strategy = float(np.clip(float(sampling_params["smote_sampling_strategy"]), 0.05, 1.0))

    if sampling in {"base", "cw"}:
        return encode_scale(scaler, ohe, X_train_imp, num_cols, cat_cols), y_arr.copy()

    if sampling in {"smote", "smotecw"}:
        X_proc = encode_scale(scaler, ohe, X_train_imp, num_cols, cat_cols)
        sampler = SMOTE(random_state=seed, k_neighbors=k_neighbors, sampling_strategy=sampling_strategy)
        return sampler.fit_resample(X_proc, y_arr)

    if not cat_cols:
        X_proc = encode_scale(scaler, ohe, X_train_imp, num_cols, cat_cols)
        sampler = SMOTE(random_state=seed, k_neighbors=k_neighbors, sampling_strategy=sampling_strategy)
        return sampler.fit_resample(X_proc, y_arr)

    encoder = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)
    X_cat_ord = encoder.fit_transform(X_train_imp[cat_cols].astype(str))
    X_num = X_train_imp[num_cols].values if num_cols else np.empty((len(X_train_imp), 0))
    X_joined = np.hstack([X_num, X_cat_ord])

    categorical_indices = list(range(len(num_cols), len(num_cols) + len(cat_cols)))
    sampler = SMOTENC(
        categorical_features=categorical_indices,
        random_state=seed,
        k_neighbors=k_neighbors,
        sampling_strategy=sampling_strategy,
    )
```

```
    X_resampled, y_resampled = sampler.fit_resample(X_joined, y_arr)

    df_num = (
        pd.DataFrame(X_resampled[:, : len(num_cols)], columns=num_cols)
        if num_cols
        else pd.DataFrame(index=range(len(X_resampled)))
    )

    X_cat_ord_res = np.round(X_resampled[:, len(num_cols) :]).astype(float)
    for j, categories in enumerate(encoder.categories_):
        X_cat_ord_res[:, j] = np.clip(X_cat_ord_res[:, j], 0, len(categories) - 1)
    X_cat_back = encoder.inverse_transform(X_cat_ord_res)
    df_cat = pd.DataFrame(X_cat_back, columns=cat_cols)

    X_rebuilt = pd.concat([df_num, df_cat], axis=1)
    X_proc = encode_scale(scaler, ohe, X_rebuilt, num_cols, cat_cols)
    return X_proc, np.asarray(y_resampled, dtype=int)


def compute_ece(y_true: np.ndarray, y_prob: np.ndarray, n_bins: int = 10) -> float:
    y_true = np.asarray(y_true, dtype=int)
    y_prob = np.clip(np.asarray(y_prob), 1e-9, 1 - 1e-9)

    bins = np.linspace(0, 1, n_bins + 1)
    bin_ids = np.digitize(y_prob, bins) - 1
    ece = 0.0

    for idx in range(n_bins):
        mask = bin_ids == idx
        if mask.sum() == 0:
            continue
        ece += mask.mean() * abs(y_true[mask].mean() - y_prob[mask].mean())

    return float(ece)


def metric_pack(y_true: np.ndarray, y_prob: np.ndarray, threshold: float = 0.5) -> dict[str, float]:
    y_pred = (np.asarray(y_prob) >= float(threshold)).astype(int)

    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
        "auc": float(roc_auc_score(y_true, y_prob)) if len(np.unique(y_true)) > 1 else 0.5,
        "logloss": float(log_loss(y_true, y_prob)),
        "ece": float(compute_ece(y_true, y_prob)),
    }


def _fit_model_with_nb_weights(model: Any, model_name: str, use_class_weight: bool, X: np.ndarray, y: np.ndarray) -> None:
    if model_name == "naive_bayes" and use_class_weight:
        pos_scale = float(getattr(model, "_cw_pos_scale", 1.0))
        y_arr = np.asarray(y, dtype=int)
        n_pos = max(1, int(y_arr.sum()))
        n_neg = max(1, int(len(y_arr) - n_pos))
        sw = np.where(y_arr == 1, (n_neg / n_pos) * pos_scale, 1.0)
        model.fit(X, y_arr, sample_weight=sw)
    else:
        model.fit(X, y)


def _apply_nb_probability_floor(model: Any, p: np.ndarray) -> np.ndarray:
    p_floor = float(getattr(model, "_probability_floor", 0.0))
    if p_floor > 0.0:
        return np.clip(p, p_floor, 1.0 - p_floor)
    return p


def fit_calibrator(method: str, p_cal: np.ndarray, y_cal: np.ndarray) -> Any:
    if method == "base":
        return None

    p_cal = np.clip(np.asarray(p_cal), 1e-9, 1 - 1e-9)
    y_cal = np.asarray(y_cal, dtype=int)

    if method == "platt":
        calibrator = LogisticRegression(max_iter=3000, random_state=42)
        calibrator.fit(p_cal.reshape(-1, 1), y_cal)
        return calibrator

    if method == "isotonic":
        calibrator = IsotonicRegression(out_of_bounds="clip")
        calibrator.fit(p_cal, y_cal)
        return calibrator

    if method == "venn_abers":
        if not HAS_VENN_ABERS:
            raise RuntimeError("venn-abers is not installed")
        calibrator = VennAbers()
        calibrator.fit(np.column_stack([1.0 - p_cal, p_cal]), y_cal)
        return calibrator

    raise ValueError(f"Unknown calibration method: {method}")


def apply_calibrator(method: str, calibrator: Any, p_eval: np.ndarray) -> np.ndarray:
    p_eval = np.clip(np.asarray(p_eval), 1e-9, 1 - 1e-9)

    if method == "base" or calibrator is None:
        return p_eval

    if method == "platt":
        return np.clip(calibrator.predict_proba(p_eval.reshape(-1, 1))[:, 1], 1e-9, 1 - 1e-9)

    if method == "isotonic":
```

```
            return np.clip(calibrator.predict(p_eval), 1e-9, 1 - 1e-9)

        if method == "venn_abers":
            pred = calibrator.predict_proba(np.column_stack([1.0 - p_eval, p_eval]))
            if isinstance(pred, tuple) and len(pred) == 2:
                calibrated_pair, interval_pair = pred
                calibrated_pair = np.asarray(calibrated_pair)
                if calibrated_pair.ndim == 2 and calibrated_pair.shape[1] >= 2:
                    return np.clip(calibrated_pair[:, 1], 1e-9, 1 - 1e-9)

                interval_pair = np.asarray(interval_pair)
                if interval_pair.ndim == 2 and interval_pair.shape[1] >= 2:
                    return np.clip(interval_pair[:, 1], 1e-9, 1 - 1e-9)
                return np.clip(interval_pair.reshape(-1), 1e-9, 1 - 1e-9)

            pred = np.asarray(pred)
            if pred.ndim == 2 and pred.shape[1] >= 2:
                return np.clip(pred[:, 1], 1e-9, 1 - 1e-9)
            return np.clip(pred.reshape(-1), 1e-9, 1 - 1e-9)

        raise ValueError(f"Unknown calibration method: {method}")


    def refine_candidates(base_params_list: list[dict[str, Any]], n_refine: int, seed: int) -> list[dict[str, Any]]:
        rng = np.random.RandomState(seed)
        candidates: list[dict[str, Any]] = []

        for params in base_params_list:
            candidates.append(deepcopy(params))
            for _ in range(n_refine):
                refined: dict[str, Any] = {}
                for key, value in params.items():
                    if isinstance(value, (int, np.integer)):
                        refined[key] = max(1, int(round(value * rng.uniform(0.7, 1.3))))
                    elif isinstance(value, (float, np.floating)):
                        refined[key] = max(1e-7, float(value * rng.uniform(0.7, 1.3)))
                    else:
                        refined[key] = value
                candidates.append(refined)

        unique: list[dict[str, Any]] = []
        seen: set[str] = set()
        for params in candidates:
            key = json.dumps(params, sort_keys=True, default=str)
            if key in seen:
                continue
            seen.add(key)
            unique.append(params)

        return unique


    def build_model_for_search(
        model_name: str,
        params: dict[str, Any],
        use_class_weight: bool,
        y_sample: np.ndarray,
        epoch_budget: int,
        seed: int,
        use_gpu_when_available: bool,
    ) -> Any:
        y_sample = np.asarray(y_sample, dtype=int)
        n_pos = int(y_sample.sum())
        n_neg = int(len(y_sample) - n_pos)
        pos_weight = float(n_neg) / max(n_pos, 1)
        p = deepcopy(params)

        if model_name == "knn":
            return KNeighborsClassifier(
                n_neighbors=max(1, int(p.get("n_neighbors", 7))),
                weights=p.get("weights", "uniform"),
                metric=p.get("metric", "euclidean"),
                n_jobs=1,
            )

        if model_name == "randomforest":
            max_features = p.get("max_features", "sqrt")
            if isinstance(max_features, float):
                max_features = float(np.clip(max_features, 0.01, 1.0))

            return RandomForestClassifier(
                n_estimators=int(max(20, epoch_budget)),
                max_depth=p.get("max_depth", None),
                min_samples_split=max(2, int(round(p.get("min_samples_split", 2)))),
                min_samples_leaf=max(1, int(round(p.get("min_samples_leaf", 1)))),
                max_features=max_features,
                random_state=seed,
                n_jobs=-1,
                class_weight="balanced_subsample" if use_class_weight else None,
            )

        if model_name == "xgboost":
            if not HAS_XGBOOST:
                raise RuntimeError("xgboost is not available")

            use_gpu = bool(use_gpu_when_available and TORCH_CUDA_AVAILABLE)
            return XGBClassifier(
                n_estimators=int(max(30, epoch_budget)),
                learning_rate=max(1e-4, float(p.get("learning_rate", 0.1))),
                max_depth=max(1, int(round(p.get("max_depth", 5)))),
                subsample=float(np.clip(p.get("subsample", 0.8), 0.1, 1.0)),
                colsample_bytree=float(np.clip(p.get("colsample_bytree", 0.8), 0.1, 1.0)),
                min_child_weight=max(1, int(round(p.get("min_child_weight", 1)))),
                gamma=max(0.0, float(p.get("gamma", 0.0))),
                reg_lambda=max(1e-4, float(p.get("reg_lambda", 1.0))),
                objective="binary:logistic",
```

```
            eval_metric="logloss",
            random_state=seed,
            tree_method="hist",
            device="cuda" if use_gpu else "cpu",
            verbosity=0,
            scale_pos_weight=pos_weight if use_class_weight else 1.0,
        )

    if model_name == "adaboost":
        base_depth = max(1, int(round(p.get("base_depth", 1))))
        base_estimator = DecisionTreeClassifier(
            max_depth=base_depth,
            random_state=seed,
            class_weight="balanced" if use_class_weight else None,
        )
        kwargs = {
            "n_estimators": max(10, int(round(p.get("n_estimators", epoch_budget)))),
            "learning_rate": max(1e-4, float(p.get("learning_rate", 0.1))),
            "random_state": seed,
        }
        try:
            return AdaBoostClassifier(estimator=base_estimator, **kwargs)
        except TypeError:
            return AdaBoostClassifier(base_estimator=base_estimator, **kwargs)

    if model_name == "logreg":
        return LogisticRegression(
            C=float(p.get("C", 1.0)),
            solver=p.get("solver", "lbfgs"),
            penalty=p.get("penalty", "l2"),
            max_iter=1000,
            random_state=seed,
            class_weight="balanced" if use_class_weight else None,
        )

    if model_name == "catboost":
        if not HAS_CATBOOST:
            raise RuntimeError("catboost is not available")

        use_gpu = bool(use_gpu_when_available and TORCH_CUDA_AVAILABLE)
        kwargs = {
            "iterations": int(max(30, epoch_budget)),
            "learning_rate": max(1e-4, float(p.get("learning_rate", 0.1))),
            "depth": max(1, int(round(p.get("depth", 6)))),
            "l2_leaf_reg": max(1e-4, float(p.get("l2_leaf_reg", 3.0))),
            "random_strength": max(1e-4, float(p.get("random_strength", 1.0))),
            "loss_function": "Logloss",
            "eval_metric": "Logloss",
            "random_seed": seed,
            "verbose": 0,
        }
        if use_class_weight:
            kwargs["auto_class_weights"] = "Balanced"
        if use_gpu:
            kwargs["task_type"] = "GPU"
            kwargs["devices"] = "0"
        return CatBoostClassifier(**kwargs)

    if model_name == "lightgbm":
        if not HAS_LIGHTGBM:
            raise RuntimeError("lightgbm is not available")

        use_gpu = bool(use_gpu_when_available and TORCH_CUDA_AVAILABLE)
        return LGBMClassifier(
            n_estimators=int(max(30, epoch_budget)),
            learning_rate=max(1e-4, float(p.get("learning_rate", 0.1))),
            max_depth=int(round(p.get("max_depth", -1))),
            num_leaves=max(2, int(round(p.get("num_leaves", 31)))),
            min_child_samples=max(1, int(round(p.get("min_child_samples", 20)))),
            subsample=float(np.clip(p.get("subsample", 0.8), 0.1, 1.0)),
            colsample_bytree=float(np.clip(p.get("colsample_bytree", 0.8), 0.1, 1.0)),
            reg_lambda=max(0.0, float(p.get("reg_lambda", 1.0))),
            reg_alpha=max(0.0, float(p.get("reg_alpha", 0.0))),
            scale_pos_weight=pos_weight if use_class_weight else 1.0,
            device_type="gpu" if use_gpu else "cpu",
            random_state=seed,
            verbose=-1,
        )

    if model_name == "naive_bayes":
        return GaussianNB(var_smoothing=max(1e-15, float(p.get("var_smoothing", 1e-9))))

    raise ValueError(f"Unknown model: {model_name}")


def evaluate_params_cv(
    model_name: str,
    params: dict[str, Any],
    sampling: str,
    X_train_imp: pd.DataFrame,
    y_train: np.ndarray,
    epoch_budget: int,
    n_splits: int,
    seed: int,
    num_cols: list[str],
    cat_cols: list[str],
    threshold_grid: np.ndarray,
    use_gpu_when_available: bool,
) -> dict[str, float]:
    use_class_weight = sampling in CW_SAMPLINGS
    y_arr = np.asarray(y_train, dtype=int)
    class_counts = np.bincount(y_arr)

    if len(class_counts) < 2 or int(class_counts.min()) < 2:
        raise ValueError("Need at least 2 samples per class for stratified CV.")
```

```
effective_splits = min(int(n_splits), int(class_counts.min()))
splitter = StratifiedKFold(n_splits=effective_splits, shuffle=True, random_state=seed)

fold_rows: list[dict[str, float]] = []

for fold_idx, (train_idx, val_idx) in enumerate(splitter.split(X_train_imp, y_arr), start=1):
    X_fold_train = X_train_imp.iloc[train_idx]
    X_fold_val = X_train_imp.iloc[val_idx]
    y_fold_train = y_arr[train_idx]
    y_fold_val = y_arr[val_idx]

    fold_scaler, fold_ohe, _ = fit_encoder_scaler(X_fold_train, num_cols, cat_cols)
    X_fold_train_sampled, y_fold_train_sampled = get_sampled_train(
        X_fold_train,
        y_fold_train,
        sampling,
        fold_scaler,
        fold_ohe,
        num_cols,
        cat_cols,
        seed,
        sampling_params=params,
    )
    X_fold_val_proc = encode_scale(fold_scaler, fold_ohe, X_fold_val, num_cols, cat_cols)

    model = build_model_for_search(
        model_name,
        params,
        use_class_weight,
        y_fold_train_sampled,
        epoch_budget,
        seed,
        use_gpu_when_available,
    )

    try:
        model.fit(X_fold_train_sampled, y_fold_train_sampled)
    except Exception as exc:
        msg = str(exc).lower()
        if ("gpu" in msg or "cuda" in msg or "device" in msg) and model_name in {
            "xgboost",
            "catboost",
            "lightgbm",
        }:
            if model_name == "xgboost":
                model.set_params(device="cpu")
            elif model_name == "catboost":
                model.set_params(task_type="CPU")
            elif model_name == "lightgbm":
                model.set_params(device_type="cpu")
            model.fit(X_fold_train_sampled, y_fold_train_sampled)
        else:
            raise

    prob_val = np.clip(model.predict_proba(X_fold_val_proc)[:, 1], 1e-9, 1 - 1e-9)
    prob_val = _apply_nb_probability_floor(model, prob_val)

    best_obj = -np.inf
    best_metrics: dict[str, float] | None = None
    best_thr = 0.5

    for thr in threshold_grid:
        preds = (prob_val >= thr).astype(int)
        acc = accuracy_score(y_fold_val, preds)
        rec = recall_score(y_fold_val, preds, zero_division=0)
        obj = 0.60 * acc + 0.40 * rec
        if obj > best_obj:
            best_obj = obj
            best_metrics = {
                "accuracy": float(acc),
                "recall": float(rec),
                "precision": float(precision_score(y_fold_val, preds, zero_division=0)),
                "f1": float(f1_score(y_fold_val, preds, zero_division=0)),
                "auc": float(roc_auc_score(y_fold_val, prob_val)) if len(np.unique(y_fold_val)) > 1 else 0.5,
                "logloss": float(log_loss(y_fold_val, prob_val)),
            }
            best_thr = float(thr)

    assert best_metrics is not None
    best_metrics["fold"] = float(fold_idx)
    best_metrics["best_threshold"] = best_thr
    fold_rows.append(best_metrics)

summary_df = pd.DataFrame(fold_rows)

summary = {
    "accuracy_mean": float(summary_df["accuracy"].mean()),
    "accuracy_std": float(summary_df["accuracy"].std(ddof=0)),
    "recall_mean": float(summary_df["recall"].mean()),
    "recall_std": float(summary_df["recall"].std(ddof=0)),
    "precision_mean": float(summary_df["precision"].mean()),
    "f1_mean": float(summary_df["f1"].mean()),
    "f1_std": float(summary_df["f1"].std(ddof=0)),
    "auc_mean": float(summary_df["auc"].mean()),
    "logloss_mean": float(summary_df["logloss"].mean()),
    "logloss_std": float(summary_df["logloss"].std(ddof=0)),
    "threshold_mean": float(summary_df["best_threshold"].mean()),
}

summary["stage_score"] = (
    0.60 * summary["accuracy_mean"]
    + 0.40 * summary["recall_mean"]
    + 0.05 * summary["f1_mean"]
    - 0.08 * summary["logloss_mean"]
    - 0.03 * summary["accuracy_std"]
    - 0.03 * summary["recall_std"]
```

```
    )

    return summary


def run_eda(
    X: pd.DataFrame,
    y: pd.Series,
    target_col: str,
    X_train_raw: pd.DataFrame,
    X_train_imp_before_filter: pd.DataFrame,
    num_cols_after_filter: list[str],
    output_dir: Path,
    collinearity_cutoff: float,
) -> None:
    eda_dir = output_dir / "eda_plots"
    eda_dir.mkdir(parents=True, exist_ok=True)

    eda_df = X.copy()
    eda_df["__target__"] = y.values
    num_cols_raw = eda_df.drop(columns=["__target__"]).select_dtypes(include=np.number).columns.tolist()
    cat_cols_raw = [c for c in eda_df.columns if c not in num_cols_raw and c != "__target__"]

    overview_rows = []
    for col in eda_df.columns:
        if col == "__target__":
            continue
        s = eda_df[col]
        overview_rows.append(
            {
                "feature": col,
                "dtype": str(s.dtype),
                "missing_n": int(s.isna().sum()),
                "missing_pct": float(s.isna().mean() * 100.0),
                "n_unique": int(s.nunique(dropna=False)),
            }
        )

    overview_df = pd.DataFrame(overview_rows).sort_values("missing_pct", ascending=False)
    overview_df.to_csv(output_dir / "eda_feature_overview.csv", index=False)

    class_counts = y.value_counts().sort_index()
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    labels = ["No Hypertension (0)", "Hypertension (1)"]
    bars = axes[0].bar(labels, class_counts.values, color=["#4878CF", "#D65F5F"], edgecolor="white")
    for bar, count in zip(bars, class_counts.values):
        axes[0].text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 15,
            f"{count:,}\n({count / len(y) * 100:.1f}%)",
            ha="center",
            va="bottom",
            fontsize=9,
            fontweight="bold",
        )
    axes[0].set_title("Class Distribution", fontweight="bold")
    axes[0].set_ylabel("Count")

    axes[1].pie(
        class_counts.values,
        labels=labels,
        colors=["#4878CF", "#D65F5F"],
        autopct="%1.1f%%",
        startangle=140,
        wedgeprops={"edgecolor": "white", "linewidth": 1.5},
    )
    axes[1].set_title("Class Proportion", fontweight="bold")

    fig.suptitle(f"Target Variable: {target_col}", fontsize=12, fontweight="bold")
    plt.tight_layout()
    plt.savefig(eda_dir / "fig01_target_distribution.png", bbox_inches="tight", dpi=200)
    plt.close(fig)

    missing_df = overview_df[overview_df["missing_n"] > 0]
    fig, axes = plt.subplots(1, 2, figsize=(14, max(4, min(12, len(missing_df) * 0.35 + 2))))
    if len(missing_df) > 0:
        axes[0].barh(missing_df["feature"], missing_df["missing_pct"], color="#E07B54", edgecolor="white")
        axes[0].axvline(5, color="#555", linestyle="--", linewidth=0.8, alpha=0.7)
        axes[0].axvline(20, color="#C22", linestyle="--", linewidth=0.8, alpha=0.7)
        axes[0].set_title("Missing Data per Feature", fontweight="bold")
        axes[0].set_xlabel("Missing (%)")
        axes[0].invert_yaxis()

        miss_matrix = eda_df.drop(columns=["__target__"]).isnull()
        show_cols = missing_df["feature"].tolist()[:30]
        sampled = miss_matrix[show_cols].astype(int)
        sample_n = min(500, len(sampled))
        sampled = sampled.sample(sample_n, random_state=42)
        sns.heatmap(
            sampled.T,
            cmap=["#EEF2FF", "#D65F5F"],
            cbar=False,
            xticklabels=False,
            yticklabels=True,
            linewidths=0,
            ax=axes[1],
        )
        axes[1].set_title(f"Missingness Pattern (sample={sample_n})", fontweight="bold")
        axes[1].set_xlabel("Observations")
    else:
        axes[0].text(0.5, 0.5, "No missing values", ha="center", va="center", transform=axes[0].transAxes)
        axes[0].set_axis_off()
        axes[1].text(0.5, 0.5, "No missingness map", ha="center", va="center", transform=axes[1].transAxes)
        axes[1].set_axis_off()
```

```
plt.tight_layout()
plt.savefig(eda_dir / "fig02_missing_data.png", bbox_inches="tight", dpi=200)
plt.close(fig)

all_num_before = X_train_raw.select_dtypes(include=np.number).columns.tolist()
knn_vis = KNNImputer(n_neighbors=5)
X_before_imp = pd.DataFrame(
    knn_vis.fit_transform(X_train_raw[all_num_before]),
    columns=all_num_before,
    index=X_train_raw.index,
)
corr_before = X_before_imp.corr()

fig, ax = plt.subplots(figsize=(max(8, len(corr_before) * 0.35 + 1), max(8, len(corr_before) * 0.35)))
mask = np.triu(np.ones_like(corr_before, dtype=bool), k=0)
sns.heatmap(
    corr_before,
    mask=mask,
    annot=(len(corr_before) <= 20),
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    vmin=-1,
    vmax=1,
    linewidths=0.3,
    linecolor="#eee",
    cbar_kws={"shrink": 0.6, "label": "Pearson r"},
    ax=ax,
    square=True,
)
ax.set_title("Correlation Matrix Before Culling", fontweight="bold")
ax.tick_params(axis="x", rotation=45, labelsize=7)
ax.tick_params(axis="y", labelsize=7)
plt.tight_layout()
plt.savefig(eda_dir / "fig03a_corr_before_culling.png", bbox_inches="tight", dpi=200)
plt.close(fig)

corr_after = X_train_imp_before_filter[num_cols_after_filter].corr()
fig, ax = plt.subplots(figsize=(max(6, len(corr_after) * 0.4 + 1), max(6, len(corr_after) * 0.4)))
mask_after = np.triu(np.ones_like(corr_after, dtype=bool), k=0)
sns.heatmap(
    corr_after,
    mask=mask_after,
    annot=(len(corr_after) <= 25),
    fmt=".2f",
    cmap="RdBu_r",
    center=0,
    vmin=-1,
    vmax=1,
    linewidths=0.3,
    linecolor="#eee",
    cbar_kws={"shrink": 0.6, "label": "Pearson r"},
    ax=ax,
    square=True,
)
ax.set_title(
    f"Correlation Matrix After Culling (cutoff={collinearity_cutoff})",
    fontweight="bold",
)
ax.tick_params(axis="x", rotation=45, labelsize=7)
ax.tick_params(axis="y", labelsize=7)
plt.tight_layout()
plt.savefig(eda_dir / "fig03b_corr_after_culling.png", bbox_inches="tight", dpi=200)
plt.close(fig)

assoc_rows: list[dict[str, Any]] = []
y_vals = eda_df["__target__"].values

for col in num_cols_raw:
    vals = pd.to_numeric(eda_df[col], errors="coerce").values
    valid = ~(np.isnan(vals) | np.isnan(y_vals.astype(float)))
    if valid.sum() < 30:
        continue
    corr, p_val = pointbiserialr(vals[valid], y_vals[valid])
    assoc_rows.append(
        {
            "feature": col,
            "type": "numeric",
            "metric": "point_biserial_r",
            "statistic": float(corr),
            "abs_stat": float(abs(corr)),
            "p_value": float(p_val),
        }
    )

def cramers_v(x: pd.Series, y_local: pd.Series) -> float:
    table = pd.crosstab(x, y_local)
    if table.empty:
        return 0.0
    chi2, _, _, _ = chi2_contingency(table)
    n = table.values.sum()
    k = min(table.shape) - 1
    if n <= 0 or k <= 0:
        return 0.0
    return float(np.sqrt(chi2 / n) / np.sqrt(k))

for col in cat_cols_raw:
    vals = eda_df[col].dropna()
    if len(vals) < 30:
        continue
    aligned_target = eda_df.loc[vals.index, "__target__"]
    v = cramers_v(vals, aligned_target)
    assoc_rows.append(
        {
            "feature": col,
            "type": "categorical",
```

```
                "metric": "cramers_v",
                "statistic": float(v),
                "abs_stat": float(abs(v)),
                "p_value": np.nan,
            }
        )

    assoc_df = pd.DataFrame(assoc_rows).sort_values("abs_stat", ascending=False).reset_index(drop=True)
    assoc_df.to_csv(output_dir / "eda_feature_association.csv", index=False)

    top_n = min(40, len(assoc_df))
    if top_n > 0:
        plot_df = assoc_df.head(top_n).iloc[::-1]
        colors = ["#D65F5F" if t == "numeric" else "#8DA0CB" for t in plot_df["type"]]

        fig, ax = plt.subplots(figsize=(8, top_n * 0.35 + 1.5))
        ax.barh(plot_df["feature"], plot_df["abs_stat"], color=colors, edgecolor="white")
        ax.axvline(0.1, color="#999", linestyle="--", linewidth=0.8)
        ax.axvline(0.3, color="#555", linestyle="--", linewidth=0.8)
        ax.set_xlabel("Association Strength")
        ax.set_title("Feature-Target Association Ranking", fontweight="bold")
        ax.tick_params(axis="y", labelsize=8)
        plt.tight_layout()
        plt.savefig(eda_dir / "fig05_feature_association_ranking.png", bbox_inches="tight", dpi=200)
        plt.close(fig)


def run_shap(
    best_model: Any,
    best_model_name: str,
    best_sampling: str,
    X_train_proc: np.ndarray,
    X_test_proc: np.ndarray,
    y_test: np.ndarray,
    feature_names: list[str],
    output_dir: Path,
    shap_test_n: int,
    shap_bg_n: int,
) -> int:
    if not HAS_SHAP:
        print("SHAP is unavailable. Skipping SHAP stage.")
        return 0

    plots_dir = output_dir / "plots"
    plots_dir.mkdir(parents=True, exist_ok=True)

    test_n = min(shap_test_n, len(X_test_proc))
    bg_n = min(shap_bg_n, len(X_train_proc))

    if test_n < 1 or bg_n < 1:
        print("SHAP skipped: empty background or test subset after parameter limits.")
        return 0

    X_test_shap = X_test_proc[:test_n]
    X_bg = X_train_proc[:bg_n]

    tree_types = {"catboost", "xgboost", "randomforest", "adaboost", "lightgbm"}

    if best_model_name in tree_types:
        explainer = shap.TreeExplainer(best_model)
    elif best_model_name == "logreg":
        explainer = shap.LinearExplainer(best_model, X_bg)
    else:
        kernel_bg = X_bg[: min(60, len(X_bg))]
        explainer = shap.KernelExplainer(lambda x: best_model.predict_proba(x)[:, 1], kernel_bg)

    shap_values_raw = explainer.shap_values(X_test_shap)

    if isinstance(shap_values_raw, list):
        shap_values = np.array(shap_values_raw[1])
        if hasattr(explainer.expected_value, "__len__"):
            expected_value = float(explainer.expected_value[1])
        else:
            expected_value = float(explainer.expected_value)
    else:
        shap_values = np.array(shap_values_raw)
        if shap_values.ndim == 3:
            shap_values = shap_values[:, :, 1]
        if hasattr(explainer.expected_value, "__len__"):
            expected_value = float(explainer.expected_value[0])
        else:
            expected_value = float(explainer.expected_value)

    if shap_values.ndim != 2:
        shap_values = np.atleast_2d(shap_values)

    plt.figure(figsize=(10, 7))
    shap.summary_plot(
        shap_values,
        X_test_shap,
        feature_names=np.array(feature_names),
        plot_type="bar",
        show=False,
        max_display=20,
    )
    plt.title(
        f"SHAP Global - Feature Importance\n{best_model_name} | {best_sampling}",
        fontweight="bold",
        fontsize=12,
    )
    plt.tight_layout()
    plt.savefig(plots_dir / "shap_global_bar.png", dpi=150, bbox_inches="tight")
    plt.close()

    plt.figure(figsize=(10, 7))
    shap.summary_plot(
```

```
        shap_values,
        X_test_shap,
        feature_names=np.array(feature_names),
        show=False,
        max_display=20,
    )
    plt.title(
        f"SHAP Global - Beeswarm\n{best_model_name} | {best_sampling}",
        fontweight="bold",
        fontsize=12,
    )
    plt.tight_layout()
    plt.savefig(plots_dir / "shap_global_beeswarm.png", dpi=150, bbox_inches="tight")
    plt.close()

    positive_indices = np.where(y_test[:test_n] == 1)[0]
    local_idx = int(positive_indices[0]) if len(positive_indices) else 0

    local_explanation = shap.Explanation(
        values=shap_values[local_idx],
        base_values=expected_value,
        data=X_test_shap[local_idx],
        feature_names=list(feature_names),
    )

    plt.figure(figsize=(10, 6))
    shap.plots.waterfall(local_explanation, show=False, max_display=15)
    plt.title(
        f"SHAP Local - Waterfall (sample #{local_idx})\n{best_model_name} | {best_sampling}",
        fontweight="bold",
        fontsize=11,
    )
    plt.tight_layout()
    plt.savefig(plots_dir / "shap_local_waterfall.png", dpi=150, bbox_inches="tight")
    plt.close()

    print(f"SHAP plots saved to: {plots_dir}")
    return local_idx


def run_lime(
    best_model: Any,
    X_train_proc: np.ndarray,
    X_test_proc: np.ndarray,
    feature_names: list[str],
    local_idx: int,
    output_dir: Path,
    lime_features: int,
    seed: int,
) -> None:
    if not HAS_LIME:
        print("LIME is unavailable. Skipping LIME stage.")
        return

    plots_dir = output_dir / "plots"
    plots_dir.mkdir(parents=True, exist_ok=True)

    if len(X_test_proc) < 1:
        print("LIME skipped: empty test set.")
        return

    explainer = lime.lime_tabular.LimeTabularExplainer(
        training_data=X_train_proc,
        feature_names=feature_names,
        class_names=["No HTN", "HTN"],
        mode="classification",
        random_state=seed,
        discretize_continuous=True,
    )

    lime_exp = explainer.explain_instance(
        data_row=X_test_proc[local_idx],
        predict_fn=best_model.predict_proba,
        num_features=min(lime_features, len(feature_names)),
        labels=(1,),
    )

    fig = lime_exp.as_pyplot_figure(label=1)
    fig.set_size_inches(10, 6)
    plt.title(
        f"LIME Local Explanation (sample #{local_idx})",
        fontweight="bold",
        fontsize=11,
    )
    plt.tight_layout()
    plt.savefig(plots_dir / "lime_local.png", dpi=150, bbox_inches="tight")
    plt.close(fig)

    lime_pairs = lime_exp.as_list(label=1)
    lime_table = pd.DataFrame(lime_pairs, columns=["Feature Condition", "LIME Weight"])
    lime_table["abs_weight"] = lime_table["LIME Weight"].abs()
    lime_table = lime_table.sort_values("abs_weight", ascending=False).reset_index(drop=True)
    lime_table.to_csv(output_dir / "lime_local_weights.csv", index=False)

    print(f"LIME outputs saved to: {plots_dir}")


def _effective_stage_value(model_name: str, key: str, cli_override: int | None) -> int:
    profile = MODEL_STAGE_DEFAULTS.get(model_name)
    if profile is None:
        raise KeyError(f"Missing stage defaults for model: {model_name}")
    return int(profile[key] if cli_override is None else cli_override)


def run_pipeline(args: argparse.Namespace) -> None:
    random.seed(args.seed)
```

```
np.random.seed(args.seed)

training_root = resolve_training_root(args.root)
output_dir = training_root / args.out_dir
models_dir = output_dir / "models"
plots_dir = output_dir / "plots"
output_dir.mkdir(parents=True, exist_ok=True)
models_dir.mkdir(parents=True, exist_ok=True)
plots_dir.mkdir(parents=True, exist_ok=True)

print("-" * 90)
print("EXP3 Unified Pipeline")
print("-" * 90)
print(f"Training root: {training_root}")
print(f"Output dir   : {output_dir}")
print(f"CUDA available (torch): {TORCH_CUDA_AVAILABLE}")

if not args.skip_merge:
    merge_datasets(training_root)

data_path = training_root / "merged_clinical_leftjoin.csv"
if not data_path.exists():
    raise FileNotFoundError(f"Required dataset not found: {data_path}")

X, y, base_meta = load_and_prepare_features(data_path)

X_train_raw, X_tmp, y_train, y_tmp = train_test_split(
    X,
    y,
    test_size=0.40,
    random_state=args.seed,
    stratify=y,
)
X_cal_raw, X_test_raw, y_cal, y_test = train_test_split(
    X_tmp,
    y_tmp,
    test_size=0.50,
    random_state=args.seed,
    stratify=y_tmp,
)

y_train_arr = np.asarray(y_train, dtype=int)
y_cal_arr = np.asarray(y_cal, dtype=int)
y_test_arr = np.asarray(y_test, dtype=int)

num_cols = X_train_raw.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = [c for c in X_train_raw.columns if c not in num_cols]

print("Split summary")
print(
    pd.DataFrame(
        {
            "Split": ["Train", "Cal", "Test"],
            "N": [len(y_train_arr), len(y_cal_arr), len(y_test_arr)],
            "Pos %": [
                f"{y_train_arr.mean() * 100:.1f}",
                f"{y_cal_arr.mean() * 100:.1f}",
                f"{y_test_arr.mean() * 100:.1f}",
            ],
        }
    ).to_string(index=False)
)

knn_imputer, cat_imputer = fit_imputers(X_train_raw, num_cols, cat_cols)

X_train_imp = impute_frame(knn_imputer, cat_imputer, X_train_raw, num_cols, cat_cols)
X_cal_imp = impute_frame(knn_imputer, cat_imputer, X_cal_raw, num_cols, cat_cols)
X_test_imp = impute_frame(knn_imputer, cat_imputer, X_test_raw, num_cols, cat_cols)

protected_numeric = sorted(
    {
        c
        for c in num_cols
        if any(alias in c.lower() for alias in ["age", "sex", "bmi", "whr"])
    }
)

corr_matrix = X_train_imp[num_cols].corr().abs() if num_cols else pd.DataFrame()
if not corr_matrix.empty:
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    drop_columns = [
        c
        for c in upper.columns
        if c not in protected_numeric and (upper[c] > args.collinearity_cutoff).any()
    ]
else:
    drop_columns = []

num_cols_reduced = [c for c in num_cols if c not in drop_columns]
keep_columns = num_cols_reduced + cat_cols

X_train_imp = X_train_imp[keep_columns]
X_cal_imp = X_cal_imp[keep_columns]
X_test_imp = X_test_imp[keep_columns]

scaler, ohe, feature_names = fit_encoder_scaler(X_train_imp, num_cols_reduced, cat_cols)
X_cal_proc = encode_scale(scaler, ohe, X_cal_imp, num_cols_reduced, cat_cols)
X_test_proc = encode_scale(scaler, ohe, X_test_imp, num_cols_reduced, cat_cols)

print(f"Numeric features before cull: {len(num_cols)}")
print(f"Numeric features after cull : {len(num_cols_reduced)}")
print(f"Dropped by collinearity     : {len(drop_columns)}")
if drop_columns:
    print(f"Dropped columns             : {drop_columns}")
print(f"Model feature count         : {len(feature_names)}")
```

```
preprocess_artifacts = PreprocessArtifacts(
    X_train_imp=X_train_imp,
    X_cal_imp=X_cal_imp,
    X_test_imp=X_test_imp,
    X_train_proc=encode_scale(scaler, ohe, X_train_imp, num_cols_reduced, cat_cols),
    X_cal_proc=X_cal_proc,
    X_test_proc=X_test_proc,
    y_train=y_train_arr,
    y_cal=y_cal_arr,
    y_test=y_test_arr,
    knn_imputer=knn_imputer,
    cat_imputer=cat_imputer,
    scaler=scaler,
    ohe=ohe,
    num_cols_full=num_cols,
    num_cols_reduced=num_cols_reduced,
    cat_cols=cat_cols,
    feature_names=feature_names,
)

if not args.skip_eda:
    run_eda(
        X=X,
        y=y,
        target_col=str(base_meta["target_col"]),
        X_train_raw=X_train_raw,
        X_train_imp_before_filter=impute_frame(knn_imputer, cat_imputer, X_train_raw, num_cols, cat_cols),
        num_cols_after_filter=num_cols_reduced,
        output_dir=output_dir,
        collinearity_cutoff=args.collinearity_cutoff,
    )

available_models = build_available_models(args.models)
model_spaces = build_model_spaces(available_models)
threshold_grid = np.round(
    np.arange(args.threshold_min, args.threshold_max, args.threshold_step),
    2,
)
if len(threshold_grid) < 1:
    raise ValueError("Threshold grid is empty. Check --threshold-min/--threshold-max/--threshold-step.")

print("Models to train:", available_models)
print("Sampling methods:", args.sampling_methods)

stage1_results: dict[tuple[str, str], pd.DataFrame] = {}
stage2_results: dict[tuple[str, str], pd.DataFrame] = {}
best_config_per_key: dict[tuple[str, str], list[dict[str, Any]]] = {}

trained_models: dict[tuple[str, str], Any] = {}
train_data_by_key: dict[tuple[str, str], tuple[np.ndarray, np.ndarray]] = {}
model_thresholds: dict[tuple[str, str], float] = {}
pre_rows: list[dict[str, Any]] = []

for sampling in args.sampling_methods:
    use_cw = sampling in CW_SAMPLINGS
    print("=" * 80)
    print(f"Sampling: {sampling} | class-weight active: {use_cw}")
    print("=" * 80)

    for model_name in available_models:
        key = (sampling, model_name)
        print(f"  Training {model_name} ...")

        s1_trials = _effective_stage_value(model_name, "s1_trials", args.s1_trials)
        s1_epochs = _effective_stage_value(model_name, "s1_epochs", args.s1_epochs)
        s1_folds = _effective_stage_value(model_name, "s1_folds", args.s1_folds)
        top_k_s1_eff = _effective_stage_value(model_name, "top_k_s1", args.top_k_s1)
        s2_refine = _effective_stage_value(model_name, "s2_refine", args.s2_refine)
        s2_epochs = _effective_stage_value(model_name, "s2_epochs", args.s2_epochs)
        s2_folds = _effective_stage_value(model_name, "s2_folds", args.s2_folds)
        top_k_s2_eff = _effective_stage_value(model_name, "top_k_s2", args.top_k_s2)
        final_epochs = _effective_stage_value(model_name, "final_epochs", args.final_epochs)

        stage1_trials = list(
            ParameterSampler(
                model_spaces[model_name],
                n_iter=s1_trials,
                random_state=args.seed,
            )
        )

        stage1_rows: list[dict[str, Any]] = []
        for trial_idx, params in enumerate(stage1_trials, start=1):
            try:
                cv_summary = evaluate_params_cv(
                    model_name=model_name,
                    params=params,
                    sampling=sampling,
                    X_train_imp=preprocess_artifacts.X_train_imp,
                    y_train=preprocess_artifacts.y_train,
                    epoch_budget=s1_epochs,
                    n_splits=s1_folds,
                    seed=args.seed,
                    num_cols=preprocess_artifacts.num_cols_reduced,
                    cat_cols=preprocess_artifacts.cat_cols,
                    threshold_grid=threshold_grid,
                    use_gpu_when_available=args.use_gpu_when_available,
                )
                stage1_rows.append({"trial": trial_idx, "params": params, **cv_summary})
            except Exception as exc:
                stage1_rows.append(
                    {
                        "trial": trial_idx,
                        "params": params,
                        "stage_score": -999.0,
                        "error": str(exc),
```

```
                }
            )

        stage1_df = pd.DataFrame(stage1_rows).sort_values("stage_score", ascending=False).reset_index(drop=True)
        stage1_results[key] = stage1_df

        if stage1_df.empty:
            print("    Stage 1 failed: no candidates")
            continue

        stage1_valid = stage1_df if "error" not in stage1_df.columns else stage1_df[stage1_df["error"].isna()]
        if stage1_valid.empty:
            print("    Stage 1 failed: all trials errored")
            continue

        top_k_s1_eff = max(1, min(top_k_s1_eff, len(stage1_valid)))
        top_stage1_params = stage1_valid.head(top_k_s1_eff)["params"].tolist()
        stage1_best_score = float(stage1_valid.iloc[0].get("stage_score", -999.0))

        stage2_candidates = refine_candidates(
            top_stage1_params,
            n_refine=s2_refine,
            seed=args.seed,
        )

        stage2_rows: list[dict[str, Any]] = []
        for trial_idx, params in enumerate(stage2_candidates, start=1):
            try:
                cv_summary = evaluate_params_cv(
                    model_name=model_name,
                    params=params,
                    sampling=sampling,
                    X_train_imp=preprocess_artifacts.X_train_imp,
                    y_train=preprocess_artifacts.y_train,
                    epoch_budget=s2_epochs,
                    n_splits=s2_folds,
                    seed=args.seed,
                    num_cols=preprocess_artifacts.num_cols_reduced,
                    cat_cols=preprocess_artifacts.cat_cols,
                    threshold_grid=threshold_grid,
                    use_gpu_when_available=args.use_gpu_when_available,
                )
                stage2_rows.append({"trial": trial_idx, "params": params, **cv_summary})
            except Exception as exc:
                stage2_rows.append(
                    {
                        "trial": trial_idx,
                        "params": params,
                        "stage_score": -999.0,
                        "error": str(exc),
                    }
                )

        stage2_df = pd.DataFrame(stage2_rows).sort_values("stage_score", ascending=False).reset_index(drop=True)
        stage2_results[key] = stage2_df

        if stage2_df.empty:
            print("    Stage 2 failed: no candidates")
            continue

        stage2_valid = stage2_df if "error" not in stage2_df.columns else stage2_df[stage2_df["error"].isna()]
        if stage2_valid.empty:
            print("    Stage 2 failed: all trials errored")
            continue

        top_k_s2_eff = max(1, min(top_k_s2_eff, len(stage2_valid)))
        best_params_list = stage2_valid.head(top_k_s2_eff)["params"].tolist()
        best_config_per_key[key] = best_params_list

        stage2_best_score = float(stage2_valid.iloc[0].get("stage_score", -999.0))
        print(f"    Stage1 best={stage1_best_score:.4f} | Stage2 best={stage2_best_score:.4f}")

        best_model = None
        best_score = -np.inf
        best_threshold = 0.5
        best_tr_data: tuple[np.ndarray, np.ndarray] | None = None

        for params in best_params_list:
            X_train_sampled, y_train_sampled = get_sampled_train(
                preprocess_artifacts.X_train_imp,
                preprocess_artifacts.y_train,
                sampling,
                preprocess_artifacts.scaler,
                preprocess_artifacts.ohe,
                preprocess_artifacts.num_cols_reduced,
                preprocess_artifacts.cat_cols,
                args.seed,
                sampling_params=params,
            )
            y_train_sampled = np.asarray(y_train_sampled, dtype=int)

            model = build_model_for_search(
                model_name=model_name,
                params=params,
                use_class_weight=use_cw,
                y_sample=y_train_sampled,
                epoch_budget=final_epochs,
                seed=args.seed,
                use_gpu_when_available=args.use_gpu_when_available,
            )

            try:
                _fit_model_with_nb_weights(
                    model,
                    model_name,
                    use_cw,
```

```
                    X_train_sampled,
                    y_train_sampled,
                )
            except Exception as exc:
                msg = str(exc).lower()
                if ("gpu" in msg or "cuda" in msg or "device" in msg) and model_name in {
                    "xgboost",
                    "catboost",
                    "lightgbm",
                }:
                    if model_name == "xgboost":
                        model.set_params(device="cpu")
                    elif model_name == "catboost":
                        model.set_params(task_type="CPU")
                    elif model_name == "lightgbm":
                        model.set_params(device_type="cpu")
                    _fit_model_with_nb_weights(
                        model,
                        model_name,
                        use_cw,
                        X_train_sampled,
                        y_train_sampled,
                    )
                else:
                    continue

            p_cal = np.clip(model.predict_proba(preprocess_artifacts.X_cal_proc)[:, 1], 1e-9, 1 - 1e-9)
            p_cal = _apply_nb_probability_floor(model, p_cal)

            local_best_score = -np.inf
            local_best_threshold = 0.5
            for threshold in threshold_grid:
                preds = (p_cal >= threshold).astype(int)
                objective = 0.60 * accuracy_score(preprocess_artifacts.y_cal, preds) + 0.40 * recall_score(
                    preprocess_artifacts.y_cal,
                    preds,
                    zero_division=0,
                )
                if objective > local_best_score:
                    local_best_score = objective
                    local_best_threshold = float(threshold)

            if local_best_score > best_score:
                best_score = local_best_score
                best_model = model
                best_threshold = local_best_threshold
                best_tr_data = (X_train_sampled, y_train_sampled)

        if best_model is None:
            print("    Final training failed: no valid model")
            continue

        p_test = np.clip(best_model.predict_proba(preprocess_artifacts.X_test_proc)[:, 1], 1e-9, 1 - 1e-9)
        p_test = _apply_nb_probability_floor(best_model, p_test)
        metrics = metric_pack(preprocess_artifacts.y_test, p_test, best_threshold)

        trained_models[key] = best_model
        train_data_by_key[key] = best_tr_data if best_tr_data is not None else (preprocess_artifacts.X_train_proc,
              preprocess_artifacts.y_train)
        model_thresholds[key] = best_threshold

        pre_rows.append(
            {
                "experiment": MODEL_EXPERIMENT.get(model_name, "U"),
                "sampling": sampling,
                "model": model_name,
                "threshold": best_threshold,
                **{k: round(v, 4) for k, v in metrics.items()},
            }
        )

        print(
            "    Test pre-cal metrics: "
            f"acc={metrics['accuracy']:.3f} rec={metrics['recall']:.3f} auc={metrics['auc']:.3f}"
        )

stage1_summary_rows: list[dict[str, Any]] = []
for (sampling, model_name), frame in stage1_results.items():
    if frame.empty:
        continue
    top = frame.iloc[0]
    stage1_summary_rows.append(
        {
            "sampling": sampling,
            "model": model_name,
            "best_stage1_score": float(top.get("stage_score", np.nan)),
            "best_acc_cv": float(top.get("accuracy_mean", np.nan)),
            "best_rec_cv": float(top.get("recall_mean", np.nan)),
        }
    )

stage2_summary_rows: list[dict[str, Any]] = []
for (sampling, model_name), frame in stage2_results.items():
    if frame.empty:
        continue
    top = frame.iloc[0]
    stage2_summary_rows.append(
        {
            "sampling": sampling,
            "model": model_name,
            "best_stage2_score": float(top.get("stage_score", np.nan)),
            "best_acc_cv": float(top.get("accuracy_mean", np.nan)),
            "best_rec_cv": float(top.get("recall_mean", np.nan)),
            "best_thr_cv": float(top.get("threshold_mean", np.nan)),
        }
    )
```

```
pd.DataFrame(stage1_summary_rows).to_csv(output_dir / "stage1_summary.csv", index=False)
pd.DataFrame(stage2_summary_rows).to_csv(output_dir / "stage2_summary.csv", index=False)

if not pre_rows:
    raise RuntimeError("No models were trained successfully.")

pre_df = pd.DataFrame(pre_rows)
pre_df["combined"] = (pre_df["accuracy"] + pre_df["recall"]) / 2.0
pre_df = pre_df.sort_values(["combined", "auc"], ascending=False).reset_index(drop=True)
pre_df.index += 1
pre_df.to_csv(output_dir / "pre_calibration_results.csv", index=True, index_label="rank")

print("Saved pre_calibration_results.csv")

calibrator_store: dict[tuple[str, str, str], Any] = {}
post_rows: list[dict[str, Any]] = []

effective_cal_methods = CAL_METHODS.copy()
if not HAS_VENN_ABERS:
    effective_cal_methods = [m for m in effective_cal_methods if m != "venn_abers"]

for (sampling, model_name), model in trained_models.items():
    p_cal = np.clip(model.predict_proba(preprocess_artifacts.X_cal_proc)[:, 1], 1e-9, 1 - 1e-9)
    p_test = np.clip(model.predict_proba(preprocess_artifacts.X_test_proc)[:, 1], 1e-9, 1 - 1e-9)
    p_cal = _apply_nb_probability_floor(model, p_cal)
    p_test = _apply_nb_probability_floor(model, p_test)

    for cal_method in effective_cal_methods:
        try:
            calibrator = fit_calibrator(cal_method, p_cal, preprocess_artifacts.y_cal)
            p_test_cal = apply_calibrator(cal_method, calibrator, p_test)
            metrics = metric_pack(preprocess_artifacts.y_test, p_test_cal, threshold=0.5)

            row = {
                "experiment": MODEL_EXPERIMENT.get(model_name, "U"),
                "sampling": sampling,
                "model": model_name,
                "calibration": cal_method,
                **{k: round(v, 4) for k, v in metrics.items()},
            }
            post_rows.append(row)
            calibrator_store[(sampling, model_name, cal_method)] = calibrator
        except Exception as exc:
            post_rows.append(
                {
                    "experiment": MODEL_EXPERIMENT.get(model_name, "U"),
                    "sampling": sampling,
                    "model": model_name,
                    "calibration": cal_method,
                    "error": str(exc),
                }
            )

post_df = pd.DataFrame([row for row in post_rows if "error" not in row])
if post_df.empty:
    raise RuntimeError("No calibrated results were produced.")

post_df["combined"] = (
    post_df["accuracy"]
    + post_df["recall"]
    + (1.0 - post_df["ece"])
    + (1.0 - post_df["logloss"])
) / 4.0
post_df = post_df.sort_values(["combined", "auc"], ascending=False).reset_index(drop=True)
post_df.index += 1
post_df.to_csv(output_dir / "post_calibration_results.csv", index=True, index_label="rank")

print("Saved post_calibration_results.csv")

best_row = post_df.iloc[0]
best_sampling = str(best_row["sampling"])
best_model_name = str(best_row["model"])
best_cal_method = str(best_row["calibration"])
best_key = (best_sampling, best_model_name)

best_model = trained_models[best_key]
best_calibrator = calibrator_store.get((best_sampling, best_model_name, best_cal_method))
best_threshold = float(model_thresholds.get(best_key, 0.5))

pre_entry = pre_df[(pre_df["sampling"] == best_sampling) & (pre_df["model"] == best_model_name)]
if not pre_entry.empty:
    best_threshold = float(pre_entry.iloc[0]["threshold"])

bundle = {
    "model": best_model,
    "knn_imputer": preprocess_artifacts.knn_imputer,
    "cat_imputer": preprocess_artifacts.cat_imputer,
    "scaler": preprocess_artifacts.scaler,
    "ohe": preprocess_artifacts.ohe,
    "num_cols_full": list(preprocess_artifacts.num_cols_full),
    "num_cols_reduced": list(preprocess_artifacts.num_cols_reduced),
    "cat_cols": list(preprocess_artifacts.cat_cols),
    "feat_names": list(preprocess_artifacts.feature_names),
    "input_feature_names": list(X.columns),
    "threshold": best_threshold,
    "sampling": best_sampling,
    "model_name": best_model_name,
    "calibration": best_cal_method,
    "metrics": {
        "accuracy": float(best_row["accuracy"]),
        "recall": float(best_row["recall"]),
        "precision": float(best_row["precision"]),
        "f1": float(best_row["f1"]),
        "auc": float(best_row["auc"]),
        "logloss": float(best_row["logloss"]),
```

```
            "ece": float(best_row["ece"]),
        },
    }

    bundle_path = models_dir / "best_model_bundle.joblib"
    joblib.dump(bundle, bundle_path, compress=3)

    if best_calibrator is not None:
        calibrator_path = models_dir / "best_calibrator.joblib"
        joblib.dump(best_calibrator, calibrator_path, compress=3)

    with open(output_dir / "best_model_summary.json", "w", encoding="utf-8") as f:
        json.dump(
            {
                "experiment": MODEL_EXPERIMENT.get(best_model_name, "U"),
                "sampling": best_sampling,
                "model": best_model_name,
                "calibration": best_cal_method,
                "threshold": best_threshold,
                "metrics": bundle["metrics"],
                "bundle_path": str(bundle_path),
            },
            f,
            indent=2,
        )

    print("Best model selected")
    print(f"  Model       : {best_model_name}")
    print(f"  Sampling    : {best_sampling}")
    print(f"  Calibration : {best_cal_method}")
    print(f"  Bundle      : {bundle_path}")

    local_idx = 0
    if not args.skip_explainability:
        try:
            local_idx = run_shap(
                best_model=best_model,
                best_model_name=best_model_name,
                best_sampling=best_sampling,
                X_train_proc=train_data_by_key[best_key][0],
                X_test_proc=preprocess_artifacts.X_test_proc,
                y_test=preprocess_artifacts.y_test,
                feature_names=preprocess_artifacts.feature_names,
                output_dir=output_dir,
                shap_test_n=args.shap_test_n,
                shap_bg_n=args.shap_bg_n,
            )
        except Exception as exc:
            print(f"SHAP stage failed: {exc}")

        try:
            run_lime(
                best_model=best_model,
                X_train_proc=train_data_by_key[best_key][0],
                X_test_proc=preprocess_artifacts.X_test_proc,
                feature_names=preprocess_artifacts.feature_names,
                local_idx=local_idx,
                output_dir=output_dir,
                lime_features=args.lime_features,
                seed=args.seed,
            )
        except Exception as exc:
            print(f"LIME stage failed: {exc}")

    print("-" * 90)
    print("Pipeline complete")
    print(f"Pre-cal results : {output_dir / 'pre_calibration_results.csv'}")
    print(f"Post-cal results: {output_dir / 'post_calibration_results.csv'}")
    print(f"Best bundle     : {bundle_path}")
    print("-" * 90)


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Unified EXP3 pipeline: merge, preprocess, EDA, train, calibrate, select best, SHAP/LIME."
    )
    parser.add_argument("--root", type=str, default=None, help="Training root containing Datasets2015.")
    parser.add_argument("--out-dir", type=str, default="exp3_unified", help="Output subdirectory name under training root.")
    parser.add_argument("--seed", type=int, default=42)

    parser.add_argument("--skip-merge", action="store_true", help="Skip dataset merge stage.")
    parser.add_argument("--skip-eda", action="store_true", help="Skip EDA stage.")
    parser.add_argument("--skip-explainability", action="store_true", help="Skip SHAP and LIME stages.")

    parser.add_argument(
        "--sampling-methods",
        nargs="*",
        default=SAMPLING_METHODS,
        choices=SAMPLING_METHODS,
        help="Sampling methods to run.",
    )
    parser.add_argument(
        "--models",
        nargs="*",
        default=None,
        choices=["knn", "randomforest", "xgboost", "adaboost", "logreg", "catboost", "lightgbm", "naive_bayes"],
        help="Subset of models to run.",
    )

    parser.add_argument("--s1-trials", type=int, default=None, help="Override Stage-1 trials for all models.")
    parser.add_argument("--s1-folds", type=int, default=None, help="Override Stage-1 folds for all models.")
    parser.add_argument("--s1-epochs", type=int, default=None, help="Override Stage-1 epochs for all models.")

    parser.add_argument("--s2-refine", type=int, default=None, help="Override Stage-2 refinement count for all models.")
    parser.add_argument("--s2-folds", type=int, default=None, help="Override Stage-2 folds for all models.")
    parser.add_argument("--s2-epochs", type=int, default=None, help="Override Stage-2 epochs for all models.")
```

```
    parser.add_argument("--top-k-s1", type=int, default=None, help="Override Stage-1 top-k carryover for all models.")
    parser.add_argument("--top-k-s2", type=int, default=None, help="Override Stage-2 top-k finalists for all models.")
    parser.add_argument("--final-epochs", type=int, default=None, help="Override final-fit epochs for all models.")

    parser.add_argument("--collinearity-cutoff", type=float, default=0.70)

    parser.add_argument("--threshold-min", type=float, default=0.35)
    parser.add_argument("--threshold-max", type=float, default=0.70)
    parser.add_argument("--threshold-step", type=float, default=0.05)

    parser.add_argument("--shap-test-n", type=int, default=300)
    parser.add_argument("--shap-bg-n", type=int, default=100)
    parser.add_argument("--lime-features", type=int, default=15)

    gpu_group = parser.add_mutually_exclusive_group()
    gpu_group.add_argument(
        "--use-gpu-when-available",
        dest="use_gpu_when_available",
        action="store_true",
        help="Enable GPU usage when CUDA is available.",
    )
    gpu_group.add_argument(
        "--no-gpu",
        dest="use_gpu_when_available",
        action="store_false",
        help="Force CPU mode even if CUDA is available.",
    )
    parser.set_defaults(use_gpu_when_available=True)

    return parser


def validate_args(args: argparse.Namespace) -> None:
    required_positive_fields = ["shap_test_n", "shap_bg_n", "lime_features"]
    for field in required_positive_fields:
        value = int(getattr(args, field))
        if value < 1:
            raise ValueError(f"--{field.replace('_', '-')} must be >= 1.")

    optional_positive_fields = [
        "s1_trials",
        "s1_folds",
        "s1_epochs",
        "s2_folds",
        "s2_epochs",
        "top_k_s1",
        "top_k_s2",
        "final_epochs",
    ]
    for field in optional_positive_fields:
        value = getattr(args, field)
        if value is not None and int(value) < 1:
            raise ValueError(f"--{field.replace('_', '-')} must be >= 1 when provided.")

    if args.s2_refine is not None and int(args.s2_refine) < 0:
        raise ValueError("--s2-refine must be >= 0 when provided.")

    if int(args.seed) < 0:
        raise ValueError("--seed must be >= 0.")

    if args.s1_folds is not None and args.s1_folds < 2:
        raise ValueError("--s1-folds must be >= 2 when provided.")
    if args.s2_folds is not None and args.s2_folds < 2:
        raise ValueError("--s2-folds must be >= 2 when provided.")

    if not (0.0 <= float(args.threshold_min) <= 1.0):
        raise ValueError("--threshold-min must be within [0, 1].")
    if not (0.0 <= float(args.threshold_max) <= 1.0):
        raise ValueError("--threshold-max must be within [0, 1].")
    if float(args.threshold_min) > float(args.threshold_max):
        raise ValueError("--threshold-min must be <= --threshold-max.")
    if float(args.threshold_step) <= 0:
        raise ValueError("--threshold-step must be > 0.")

    if not (0.0 < float(args.collinearity_cutoff) < 1.0):
        raise ValueError("--collinearity-cutoff must be between 0 and 1 (exclusive).")

    if args.models is not None and len(args.models) == 0:
        raise ValueError("--models was provided but empty.")
    if args.sampling_methods is not None and len(args.sampling_methods) == 0:
        raise ValueError("--sampling-methods was provided but empty.")

    # top-k vs trial-count consistency is applied per-model during run-time
    # after notebook-faithful defaults and optional CLI overrides are resolved.


def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    validate_args(args)

    run_pipeline(args)


if __name__ == "__main__":
    main()
```

10.1.2 EXP G sampling exp.py

```
# Install required packages (idempotent, safe to re-run)
import sys
```

```
!{sys.executable} -m pip install --quiet lightgbm scikit-learn pandas numpy matplotlib joblib imblearn

# # EXP 3 G Sampling Experiment
#
# Reliability check for ‘base‘, ‘cw‘, ‘smote‘, ‘smote+cw‘, ‘smotenc‘, and ‘smote+nc‘ using the same EXP3 preprocessing bundle
      used by the app.
# Focus: detect physiologically/medically impossible synthetic patients.


import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from imblearn.over_sampling import SMOTE, SMOTENC
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import OrdinalEncoder
from sklearn.utils import resample
import lightgbm as lgb
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 200)

# -- Utility helpers (mirrors Main_2015_Training_Core + EXP3-B) --------------

def _find_col(columns, aliases):
    lc = {c.lower(): c for c in columns}
    for a in aliases:
        if a.lower() in lc:
            return lc[a.lower()]
    for c in columns:
        if any(a.lower() in c.lower() for a in aliases):
            return c
    return None

def _to_num(s):
    return pd.to_numeric(s, errors='coerce').replace([9, 99, 888888, 999999], np.nan)

def _build_smoking_level(df_):
    sl = _find_col(df_.columns, ['smoking_level'])
    if sl:
        return _to_num(df_[sl]).clip(0, 3).astype(float), [sl]
    ss  = _find_col(df_.columns, ['smoke_status'])
    cs  = _find_col(df_.columns, ['current_smoking', 'currentsmoking'])
    es  = _find_col(df_.columns, ['ever_smk'])
    used, out = [], pd.Series(np.nan, index=df_.index, dtype=float)
    if ss:
        status = _to_num(df_[ss]); used.append(ss)
        out[status == 0] = 0; out[status == 2] = 1; out[status == 1] = 2
        if cs:
            cur = _to_num(df_[cs]); used.append(cs)
            out[(status == 1) & (cur == 3)] = 3
        return out, used
    if cs:
        cur = _to_num(df_[cs]); used.append(cs)
        out[cur == 0] = 0; out[cur.isin([1, 2])] = 2; out[cur == 3] = 3
        return out, used
    if es:
        ever = _to_num(df_[es]); used.append(es)
        out[ever == 0] = 0; out[ever > 0] = 1
        return out, used
    return None, []

def _build_alcohol_level(df_):
    al = _find_col(df_.columns, ['alcohol_level'])
    if al:
        return _to_num(df_[al]).clip(0, 3).astype(float), [al]
    as_ = _find_col(df_.columns, ['alcohol_status'])
    bg  = _find_col(df_.columns, ['binge_drink', 'binge_drinking'])
    ca  = _find_col(df_.columns, ['con_alcohol'])
    d30 = _find_col(df_.columns, ['drnk_30days'])
    ae  = _find_col(df_.columns, ['alcohol'])
    used, out = [], pd.Series(np.nan, index=df_.index, dtype=float)
    if as_:
        status = _to_num(df_[as_]); used.append(as_)
        out[status == 0] = 0; out[status == 2] = 1; out[status == 1] = 2
        if bg:
            binge = _to_num(df_[bg]); used.append(bg)
            out[(status == 1) & (binge == 1)] = 3
        return out, used
    out[:] = 0
    if ae:
        ever = _to_num(df_[ae]);   used.append(ae);   out[ever > 0] = 1
    if ca:
        cur  = _to_num(df_[ca]);   used.append(ca);   out[cur == 1] = np.maximum(out[cur == 1], 2)
    if d30:
        d    = _to_num(df_[d30]);  used.append(d30);  out[d == 1]   = np.maximum(out[d == 1], 2)
    if bg:
        b    = _to_num(df_[bg]);   used.append(bg);   out[b == 1]   = 3
    return (out, used) if used else (None, [])

def _build_bmi(df_):
    w = _find_col(df_.columns, ['weight'])
    h = _find_col(df_.columns, ['height'])
    if not w or not h:
        return None, []
    wt = pd.to_numeric(df_[w], errors='coerce')
    ht = pd.to_numeric(df_[h], errors='coerce')
    if pd.notna(ht.median()) and float(ht.median()) > 3.0:
        ht = ht / 100.0
    return (wt / ht**2).replace([np.inf, -np.inf], np.nan).astype(float), [w, h]

def _build_whr(df_):
```

```
    wc = _find_col(df_.columns, ['waist'])
    hc = _find_col(df_.columns, ['hip'])
    if not wc or not hc:
        return None, []
    waist = pd.to_numeric(df_[wc], errors='coerce')
    hip   = pd.to_numeric(df_[hc], errors='coerce').replace(0, np.nan)
    return (waist / hip).replace([np.inf, -np.inf], np.nan).astype(float), [wc, hc]

# -- Load fixed merged dataset from training subfolder ------------------------
CWD = Path.cwd()
if (CWD / 'training').exists():
    ROOT = CWD
elif (CWD / 'FINAL' / 'training').exists():
    ROOT = CWD / 'FINAL'
elif (CWD.parent / 'training').exists():
    ROOT = CWD.parent
else:
    ROOT = CWD

DATA_PATH = ROOT / 'training' / 'merged_clinical_leftjoin.csv'
if not DATA_PATH.exists():
    raise FileNotFoundError(f'Required dataset not found: {DATA_PATH}')

df = pd.read_csv(DATA_PATH, low_memory=False)
print(f'Using merged dataset: {DATA_PATH}')
print(f'Final merged shape : {df.shape}')

# -- Target engineering (Hypertension from SBP/DBP, mirrors Main_2015_Training_Core) --
sbp_col = _find_col(df.columns, ['ave_sbp', 'sbp', 'systolic', 'sysbp'])
dbp_col = _find_col(df.columns, ['ave_dbp', 'dbp', 'diastolic', 'diabp'])

if sbp_col and dbp_col:
    sbp = pd.to_numeric(df[sbp_col], errors='coerce')
    dbp = pd.to_numeric(df[dbp_col], errors='coerce')
    df['Hypertension'] = ((sbp >= 140) | (dbp >= 90)).fillna(False).astype(int)
    TARGET_COL = 'Hypertension'
    print(f'Target created from {sbp_col}, {dbp_col}  (>=140/90 rule)')
else:
    TARGET_COL = _find_col(df.columns, ['hypertension', 'htn', 'target', 'label', 'outcome'])
    if TARGET_COL is None:
        raise ValueError(f'Cannot derive target. Columns: {list(df.columns[:30])}')
    print(f'Target column found : {TARGET_COL}')

df = df.dropna(subset=[TARGET_COL]).copy()
print('Class balance:', df[TARGET_COL].value_counts(normalize=True).round(3).to_dict())

# -- Feature engineering (mirrors EXP3-B) -------------------------------------
X_raw = df.drop(columns=[TARGET_COL]).copy()
y     = df[TARGET_COL].astype(int).copy()

smk, smk_src = _build_smoking_level(X_raw)
if smk is not None:
    X_raw['fe_smoking_level'] = smk

alc, alc_src = _build_alcohol_level(X_raw)
if alc is not None:
    X_raw['fe_alcohol_level'] = alc

bmi, bmi_src = _build_bmi(X_raw)
if bmi is not None:
    X_raw['bmi'] = bmi

whr, whr_src = _build_whr(X_raw)
if whr is not None:
    X_raw['whr'] = whr

# -- Drop non-predictive / raw source columns (mirrors EXP3-B manual_non_predictive) --
DROP_ALWAYS = [
    sbp_col, dbp_col,
    'height', 'weight', 'waist', 'hip',
    'enns_year', 'hhnum', 'member_code',
    'regcode', 'provcode', 'psc', 'csc', 'rhc', 'psurec', 'strrec',
    'wgts', 'finalwgt', 'finalwgt1', 'finalwgt4',
    'rep_natl', 'rep_prov', 'ms_psucode', 'wrkplace',
    'interview_status', 'intdate', 'enumcode',
    'alcohol', 'con_alcohol', 'drnk_30days', 'drnk_30d_num', 'alcohol_status',
    'binge_drinking', 'current_smoking', 'ever_smk', 'smoke_status', 'smoking_level', 'alcohol_level',
]
DROP_ALWAYS += list(set(bmi_src + whr_src + smk_src + alc_src))
lc_map = {c.lower(): c for c in X_raw.columns}
to_drop = sorted({lc_map[d.lower()] for d in DROP_ALWAYS if d and d.lower() in lc_map})
X_raw = X_raw.drop(columns=to_drop, errors='ignore')

# Deduplicate age/sex alias columns from anthro join
for _alias in ['age', 'sex']:
    _cands = [c for c in X_raw.columns if _alias in c.lower()]
    if len(_cands) > 1:
        _keep  = next((c for c in _cands if c.lower() == _alias), _cands[0])
        X_raw  = X_raw.drop(columns=[c for c in _cands if c != _keep], errors='ignore')

X = X_raw.copy()
print(f'Features: {X.shape[1]}  | Rows: {len(X)}  | Positives: {int(y.sum())}')

# -- EXP3-B style preprocessing: 3-way split + KNN impute + scale/OHE ---------
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

def _ohe():
    try:
        return OneHotEncoder(handle_unknown='ignore', sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown='ignore', sparse=False)

def e3_fit_imputers(X_tr, num_cols, cat_cols):
    knn = KNNImputer(n_neighbors=5)
    if num_cols:
```

```
        knn.fit(X_tr[num_cols])
    cat_imp = None
    if cat_cols:
        cat_imp = SimpleImputer(strategy='most_frequent')
        cat_imp.fit(X_tr[cat_cols])
    return knn, cat_imp

def e3_impute(knn, cat_imp, X_df, num_cols, cat_cols):
    frames = []
    if num_cols:
        frames.append(pd.DataFrame(knn.transform(X_df[num_cols]), columns=num_cols, index=X_df.index))
    if cat_cols and cat_imp is not None:
        frames.append(pd.DataFrame(cat_imp.transform(X_df[cat_cols]), columns=cat_cols, index=X_df.index))
    return pd.concat(frames, axis=1) if frames else pd.DataFrame(index=X_df.index)

def e3_fit_enc_scaler(X_imp, num_cols, cat_cols):
    scaler = StandardScaler()
    ohe_enc = None
    if num_cols:
        scaler.fit(X_imp[num_cols])
    if cat_cols:
        ohe_enc = _ohe()
        ohe_enc.fit(X_imp[cat_cols].astype(str))
    return scaler, ohe_enc

def e3_encode(scaler, ohe_enc, X_imp, num_cols, cat_cols):
    parts = []
    if num_cols:
        parts.append(scaler.transform(X_imp[num_cols]))
    if cat_cols and ohe_enc is not None:
        parts.append(ohe_enc.transform(X_imp[cat_cols].astype(str)))
    return np.hstack(parts) if parts else np.empty((len(X_imp), 0))

# 3-way split: 60% train, 20% cal, 20% test (mirrors EXP3-B)
X_tr_raw, X_tmp_raw, y_train, y_tmp = train_test_split(
    X, y, test_size=0.40, random_state=42, stratify=y
)
X_cal_raw, X_test_raw, y_cal, y_test = train_test_split(
    X_tmp_raw, y_tmp, test_size=0.50, random_state=42, stratify=y_tmp
)

# Detect numeric / categorical on training split

# Always treat these as categorical for SMOTENC, even if numeric
force_cat = ['fe_smoking_level', 'fe_alcohol_level']
# Try to find an ethnicity column (case-insensitive, partial match)
ethnicity_col = next((c for c in X_tr_raw.columns if 'ethnic' in c.lower()), None)
if ethnicity_col:
    force_cat.append(ethnicity_col)

e3_num_cols = X_tr_raw.select_dtypes(include=[np.number]).columns.tolist()
e3_cat_cols = [c for c in X_tr_raw.columns if c not in e3_num_cols or c in force_cat]
e3_num_cols = [c for c in e3_num_cols if c not in force_cat]

# Fit imputers and scalers on train only
knn_imp, cat_imp = e3_fit_imputers(X_tr_raw, e3_num_cols, e3_cat_cols)

X_tr_imp  = e3_impute(knn_imp, cat_imp, X_tr_raw,   e3_num_cols, e3_cat_cols)
X_cal_imp = e3_impute(knn_imp, cat_imp, X_cal_raw,  e3_num_cols, e3_cat_cols)
X_te_imp  = e3_impute(knn_imp, cat_imp, X_test_raw, e3_num_cols, e3_cat_cols)

scaler_e3, ohe_e3 = e3_fit_enc_scaler(X_tr_imp, e3_num_cols, e3_cat_cols)

# For SMOTENC: build ordinal-encoded train matrix (cat preserved as ordinal)
oe_sampling = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
if e3_cat_cols:
    Xc_ord = pd.DataFrame(
        oe_sampling.fit_transform(X_tr_imp[e3_cat_cols].astype(str)),
        columns=e3_cat_cols, index=X_tr_imp.index
    )
else:
    Xc_ord = pd.DataFrame(index=X_tr_imp.index)

X_train_sampling = pd.concat([X_tr_imp[e3_num_cols] if e3_num_cols else pd.DataFrame(index=X_tr_imp.index), Xc_ord], axis=1)
feature_cols_sampling = list(X_train_sampling.columns)
cat_idx = list(range(len(e3_num_cols), len(e3_num_cols) + len(e3_cat_cols)))

print(pd.DataFrame({
    'Split': ['Train', 'Cal', 'Test'],
    'N':     [len(y_train), len(y_cal), len(y_test)],
    'Pos%':  [f"{y_train.mean()*100:.1f}", f"{y_cal.mean()*100:.1f}", f"{y_test.mean()*100:.1f}"],
}).to_string(index=False))
print(f'\nNumeric: {len(e3_num_cols)} | Categorical: {len(e3_cat_cols)}')
print(f'Sampling matrix shape: {X_train_sampling.shape} | cat_idx for SMOTENC: {len(cat_idx)}')

def impossible_flags(df_sample: pd.DataFrame) -> pd.Series:
    f = pd.Series(False, index=df_sample.index)

    if 'age' in df_sample.columns:
        f |= (df_sample['age'] < 18) | (df_sample['age'] > 120)
    if 'sex' in df_sample.columns:
        f |= ~df_sample['sex'].round().isin([1, 2])
    if 'height' in df_sample.columns:
        f |= (df_sample['height'] <= 0) | (df_sample['height'] > 260)
    if 'weight' in df_sample.columns:
        f |= (df_sample['weight'] <= 0) | (df_sample['weight'] > 350)
    if 'waist' in df_sample.columns:
        f |= (df_sample['waist'] <= 0) | (df_sample['waist'] > 200)
    if 'hip' in df_sample.columns:
        f |= (df_sample['hip'] <= 0) | (df_sample['hip'] > 220)
    if 'BMI' in df_sample.columns:
        f |= (df_sample['BMI'] < 10) | (df_sample['BMI'] > 50)
    if 'bmi' in df_sample.columns:
        f |= (df_sample['bmi'] < 10) | (df_sample['bmi'] > 50)
    if 'whr' in df_sample.columns:
        f |= (df_sample['whr'] < 0.4) | (df_sample['whr'] > 2.0)
```

```
    nonneg_prefixes = ('Total_', 'fg', 'epwt_fg')
    for c in df_sample.columns:
        if c.startswith(nonneg_prefixes):
            f |= (pd.to_numeric(df_sample[c], errors='coerce') < 0)

    return f.fillna(True)

def resample_method(name: str, Xs: pd.DataFrame, ys: pd.Series) -> tuple[pd.DataFrame, pd.Series]:
    if name in {'base', 'cw'}:
        return Xs.copy(), ys.copy()
    if name in {'smote', 'smote+cw'}:
        sampler = SMOTE(random_state=42, k_neighbors=5)
        Xr, yr = sampler.fit_resample(Xs, ys)
        return pd.DataFrame(Xr, columns=Xs.columns), pd.Series(yr)
    if name in {'smotenc', 'smote+nc'}:
        sampler = SMOTENC(categorical_features=cat_idx, random_state=42, k_neighbors=5)
        Xr, yr = sampler.fit_resample(Xs, ys)
        return pd.DataFrame(Xr, columns=Xs.columns), pd.Series(yr)
    raise ValueError(name)

methods = ['base', 'cw', 'smote', 'smote+cw', 'smotenc', 'smote+nc']
summary_rows = []
examples = {}

n_orig = len(X_train_sampling)

for m in methods:
    Xr, yr = resample_method(m, X_train_sampling, y_train)
    flags_all = impossible_flags(Xr)

    n_all = int(len(Xr))
    n_bad_all = int(flags_all.sum())

    # For over-sampling methods, imblearn appends synthetic rows after originals.
    n_syn = max(n_all - n_orig, 0)
    if n_syn > 0:
        flags_syn = flags_all.iloc[n_orig:]
        n_bad_syn = int(flags_syn.sum())
        syn_bad_pct = 100.0 * n_bad_syn / n_syn
        if n_bad_syn > 0:
            examples[m] = Xr.iloc[n_orig:].loc[flags_syn].head(5).copy()
    else:
        n_bad_syn = 0
        syn_bad_pct = np.nan

    summary_rows.append({
        'method': m,
        'rows_total': n_all,
        'rows_synthetic': n_syn,
        'positive_rate': float(np.mean(yr)),
        'impossible_all_rows': n_bad_all,
        'impossible_all_pct': (100.0 * n_bad_all / max(n_all, 1)),
        'impossible_synth_rows': n_bad_syn,
        'impossible_synth_pct': syn_bad_pct,
    })

summary_df = pd.DataFrame(summary_rows)
summary_df = summary_df.sort_values(['impossible_synth_pct', 'impossible_all_pct', 'method'], na_position='last').reset_index(
      drop=True)
summary_df

from sklearn.metrics import roc_auc_score

def build_sampling_matrix(X_imp: pd.DataFrame) -> pd.DataFrame:
    if e3_cat_cols:
        Xc = pd.DataFrame(
            oe_sampling.transform(X_imp[e3_cat_cols].astype(str)),
            columns=e3_cat_cols,
            index=X_imp.index,
        )
    else:
        Xc = pd.DataFrame(index=X_imp.index)

    Xn = X_imp[e3_num_cols] if e3_num_cols else pd.DataFrame(index=X_imp.index)
    return pd.concat([Xn, Xc], axis=1)[feature_cols_sampling]

X_cal_sampling = build_sampling_matrix(X_cal_imp)
X_test_sampling = build_sampling_matrix(X_te_imp)

def make_lgbm(method_name: str) -> lgb.LGBMClassifier:
    params = dict(
        objective='binary',
        random_state=42,
        n_estimators=400,
        learning_rate=0.03,
        num_leaves=31,
        subsample=0.9,
        colsample_bytree=0.8,
        min_child_samples=20,
        reg_alpha=0.0,
        reg_lambda=0.0,
        verbosity=-1,
    )
    if method_name in {'cw', 'smote+cw'}:
        params['class_weight'] = 'balanced'

    # Try GPU first; fallback to CPU for environments without OpenCL/CUDA support.
    try:
        return lgb.LGBMClassifier(device='gpu', **params)
    except Exception:
        return lgb.LGBMClassifier(device='cpu', **params)

metric_rows = []
for m in methods:
    Xr, yr = resample_method(m, X_train_sampling, y_train)
```

```
    model = make_lgbm(m)
    try:
        model.fit(Xr, yr)
    except Exception:
        # If GPU fit fails at runtime, retry on CPU.
        model = lgb.LGBMClassifier(
            device='cpu',
            objective='binary',
            random_state=42,
            n_estimators=400,
            learning_rate=0.03,
            num_leaves=31,
            subsample=0.9,
            colsample_bytree=0.8,
            min_child_samples=20,
            reg_alpha=0.0,
            reg_lambda=0.0,
            verbosity=-1,
            class_weight='balanced' if m in {'cw', 'smote+cw'} else None,
        )
        model.fit(Xr, yr)

    proba = model.predict_proba(X_test_sampling)[:, 1]
    pred = (proba >= 0.5).astype(int)

    metric_rows.append({
        'method': m,
        'accuracy': accuracy_score(y_test, pred),
        'precision': precision_score(y_test, pred, zero_division=0),
        'recall': recall_score(y_test, pred, zero_division=0),
        'f1': f1_score(y_test, pred, zero_division=0),
        'roc_auc': roc_auc_score(y_test, proba),
    })

metrics_df = pd.DataFrame(metric_rows).sort_values('f1', ascending=False).reset_index(drop=True)
comparison_df = summary_df.merge(metrics_df, on='method', how='left')
comparison_df[['method', 'rows_synthetic', 'impossible_synth_pct', 'impossible_all_pct', 'accuracy', 'precision', 'recall', '
      f1', 'roc_auc']].sort_values('f1', ascending=False)

import matplotlib.pyplot as plt

plot_df = summary_df.copy()
plot_df['impossible_synth_pct_plot'] = plot_df['impossible_synth_pct'].fillna(0.0)

plt.figure(figsize=(9, 4))
plt.bar(plot_df['method'], plot_df['impossible_synth_pct_plot'])
plt.ylabel('Impossible synthetic patients (%)')
plt.xlabel('Sampling strategy')
plt.title('Synthetic-only Plausibility Check per Sampling Method')
plt.xticks(rotation=25, ha='right')
plt.tight_layout()
plt.show()

# Inspect first impossible examples per method (if any).
for m in methods:
    print('\n===', m, '===')
    if m not in examples:
        print('No impossible rows detected.')
    else:
        display(examples[m])
```

10.1.3 EXP H threshold exp.py

```
# Install required packages (idempotent, safe to re-run)
import sys
!{sys.executable} -m pip install --quiet lightgbm scikit-learn pandas numpy matplotlib

# # EXP H Threshold Experiment
#
# Threshold sweep for: `0.5, 0.45, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.1` using the same EXP3 preprocessing/model bundle.


from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

# -- Utility helpers (mirrors Main_2015_Training_Core + EXP3-B) --------------

def _find_col(columns, aliases):
    lc = {c.lower(): c for c in columns}
    for a in aliases:
        if a.lower() in lc:
            return lc[a.lower()]
    for c in columns:
        if any(a.lower() in c.lower() for a in aliases):
            return c
    return None

def _to_num(s):
    return pd.to_numeric(s, errors='coerce').replace([9, 99, 888888, 999999], np.nan)

def _build_smoking_level(df_):
    sl = _find_col(df_.columns, ['smoking_level'])
    if sl:
        return _to_num(df_[sl]).clip(0, 3).astype(float), [sl]
    ss  = _find_col(df_.columns, ['smoke_status'])
```

```
    cs  = _find_col(df_.columns, ['current_smoking', 'currentsmoking'])
    es  = _find_col(df_.columns, ['ever_smk'])
    used, out = [], pd.Series(np.nan, index=df_.index, dtype=float)
    if ss:
        status = _to_num(df_[ss]); used.append(ss)
        out[status == 0] = 0; out[status == 2] = 1; out[status == 1] = 2
        if cs:
            cur = _to_num(df_[cs]); used.append(cs)
            out[(status == 1) & (cur == 3)] = 3
        return out, used
    if cs:
        cur = _to_num(df_[cs]); used.append(cs)
        out[cur == 0] = 0; out[cur.isin([1, 2])] = 2; out[cur == 3] = 3
        return out, used
    if es:
        ever = _to_num(df_[es]); used.append(es)
        out[ever == 0] = 0; out[ever > 0] = 1
        return out, used
    return None, []

def _build_alcohol_level(df_):
    al = _find_col(df_.columns, ['alcohol_level'])
    if al:
        return _to_num(df_[al]).clip(0, 3).astype(float), [al]
    as_ = _find_col(df_.columns, ['alcohol_status'])
    bg  = _find_col(df_.columns, ['binge_drink', 'binge_drinking'])
    ca  = _find_col(df_.columns, ['con_alcohol'])
    d30 = _find_col(df_.columns, ['drnk_30days'])
    ae  = _find_col(df_.columns, ['alcohol'])
    used, out = [], pd.Series(np.nan, index=df_.index, dtype=float)
    if as_:
        status = _to_num(df_[as_]); used.append(as_)
        out[status == 0] = 0; out[status == 2] = 1; out[status == 1] = 2
        if bg:
            binge = _to_num(df_[bg]); used.append(bg)
            out[(status == 1) & (binge == 1)] = 3
        return out, used
    out[:] = 0
    if ae:
        ever = _to_num(df_[ae]);   used.append(ae);   out[ever > 0] = 1
    if ca:
        cur  = _to_num(df_[ca]);   used.append(ca);   out[cur == 1] = np.maximum(out[cur == 1], 2)
    if d30:
        d    = _to_num(df_[d30]);  used.append(d30);  out[d == 1]   = np.maximum(out[d == 1], 2)
    if bg:
        b    = _to_num(df_[bg]);   used.append(bg);   out[b == 1]   = 3
    return (out, used) if used else (None, [])

def _build_bmi(df_):
    w = _find_col(df_.columns, ['weight'])
    h = _find_col(df_.columns, ['height'])
    if not w or not h:
        return None, []
    wt = pd.to_numeric(df_[w], errors='coerce')
    ht = pd.to_numeric(df_[h], errors='coerce')
    if pd.notna(ht.median()) and float(ht.median()) > 3.0:
        ht = ht / 100.0
    return (wt / ht**2).replace([np.inf, -np.inf], np.nan).astype(float), [w, h]

def _build_whr(df_):
    wc = _find_col(df_.columns, ['waist'])
    hc = _find_col(df_.columns, ['hip'])
    if not wc or not hc:
        return None, []
    waist = pd.to_numeric(df_[wc], errors='coerce')
    hip   = pd.to_numeric(df_[hc], errors='coerce').replace(0, np.nan)
    return (waist / hip).replace([np.inf, -np.inf], np.nan).astype(float), [wc, hc]

# -- Load fixed merged dataset from training subfolder -------------------------
CWD = Path.cwd()
if (CWD / 'training').exists():
    ROOT = CWD
elif (CWD / 'FINAL' / 'training').exists():
    ROOT = CWD / 'FINAL'
elif (CWD.parent / 'training').exists():
    ROOT = CWD.parent
else:
    ROOT = CWD

DATA_PATH = ROOT / 'training' / 'merged_clinical_leftjoin.csv'
if not DATA_PATH.exists():
    raise FileNotFoundError(f'Required dataset not found: {DATA_PATH}')

df = pd.read_csv(DATA_PATH, low_memory=False)
print(f'Using merged dataset: {DATA_PATH}')
print(f'Final merged shape : {df.shape}')

# -- Target engineering (Hypertension from SBP/DBP, mirrors Main_2015_Training_Core) --
sbp_col = _find_col(df.columns, ['ave_sbp', 'sbp', 'systolic', 'sysbp'])
dbp_col = _find_col(df.columns, ['ave_dbp', 'dbp', 'diastolic', 'diabp'])

if sbp_col and dbp_col:
    sbp = pd.to_numeric(df[sbp_col], errors='coerce')
    dbp = pd.to_numeric(df[dbp_col], errors='coerce')
    df['Hypertension'] = ((sbp >= 140) | (dbp >= 90)).fillna(False).astype(int)
    TARGET_COL = 'Hypertension'
    print(f'Target created from {sbp_col}, {dbp_col}  (>=140/90 rule)')
else:
    TARGET_COL = _find_col(df.columns, ['hypertension', 'htn', 'target', 'label', 'outcome'])
    if TARGET_COL is None:
        raise ValueError(f'Cannot derive target. Columns: {list(df.columns[:30])}')
    print(f'Target column found : {TARGET_COL}')

df = df.dropna(subset=[TARGET_COL]).copy()
print('Class balance:', df[TARGET_COL].value_counts(normalize=True).round(3).to_dict())
```

```
# -- Feature engineering (mirrors EXP3-B) ---------------------------------------
X_raw = df.drop(columns=[TARGET_COL]).copy()
y     = df[TARGET_COL].astype(int).copy()

smk, smk_src = _build_smoking_level(X_raw)
if smk is not None:
    X_raw['fe_smoking_level'] = smk

alc, alc_src = _build_alcohol_level(X_raw)
if alc is not None:
    X_raw['fe_alcohol_level'] = alc

bmi, bmi_src = _build_bmi(X_raw)
if bmi is not None:
    X_raw['bmi'] = bmi

whr, whr_src = _build_whr(X_raw)
if whr is not None:
    X_raw['whr'] = whr

# -- Drop non-predictive / raw source columns (mirrors EXP3-B manual_non_predictive) --
DROP_ALWAYS = [
    sbp_col, dbp_col,
    'height', 'weight', 'waist', 'hip',
    'enns_year', 'hhnum', 'member_code',
    'regcode', 'provcode', 'psc', 'csc', 'rhc', 'psurec', 'strrec',
    'wgts', 'finalwgt', 'finalwgt1', 'finalwgt4',
    'rep_natl', 'rep_prov', 'ms_psucode', 'wrkplace',
    'interview_status', 'intdate', 'enumcode',
    'alcohol', 'con_alcohol', 'drnk_30days', 'drnk_30d_num', 'alcohol_status',
    'binge_drinking', 'current_smoking', 'ever_smk', 'smoke_status', 'smoking_level', 'alcohol_level',
]
DROP_ALWAYS += list(set(bmi_src + whr_src + smk_src + alc_src))
lc_map = {c.lower(): c for c in X_raw.columns}
to_drop = sorted({lc_map[d.lower()] for d in DROP_ALWAYS if d and d.lower() in lc_map})
X_raw = X_raw.drop(columns=to_drop, errors='ignore')

# Deduplicate age/sex alias columns from anthro join
for _alias in ['age', 'sex']:
    _cands = [c for c in X_raw.columns if _alias in c.lower()]
    if len(_cands) > 1:
        _keep  = next((c for c in _cands if c.lower() == _alias), _cands[0])
        X_raw  = X_raw.drop(columns=[c for c in _cands if c != _keep], errors='ignore')

X = X_raw.copy()
print(f'Features: {X.shape[1]}  | Rows: {len(X)}  | Positives: {int(y.sum())}')


# -- EXP3-B style preprocessing: 3-way split + KNN impute + StandardScaler + OHE --
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

def _ohe():
    try:
        return OneHotEncoder(handle_unknown='ignore', sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown='ignore', sparse=False)

def e3_fit_imputers(X_tr, num_cols, cat_cols):
    knn = KNNImputer(n_neighbors=5)
    if num_cols:
        knn.fit(X_tr[num_cols])
    cat_imp = None
    if cat_cols:
        cat_imp = SimpleImputer(strategy='most_frequent')
        cat_imp.fit(X_tr[cat_cols])
    return knn, cat_imp

def e3_impute(knn, cat_imp, X_df, num_cols, cat_cols):
    frames = []
    if num_cols:
        frames.append(pd.DataFrame(knn.transform(X_df[num_cols]), columns=num_cols, index=X_df.index))
    if cat_cols and cat_imp is not None:
        frames.append(pd.DataFrame(cat_imp.transform(X_df[cat_cols]), columns=cat_cols, index=X_df.index))
    return pd.concat(frames, axis=1) if frames else pd.DataFrame(index=X_df.index)

def e3_fit_enc_scaler(X_imp, num_cols, cat_cols):
    scaler  = StandardScaler()
    ohe_enc = None
    if num_cols:
        scaler.fit(X_imp[num_cols])
    if cat_cols:
        ohe_enc = _ohe()
        ohe_enc.fit(X_imp[cat_cols].astype(str))
    return scaler, ohe_enc

def e3_encode(scaler, ohe_enc, X_imp, num_cols, cat_cols):
    parts = []
    if num_cols:
        parts.append(scaler.transform(X_imp[num_cols]))
    if cat_cols and ohe_enc is not None:
        parts.append(ohe_enc.transform(X_imp[cat_cols].astype(str)))
    return np.hstack(parts) if parts else np.empty((len(X_imp), 0))

# 3-way split: 60% train, 20% cal, 20% test (mirrors EXP3-B)
X_tr_raw, X_tmp_raw, y_train, y_tmp = train_test_split(
    X, y, test_size=0.40, random_state=42, stratify=y
)
X_cal_raw, X_test_raw, y_cal, y_test = train_test_split(
    X_tmp_raw, y_tmp, test_size=0.50, random_state=42, stratify=y_tmp
)

# Always treat these as categorical for SMOTENC, even if numeric
force_cat = ['fe_smoking_level', 'fe_alcohol_level']
# Try to find an ethnicity column (case-insensitive, partial match)
ethnicity_col = next((c for c in X_tr_raw.columns if 'ethnic' in c.lower()), None)
```

```
if ethnicity_col:
    force_cat.append(ethnicity_col)

e3_num_cols = X_tr_raw.select_dtypes(include=[np.number]).columns.tolist()
e3_cat_cols = [c for c in X_tr_raw.columns if c not in e3_num_cols or c in force_cat]
e3_num_cols = [c for c in e3_num_cols if c not in force_cat]

# Fit on train only, transform all splits
knn_imp, cat_imp_h = e3_fit_imputers(X_tr_raw, e3_num_cols, e3_cat_cols)
X_tr_imp  = e3_impute(knn_imp, cat_imp_h, X_tr_raw,   e3_num_cols, e3_cat_cols)
X_cal_imp = e3_impute(knn_imp, cat_imp_h, X_cal_raw,  e3_num_cols, e3_cat_cols)
X_te_imp  = e3_impute(knn_imp, cat_imp_h, X_test_raw, e3_num_cols, e3_cat_cols)

scaler_h, ohe_h = e3_fit_enc_scaler(X_tr_imp, e3_num_cols, e3_cat_cols)

# Encoded matrices ready for LightGBM
X_tr_enc  = e3_encode(scaler_h, ohe_h, X_tr_imp,  e3_num_cols, e3_cat_cols)
X_cal_enc = e3_encode(scaler_h, ohe_h, X_cal_imp, e3_num_cols, e3_cat_cols)
X_te_enc  = e3_encode(scaler_h, ohe_h, X_te_imp,  e3_num_cols, e3_cat_cols)

print(pd.DataFrame({
    'Split':  ['Train', 'Cal', 'Test'],
    'N':      [len(y_train), len(y_cal), len(y_test)],
    'Pos%':   [f"{y_train.mean()*100:.1f}", f"{y_cal.mean()*100:.1f}", f"{y_test.mean()*100:.1f}"],
    'Shape':  [str(X_tr_enc.shape), str(X_cal_enc.shape), str(X_te_enc.shape)],
}).to_string(index=False))


# -- EXP3-C style LIGHTGBM-only training + expanded threshold experiment ------
# Strictly follows EXP3-C pattern:
#   1) Stage 1 random CV search
#   2) Stage 2 refined CV search
#   3) Final train on full train split
#   4) Best threshold chosen on calibration split
#   5) Expanded exploratory threshold sweep on held-out test split

from copy import deepcopy
from lightgbm import LGBMClassifier
from sklearn.model_selection import StratifiedKFold, ParameterSampler
from sklearn.metrics import roc_auc_score, log_loss
from scipy.stats import loguniform, randint, uniform as sp_uniform

# Detect GPU availability (same spirit as EXP3-C GPU-aware setup)
try:
    import torch
    _lgb_device = 'gpu' if torch.cuda.is_available() else 'cpu'
except Exception:
    _lgb_device = 'cpu'

E3_SEED = 42
np.random.seed(E3_SEED)

# EXP3-C-like optimization controls
E3_S1_TRIALS = 20
E3_S1_EPOCHS = 120
E3_S1_FOLDS = 3
E3_TOP_K_S1 = 5

E3_S2_REFINE = 3
E3_S2_EPOCHS = 320
E3_S2_FOLDS = 5
E3_TOP_K_S2 = 2

E3_FINAL_EPOCHS = 700
THRESHOLD_GRID_CAL = np.round(np.arange(0.35, 0.70, 0.05), 2)  # EXP3-C style for selection

# Expanded exploratory thresholds for this dedicated threshold experiment
thresholds = [0.50, 0.45, 0.40, 0.35, 0.30, 0.25, 0.20, 0.15, 0.10]

_LGB_SPACE = {
    'learning_rate':     loguniform(0.005, 0.40),
    'max_depth':         randint(3, 12),
    'num_leaves':        randint(20, 200),
    'min_child_samples': randint(5, 50),
    'subsample':         sp_uniform(0.5, 0.5),
    'colsample_bytree':  sp_uniform(0.5, 0.5),
    'reg_lambda':        loguniform(1e-3, 100),
    'reg_alpha':         loguniform(1e-3, 10),
}

def _build_lgb(params, epoch_budget, seed=E3_SEED):
    p = deepcopy(params)
    md = p.get('max_depth', 6)
    return LGBMClassifier(
        n_estimators=int(epoch_budget),
        learning_rate=max(1e-4, float(p.get('learning_rate', 0.1))),
        max_depth=int(md) if md != -1 else -1,
        num_leaves=max(2, int(round(p.get('num_leaves', 31)))),
        subsample=float(np.clip(p.get('subsample', 0.8), 0.1, 1.0)),
        colsample_bytree=float(np.clip(p.get('colsample_bytree', 0.8), 0.1, 1.0)),
        reg_lambda=max(0.0, float(p.get('reg_lambda', 1.0))),
        reg_alpha=max(0.0, float(p.get('reg_alpha', 0.0))),
        min_child_samples=max(1, int(round(p.get('min_child_samples', 20)))),
        objective='binary',
        device_type=_lgb_device,
        random_state=seed,
        verbose=-1,
    )

def _refine_candidates(base_params_list, n_refine=3, seed=E3_SEED):
    rng = np.random.RandomState(seed)
    out = []
    for b in base_params_list:
        out.append(deepcopy(b))
        for _ in range(n_refine):
            c = {}
```

```
            for k, v in b.items():
                if isinstance(v, (int, np.integer)):
                    c[k] = max(1, int(round(v * rng.uniform(0.7, 1.3))))
                elif isinstance(v, (float, np.floating)):
                    c[k] = max(1e-7, float(v * rng.uniform(0.7, 1.3)))
                else:
                    c[k] = v
            out.append(c)
    seen, unique = set(), []
    for item in out:
        key = str(sorted(item.items()))
        if key not in seen:
            seen.add(key)
            unique.append(item)
    return unique

def _evaluate_params_cv(params, epoch_budget, n_splits, seed=E3_SEED):
    splitter = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    y_arr = np.asarray(y_train, dtype=int)
    fold_rows = []

    for fold_i, (tr_idx, va_idx) in enumerate(splitter.split(X_tr_enc, y_arr), start=1):
        Xf_tr, Xf_va = X_tr_enc[tr_idx], X_tr_enc[va_idx]
        yf_tr, yf_va = y_arr[tr_idx], y_arr[va_idx]

        mdl = _build_lgb(params, epoch_budget, seed=seed)
        try:
            mdl.fit(Xf_tr, yf_tr)
        except Exception as exc:
            if 'gpu' in str(exc).lower() or 'cuda' in str(exc).lower() or 'device' in str(exc).lower():
                mdl.set_params(device_type='cpu')
                mdl.fit(Xf_tr, yf_tr)
            else:
                raise

        p_val = np.clip(mdl.predict_proba(Xf_va)[:, 1], 1e-9, 1 - 1e-9)

        best_obj, best_met, best_thr = -np.inf, None, 0.5
        for thr in THRESHOLD_GRID_CAL:
            yp = (p_val >= thr).astype(int)
            acc = accuracy_score(yf_va, yp)
            rec = recall_score(yf_va, yp, zero_division=0)
            obj = 0.60 * acc + 0.40 * rec
            if obj > best_obj:
                best_obj = obj
                best_met = dict(
                    accuracy=acc,
                    recall=rec,
                    precision=precision_score(yf_va, yp, zero_division=0),
                    f1=f1_score(yf_va, yp, zero_division=0),
                    auc=roc_auc_score(yf_va, p_val) if np.unique(yf_va).size > 1 else 0.5,
                    logloss=log_loss(yf_va, p_val),
                )
                best_thr = float(thr)

        best_met['fold'] = fold_i
        best_met['best_threshold'] = best_thr
        fold_rows.append(best_met)

    d = pd.DataFrame(fold_rows)
    s = {
        'accuracy_mean': float(d['accuracy'].mean()),
        'accuracy_std': float(d['accuracy'].std(ddof=0)),
        'recall_mean': float(d['recall'].mean()),
        'recall_std': float(d['recall'].std(ddof=0)),
        'precision_mean': float(d['precision'].mean()),
        'f1_mean': float(d['f1'].mean()),
        'f1_std': float(d['f1'].std(ddof=0)),
        'auc_mean': float(d['auc'].mean()),
        'logloss_mean': float(d['logloss'].mean()),
        'logloss_std': float(d['logloss'].std(ddof=0)),
        'threshold_mean': float(d['best_threshold'].mean()),
    }
    s['stage_score'] = (
        0.60 * s['accuracy_mean']
        + 0.40 * s['recall_mean']
        + 0.05 * s['f1_mean']
        - 0.08 * s['logloss_mean']
        - 0.03 * s['accuracy_std']
        - 0.03 * s['recall_std']
    )
    return s

print(f'LightGBM device: {_lgb_device}')
print(f'Stage 1: {E3_S1_TRIALS} trials x {E3_S1_FOLDS}-fold CV')

# -- Stage 1 ---------------------------------------------------------------
s1_trials = list(ParameterSampler(_LGB_SPACE, n_iter=E3_S1_TRIALS, random_state=E3_SEED))
s1_rows = []
for i, params in enumerate(s1_trials, start=1):
    try:
        cv_met = _evaluate_params_cv(params, E3_S1_EPOCHS, E3_S1_FOLDS, seed=E3_SEED)
        s1_rows.append({'trial': i, 'params': params, **cv_met})
        print(f'[S1 {i:02d}/{E3_S1_TRIALS}] score={cv_met["stage_score"]:.4f}')
    except Exception as exc:
        s1_rows.append({'trial': i, 'params': params, 'stage_score': -999.0, 'error': str(exc)})
        print(f'[S1 {i:02d}/{E3_S1_TRIALS}] failed: {exc}')

df_s1 = pd.DataFrame(s1_rows).sort_values('stage_score', ascending=False).reset_index(drop=True)
top_s1_params = df_s1.head(E3_TOP_K_S1)['params'].tolist()
print(f'Best Stage-1 score: {float(df_s1.iloc[0]["stage_score"]):.4f}')

# -- Stage 2 ---------------------------------------------------------------
print(f'\nStage 2: refine top-{E3_TOP_K_S1}, {E3_S2_FOLDS}-fold CV')
s2_candidates = _refine_candidates(top_s1_params, n_refine=E3_S2_REFINE, seed=E3_SEED)
s2_rows = []
```

```
for i, params in enumerate(s2_candidates, start=1):
    try:
        cv_met = _evaluate_params_cv(params, E3_S2_EPOCHS, E3_S2_FOLDS, seed=E3_SEED)
        s2_rows.append({'trial': i, 'params': params, **cv_met})
        print(f'[S2 {i:02d}/{len(s2_candidates)}] score={cv_met["stage_score"]:.4f}')
    except Exception as exc:
        s2_rows.append({'trial': i, 'params': params, 'stage_score': -999.0, 'error': str(exc)})
        print(f'[S2 {i:02d}/{len(s2_candidates)}] failed: {exc}')

df_s2 = pd.DataFrame(s2_rows).sort_values('stage_score', ascending=False).reset_index(drop=True)
best_params_list = df_s2.head(E3_TOP_K_S2)['params'].tolist()
print(f'Best Stage-2 score: {float(df_s2.iloc[0]["stage_score"]):.4f}')

# -- Final training + threshold selection on calibration split -----------------
final_model = None
final_score = -np.inf
best_cal_thr = 0.5

for params in best_params_list:
    mdl = _build_lgb(params, E3_FINAL_EPOCHS, seed=E3_SEED)
    try:
        mdl.fit(X_tr_enc, np.asarray(y_train, dtype=int))
    except Exception as exc:
        if 'gpu' in str(exc).lower() or 'cuda' in str(exc).lower() or 'device' in str(exc).lower():
            mdl.set_params(device_type='cpu')
            mdl.fit(X_tr_enc, np.asarray(y_train, dtype=int))
        else:
            continue

    p_cal = np.clip(mdl.predict_proba(X_cal_enc)[:, 1], 1e-9, 1 - 1e-9)
    local_score, local_thr = -np.inf, 0.5
    for thr in THRESHOLD_GRID_CAL:
        yp = (p_cal >= thr).astype(int)
        s = 0.60 * accuracy_score(y_cal, yp) + 0.40 * recall_score(y_cal, yp, zero_division=0)
        if s > local_score:
            local_score, local_thr = s, float(thr)

    if local_score > final_score:
        final_score = local_score
        best_cal_thr = local_thr
        final_model = mdl

if final_model is None:
    raise RuntimeError('No valid final LightGBM model found after Stage-2 selection.')

print(f'\nSelected final cal threshold (EXP3-C grid): {best_cal_thr:.2f}')

# -- Expanded exploratory threshold sweep on held-out test set -----------------
proba = np.clip(final_model.predict_proba(X_te_enc)[:, 1], 1e-9, 1 - 1e-9)
rows = []
for th in thresholds:
    pred = (proba >= th).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    specificity = tn / (tn + fp) if (tn + fp) else 0.0
    rows.append({
        'threshold': th,
        'accuracy': accuracy_score(y_test, pred),
        'precision': precision_score(y_test, pred, zero_division=0),
        'recall': recall_score(y_test, pred, zero_division=0),
        'f1': f1_score(y_test, pred, zero_division=0),
        'specificity': specificity,
        'tp': int(tp), 'fp': int(fp), 'tn': int(tn), 'fn': int(fn),
        'model': 'lightgbm',
        'cal_best_threshold': float(best_cal_thr),
    })

threshold_df = pd.DataFrame(rows).sort_values('threshold', ascending=False).reset_index(drop=True)
threshold_df

plt.figure(figsize=(10, 4.5))
for metric in ['accuracy', 'precision', 'recall', 'f1', 'specificity']:
    plt.plot(threshold_df['threshold'], threshold_df[metric], marker='o', label=metric)
plt.gca().invert_xaxis()
plt.ylim(0.0, 1.0)
plt.xlabel('Threshold')
plt.ylabel('Metric value')
plt.title('LightGBM (GPU) Metric Behavior Across Thresholds')
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
plt.show()
```

10.1.4 app.py

```
from __future__ import annotations

import os

# Must be set BEFORE numpy/sklearn/xgboost are imported.
# Prevents Intel MKL from calling os._exit() when its OpenMP thread pool
# clashes with XGBoost's own OpenMP threads inside the Streamlit process.
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st
```

```
import streamlit.components.v1 as components

from backend import (
    DEFAULT_CALIBRATOR_PATH,
    DEFAULT_MODEL_PATH,
    DEFAULT_PREPROCESSOR_PATH,
    build_input_values_from_widgets,
    feature_default,
    feature_range,
    group_feature_names,
    load_calibrator,
    load_input_feature_names,
    load_model_feature_names,
    load_model,
    load_preprocessor,
    make_input_frame,
    prepare_model_input,
    predict_with_venn_abers,
    predict_with_venn_abers_safe,
    RANGE_HINTS,
    unwrap_model,
)
from app_constants import (
    AUTO_COMPUTED_TOTAL_FIELDS,
    CONDITIONALLY_ALLOWED_NA_CODES,
    DISPLAY_LABEL_OVERRIDES,
    FEATURE_UNITS,
    FEATURE_DICTIONARY_ALIASES,
    FOOD_GROUP_COMPONENT_TOTALS,
    MISSING_INPUT_CODES,
    NO_HELP_FEATURES,
    VALUE_LABEL_OVERRIDES,
    VARIABLE_DEFINITION_OVERRIDES,
    get_dictionary_paths,
)
from counterfactuals import compute_counterfactuals, compute_counterfactuals_safe
from explainability import compute_explainability_safe, try_compute_lime, try_compute_shap
from styles import apply_global_styles


PLOTLY_STATIC_CONFIG = {"displayModeBar": False}


st.set_page_config(page_title="Hypertension Risk Assessment", page_icon="", layout="wide", initial_sidebar_state="collapsed")


PROJECT_ROOT = Path(__file__).resolve().parents[1]
DICTIONARY_PATHS = get_dictionary_paths(PROJECT_ROOT)

apply_global_styles()


@st.cache_resource(show_spinner=False)
def load_artifacts(model_path: str, calibrator_path: str, preprocessor_path: str):
    model = load_model(Path(model_path))
    calibrator = load_calibrator(Path(calibrator_path))
    preprocessor = load_preprocessor(Path(preprocessor_path))
    input_feature_names = load_input_feature_names(model, preprocessor)
    model_feature_names = load_model_feature_names(model, preprocessor)
    return model, calibrator, preprocessor, input_feature_names, model_feature_names


@st.cache_data(show_spinner=False)
def load_dataset_dictionaries() -> tuple[dict[str, str], dict[str, dict[str, str]]]:
    labels: dict[str, str] = {}
    value_labels: dict[str, dict[str, str]] = {}

    for path in DICTIONARY_PATHS:
        if not path.exists():
            continue

        text = path.read_text(encoding="utf-8", errors="ignore")
        lines = text.splitlines()
        if not lines:
            continue

        for line in lines[1:]:
            if not line.strip():
                continue

            parts = line.split(",", 4)
            if len(parts) < 5:
                continue

            _, variable_name, variable_label, var_value, value_label = [part.strip() for part in parts]
            if not variable_name:
                continue

            labels.setdefault(variable_name, variable_label)

            if var_value:
                if variable_name not in value_labels:
                    value_labels[variable_name] = {}
                value_labels[variable_name].setdefault(var_value, value_label)

    return labels, value_labels


def dictionary_name_candidates(feature_name: str) -> list[str]:
    candidates = [feature_name]

    if feature_name.startswith("epwt_fg"):
        candidates.append(feature_name.replace("epwt_", ""))

    if feature_name in FEATURE_DICTIONARY_ALIASES:
```

```
        candidates.extend(FEATURE_DICTIONARY_ALIASES[feature_name])

    return candidates


def field_help_text(feature_name: str, dictionary_labels: dict[str, str]) -> str | None:
    candidates = dictionary_name_candidates(feature_name)
    unit_text = None
    for candidate in candidates:
        if candidate in FEATURE_UNITS:
            unit_text = FEATURE_UNITS[candidate]
            break

    def _augment(base: str) -> str:
        text = base.strip()
        if unit_text and "(" not in text:
            text += f" Unit: {unit_text}."
        if feature_name in RANGE_HINTS:
            fmin, fmax, _ = RANGE_HINTS[feature_name]
            text += f" Typical entry range: {fmin:g} to {fmax:g}."
        return text

    for candidate in candidates:
        if candidate in VARIABLE_DEFINITION_OVERRIDES:
            return _augment(VARIABLE_DEFINITION_OVERRIDES[candidate])

    for candidate in candidates:
        if candidate in dictionary_labels and dictionary_labels[candidate]:
            return _augment(dictionary_labels[candidate])
    return None


def field_value_labels(feature_name: str, dictionary_value_labels: dict[str, dict[str, str]]) -> dict[str, str]:
    candidates = dictionary_name_candidates(feature_name)
    for candidate in candidates:
        if candidate in VALUE_LABEL_OVERRIDES:
            return VALUE_LABEL_OVERRIDES[candidate]

    for candidate in candidates:
        if candidate in dictionary_value_labels and dictionary_value_labels[candidate]:
            return dictionary_value_labels[candidate]

    return {}


def _sorted_value_label_keys(value_label_map: dict[str, str]) -> list[str]:
    def _key(value: str):
        try:
            return (0, int(float(value)))
        except ValueError:
            return (1, value)

    return sorted(value_label_map.keys(), key=_key)


def _clean_label_text(text: str) -> str:
    return text.split(":", 1)[0].strip().replace("  ", " ")


def field_display_label(feature_name: str, dictionary_labels: dict[str, str]) -> str:
    candidates = dictionary_name_candidates(feature_name)
    for candidate in candidates:
        if candidate in DISPLAY_LABEL_OVERRIDES:
            return DISPLAY_LABEL_OVERRIDES[candidate]

    for candidate in candidates:
        if candidate in dictionary_labels and dictionary_labels[candidate]:
            return _clean_label_text(dictionary_labels[candidate])

    help_text = field_help_text(feature_name, dictionary_labels)
    if help_text:
        return _clean_label_text(help_text)

    fallback = feature_name.replace("epwt_", "").replace("_", " ").strip()
    return fallback.title()


def field_label_with_unit(feature_name: str, dictionary_labels: dict[str, str]) -> str:
    label = field_display_label(feature_name, dictionary_labels)
    if "gram" in label.lower():
        return label
    unit_text = None
    for candidate in dictionary_name_candidates(feature_name):
        if candidate in FEATURE_UNITS:
            unit_text = FEATURE_UNITS[candidate]
            break
    if not unit_text:
        return label
    if f"({unit_text.lower()})" in label.lower():
        return label
    return f"{label} ({unit_text})"

def is_missing_input_value(value: Any) -> bool:
    if value is None:
        return True
    try:
        numeric_value = float(value)
    except (TypeError, ValueError):
        return False
    return np.isnan(numeric_value) or numeric_value in MISSING_INPUT_CODES


def is_conditionally_allowed_na_value(feature_name: str, value: Any) -> bool:
    allowed_codes = CONDITIONALLY_ALLOWED_NA_CODES.get(feature_name)
    if not allowed_codes:
        return False
```

```
    try:
        numeric_value = float(value)
    except (TypeError, ValueError):
        return False
    return not np.isnan(numeric_value) and numeric_value in allowed_codes


def render_quick_identity_fields(
    widget_values: dict[str, float],
    dictionary_labels: dict[str, str],
    dictionary_value_labels: dict[str, dict[str, str]],
    feature_names: list[str],
    rendered_feature_names: set[str],
):
    top_cols = st.columns(3)
    if "age" in feature_names:
        with top_cols[0]:
            age_text = st.text_input(
                field_label_with_unit("age", dictionary_labels),
                value=st.session_state.get("input_age_text", ""),
                key="input_age_text",
                placeholder="missing",
            )
            if age_text is None or not str(age_text).strip():
                widget_values["age"] = float("nan")
            else:
                try:
                    age_value = int(round(float(age_text)))
                except ValueError:
                    st.warning("Please enter a valid numeric age or leave it blank.")
                    widget_values["age"] = float("nan")
                else:
                    minimum, maximum, _ = feature_range("age")
                    if age_value < int(round(minimum)) or age_value > int(round(maximum)):
                        st.warning(f"Age must be between {int(round(minimum))} and {int(round(maximum))}.")
                        widget_values["age"] = float("nan")
                    else:
                        widget_values["age"] = float(age_value)
            rendered_feature_names.add("age")
    else:
        with top_cols[0]:
            widget_values["age"] = st.number_input("Age", min_value=0, max_value=120, value=40, step=1)
            rendered_feature_names.add("age")

    if "sex" in feature_names:
        with top_cols[1]:
            sex_choice = st.selectbox(
                "Sex",
                options=[None, 1, 2],
                index=0,
                format_func=lambda v: "missing" if v is None else ("Male" if v == 1 else "Female"),
            )
            widget_values["sex"] = float("nan") if sex_choice is None else float(sex_choice)
            rendered_feature_names.add("sex")
    else:
        with top_cols[1]:
            sex_choice = st.selectbox(
                "Sex",
                options=[None, 1, 2],
                index=0,
                format_func=lambda v: "missing" if v is None else ("Male" if v == 1 else "Female"),
            )
            widget_values["sex"] = float("nan") if sex_choice is None else float(sex_choice)
            rendered_feature_names.add("sex")

    if "ethnicity" in feature_names:
        with top_cols[2]:
            widget_values["ethnicity"] = render_number_input("ethnicity", dictionary_labels, dictionary_value_labels,
                  show_help=False)
            rendered_feature_names.add("ethnicity")


def _format_numeric_text(value: float, step: float) -> str:
    if not np.isfinite(float(value)):
        return ""
    step_text = format(float(step), "f").rstrip("0").rstrip(".")
    decimals = 0
    if "." in step_text:
        decimals = len(step_text.split(".", 1)[1])
    return f"{float(value):.{decimals}f}" if decimals else str(int(round(float(value))))


def render_editable_numeric_input(
    label: str,
    minimum: float,
    maximum: float,
    default_value: float,
    step: float,
    widget_key: str,
    help_text: str | None,
):
    default_text = _format_numeric_text(float(default_value), float(step))
    existing_text = str(st.session_state.get(widget_key, default_text)).strip()

    text_value = st.text_input(
        label,
        value=existing_text if existing_text != "" else default_text,
        key=widget_key,
        help=help_text,
        placeholder=default_text,
    )

    cleaned_text = text_value.strip()
    if cleaned_text == "":
        return float(default_value)
```

```
    try:
        numeric_value = float(cleaned_text)
    except ValueError:
        st.warning(f"Please enter a valid numeric value for {label}.")
        return float("nan")

    if numeric_value < float(minimum) or numeric_value > float(maximum):
        st.warning(f"{label} must be between {minimum:g} and {maximum:g}.")
        return float("nan")

    return float(numeric_value)


def render_number_input(
    feature_name: str,
    dictionary_labels: dict[str, str],
    dictionary_value_labels: dict[str, dict[str, str]],
    show_help: bool = True,
):
    minimum, maximum, step = feature_range(feature_name)
    default_value = feature_default(feature_name)
    raw_help = field_help_text(feature_name, dictionary_labels)
    help_text = None if not show_help or feature_name in NO_HELP_FEATURES else raw_help
    display_label = field_label_with_unit(feature_name, dictionary_labels)
    widget_key = f"input_{feature_name}"
    is_food_group_field = feature_name.startswith("fg") or feature_name.startswith("epwt_fg")
    is_total_field = feature_name.startswith("Total_")
    force_numeric_input = is_food_group_field or is_total_field

    value_label_map = field_value_labels(feature_name, dictionary_value_labels)

    if feature_name == "age":
        age_text = st.text_input(
            display_label,
            value=st.session_state.get("input_age_text", ""),
            key="input_age_text",
            help=help_text,
            placeholder="missing",
        )
        if age_text is None or not age_text.strip():
            return float("nan")
        try:
            age_value = int(round(float(age_text)))
        except ValueError:
            st.warning("Please enter a valid numeric age or leave it blank.")
            return float("nan")
        if age_value < int(round(minimum)) or age_value > int(round(maximum)):
            st.warning(f"Age must be between {int(round(minimum))} and {int(round(maximum))}.")
            return float("nan")
        return float(age_value)

    if feature_name in AUTO_COMPUTED_TOTAL_FIELDS:
        st.number_input(
            display_label,
            min_value=float(minimum),
            max_value=float(maximum),
            value=float(default_value),
            step=float(step),
            format="%.2f",
            help=(help_text + " Auto-computed from dietary components above when Predict is pressed.") if help_text else "Auto
                -computed from dietary components above when Predict is pressed.",
            disabled=True,
        )
        st.caption("Auto-computed from prior dietary components above.")
        return float(default_value)

    if value_label_map and len(value_label_map) <= 30 and not force_numeric_input:
        if feature_name == "ethnicity":
            option_values_eth = _sorted_value_label_keys(value_label_map)
            selected_eth = st.selectbox(
                display_label,
                options=option_values_eth,
                index=option_values_eth.index("0") if "0" in option_values_eth else 0,
                key=f"input_{feature_name}",
                help=help_text,
                format_func=lambda choice: f"{choice} - {value_label_map.get(choice, '')}".strip(" -"),
            )
            try:
                return float(int(float(selected_eth)))
            except ValueError:
                return 0.0

        option_values: list[str | None] = [None] + _sorted_value_label_keys(value_label_map)

        def _format_choice(choice: str | None) -> str:
            if choice is None:
                return "missing"
            return f"{choice} - {value_label_map.get(choice, '')}".strip(" -")

        selected = st.selectbox(
            display_label,
            options=option_values,
            index=0,
            key=f"input_{feature_name}",
            help=help_text,
            format_func=_format_choice,
        )
        if selected is None:
            return float("nan")
        try:
            return float(int(float(selected)))
        except ValueError:
            return float("nan")

    numeric_value = render_editable_numeric_input(
        label=display_label,
```

```
            minimum=float(minimum),
            maximum=float(maximum),
            default_value=float(default_value),
            step=float(step),
            widget_key=widget_key,
            help_text=help_text,
        )

        return float(numeric_value)


    def render_behavioral_selectors(dictionary_labels: dict[str, str], dictionary_value_labels: dict[str, dict[str, str]]):

        def _optional_code(
            variable_name: str,
            options: list[int],
            key: str,
            fallback_help: str,
            *,
            auto_value: int | None = None,
            disabled: bool = False,
        ):
            value_label_map = VALUE_LABEL_OVERRIDES.get(variable_name, dictionary_value_labels.get(variable_name, {}))
            variable_help = dictionary_labels.get(variable_name, fallback_help)

            option_values: list[int | None] = [None] + options

            def _format_choice(choice: int | None) -> str:
                if choice is None:
                    return "missing"
                label = value_label_map.get(str(choice), "")
                if "not applicable" in label.lower():
                    return "N/A - Not Applicable"
                if label:
                    return f"{choice} - {label}"
                return str(choice)

            # For disabled/auto fields: render a display-only selectbox with a
            # run-scoped key (incorporates the auto_value so it reinitialises when
            # the cascade changes it) and return the auto_value directly.  This
            # avoids the session-state persistence problem where a stale None from
            # a prior run overrides the computed auto_value.
            if auto_value is not None and disabled:
                display_key = f"{key}_locked_{auto_value}"
                _sel_index = option_values.index(auto_value) if auto_value in option_values else 0
                st.selectbox(
                    field_label_with_unit(variable_name, dictionary_labels),
                    options=option_values,
                    index=_sel_index,
                    key=display_key,
                    format_func=_format_choice,
                    help=variable_help,
                    disabled=True,
                )
                return float(int(auto_value))

            # Normal user-editable field: persist selection across reruns via key.
            if key not in st.session_state or st.session_state[key] not in option_values:
                _sel_index = 0
            else:
                _sel_index = option_values.index(st.session_state[key])

            selected = st.selectbox(
                field_label_with_unit(variable_name, dictionary_labels),
                options=option_values,
                index=_sel_index,
                key=key,
                format_func=_format_choice,
                help=variable_help,
                disabled=False,
            )
            return float("nan") if selected is None else float(int(selected))

        def _is_code(value: float, target: int) -> bool:
            return not (value is None or (isinstance(value, float) and np.isnan(value))) and int(value) == target

        smoke_left, smoke_mid, smoke_right = st.columns(3)
        with smoke_left:
            smoke_status = _optional_code(
                variable_name="smoke_status",
                options=[0, 1, 2],
                key="raw_smoke_status",
                fallback_help="Smoking status code.",
            )

        smoke_current_options = [0, 1, 2, 3, 888888]
        smoke_history_options = [0, 1, 2, 3, 4, 5, 888888]
        smoke_current_auto: int | None = None
        smoke_history_auto: int | None = None
        smoke_current_disabled = False
        smoke_history_disabled = False

        if _is_code(smoke_status, 0):
            smoke_current_auto = 0
            smoke_history_auto = 0
            smoke_current_disabled = True
            smoke_history_disabled = True
        elif _is_code(smoke_status, 1):
            smoke_current_options = [1, 2, 3]
            smoke_history_auto = 888888
            smoke_history_disabled = True
        elif _is_code(smoke_status, 2):
            smoke_current_auto = 0
            smoke_history_options = [1, 2, 3, 4, 5]
            smoke_current_disabled = True
            smoke_history_disabled = False
```

```
with smoke_mid:
    current_smoking = _optional_code(
        variable_name="current_smoking",
        options=smoke_current_options,
        key="raw_current_smoking",
        fallback_help="Current smoking code used by the engineering logic.",
        auto_value=smoke_current_auto,
        disabled=smoke_current_disabled,
    )

if smoke_history_auto is None and not smoke_history_disabled and _is_code(current_smoking, 1):
    smoke_history_auto = 1
    smoke_history_disabled = True
if smoke_history_auto is None and not smoke_history_disabled and _is_code(current_smoking, 2):
    smoke_history_auto = 2
    smoke_history_disabled = True
if smoke_history_auto is None and not smoke_history_disabled and _is_code(current_smoking, 3):
    smoke_history_auto = 3
    smoke_history_disabled = True

with smoke_right:
    ever_smk = _optional_code(
        variable_name="ever_smk",
        options=smoke_history_options,
        key="raw_ever_smk",
        fallback_help="Ever-smoking history code.",
        auto_value=smoke_history_auto,
        disabled=smoke_history_disabled,
    )

alc_left, alc_mid, alc_right = st.columns(3)
with alc_left:
    alcohol_status = _optional_code(
        variable_name="alcohol_status",
        options=[0, 1, 2],
        key="raw_alcohol_status",
        fallback_help="Alcohol status code.",
    )

alcohol_options = [0, 1, 2, 888888]
con_alcohol_options = [0, 1, 999999]
drnk_30days_options = [0, 1, 999999]
alcohol_auto: int | None = None
con_alcohol_auto: int | None = None
drnk_30days_auto: int | None = None
binge_auto: int | None = None
alcohol_disabled = False
con_alcohol_disabled = False
drnk_30days_disabled = False
binge_disabled = False

if _is_code(alcohol_status, 0):
    alcohol_auto = 0
    con_alcohol_auto = 999999
    drnk_30days_auto = 999999
    binge_auto = 99
    alcohol_disabled = True
    con_alcohol_disabled = True
    drnk_30days_disabled = True
    binge_disabled = True
elif _is_code(alcohol_status, 1):
    alcohol_auto = 1
    con_alcohol_auto = 1
    drnk_30days_options = [0, 1]
    alcohol_disabled = True
    con_alcohol_disabled = True
elif _is_code(alcohol_status, 2):
    alcohol_auto = 1
    alcohol_disabled = True
    drnk_30days_auto = 999999
    binge_auto = 99
    drnk_30days_disabled = True
    binge_disabled = True

with alc_mid:
    alcohol = _optional_code(
        variable_name="alcohol",
        options=alcohol_options,
        key="raw_alcohol",
        fallback_help="Ever alcohol-use code used in fallback logic.",
        auto_value=alcohol_auto,
        disabled=alcohol_disabled,
    )
with alc_right:
    con_alcohol = _optional_code(
        variable_name="con_alcohol",
        options=con_alcohol_options,
        key="raw_con_alcohol",
        fallback_help="Current alcohol indicator for fallback logic.",
        auto_value=con_alcohol_auto,
        disabled=con_alcohol_disabled,
    )

binge_left, binge_mid, _ = st.columns(3)
with binge_left:
    drnk_30days = _optional_code(
        variable_name="drnk_30days",
        options=drnk_30days_options,
        key="raw_drnk_30days",
        fallback_help="Drank an alcoholic drink within the past 30 days.",
        auto_value=drnk_30days_auto,
        disabled=drnk_30days_disabled,
    )

if not binge_disabled and _is_code(drnk_30days, 0):
```

```
            binge_auto = 0
            binge_disabled = True
        elif not binge_disabled and _is_code(drnk_30days, 999999):
            binge_auto = 99
            binge_disabled = True

        with binge_mid:
            binge_drink = _optional_code(
                variable_name="binge_drink",
                options=[0, 1, 99],
                key="raw_binge_drink",
                fallback_help="Binge-drink indicator used to set highest engineered level.",
                auto_value=binge_auto,
                disabled=binge_disabled,
            )


        return {
            "smoke_status": smoke_status,
            "current_smoking": current_smoking,
            "ever_smk": ever_smk,
            "alcohol_status": alcohol_status,
            "alcohol": alcohol,
            "con_alcohol": con_alcohol,
            "drnk_30days": drnk_30days,
            "binge_drink": binge_drink,
        }


    def render_anthro_origin_inputs(dictionary_labels: dict[str, str], rendered_feature_names: set[str]):
        raw_feature_order = ["weight", "height", "waist", "hip"]
        missing_raw = [feature for feature in raw_feature_order if feature not in rendered_feature_names]
        if not missing_raw:
            return {}

        cols = st.columns(3)
        values: dict[str, float] = {}

        for index, feature in enumerate(missing_raw):
            with cols[index % 3]:
                if feature == "weight":
                    values["weight"] = render_editable_numeric_input(
                        label=field_label_with_unit("weight", dictionary_labels),
                        minimum=0.0,
                        maximum=300.0,
                        default_value=0.0,
                        step=0.1,
                        widget_key="raw_weight",
                        help_text=None,
                    )
                elif feature == "height":
                    values["height"] = render_editable_numeric_input(
                        label=field_label_with_unit("height", dictionary_labels),
                        minimum=0.0,
                        maximum=260.0,
                        default_value=0.0,
                        step=0.1,
                        widget_key="raw_height",
                        help_text=None,
                    )
                elif feature == "waist":
                    values["waist"] = render_editable_numeric_input(
                        label=field_label_with_unit("waist", dictionary_labels),
                        minimum=0.0,
                        maximum=200.0,
                        default_value=0.0,
                        step=0.1,
                        widget_key="raw_waist",
                        help_text=None,
                    )
                elif feature == "hip":
                    values["hip"] = render_editable_numeric_input(
                        label=field_label_with_unit("hip", dictionary_labels),
                        minimum=0.0,
                        maximum=200.0,
                        default_value=0.0,
                        step=0.1,
                        widget_key="raw_hip",
                        help_text=None,
                    )

        return values


    def make_gauge_chart(score: float):
        threshold_pct = RISK_THRESHOLD * 100.0
        fig = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=score * 100.0,
                number={"suffix": "%", "font": {"size": 42, "color": "#0f172a"}},
                gauge={
                    "axis": {"range": [0, 100], "tickwidth": 1, "tickcolor": "#94a3b8"},
                    "bar": {"color": "#dc2626"},
                    "steps": [
                        {"range": [0, threshold_pct], "color": "#dcfce7"},
                        {"range": [threshold_pct, 100], "color": "#fee2e2"},
                    ],
                    "threshold": {"line": {"color": "#111827", "width": 4}, "thickness": 0.75, "value": threshold_pct},
                },
            )
        )
        fig.update_layout(height=290, margin=dict(l=18, r=18, t=10, b=10), paper_bgcolor="rgba(0,0,0,0)")
        return fig
```

```
def make_carla_cf_chart(cf_df: pd.DataFrame):
    """Divergent bar chart in the style of CARLA reference figures.

    Each bar shows the delta (Suggested - Current) for a feature.
    Blue bars = reduce the value; amber bars = increase the value.
    Bar labels show the risk reduction percentage.
    """
    features = cf_df["Feature"].tolist()
    currents = cf_df["Current Value"].tolist()
    suggesteds = cf_df["Suggested Value"].tolist()
    deltas = [s - c for s, c in zip(suggesteds, currents)]
    reductions = cf_df["Risk Reduction"].tolist()
    colors = ["#2563eb" if d < 0 else "#f59e0b" for d in deltas]

    bar_labels = [
        f"-{r}  ({cur:.1f} -> {sug:.1f})"
        for r, cur, sug in zip(reductions, currents, suggesteds)
    ]

    # Pad the x-axis so outside labels are never clipped regardless of bar length.
    # Estimate ~7 px per character at 12 px font; convert to data units using the
    # axis span so the padding scales with the actual data range.
    max_abs = max((abs(d) for d in deltas), default=1.0) or 1.0
    longest_label = max((len(lbl) for lbl in bar_labels), default=10)
    # Add 55 % of the full span per side for short values; scale by label length.
    label_pad = max_abs * max(0.55, longest_label * 0.035)
    x_min = min(min(deltas), 0) - label_pad
    x_max = max(max(deltas), 0) + label_pad

    fig = go.Figure(
        go.Bar(
            x=deltas,
            y=features,
            orientation="h",
            marker_color=colors,
            text=bar_labels,
            textposition="outside",
            cliponaxis=False,
            hovertemplate=(
                "<b>%{y}</b><br>"
                "Current: %{customdata[0]:.2f}<br>"
                "Suggested: %{customdata[1]:.2f}<br>"
                "Change: %{x:+.2f}<br>"
                "Risk Reduction: %{customdata[2]}"
                "<extra></extra>"
            ),
            customdata=list(zip(currents, suggesteds, reductions)),
        )
    )
    fig.add_vline(x=0, line_width=1, line_color="#64748b")
    fig.update_layout(
        height=max(220, 36 * len(features)),
        margin=dict(l=10, r=20, t=30, b=10),
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(0,0,0,0)",
        xaxis=dict(
            title="Suggested - Current Value",
            range=[x_min, x_max],
            zeroline=False,
            showgrid=True,
            gridcolor="#e2e8f0",
        ),
        yaxis=dict(autorange="reversed", tickfont=dict(size=12)),
        showlegend=False,
    )
    return fig


def _food_group_prefix_and_index(name: str) -> tuple[str, int] | None:
    for prefix in ("epwt_fg", "fg"):
        if name.startswith(prefix):
            suffix = name[len(prefix):]
            return (prefix, int(suffix)) if suffix.isdigit() else None
    return None


def _food_group_is_component_total(name: str) -> bool:
    return (parsed := _food_group_prefix_and_index(name)) is not None and parsed[1] in FOOD_GROUP_COMPONENT_TOTALS


def _food_group_is_addend(name: str) -> bool:
    return (parsed := _food_group_prefix_and_index(name)) is not None and parsed[1] in {num for nums in
          FOOD_GROUP_COMPONENT_TOTALS.values() for num in nums}


def _food_group_addend_names_for_total(total_name: str, available_names: set[str]) -> list[str]:
    if (parsed := _food_group_prefix_and_index(total_name)) is None:
        return []
    prefix, total_idx = parsed
    return [f"{prefix}{idx}" for idx in FOOD_GROUP_COMPONENT_TOTALS.get(total_idx, []) if f"{prefix}{idx}" in available_names]


def apply_dietary_derived_totals(widget_values: dict[str, float], feature_names: list[str]) -> dict[str, float]:
    values = dict(widget_values)
    feature_name_set = set(feature_names)

    # Enforce dictionary-defined subtotal fields from their component food-group inputs.
    for name in feature_names:
        if not _food_group_is_component_total(name):
            continue
        addends = _food_group_addend_names_for_total(name, feature_name_set)
        if not addends:
            continue
        total_val = 0.0
        for addend_name in addends:
            raw = values.get(addend_name)
```

```
            if raw is None or (isinstance(raw, float) and np.isnan(raw)):
                continue
            total_val += float(raw)
        values[name] = float(total_val)

    food_group_names = [
        name
        for name in feature_names
        if (name.startswith("epwt_fg") or name.startswith("fg")) and name not in {"fg", "epwt_fg"}
    ]
    food_group_total = 0.0
    for name in food_group_names:
        # Sum subtotal groups and standalone groups, but exclude addends already represented by subtotal groups.
        if _food_group_is_addend(name):
            continue
        raw = values.get(name)
        if raw is None or (isinstance(raw, float) and np.isnan(raw)):
            continue
        food_group_total += float(raw)

    if "Total_FoodIntake" in feature_names:
        values["Total_FoodIntake"] = float(food_group_total)
    if "Total_Food_epwt" in feature_names:
        values["Total_Food_epwt"] = float(food_group_total)

    cho = float(values.get("Total_CHO", 0.0) or 0.0)
    fat = float(values.get("Total_Fat", 0.0) or 0.0)

    # Approximate protein from protein-heavy food groups when explicit protein isn't available.
    protein_group_suffixes = {"7", "14", "15", "16", "17", "18", "19", "20", "21"}
    protein_source_total = 0.0
    for name in food_group_names:
        if _food_group_is_addend(name):
            continue
        suffix = ""
        if name.startswith("epwt_fg"):
            suffix = name.replace("epwt_fg", "")
        elif name.startswith("fg"):
            suffix = name.replace("fg", "")
        if suffix in protein_group_suffixes:
            raw = values.get(name)
            if raw is None or (isinstance(raw, float) and np.isnan(raw)):
                continue
            protein_source_total += float(raw)

    protein_val = 0.0
    if "Total_Protein" in feature_names and "Total_Protein" in values:
        protein_val = float(values.get("Total_Protein", 0.0) or 0.0)
    elif "Total_Prot" in feature_names and "Total_Prot" in values:
        protein_val = float(values.get("Total_Prot", 0.0) or 0.0)
    else:
        protein_val = float(protein_source_total * 0.18)

    if cho == 0.0 and fat == 0.0 and food_group_total > 0.0:
        cho = float(food_group_total * 0.30)
        fat = float(food_group_total * 0.04)

    energy_val = float((4.0 * cho) + (4.0 * protein_val) + (9.0 * fat))
    if "Total_Energy" in feature_names:
        values["Total_Energy"] = energy_val
    if "Total_Ener" in feature_names:
        values["Total_Ener"] = energy_val

    if "Total_Protein" in feature_names:
        values["Total_Protein"] = protein_val
    if "Total_Prot" in feature_names:
        values["Total_Prot"] = protein_val

    if "Total_Protein" in feature_names and "Total_Protein" not in values and "Total_Prot" in values:
        values["Total_Protein"] = float(values.get("Total_Prot", 0.0) or 0.0)
    if "Total_Prot" in feature_names and "Total_Prot" not in values and "Total_Protein" in values:
        values["Total_Prot"] = float(values.get("Total_Protein", 0.0) or 0.0)

    return values


RISK_THRESHOLD = 0.35

def risk_label_from_score(probability: float) -> str:
    return "At Risk of Hypertension" if probability > RISK_THRESHOLD else "Not at Risk"


def _model_column_source_input_feature(model_column: str, input_features: list[str]) -> str | None:
    # Map transformed model columns (e.g., num__age, cat__ethnicity_1.0) back to raw input features.
    token = model_column.split("__", 1)[1] if "__" in model_column else model_column
    lowered_features = sorted({str(name).lower() for name in input_features}, key=len, reverse=True)
    token_lc = token.lower()

    if token_lc in lowered_features:
        return token_lc

    for feature_name in lowered_features:
        if token_lc.startswith(f"{feature_name}_"):
            return feature_name

    # Alias lookup: model may use short names (Total_Ribo) while the input
    # form uses the full survey name (Total_Riboflavin).  Check both directions.
    for key, aliases in FEATURE_DICTIONARY_ALIASES.items():
        group = {key.lower()} | {a.lower() for a in aliases}
        if token_lc in group:
            for member_lc in group:
                if member_lc in lowered_features:
                    return member_lc

    return None
```

```
def resolve_explainability_columns(
    model_columns: list[str],
    input_features: list[str],
    rendered_input_features: set[str],
) -> list[str]:
    rendered_lc = {str(name).lower() for name in rendered_input_features}
    return [str(c) for c in model_columns if (s := _model_column_source_input_feature(str(c), input_features)) and s in
          rendered_lc]


def _raw_value_for_feature(model_col: str, input_features: list[str], widget_values: dict) -> float | None:
    """Return the user's raw widget value for a model column, or None if not found."""
    source = _model_column_source_input_feature(model_col, input_features)
    if source is None:
        return None
    # widget_values keys may be mixed-case or prefixed with "input_".
    # Use a case-insensitive lookup so "total_riboflavin" matches "Total_Riboflavin".
    wv_lower = {k.lower(): v for k, v in widget_values.items()}
    for key in (source, f"input_{source}"):
        val = wv_lower.get(key.lower())
        if val is not None:
            try:
                return float(val)
            except (TypeError, ValueError):
                return None
    return None


def _fmt_raw(value: float | None) -> str:
    if value is None:
        return "-"
    if value == int(value):
        return str(int(value))
    return f"{value:.4g}"


def _parse_lime_feature(rule: str, input_features: list[str]) -> str | None:
    """Extract which input feature a LIME rule string refers to."""
    rule_lc = rule.lower()
    # Try longest match first so e.g. 'epwt_fg1' beats 'fg1'
    for name in sorted(input_features, key=len, reverse=True):
        if name.lower() in rule_lc:
            return name
    return None


def _resolve_lime_source_feature(
    model_like_name: str,
    rule_text: str,
    input_features: list[str],
) -> str | None:
    """Map LIME feature text back to a raw user-input feature when possible."""
    if model_like_name in input_features:
        return model_like_name
    mapped = _model_column_source_input_feature(model_like_name, input_features)
    if mapped is not None:
        return mapped
    parsed = _parse_lime_feature(rule_text, input_features)
    if parsed is not None:
        return parsed
    return None


print("[DEBUG] >>> script top - new rerun starting", flush=True)

if "show_form" not in st.session_state:
    st.session_state.show_form = False
if "scored" not in st.session_state:
    st.session_state.scored = False


model_path = str(DEFAULT_MODEL_PATH)
calibrator_path = str(DEFAULT_CALIBRATOR_PATH)
preprocessor_path = str(DEFAULT_PREPROCESSOR_PATH)

with st.sidebar:
    st.subheader("Model artifacts")
    model_path = st.text_input("Stand-in model joblib", value=model_path)
    preprocessor_path = st.text_input("Preprocessor joblib", value=preprocessor_path)
    calibrator_path = st.text_input("Venn-Abers calibrator", value=calibrator_path)
    st.caption("Replace these paths when your final exported artifacts are ready.")
    st.divider()
    st.caption("Form is driven from the model's `feature_names_in_` metadata.")
    st.caption("Install `shap` and `lime` to enable explainability outputs.")


model, calibrator, preprocessor, input_feature_names, model_feature_names = load_artifacts(
    model_path,
    calibrator_path,
    preprocessor_path,
)
explain_model = unwrap_model(model)
feature_names = input_feature_names
grouped_features = group_feature_names(feature_names)
dictionary_labels, dictionary_value_labels = load_dataset_dictionaries()


if not st.session_state.show_form:
    st.markdown(
        """
        <div class="landing-hero">
          <div class="landing-badge">Philippine 2015 DOST-FNRI Thesis Model</div>
                    <h1>HRP-AI: A WEB APPLICATION FOR HYPERTENSION<br>RISK PREDICTION WITH CALIBRATED<br>EXPLAINABLE AI</h1>
          <p class="landing-sub" style="text-align:center;margin:0 auto;width:100%;">
                       A thesis-focused decision support interface for estimating hypertension risk using dietary,
```

```
                            anthropometric, and clinical inputs, with calibrated probabilities and transparent
                            explainability outputs. The AI model was trained on the DOST-FNRI 2015 NNS dataset.
              </p>
            </div>
            """,
            unsafe_allow_html=True,
        )

        _, landing_cta_col, _ = st.columns([1.2, 2.6, 1.2])
        with landing_cta_col:
            st.markdown(
                            """
                            <div style="width:100%;margin:0.5rem 0 2.6rem;background:rgba(255,255,255,0.82);
                                                border:1px solid rgba(24,33,47,0.09);border-radius:22px;
                                                padding:1.6rem 2rem;box-shadow:0 8px 30px rgba(15,23,42,0.07);">
                                <div style="text-align:center;font-size:0.74rem;text-transform:uppercase;
                                                    letter-spacing:0.15em;color:#1e293b;margin-bottom:1rem;">Model Performance
                                                        </div>
                                <div style="display:flex;justify-content:center;gap:3rem;margin-bottom:1.2rem;">
                                    <div style="text-align:center;">
                                        <div style="font-size:2rem;font-weight:800;color:#0f172a;">76.9%</div>
                                        <div style="font-size:0.82rem;color:#1e293b;margin-top:0.2rem;">Accuracy</div>
                                    </div>
                                    <div style="text-align:center;">
                                        <div style="font-size:2rem;font-weight:800;color:#dc2626;">76.5%</div>
                                        <div style="font-size:0.82rem;color:#1e293b;margin-top:0.2rem;">Recall</div>
                                    </div>
                                    <div style="text-align:center;">
                                        <div style="font-size:2rem;font-weight:800;color:#0f172a;">84.7%</div>
                                        <div style="font-size:0.82rem;color:#1e293b;margin-top:0.2rem;">AUC</div>
                                    </div>
                                </div>
                                <div style="font-size:0.83rem;color:#1f2937;line-height:1.65;text-align:justify;text-justify:inter
                                      -word;">
                                    <strong>Accuracy</strong> is the share of all individuals the model classifies correctly.
                                    <strong>Recall</strong> (sensitivity) is the share of true hypertensive cases the model
                                            catches -
                                    prioritised here because missing a true positive in health screening carries greater risk than
                                           a false alarm.
                                    <strong>AUC</strong> reflects overall discriminative power across all thresholds.
                                    Metrics are from the best updated EXP3 run: XGBoost with class-weight balancing (base
                                            calibration) evaluated on the held-out test set.
                                </div>
                            </div>
                            """,
                            unsafe_allow_html=True,
            )

            _begin = st.button("Begin Assessment", width='stretch', type="primary")
        if _begin:
            st.session_state.show_form = True
            st.rerun()

        st.markdown(
            """
            <div class="landing-disclaimer">
              For research and academic use only. This interface does not replace clinical judgment.
            </div>
            """,
            unsafe_allow_html=True,
        )
    else:
        nav_left, nav_right = st.columns([5, 1])
        with nav_left:
            st.markdown(
                "<div style='display:flex;align-items:center;gap:0.7rem;padding:0.5rem 0;'>"
                "<span style='font-size:1.2rem;font-weight:700;color:#0f172a;'>Hypertension Risk Assessment</span>"
                "</div>",
                unsafe_allow_html=True,
            )
        with nav_right:
            if st.button("Back Home"):
                st.session_state.show_form = False
                st.session_state.scored = False
                st.rerun()

        st.divider()

        st.markdown(
            "<div id='input-section' class='section-header'>"
            "<h2>Patient Details</h2>"
            "<p>Complete all sections below."
            " Click Predict My Risk to see the output below this form on the same page.</p>"
            "</div>",
            unsafe_allow_html=True,
        )
        st.caption("Enter the requested values below. Hover the ? icon beside each field to view hints and definitions.")

        # No st.form wrapper - inputs update live so dietary totals auto-compute on every re-render.
        submitted = False
        widget_values: dict[str, float] = {}
        dietary_addend_fields: list[str] = []
        required_field_labels: dict[str, str] = {}
        rendered_feature_names: set[str] = set()
        engineered_names = {"BMI", "bmi", "whr", "alcohol_level", "smoking_level",
                            "fe_smoking_level", "fe_alcohol_level"}
        has_engineered_features = any(
            name in engineered_names or name.startswith("alcohol_level_") or name.startswith("smoking_level_")
            for name in feature_names
        )

        st.markdown("#### Quick Identity and Key Inputs")
        render_quick_identity_fields(widget_values, dictionary_labels, dictionary_value_labels, feature_names,
              rendered_feature_names)
        required_field_labels["age"] = field_display_label("age", dictionary_labels)
        required_field_labels["sex"] = field_display_label("sex", dictionary_labels)
```

```
if "ethnicity" in feature_names:
    required_field_labels["ethnicity"] = field_display_label("ethnicity", dictionary_labels)

if has_engineered_features:
    st.markdown("#### Behavioral (Smoking and Alcohol)")
    st.caption("Provide original smoking and alcohol variables. The backend computes engineered levels.")
    widget_values.update(render_behavioral_selectors(dictionary_labels, dictionary_value_labels))
    for feature_name in ("smoke_status", "current_smoking", "ever_smk", "alcohol_status", "alcohol", "con_alcohol", "
          drnk_30days", "binge_drink"):
        required_field_labels[feature_name] = field_display_label(feature_name, dictionary_labels)

if "Anthropometrics" in grouped_features:
    st.markdown("#### Anthropometrics")
    anthro_cols = st.columns(3)
    for index, feature_name in enumerate(grouped_features["Anthropometrics"]):
        if feature_name in engineered_names:
            continue
        if feature_name in rendered_feature_names:
            continue
        with anthro_cols[index % 3]:
            widget_values[feature_name] = render_number_input(feature_name, dictionary_labels, dictionary_value_labels,
                  show_help=False)
            rendered_feature_names.add(feature_name)
            required_field_labels[feature_name] = field_display_label(feature_name, dictionary_labels)

if has_engineered_features:
    st.caption("For engineered BMI and WHR, provide these origin anthropometric measurements.")
    widget_values.update(render_anthro_origin_inputs(dictionary_labels, rendered_feature_names))

if "Dietary pattern" in grouped_features:
    st.markdown("#### Dietary")
    st.caption("Enter each dietary value as your daily intake. Hover the ? icon on each field for definitions and examples
          .")
    dietary_features = list(grouped_features["Dietary pattern"])
    if "vita" in feature_names and "vita" not in dietary_features:
        dietary_features.append("vita")

    def _food_group_position_key(name: str) -> float | None:
        parsed = _food_group_prefix_and_index(name)
        if parsed is None:
            return None
        _, idx = parsed
        if idx in FOOD_GROUP_COMPONENT_TOTALS:
            return float(max(FOOD_GROUP_COMPONENT_TOTALS[idx])) + 0.5
        return float(idx)

    # Match dictionary-style ordering, with subtotal fields moved after their component addends.
    original_positions = {name: idx for idx, name in enumerate(dietary_features)}
    dietary_features = sorted(
        dietary_features,
        key=lambda name: (
            0 if _food_group_position_key(name) is not None else 1,
            _food_group_position_key(name) if _food_group_position_key(name) is not None else 10_000,
            original_positions[name],
        ),
    )

    dietary_input_features: list[str] = []
    for feature_name in dietary_features:
        if feature_name in rendered_feature_names:
            continue
        if feature_name in AUTO_COMPUTED_TOTAL_FIELDS:
            widget_values[feature_name] = float(feature_default(feature_name))
            rendered_feature_names.add(feature_name)
            continue
        dietary_input_features.append(feature_name)

    dietary_addend_fields = [
        name
        for name in dietary_input_features
        if _food_group_prefix_and_index(name) is not None and not _food_group_is_component_total(name)
    ]

    for start_index in range(0, len(dietary_input_features), 3):
        dietary_cols = st.columns(3)
        for offset, feature_name in enumerate(dietary_input_features[start_index:start_index + 3]):
            with dietary_cols[offset]:
                if _food_group_is_component_total(feature_name):
                    addend_names = _food_group_addend_names_for_total(feature_name, set(dietary_features))
                    subtotal_value = 0.0
                    for addend_name in addend_names:
                        raw = widget_values.get(addend_name)
                        if raw is None or (isinstance(raw, float) and np.isnan(raw)):
                            continue
                        subtotal_value += float(raw)

                    minimum, maximum, step = feature_range(feature_name)
                    display_label = field_label_with_unit(feature_name, dictionary_labels)
                    help_text = field_help_text(feature_name, dictionary_labels)
                    st.number_input(
                        display_label,
                        min_value=float(minimum),
                        max_value=float(maximum),
                        value=float(subtotal_value),
                        step=float(step),
                        format="%.2f",
                        disabled=True,
                        help=(help_text + " Auto-computed from the component food groups.") if help_text else "Auto-
                              computed from the component food groups.",
                    )
                    st.caption("Auto-summed from the food-group inputs above.")
                    widget_values[feature_name] = float(subtotal_value)
                else:
                    widget_values[feature_name] = render_number_input(feature_name, dictionary_labels,
                          dictionary_value_labels)
                rendered_feature_names.add(feature_name)
```

```
                required_field_labels[feature_name] = field_display_label(feature_name, dictionary_labels)
                st.markdown("<div style='height: 1.25rem;'></div>", unsafe_allow_html=True)

    _protein_group_sfx = {"7", "14", "15", "16", "17", "18", "19", "20", "21"}
    _live_food = 0.0
    _live_protein_src = 0.0
    for _fn in dietary_features:
        if _fn not in widget_values:
            continue
        _raw = widget_values[_fn]
        if _raw is None or (isinstance(_raw, float) and np.isnan(_raw)):
            continue
        _is_fg = _fn.startswith("epwt_fg") or (_fn.startswith("fg") and not _fn.startswith("epwt_"))
        if _is_fg:
            if _food_group_is_addend(_fn):
                continue
            _live_food += float(_raw)
            _sfx = _fn.replace("epwt_fg", "").replace("fg", "")
            if _sfx in _protein_group_sfx:
                _live_protein_src += float(_raw)
    _live_protein = _live_protein_src * 0.18
    _live_cho = float(widget_values.get("Total_CHO") or 0.0)
    _live_fat = float(widget_values.get("Total_Fat") or 0.0)
    if _live_cho == 0.0 and _live_fat == 0.0 and _live_food > 0.0:
        _live_cho = _live_food * 0.30
        _live_fat = _live_food * 0.04
    _live_energy = 4.0 * _live_cho + 4.0 * _live_protein + 9.0 * _live_fat
    # Store live values so submit handler picks them up
    for _alias in ("Total_FoodIntake", "Total_Food_epwt"):
        if _alias in feature_names:
            widget_values[_alias] = _live_food
    for _alias in ("Total_Energy", "Total_Ener"):
        if _alias in feature_names:
            widget_values[_alias] = _live_energy
    for _alias in ("Total_Protein", "Total_Prot"):
        if _alias in feature_names:
            widget_values[_alias] = _live_protein

    _tot_a, _tot_b, _tot_c = st.columns(3)
    with _tot_a:
        st.number_input("Total Food Intake (g)", value=round(_live_food, 1), disabled=True)
        st.caption("Auto-updates from the dietary values above.")
    with _tot_b:
        st.number_input("Total Energy (kcal)", value=round(_live_energy, 1), disabled=True)
        st.caption("Auto-updates from the dietary values above.")
    with _tot_c:
        st.number_input("Total Protein (g)", value=round(_live_protein, 1), disabled=True)
        st.caption("Auto-updates from the dietary values above.")

missing_field_labels = [
    label
    for feature_name, label in required_field_labels.items()
    if is_missing_input_value(widget_values.get(feature_name))
    and not is_conditionally_allowed_na_value(feature_name, widget_values.get(feature_name))
]
if missing_field_labels:
    missing_preview = ", ".join(missing_field_labels[:8])
    if len(missing_field_labels) > 8:
        missing_preview += f", and {len(missing_field_labels) - 8} more"
    st.warning(f"Complete all missing inputs before predicting: {missing_preview}.")

dietary_all_zero = False
if dietary_addend_fields:
    dietary_all_zero = all(
        float(widget_values.get(name, 0.0) or 0.0) == 0.0 for name in dietary_addend_fields
    )
    if dietary_all_zero:
        st.warning("Enter at least one non-zero food-group dietary intake value before predicting.")

st.markdown("<br>", unsafe_allow_html=True)
_, submit_col, _ = st.columns([1.3, 1.2, 1.3])
with submit_col:
    submitted = st.button("Predict My Risk", width='stretch', type="primary", disabled=bool(missing_field_labels) or
          dietary_all_zero)

st.html(
            """
            <script>
            (function() {
                const doc = window.parent.document;
                const submitButtons = Array.from(doc.querySelectorAll('button')).filter(
                    (btn) => btn.innerText && btn.innerText.trim() === 'Predict My Risk'
                );
                if (!submitButtons.length) return;

                const submitBtn = submitButtons[submitButtons.length - 1];
                const formContainer = doc;
                const selector = 'input:not([type="hidden"]):not([disabled]), textarea';
                const numericPlaceholderPattern = /^-?\\d+(?:\\.\\d+)?$/;

                function visibleFields() {
                    return Array.from(formContainer.querySelectorAll(selector)).filter((el) => el.offsetParent !== null);
                }

                function bindClearOnFocus(el) {
                    if (el.dataset.clearOnFocusBound === '1') return;
                    if (el.tagName !== 'INPUT' || el.type !== 'text') return;
                    const placeholder = (el.getAttribute('placeholder') || '').trim();
                    if (!numericPlaceholderPattern.test(placeholder)) return;
                    el.dataset.clearOnFocusBound = '1';
                    el.addEventListener('focus', function() {
                        el.value = '';
                    });
                    el.addEventListener('blur', function() {
                        if (el.value.trim() !== '') return;
                        el.value = placeholder;
```

```
                        el.dispatchEvent(new Event('input', { bubbles: true }));
                        el.dispatchEvent(new Event('change', { bubbles: true }));
                    });
                }

                visibleFields().forEach((el) => {
                    bindClearOnFocus(el);
                    if (el.dataset.enterNavBound === '1') return;
                    el.dataset.enterNavBound = '1';
                    el.addEventListener('keydown', function(e) {
                        if (e.key !== 'Enter' || e.shiftKey) return;
                        e.preventDefault();
                        const fields = visibleFields();
                        const idx = fields.indexOf(el);
                        if (idx >= 0 && idx < fields.length - 1) {
                            fields[idx + 1].focus();
                            if (typeof fields[idx + 1].select === 'function') {
                                fields[idx + 1].select();
                            }
                        } else {
                            submitBtn.click();
                        }
                    });
                });
            })();
            </script>
            """
    )

if submitted and not missing_field_labels and not dietary_all_zero:
    derived_widget_values = apply_dietary_derived_totals(widget_values, feature_names)
    input_values = build_input_values_from_widgets(feature_names, derived_widget_values)
    raw_input_frame = make_input_frame(feature_names, input_values)
    model_input_frame = prepare_model_input(raw_input_frame, preprocessor)
    print("[DEBUG] >>> calling predict_with_venn_abers_safe (subprocess)", flush=True)
    result = predict_with_venn_abers_safe(model, model_input_frame, calibrator)
    print(f"[DEBUG] >>> prediction done: calibrated_prob={result.calibrated_probability:.4f}", flush=True)
    st.session_state.scored = True
    st.session_state._result = result
    st.session_state._input_frame = model_input_frame
    st.session_state._model_feature_names = list(model_input_frame.columns)
    st.session_state._input_feature_names = feature_names
    st.session_state._rendered_input_features = sorted(rendered_feature_names)
    st.session_state._all_widget_values = dict(derived_widget_values)
    st.session_state._scroll_to_output = True
    st.session_state.pop("_exp_cache", None)   # force SHAP/LIME recompute for new prediction
    st.session_state.pop("_cf_cache", None)    # force counterfactuals recompute for new prediction

if st.session_state.scored and hasattr(st.session_state, "_result"):
    print("[DEBUG] >>> entering output section", flush=True)
    result = st.session_state._result
    input_frame = st.session_state._input_frame

    st.markdown('<div id="output-section"></div>', unsafe_allow_html=True)
    st.markdown(
        "<div class='section-header' style='margin-top:2.6rem;'>"
        "<h2>Output</h2>"
        "<p>The form stays above. Scroll up anytime to edit the inputs.</p>"
        "</div>",
        unsafe_allow_html=True,
    )

    score_pct = result.calibrated_probability * 100.0
    mapped_risk_label = risk_label_from_score(result.calibrated_probability)
    tier_css = "risk-at-risk" if result.calibrated_probability > RISK_THRESHOLD else "risk-not-at-risk"

    kpi_a, kpi_b, kpi_c = st.columns(3)
    with kpi_a:
        st.markdown(
            f"<div class='output-hero'><div class='oh-label'>Risk Score</div><div class='oh-value'>{score_pct:.1f}%</div><
                div class='oh-sub'>Calibrated probability</div></div>",
            unsafe_allow_html=True,
        )
    with kpi_b:
        if result.lower_bound != result.upper_bound:
            st.markdown(
                f"<div class='output-hero'><div class='oh-label'>Uncertainty Interval</div><div class='oh-value' style='
                    font-size:1.6rem;'>{result.lower_bound*100:.1f}% \u2013 {result.upper_bound*100:.1f}%</div><div
                    class='oh-sub'>Venn-Abers p0/p1 bounds</div></div>",
                unsafe_allow_html=True,
            )
        else:
            _margin = abs(result.calibrated_probability - RISK_THRESHOLD) * 100.0
            _margin_dir = "above" if result.calibrated_probability > RISK_THRESHOLD else "below"
            st.markdown(
                f"<div class='output-hero'><div class='oh-label'>Margin from Threshold</div><div class='oh-value' style='
                    font-size:1.6rem;'>{_margin:.1f}%</div><div class='oh-sub'>{_margin:.1f} % points {_margin_dir} the
                    {int(RISK_THRESHOLD*100)}% threshold</div></div>",
                unsafe_allow_html=True,
            )
    with kpi_c:
        st.markdown(
            f"<div class='output-hero'><div class='oh-label'>Risk Classification</div><div class='oh-value' style='font-
                size:1.15rem;'><span class='risk-badge {tier_css}'>{mapped_risk_label}</span></div><div class='oh-sub'>
                Based on calibrated probability</div></div>",
            unsafe_allow_html=True,
        )

    print("[DEBUG] >>> rendering KPI cards done, starting charts", flush=True)
    chart_left, chart_right = st.columns([1, 1.5])
    with chart_left:
        st.markdown("**Risk Gauge**")
        st.plotly_chart(make_gauge_chart(result.calibrated_probability), width='stretch', config=PLOTLY_STATIC_CONFIG)
    with chart_right:
        st.markdown("**Prediction Summary**")
        _margin_pp = abs(result.calibrated_probability - RISK_THRESHOLD) * 100.0
```

```
    _margin_dir = "above" if result.calibrated_probability > RISK_THRESHOLD else "below"
    summary_frame = pd.DataFrame(
        {
            "metric": [
                "Calibrated Probability",
                "Risk Classification",
                "Uncertainty Interval (Venn-Abers p0/p1 bounds)",
                "Uncertainty Range (interval width)",
                "Margin from 35% Threshold",
            ],
            "value": [
                f"{result.calibrated_probability:.4f}",
                mapped_risk_label,
                f"{result.lower_bound*100:.1f}% - {result.upper_bound*100:.1f}%" if result.uncertainty_width > 0.001
                    else "N/A",
                f"{result.uncertainty_width*100:.1f} % points" if result.uncertainty_width > 0.001 else "N/A",
                f"{_margin_pp:.1f} % points {_margin_dir} threshold",
            ],
        }
    )
    st.table(summary_frame)

st.markdown(
    "<div class='section-header' style='margin-top:1.8rem;'>"
    "<h2>Understanding Your Risk Score and Alert Threshold</h2>"
    "</div>",
    unsafe_allow_html=True,
)
st.markdown(
    """
    <div style="background:rgba(255,255,255,0.88);border:1px solid rgba(24,33,47,0.09);
                border-radius:18px;padding:1.6rem 2rem;margin-bottom:1.4rem;
                box-shadow:0 4px 18px rgba(15,23,42,0.06);">

      <h4 style="margin-top:0;color:#0f172a;">1   What does your percentage score actually mean?</h4>
                    <p style="color:#1f2937;line-height:1.7;">
        The percentage you see is a <strong>Mathematically Honest Risk Score</strong>. Our system processes
        your data through a statistical safety filter called a <strong>Venn-Abers Predictor</strong> to ensure
        your score matches real-world reality. If the app gives you a <strong>35% risk score</strong>, it is a
        statistical guarantee: out of 100 people with your exact age, body type, and diet, exactly 35 of them
        currently have clinical hypertension.
      </p>

      <h4 style="color:#0f172a;">2   Why does the app flag me as &ldquo;High Risk&rdquo; at 35% instead of 50%?</
            h4>
                    <p style="color:#1f2937;line-height:1.7;">
        It is a common misconception that 50% is the standard tipping point. In reality, the baseline average
        for clinical hypertension in the Philippines is only <strong>15.20%</strong>.
      </p>
                    <p style="color:#1f2937;line-height:1.7;">
        Because our model provides perfectly calibrated, real-world numbers, waiting until your score reaches
        50% means your risk is <strong>more than triple the national average</strong>. By that point, severe
        cardiovascular damage may already be occurring. We set our &ldquo;High Risk&rdquo; alert at
        <strong>35%</strong> because it represents the critical point where your absolute risk is more than
        double the baseline average. This ensures we catch escalating danger early, giving you the exact
        lifestyle recommendations needed to reverse your risk before it is too late.
      </p>

    </div>
    """,
    unsafe_allow_html=True,
)

print("[DEBUG] >>> starting counterfactual section", flush=True)
st.markdown(
    "<div class='section-header' style='margin-top:1.8rem;'>"
    "<h2>Counterfactual Analysis</h2>"
    "<p>What single-feature changes would most reduce your hypertension risk score?</p>"
    "</div>",
    unsafe_allow_html=True,
)

_all_wvals = st.session_state.get("_all_widget_values", {})
_input_fnames = st.session_state.get("_input_feature_names", [])
if _all_wvals and _input_fnames:
    _cf_cache_key = hash(tuple(sorted(_all_wvals.items())))
    _cf_cached = st.session_state.get("_cf_cache")
    if _cf_cached is not None and _cf_cached.get("key") == _cf_cache_key:
        print("[DEBUG] >>> counterfactuals: cache hit", flush=True)
        cf_df = _cf_cached["cf_df"]
    else:
        print("[DEBUG] >>> counterfactuals: computing (no cache)", flush=True)
        with st.spinner("Computing counterfactuals..."):
            cf_df = compute_counterfactuals_safe(
                model=unwrap_model(model),
                preprocessor=preprocessor,
                calibrator=calibrator,
                all_widget_values=_all_wvals,
                input_feature_names=_input_fnames,
                dictionary_labels=dictionary_labels,
                current_probability=result.calibrated_probability,
            )
        st.session_state["_cf_cache"] = {"key": _cf_cache_key, "cf_df": cf_df}
        print("[DEBUG] >>> counterfactuals: done", flush=True)
    if cf_df.empty:
        st.info("No single-feature change was found to reduce risk further.")
    else:
        st.caption(
            "Each row shows the optimal value for one actionable feature that reduces hypertension risk. "
            "Where possible, the model finds the minimal change needed to push predicted probability below 35% "
            "(the decision boundary). If crossing 35% is not achievable for a feature, the suggestion instead "
            "shows the value that minimises risk as much as possible. All other features are held constant."
        )
        st.dataframe(cf_df, width='stretch', hide_index=True)

        if "CARLA Score" in cf_df.columns and not cf_df.empty:
```

```
            best_row = cf_df.loc[cf_df["CARLA Score"].astype(float).idxmax()]
            st.markdown("##### CARLA Counterfactual - Feature Change Chart")
            st.caption(
                "Each bar shows how much a feature must change (Suggested - Current) to reduce hypertension risk. "
                "Blue bars = lower the value; amber bars = raise the value. "
                "Labels show the projected risk reduction. "
                "Rows are ranked by CARLA score (high reduction with minimal change scores highest)."
            )
            st.plotly_chart(
                make_carla_cf_chart(cf_df),
                width='stretch',
                config=PLOTLY_STATIC_CONFIG,
            )
            st.markdown(
                f"**Top recommendation:** {best_row['Feature']}  \n"
                f"Change from **{best_row['Current Value']}** -> **{best_row['Suggested Value']}** "
                f"to reduce predicted risk by **{best_row['Risk Reduction']}**."
            )

print("[DEBUG] >>> starting explainability section", flush=True)
st.markdown(
    "<div class='section-header' style='margin-top:1.8rem;'>"
    "<h2>Explainability</h2>"
    "<p>Local explanations first (this patient), then global model explanations below.</p>"
    "</div>",
    unsafe_allow_html=True,
)

# Stable cache key: hash of the actual input values so the cache
# survives Streamlit reruns (id(result) changes every rerun).
_input_vals_for_key = tuple(sorted(st.session_state.get("_all_widget_values", {}).items()))
_exp_cache_key = hash(_input_vals_for_key)
_cached = st.session_state.get("_exp_cache")
if _cached is not None and _cached.get("key") == _exp_cache_key:
    print("[DEBUG] >>> explainability: cache hit", flush=True)
    shap_local_df  = _cached["shap_local"]
    shap_global_df = _cached["shap_global"]
    shap_error     = _cached["shap_error"]
    lime_df        = _cached["lime_df"]
    lime_error     = _cached["lime_error"]
else:
    shap_local_df, shap_global_df, shap_error = None, None, None
    lime_df, lime_error = None, None
    _control_flow_exc = None
    print("[DEBUG] >>> explainability: computing (no cache)", flush=True)
    with st.spinner("Computing SHAP and LIME explanations..."):
        try:
            import warnings as _warnings
            explain_feature_names = st.session_state.get("_model_feature_names", model_feature_names)
            rendered_inputs = set(st.session_state.get("_rendered_input_features", []))
            input_features_for_mapping = st.session_state.get("_input_feature_names", feature_names)
            explain_feature_names = resolve_explainability_columns(
                model_columns=list(explain_feature_names),
                input_features=list(input_features_for_mapping),
                rendered_input_features=rendered_inputs,
            )

            # compute_explainability_safe runs SHAP+LIME in a subprocess,
            # so we must NOT touch the model's C-level state here -
            # calling get_booster() in the main process activates the
            # OpenMP thread pool and triggers os._exit() on Windows.
            _explain_model = explain_model

            if explain_feature_names:
                print(f"[DEBUG] >>> explainability: calling compute_explainability_safe ({len(explain_feature_names)}
                      features)", flush=True)
                explain_input_frame = input_frame.loc[:, explain_feature_names].copy()
                with _warnings.catch_warnings():
                    _warnings.simplefilter("ignore")
                    shap_local_df, shap_global_df, shap_error, lime_df, lime_error = compute_explainability_safe(
                        model=_explain_model,
                        feature_names=explain_feature_names,
                        input_frame=explain_input_frame,
                        base_input_frame=input_frame,
                    )
                print(f"[DEBUG] >>> explainability: subprocess returned - shap_error={shap_error!r}  lime_error={
                      lime_error!r}", flush=True)
            else:
                shap_error = "No explainability columns are currently mapped to visible webpage inputs."
                lime_error = shap_error
        except BaseException as _exp_exc:
            if not isinstance(_exp_exc, Exception) and not isinstance(_exp_exc, SystemExit):
                _control_flow_exc = _exp_exc
            else:
                _exp_msg = f"sys.exit({_exp_exc.code})" if isinstance(_exp_exc, SystemExit) else str(_exp_exc)
                if shap_error is None:
                    shap_error = f"Explainability error: {_exp_msg}"
                if lime_error is None:
                    lime_error = f"Explainability error: {_exp_msg}"
        finally:
            st.session_state["_exp_cache"] = {
                "key":        _exp_cache_key,
                "shap_local":  shap_local_df,
                "shap_global": shap_global_df,
                "shap_error":  shap_error,
                "lime_df":     lime_df,
                "lime_error":  lime_error,
            }
    if _control_flow_exc is not None:
        raise _control_flow_exc

print("[DEBUG] >>> rendering explainability plots", flush=True)
_wvals_exp = st.session_state.get("_all_widget_values", {})
_ifnames_exp = st.session_state.get("_input_feature_names", feature_names)

st.markdown("#### Local Explanations (This Patient)")
```

```
st.caption(
    "Local charts explain this single prediction. Red bars increase predicted risk, blue bars decrease predicted risk.
        "
    "Bar length means contribution strength for this patient."
)
loc_a, loc_b = st.columns(2)

with loc_a:
    try:
        st.markdown("##### SHAP Local")
        if shap_error:
            st.warning(shap_error)
        elif shap_local_df is not None and not shap_local_df.empty:
            local_top = shap_local_df.head(12).copy()
            local_top["your_input"] = [
                _fmt_raw(_raw_value_for_feature(f, _ifnames_exp, _wvals_exp))
                for f in local_top["feature"]
            ]
            local_top["Feature"] = [
                field_display_label(f, dictionary_labels)
                for f in local_top["feature"]
            ]
            st.table(
                local_top[["Feature", "your_input", "shap_value"]]
                .rename(columns={"your_input": "Your Input", "shap_value": "SHAP Value"})
            )
            chart_df = local_top.head(10).iloc[::-1]
            st.plotly_chart(
                go.Figure(
                    go.Bar(
                        x=chart_df["shap_value"],
                        y=chart_df["Feature"],
                        orientation="h",
                        marker_color=["#dc2626" if value >= 0 else "#2563eb" for value in chart_df["shap_value"]],
                        customdata=chart_df["your_input"],
                        hovertemplate="<b>%{y}</b><br>SHAP value: %{x:.4f}<br>Your input: %{customdata}<extra></extra>
                            ",
                    )
                ).update_layout(height=max(320, 28 * len(chart_df)), margin=dict(l=10, r=10, t=10, b=10)),
                width='stretch',
                config=PLOTLY_STATIC_CONFIG,
            )
        else:
            st.info("No SHAP local output produced.")
    except Exception as _e:
        st.warning(f"SHAP Local render error: {_e}")

with loc_b:
    try:
        st.markdown("##### LIME Local")
        if lime_error:
            st.warning(lime_error)
        elif lime_df is None:
            st.info("LIME result is None - computation may have been skipped or failed silently.")
        elif lime_df.empty:
            st.info("LIME returned an empty result (no feature contributions).")
        else:
            lime_rows = []
            for _, row in lime_df.head(12).iterrows():
                weight = float(row["weight"])
                raw_or_model_name = str(row.get("feature", "")).strip()
                rule_text = str(row.get("rule", ""))
                if not raw_or_model_name:
                    raw_or_model_name = rule_text

                resolved_input_feature = _resolve_lime_source_feature(
                    model_like_name=raw_or_model_name,
                    rule_text=rule_text,
                    input_features=_ifnames_exp,
                )

                raw_val = _raw_value_for_feature(
                    resolved_input_feature if resolved_input_feature else raw_or_model_name,
                    _ifnames_exp,
                    _wvals_exp,
                )
                display_name = (
                    field_display_label(resolved_input_feature, dictionary_labels)
                    if resolved_input_feature and resolved_input_feature in _ifnames_exp
                    else raw_or_model_name
                )
                lime_rows.append({"feature": display_name, "your_input": _fmt_raw(raw_val), "lime_weight": weight})

            lime_display_df = pd.DataFrame(lime_rows)
            st.table(lime_display_df)

            chart_df_lime = lime_display_df.head(10).iloc[::-1]
            st.plotly_chart(
                go.Figure(
                    go.Bar(
                        x=chart_df_lime["lime_weight"],
                        y=chart_df_lime["feature"],
                        orientation="h",
                        marker_color=["#dc2626" if v >= 0 else "#2563eb" for v in chart_df_lime["lime_weight"]],
                        customdata=chart_df_lime["your_input"],
                        hovertemplate="<b>%{y}</b><br>Weight: %{x:.4f}<br>Your input: %{customdata}<extra></extra>",
                    )
                ).update_layout(height=320, margin=dict(l=10, r=10, t=10, b=10)),
                width='stretch',
                config=PLOTLY_STATIC_CONFIG,
            )
    except Exception as _e:
        st.warning(f"LIME Local render error: {_e}")

st.markdown("#### Global Explanation (Model Behavior Overall)")
st.caption(
```

```
            "Global SHAP ranks features by average absolute impact across background samples. "
            "Higher bars indicate features the model generally relies on most."
        )
        try:
            st.markdown("##### SHAP Global")
            if shap_error:
                st.warning(shap_error)
            elif shap_global_df is not None and not shap_global_df.empty:
                global_top = shap_global_df.head(12)
                st.table(global_top)
                chart_df = global_top.head(10).iloc[::-1]
                st.plotly_chart(
                    go.Figure(
                        go.Bar(
                            x=chart_df["mean_abs_shap"],
                            y=chart_df["feature"],
                            orientation="h",
                            marker_color="#2563eb",
                            hovertemplate="<b>%{y}</b><br>Mean |SHAP|: %{x:.4f}<extra></extra>",
                        )
                    ).update_layout(
                        height=320,
                        margin=dict(l=10, r=10, t=10, b=10),
                        xaxis_title="Mean |SHAP value|",
                        showlegend=False,
                    ),
                    width='stretch',
                    config=PLOTLY_STATIC_CONFIG,
                )
            else:
                st.info("No SHAP global output produced.")
        except Exception as _e:
            st.warning(f"SHAP Global render error: {_e}")

        print("[DEBUG] >>> output section fully rendered", flush=True)
        if st.session_state.get("_scroll_to_output", False):
            st.html(
                "<script>(function(){function s(){try{var w=window.parent||window,e=w.document.getElementById('output-section
                    ');if(e){e.scrollIntoView({behavior:'smooth',block:'start'});}else{w.scrollTo({top:w.document.body.
                    scrollHeight,behavior:'smooth'});}}catch(ex){}}setTimeout(s,100);setTimeout(s,600);})();</script>"
            )
            st.session_state._scroll_to_output = False
```

10.1.5 backend.py

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import joblib
import numpy as np
import pandas as pd
from venn_abers import VennAbers


PROJECT_ROOT = Path(__file__).resolve().parents[1]
WORKSPACE_ROOT = Path(__file__).resolve().parents[2]
TRAINING_ROOT = PROJECT_ROOT / "training"


def _first_existing(paths: list[Path], fallback: Path) -> Path:
    return next((path for path in paths if path.exists()), fallback)


# -- Auto-select best EXP3 bundle from updated runs (A/B/C/D) -----------------
_EXP3_CANDIDATES = [
    ("exp3_knn_rf", TRAINING_ROOT / "exp3_knn_rf"),
    ("exp3_xgb_ada", TRAINING_ROOT / "exp3_xgb_ada"),
    ("exp3_logreg_cat", TRAINING_ROOT / "exp3_logreg_cat"),
    ("exp3_naive_bayes", TRAINING_ROOT / "exp3_naive_bayes"),
    ("exp3_knn_rf", PROJECT_ROOT / "exp3_knn_rf"),
    ("exp3_xgb_ada", PROJECT_ROOT / "exp3_xgb_ada"),
    ("exp3_logreg_cat", PROJECT_ROOT / "exp3_logreg_cat"),
    ("exp3_naive_bayes", PROJECT_ROOT / "exp3_naive_bayes"),
]


def _select_best_exp3_bundle() -> Path | None:
    best_path: Path | None = None
    best_combined = -np.inf

    for _, exp_dir in _EXP3_CANDIDATES:
        bundle_path = exp_dir / "models" / "best_model_bundle.joblib"
        post_cal_path = exp_dir / "post_calibration_results.csv"

        if not bundle_path.exists() or not post_cal_path.exists():
            continue

        try:
            top_row = pd.read_csv(post_cal_path, nrows=1)
            if top_row.empty or "combined" not in top_row.columns:
                continue
            combined = float(top_row.iloc[0]["combined"])
        except Exception:
            continue

        if combined > best_combined:
            best_combined = combined
            best_path = bundle_path

    return best_path
```

```
EXP3_BUNDLE_PATH = _select_best_exp3_bundle()

# Fall back to old GPU-exp2 artifacts when EXP3 bundles are unavailable.
_OLD_MODEL_CANDIDATES = [
    PROJECT_ROOT / "gpu_rf_xgb_cat_exp2_artifacts" / "models" / "calibrated_top_models" / "top3_catboost_isotonic.joblib",
    PROJECT_ROOT.parent / "gpu_rf_xgb_cat_exp2_artifacts" / "models" / "calibrated_top_models" / "top3_catboost_isotonic.
        joblib",
]
_OLD_PREPROCESSOR_CANDIDATES = [
    PROJECT_ROOT / "gpu_rf_xgb_cat_exp2_artifacts" / "preprocessor.joblib",
    PROJECT_ROOT.parent / "gpu_rf_xgb_cat_exp2_artifacts" / "preprocessor.joblib",
]

_OLD_MODEL_PATH = _first_existing(_OLD_MODEL_CANDIDATES, _OLD_MODEL_CANDIDATES[0])
_OLD_PREPROCESSOR_PATH = _first_existing(_OLD_PREPROCESSOR_CANDIDATES, _OLD_PREPROCESSOR_CANDIDATES[0])

# Pinned to EXP3-B (XGBoost cw base) - best recall (76.5%) across updated runs.
_PINNED_BUNDLE_CANDIDATES = [
    TRAINING_ROOT / "exp3_xgb_ada" / "models" / "best_model_bundle.joblib",
    PROJECT_ROOT / "exp3_xgb_ada" / "models" / "best_model_bundle.joblib",
]
_PINNED_BUNDLE = _first_existing(_PINNED_BUNDLE_CANDIDATES, _PINNED_BUNDLE_CANDIDATES[0])

DEFAULT_MODEL_PATH = _PINNED_BUNDLE if _PINNED_BUNDLE.exists() else (EXP3_BUNDLE_PATH if EXP3_BUNDLE_PATH is not None and
      EXP3_BUNDLE_PATH.exists() else _OLD_MODEL_PATH)
DEFAULT_PREPROCESSOR_PATH = _PINNED_BUNDLE if _PINNED_BUNDLE.exists() else (EXP3_BUNDLE_PATH if EXP3_BUNDLE_PATH is not None
      and EXP3_BUNDLE_PATH.exists() else _OLD_PREPROCESSOR_PATH)

_calibrator_candidates = [
    PROJECT_ROOT / "main_2015_balanced_gpu_artifacts" / "models" / "venn_abers_calibrator.joblib",
    WORKSPACE_ROOT / "main_2015_balanced_gpu_artifacts" / "models" / "venn_abers_calibrator.joblib",
]

# EXP3 bundles already encode their selected calibration strategy in the bundle
# metadata, so avoid applying an unrelated external calibrator by default.
if DEFAULT_MODEL_PATH == _OLD_MODEL_PATH:
    DEFAULT_CALIBRATOR_PATH = next((path for path in _calibrator_candidates if path.exists()), _calibrator_candidates[0])
else:
    DEFAULT_CALIBRATOR_PATH = PROJECT_ROOT / "__no_external_calibrator__.joblib"

ONE_HOT_GROUPS = {
    "alcohol_level": ["0.0", "1.0", "2.0", "3.0", "nan"], "smoking_level": ["0.0", "1.0", "2.0", "3.0", "nan"],
}

# EXP3-C uses numeric fe_smoking_level / fe_alcohol_level instead of OHE columns.
ENGINEERED_TARGET_FEATURES = {
    "alcohol_level", "smoking_level",
    "fe_alcohol_level", "fe_smoking_level",
    "BMI", "bmi", "whr",
}

NUMERIC_DEFAULTS = {
    "ethnicity": 1.0, "waist": 0.0, "hip": 0.0, "BMI": 0.0, "bmi": 0.0, "whr": 0.0,
    "fe_smoking_level": 0.0, "fe_alcohol_level": 0.0,
    "Total_Food_epwt": 0.0, "Total_Ener": 0.0, "Total_Prot": 0.0, "Total_Calc": 0.0,
    "Total_Iron": 0.0, "Total_VitA": 0.0, "Total_VitC": 0.0,
    "Total_Thia": 0.0, "Total_Ribo": 0.0, "Total_Nia": 0.0, "Total_CHO": 0.0, "Total_Fat": 0.0,
}

RANGE_HINTS = {
    "ethnicity": (0.0, 10.0, 1.0), "waist": (40.0, 180.0, 0.5), "hip": (40.0, 180.0, 0.5),
    "BMI": (10.0, 60.0, 0.1), "bmi": (10.0, 60.0, 0.1), "whr": (0.5, 2.0, 0.01),
    "fe_smoking_level": (0.0, 3.0, 1.0), "fe_alcohol_level": (0.0, 3.0, 1.0),
    "Total_Food_epwt": (0.0, 1760.0, 5.0), "Total_Ener": (0.0, 3890.0, 10.0), "Total_Prot": (0.0, 150.0, 1.0),
    "Total_Calc": (0.0, 1270.0, 10.0), "Total_Iron": (0.0, 30.0, 0.5), "Total_VitA": (0.0, 4810.0, 10.0),
    "Total_VitC": (0.0, 190.0, 1.0), "Total_Thia": (0.0, 10.0, 0.05), "Total_Ribo": (0.0, 10.0, 0.05),
    "Total_Nia": (0.0, 50.0, 0.5), "Total_CHO": (0.0, 710.0, 5.0), "Total_Fat": (0.0, 120.0, 1.0),
    # Dietary food-group weights (edible-portion grams) - maxima from dataset p99 x 1.1
    "epwt_fg1": (0.0, 870.0, 1.0), "epwt_fg2": (0.0, 790.0, 1.0), "epwt_fg3": (0.0, 480.0, 1.0),
    "epwt_fg4": (0.0, 340.0, 1.0), "epwt_fg5": (0.0, 480.0, 1.0), "epwt_fg6": (0.0, 690.0, 1.0),
    "epwt_fg7": (0.0, 200.0, 1.0), "epwt_fg8": (0.0, 440.0, 1.0), "epwt_fg9": (0.0, 260.0, 1.0),
    "epwt_fg10": (0.0, 400.0, 1.0), "epwt_fg11": (0.0, 630.0, 1.0), "epwt_fg12": (0.0, 550.0, 1.0),
    "epwt_fg13": (0.0, 600.0, 1.0), "epwt_fg14": (0.0, 530.0, 1.0), "epwt_fg15": (0.0, 360.0, 1.0),
    "epwt_fg16": (0.0, 510.0, 1.0), "epwt_fg17": (0.0, 380.0, 1.0), "epwt_fg18": (0.0, 180.0, 1.0),
    "epwt_fg19": (0.0, 270.0, 1.0), "epwt_fg20": (0.0, 240.0, 1.0), "epwt_fg21": (0.0, 320.0, 1.0),
    "epwt_fg23": (0.0, 60.0, 0.5), "epwt_fg24": (0.0, 590.0, 1.0), "epwt_fg25": (0.0, 590.0, 1.0),
    "epwt_fg26": (0.0, 60.0, 0.5), "epwt_fg27": (0.0, 550.0, 1.0),
}


@dataclass(frozen=True)
class PredictionResult:
    raw_probability: float
    calibrated_probability: float
    lower_bound: float
    upper_bound: float
    uncertainty_width: float


def _compute_bootstrap_interval(
    base_model: Any,
    input_frame: pd.DataFrame,
    n_samples: int = 60,
    noise_std: float = 0.25,
    seed: int = 42,
) -> tuple[float, float]:
    """Estimate a 10th-90th percentile prediction interval via input perturbation.

    Adds Gaussian noise (+/-noise_std in model-feature / standardised space) to
    the single input row, obtains n_samples predictions in one batched call,
    and returns (p10, p90).  This reflects how sensitive the risk score is to
    typical measurement uncertainty in the input features.
    """
    rng = np.random.default_rng(seed)
```

```
    base = input_frame.values  # (1, n_features)
    noise = rng.normal(0.0, noise_std, size=(n_samples, base.shape[1]))
    perturbed = pd.DataFrame(base + noise, columns=input_frame.columns)
    try:
        probs = np.asarray(base_model.predict_proba(perturbed))[:, 1]
        return float(np.percentile(probs, 10)), float(np.percentile(probs, 90))
    except Exception:
        p = float(np.asarray(base_model.predict_proba(input_frame))[0, 1])
        return p, p


class Exp3BundlePreprocessor:
    """Sklearn-compatible transformer wrapping the EXP3-C multi-step preprocessing bundle.

    Pipeline: KNN impute (full num cols) -> collinearity select (reduced num cols)
              -> StandardScaler + OneHotEncoder -> hstack output.
    """

    def __init__(self, bundle: dict) -> None:
        self._b = bundle
        self.feature_names_in_ = np.array(bundle["input_feature_names"])

    # ------------------------------------------------------------------
    def transform(self, X: pd.DataFrame) -> np.ndarray:
        b = self._b
        num_full = b["num_cols_full"]
        num_red  = b["num_cols_reduced"]
        cat      = b["cat_cols"]

        # --- impute numerics (full set) ---
        if num_full:
            num_imp = pd.DataFrame(
                b["knn_imputer"].transform(X[num_full]),
                columns=num_full, index=X.index,
            )
        else:
            num_imp = pd.DataFrame(index=X.index)

        # --- impute categoricals ---
        if cat and b.get("cat_imputer") is not None:
            cat_imp = pd.DataFrame(
                b["cat_imputer"].transform(X[cat]),
                columns=cat, index=X.index,
            )
        else:
            cat_imp = pd.DataFrame(index=X.index)

        # --- scale surviving numeric cols + OHE categoricals ---
        parts: list[np.ndarray] = []
        if num_red:
            parts.append(b["scaler"].transform(num_imp[num_red]))
        if cat and b.get("ohe") is not None:
            parts.append(b["ohe"].transform(cat_imp[cat].astype(str)))

        return np.hstack(parts) if parts else np.empty((len(X), 0), dtype=float)

    def get_feature_names_out(self) -> np.ndarray:
        return np.array(self._b["feat_names"])


def _is_exp3_bundle(obj: Any) -> bool:
    return isinstance(obj, dict) and "knn_imputer" in obj


def load_model(model_path: Path | str = DEFAULT_MODEL_PATH):
    obj = joblib.load(Path(model_path))
    if _is_exp3_bundle(obj):
        # Wrap as the dict format predict_with_venn_abers expects.
        return {
            "base_model":         obj["model"],
            "feature_names":      obj["feat_names"],
            "calibration_method": obj.get("calibration", "base"),
            "calibrator":         None,
            "threshold":          obj.get("threshold", 0.5),
        }
    return obj


def load_calibrator(calibrator_path: Path | str = DEFAULT_CALIBRATOR_PATH):
    return joblib.load(path) if (path := Path(calibrator_path)).exists() else None


def load_preprocessor(preprocessor_path: Path | str = DEFAULT_PREPROCESSOR_PATH):
    path = Path(preprocessor_path)
    if not path.exists():
        return None
    obj = joblib.load(path)
    if _is_exp3_bundle(obj):
        return Exp3BundlePreprocessor(obj)
    return obj


def load_feature_names(model: Any) -> list[str]:
    if isinstance(model, dict):
        feature_names = model.get("feature_names")
        if feature_names:
            return [str(name) for name in feature_names]
        if "base_model" in model:
            return load_feature_names(model["base_model"])
    if hasattr(model, "feature_names_in_"):
        return [str(name) for name in model.feature_names_in_]
    if hasattr(model, "named_steps"):
        for step in reversed(list(model.named_steps.values())):
            if hasattr(step, "feature_names_in_"):
                return [str(name) for name in step.feature_names_in_]
    raise ValueError("The model does not expose feature names.")
```

```
def unwrap_model(model: Any) -> Any:
    return model["base_model"] if isinstance(model, dict) and "base_model" in model else model


def load_input_feature_names(model: Any, preprocessor: Any | None = None) -> list[str]:
    return [str(name) for name in preprocessor.feature_names_in_] if preprocessor is not None and hasattr(preprocessor, "
          feature_names_in_") else load_feature_names(model)


def load_model_feature_names(model: Any, preprocessor: Any | None = None) -> list[str]:
    if preprocessor is not None and hasattr(preprocessor, "get_feature_names_out"):
        try:
            return [str(name) for name in preprocessor.get_feature_names_out()]
        except Exception:
            pass
    return load_feature_names(model)


def make_input_frame(feature_names: list[str], values: dict[str, Any]) -> pd.DataFrame:
    return pd.DataFrame([{name: values.get(name, 0.0) for name in feature_names}], columns=feature_names)


def prepare_model_input(input_frame: pd.DataFrame, preprocessor: Any | None = None) -> pd.DataFrame:
    if preprocessor is None:
        return input_frame

    transformed = preprocessor.transform(input_frame)
    if hasattr(transformed, "toarray"):
        transformed = transformed.toarray()

    transformed = np.asarray(transformed)
    if transformed.ndim == 1:
        transformed = transformed.reshape(1, -1)

    if hasattr(preprocessor, "get_feature_names_out"):
        try:
            columns = [str(name) for name in preprocessor.get_feature_names_out()]
        except Exception:
            columns = [f"feature_{index}" for index in range(transformed.shape[1])]
    else:
        columns = [f"feature_{index}" for index in range(transformed.shape[1])]

    return pd.DataFrame(transformed, columns=columns)


def predict_with_venn_abers(
    model: Any,
    input_frame: pd.DataFrame,
    calibrator: Any | None = None,
) -> PredictionResult:
    raw_probability = float(np.asarray(unwrap_model(model).predict_proba(input_frame))[0, 1])

    # Prefer explicit external Venn-Abers calibrator when supplied.
    # This keeps uncertainty intervals available even if the wrapped model
    # uses a point calibrator (e.g., isotonic/platt) internally.
    if calibrator is not None:
        probability_pair = np.array([[1.0 - float(np.clip(raw_probability, 1e-9, 1 - 1e-9)), float(np.clip(raw_probability, 1e
              -9, 1 - 1e-9))]], dtype=float)
        calibrated_pair, p0_p1 = calibrator.predict_proba(p_test=probability_pair)

        calibrated_probability = float(calibrated_pair[0, 1])
        lower_bound = float(np.clip(min(p0_p1[0, 0], p0_p1[0, 1], calibrated_probability), 0.0, 1.0))
        upper_bound = float(np.clip(max(p0_p1[0, 0], p0_p1[0, 1], calibrated_probability), 0.0, 1.0))

        return PredictionResult(
            raw_probability=raw_probability,
            calibrated_probability=calibrated_probability,
            lower_bound=lower_bound,
            upper_bound=upper_bound,
            uncertainty_width=max(0.0, upper_bound - lower_bound),
        )

    if isinstance(model, dict):
        method = str(model.get("calibration_method", "base"))
        wrapped_calibrator = model.get("calibrator")

        if method == "base" or wrapped_calibrator is None:
            calibrated_probability = raw_probability
            lower_bound, upper_bound = _compute_bootstrap_interval(
                unwrap_model(model), input_frame
            )
        elif method in {"isotonic", "platt", "sigmoid"}:
            calibrated_probability = float(np.clip(wrapped_calibrator.predict(np.array([raw_probability]))[0], 0.0, 1.0))
            lower_bound = calibrated_probability
            upper_bound = calibrated_probability
        elif method == "venn_abers":
            probability_pair = np.array([[1.0 - float(np.clip(raw_probability, 1e-9, 1 - 1e-9)), float(np.clip(raw_probability
                  , 1e-9, 1 - 1e-9))]], dtype=float)
            calibrated_pair, p0_p1 = wrapped_calibrator.predict_proba(p_test=probability_pair)
            calibrated_probability = float(calibrated_pair[0, 1])
            lower_bound = float(np.clip(min(p0_p1[0, 0], p0_p1[0, 1], calibrated_probability), 0.0, 1.0))
            upper_bound = float(np.clip(max(p0_p1[0, 0], p0_p1[0, 1], calibrated_probability), 0.0, 1.0))
        else:
            calibrated_probability = raw_probability
            lower_bound = raw_probability
            upper_bound = raw_probability

        return PredictionResult(
            raw_probability=raw_probability,
            calibrated_probability=calibrated_probability,
            lower_bound=lower_bound,
            upper_bound=upper_bound,
            uncertainty_width=max(0.0, upper_bound - lower_bound),
        )
```

```
    return PredictionResult(
        raw_probability=raw_probability,
        calibrated_probability=raw_probability,
        lower_bound=raw_probability,
        upper_bound=raw_probability,
        uncertainty_width=0.0,
    )


def predict_with_venn_abers_safe(
    model: Any,
    input_frame: "pd.DataFrame",
    calibrator: Any | None = None,
    timeout: int = 60,
) -> "PredictionResult":
    """Run predict_with_venn_abers in an isolated subprocess.

    Prevents XGBoost's OpenMP thread pool from being initialised in the
    Streamlit main process, which would cause os._exit() on Windows when
    Streamlit's own threads try to run concurrently.

    Falls back to in-process prediction if the payload cannot be pickled.
    """
    import pathlib
    import pickle
    import subprocess
    import sys
    import tempfile
    import uuid

    _dir = pathlib.Path(__file__).parent
    worker = _dir / "_predict_worker.py"

    payload = {"model": model, "input_frame": input_frame, "calibrator": calibrator}
    try:
        _bytes = pickle.dumps(payload)
        del _bytes
    except Exception:
        return predict_with_venn_abers(model, input_frame, calibrator)

    uid = uuid.uuid4().hex
    tmp_dir = pathlib.Path(tempfile.gettempdir())
    tmp_in = tmp_dir / f"_pred_in_{uid}.pkl"
    tmp_out = tmp_dir / f"_pred_out_{uid}.pkl"

    try:
        with open(tmp_in, "wb") as fh:
            pickle.dump(payload, fh)

        proc = subprocess.Popen(
            [sys.executable, str(worker), str(tmp_in), str(tmp_out)],
            cwd=str(_dir),
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
        )

        try:
            returncode = proc.wait(timeout=timeout)
        except subprocess.TimeoutExpired:
            proc.kill()
            proc.wait()
            return predict_with_venn_abers(model, input_frame, calibrator)

        if not tmp_out.exists():
            return predict_with_venn_abers(model, input_frame, calibrator)

        with open(tmp_out, "rb") as fh:
            result = pickle.load(fh)

        return result

    finally:
        for f in (tmp_in, tmp_out):
            try:
                f.unlink(missing_ok=True)
            except Exception:
                pass


def feature_default(feature_name: str) -> float:
    return float(NUMERIC_DEFAULTS.get(feature_name, 0.0))


def feature_range(feature_name: str) -> tuple[float, float, float]:
    if feature_name in RANGE_HINTS:
        return RANGE_HINTS[feature_name]
    if feature_name.startswith("epwt_fg") or feature_name.startswith("fg"):
        return (0.0, 500.0, 1.0)
    if feature_name.startswith("Total_"):
        return (0.0, 5000.0, 1.0)
    if feature_name.endswith("_nan"):
        return (0.0, 1.0, 1.0)
    return (0.0, 100.0, 1.0)


def group_feature_names(feature_names: list[str]) -> dict[str, list[str]]:
    grouped = {
        "Core clinical": [],
        "Anthropometrics": [],
        "Dietary pattern": [],
        "Lifestyle encodings": [],
        "Other model inputs": [],
    }

    for feature_name in feature_names:
```

```
        if feature_name.startswith("alcohol_level_") or feature_name.startswith("smoking_level_"):
            grouped["Lifestyle encodings"].append(feature_name)
        elif feature_name in {"fe_smoking_level", "fe_alcohol_level"}:
            # EXP3-C engineered behavioral features - computed, not shown directly.
            grouped["Lifestyle encodings"].append(feature_name)
        elif feature_name in {"pa_met", "fbs", "chol", "tri", "hdl", "ldl", "ethnicity",
                              "hemoglobin", "uic", "vita"}:
            grouped["Core clinical"].append(feature_name)
        elif feature_name in {"waist", "hip", "BMI", "bmi", "whr"}:
            # bmi and whr are computed from raw inputs, not shown directly.
            grouped["Anthropometrics"].append(feature_name)
        elif feature_name.startswith("epwt_fg") or feature_name.startswith("fg") or feature_name.startswith("Total_"):
            grouped["Dietary pattern"].append(feature_name)
        else:
            grouped["Other model inputs"].append(feature_name)

    return {group: names for group, names in grouped.items() if names}


def build_input_values_from_widgets(feature_names: list[str], widget_values: dict[str, Any]) -> dict[str, float]:
    values: dict[str, float] = {}

    alcohol_level = _compute_alcohol_level(widget_values)
    smoking_level = _compute_smoking_level(widget_values)
    bmi_value = _compute_bmi(widget_values)
    whr_value = _compute_whr(widget_values)

    _populate_one_hot_group(values, feature_names, "alcohol_level", alcohol_level)
    _populate_one_hot_group(values, feature_names, "smoking_level", smoking_level)

    if "alcohol_level" in feature_names:
        values["alcohol_level"] = float(alcohol_level if alcohol_level is not None else feature_default("alcohol_level"))
    if "smoking_level" in feature_names:
        values["smoking_level"] = float(smoking_level if smoking_level is not None else feature_default("smoking_level"))

    # EXP3-C numeric engineered behavioral features.
    if "fe_smoking_level" in feature_names:
        values["fe_smoking_level"] = float(smoking_level if smoking_level is not None else 0.0)
    if "fe_alcohol_level" in feature_names:
        values["fe_alcohol_level"] = float(alcohol_level if alcohol_level is not None else 0.0)

    if bmi_value is not None:
        if "BMI" in feature_names:
            values["BMI"] = float(bmi_value)
        if "bmi" in feature_names:
            values["bmi"] = float(bmi_value)

    if whr_value is not None and "whr" in feature_names:
        values["whr"] = float(whr_value)

    for feature_name in feature_names:
        if feature_name in values:
            continue
        if feature_name.startswith("alcohol_level_") or feature_name.startswith("smoking_level_"):
            values[feature_name] = 0.0
            continue
        if feature_name in ENGINEERED_TARGET_FEATURES:
            values[feature_name] = float(feature_default(feature_name))
            continue
        values[feature_name] = float(widget_values.get(feature_name, feature_default(feature_name)))

    return values


def _to_numeric_clean_scalar(value: Any) -> float | None:
    if value is None:
        return None
    try:
        numeric = float(value)
    except (TypeError, ValueError):
        return None
    return None if np.isnan(numeric) or numeric in {9, 99, 888888, 999999} else numeric


def _compute_smoking_level(raw_values: dict[str, Any]) -> float | None:
    direct = _to_numeric_clean_scalar(raw_values.get("smoking_level"))
    if direct is not None:
        return float(np.clip(direct, 0.0, 3.0))

    smoke_status = _to_numeric_clean_scalar(raw_values.get("smoke_status"))
    current_smoking = _to_numeric_clean_scalar(raw_values.get("current_smoking"))
    ever_smk = _to_numeric_clean_scalar(raw_values.get("ever_smk"))

    if smoke_status is not None:
        level: float | None = None
        if smoke_status == 0:
            level = 0.0
        elif smoke_status == 2:
            level = 1.0
        elif smoke_status == 1:
            level = 2.0

        if smoke_status == 1 and current_smoking == 3:
            level = 3.0
        return None if level is None else float(np.clip(level, 0.0, 3.0))

    if current_smoking is not None:
        level: float | None = None
        if current_smoking == 0:
            level = 0.0
        elif current_smoking in {1, 2}:
            level = 2.0
        elif current_smoking == 3:
            level = 3.0

        if current_smoking == 0 and ever_smk is not None and ever_smk > 0:
```

```
            level = 1.0
        return None if level is None else float(np.clip(level, 0.0, 3.0))

    if ever_smk is not None:
        return 0.0 if ever_smk == 0 else 1.0

    return None


def _compute_alcohol_level(raw_values: dict[str, Any]) -> float | None:
    direct = _to_numeric_clean_scalar(raw_values.get("alcohol_level"))
    if direct is not None:
        return float(np.clip(direct, 0.0, 3.0))

    alcohol_status = _to_numeric_clean_scalar(raw_values.get("alcohol_status"))
    alcohol_ever = _to_numeric_clean_scalar(raw_values.get("alcohol"))
    con_alcohol = _to_numeric_clean_scalar(raw_values.get("con_alcohol"))
    drnk_30days = _to_numeric_clean_scalar(raw_values.get("drnk_30days"))
    binge_drink = _to_numeric_clean_scalar(raw_values.get("binge_drink"))

    if alcohol_status is not None:
        level: float | None = None
        if alcohol_status == 0:
            level = 0.0
        elif alcohol_status == 2:
            level = 1.0
        elif alcohol_status == 1:
            level = 2.0

        if alcohol_status == 1 and binge_drink == 1:
            level = 3.0
        return None if level is None else float(np.clip(level, 0.0, 3.0))

    level = 0.0
    used = False

    if alcohol_ever is not None:
        used = True
        if alcohol_ever > 0:
            level = max(level, 1.0)

    if con_alcohol is not None:
        used = True
        if con_alcohol == 1:
            level = max(level, 2.0)

    if drnk_30days is not None:
        used = True
        if drnk_30days == 1:
            level = max(level, 2.0)

    if binge_drink is not None:
        used = True
        if binge_drink == 1:
            level = 3.0

    if used:
        return float(np.clip(level, 0.0, 3.0))
    return None


def _compute_bmi(raw_values: dict[str, Any]) -> float | None:
    weight = _to_numeric_clean_scalar(raw_values.get("weight"))
    height = _to_numeric_clean_scalar(raw_values.get("height"))
    if weight is None or height is None or height <= 0:
        return None

    height_m = height / 100.0 if height > 3.0 else height
    if height_m <= 0:
        return None

    bmi = weight / (height_m**2)
    return float(bmi) if np.isfinite(bmi) else None


def _compute_whr(raw_values: dict[str, Any]) -> float | None:
    waist = _to_numeric_clean_scalar(raw_values.get("waist"))
    hip = _to_numeric_clean_scalar(raw_values.get("hip"))
    if waist is None or hip is None or hip == 0:
        return None

    whr = waist / hip
    return float(whr) if np.isfinite(whr) else None


def _populate_one_hot_group(values: dict[str, float], feature_names: list[str], group_name: str, level: float | None):
    group_prefix = f"{group_name}_"
    model_suffixes = [name[len(group_prefix) :] for name in feature_names if name.startswith(group_prefix)]
    suffixes = model_suffixes if model_suffixes else ONE_HOT_GROUPS[group_name]
    selected_suffix = "nan" if level is None else f"{float(level):.1f}"

    if selected_suffix not in suffixes and "nan" in suffixes:
        selected_suffix = "nan"

    for suffix in suffixes:
        values[f"{group_name}_{suffix}"] = 1.0 if suffix == selected_suffix else 0.0
```

10.1.6 app_constants.py

```
from __future__ import annotations

from pathlib import Path
```

```
def get_dictionary_paths(project_root: Path) -> list[Path]:
    return [project_root / "Datasets2015" / "Clinical" / "Jonathan Ralph_Baes_2026-03-26141903_data-dictionary_clinical.csv",
          project_root / "Datasets2015" / "Dietary" / "Jonathan Ralph_Baes_2026-03-26141801_data-dictionary_dietary.csv",
          project_root / "Datasets2015" / "Anthropometric" / "Jonathan Ralph_Baes_2026-03-26141834_data-dictionary_anthrop.csv
          "]


FEATURE_DICTIONARY_ALIASES = {"Total_Ener": ["Total_Energy"], "Total_Prot": ["Total_Protein"], "Total_Calc": ["Total_Calcium"
      ], "Total_VitA": ["Total_VitaminA"], "Total_VitC": ["Total_VitaminC"], "Total_Thia": ["Total_Thiamin"], "Total_Ribo": ["
      Total_Riboflavin"], "Total_Nia": ["Total_Niacin"], "Total_Food_epwt": ["Total_FoodIntake"]}


VARIABLE_DEFINITION_OVERRIDES = {
    "age": "Age of Respondent: exact age as of last birthday.",
    "sex": "Sex of Respondent: sex of household member.",
    "ethnicity": "Ethnicity code: 0 Not IP/without foreign blood, 1 Indigenous People, 2 2/3 Filipino, 3 with 1/2 foreign
          blood.",
    "current_smoking": "Presently smoke cigarettes/cigars/pipes/tobacco products: current smoking frequency code.",
    "ever_smk": "Ever smoked in the past: former smoking behavior code.",
    "alcohol": "Ever consumed alcoholic drink such as beer, wine, or spirits.",
    "con_alcohol": "Consumed alcoholic drink within the past 12 months (current drinkers).",
    "drnk_30days": "Consumed alcoholic drink within the past 30 days.",
    "smoke_status": "Smoking Status (Generated): 0 Never, 1 Current, 2 Former, 9 Not Applicable.",
    "alcohol_status": "Alcohol Status (Generated): 0 Never, 1 Current, 2 Former, 9 Not Applicable.",
    "binge_drink": "Binge Drinking Status (Generated): female >=4 standard drinks in a row; male >=5, among those who drank in
           past 30 days.",
    "weight": "Ave Weight (kg): body heaviness from muscle, fat, bone, organs and related conditions.",
    "height": "Ave Height (cm): standing height (or recumbent length for very young children in source survey).",
    "waist": "Ave Waist Circumference (cm): perimeter around natural waist/abdomen.",
    "hip": "Ave Hip Circumference (cm): distance around largest hip/buttocks area.",
    "fg1": "Cereals and Cereal Products (in grams): include rice and rice products, corn and corn products, and other cereal
          products.",
    "fg2": "Rice and Rice Products (in grams): include rice (ordinary, special, glutinous) and rice products like bihon, puto,
           biko, suman, arroz caldo, champorado, and others.",
    "fg3": "Corn and Corn Products (in grams): refer to milled corn, corn on cob, and products like cornstarch, maja blanca,
          popcorn, corn chips, and others.",
    "fg4": "Other Cereal Products (in grams): refer to pandesal, bread, cookies/biscuits, cakes/pastries, noodles, flour, and
          others.",
    "fg5": "Starchy Roots and Tubers (in grams): consist of sweet potatoes and products, potatoes and products, cassava and
          products, and other roots/tubers such as yam, taro, arrowroot, and yakon.",
    "fg6": "Sugar and Syrups (in grams): consist of refined/second class/brown/crude sugars, jams, candies, honey, sweetened
          soda, sherbet, ice candy, chocolates, and others.",
    "fg7": "Dried Beans, Nuts and Seeds (in grams): include mungbeans, soybeans, nuts, and other dried beans/seeds and
          products like almond, peas, garbanzos, sesame seed, patani, mani, taho/tofu/tokwa, and others.",
    "fg8": "Vegetables (in grams): refer to green leafy and yellow vegetables and other vegetables.",
    "fg9": "Green Leafy and Yellow Vegetables (in grams): include camote tops, kangkong, malunggay, alugbati, pechay, squash
          fruit/flower, carrot, and other yellow vegetables.",
    "fg10": "Other Vegetables (in grams): include eggplant, stringbeans, abitsuelas, ampalaya, wild vegetables, and canned/
          processed vegetables like canned mushroom and pickled cucumber.",
    "fg11": "Fruits (in grams): include vitamin C-rich fruits and other fruits.",
    "fg12": "Vitamin C-Rich Fruits (in grams): include mangoes, papaya, citrus fruits, strawberry, guava, and others.",
    "fg13": "Other Fruits (in grams): include bananas, watermelon, melon, jackfruit, pineapple, young coconut, kaimito, and
          others.",
    "fg14": "Fish, Meat and Poultry (in grams): refer to fresh fish, dried fish, processed fish, crustaceans and mollusks,
          fresh meat, organ meat, processed meat, poultry, and others.",
    "fg15": "Fish and Fish Products (in grams): refer to fresh fish (tulingan, bangus, galunggong, tilapia, and others), dried
           fish, processed fish (bagoong isda, patis, canned/smoked fish), and crustaceans/mollusks.",
    "fg16": "Meat and Meat Products (in grams): refer to fresh meat (pork, beef, carabeef, and others), organ meat, and
          processed meat (hotdog, longganisa, tocino, ham, meat loaf, and others).",
    "fg17": "Poultry (in grams): refer to chicken and other fowls like duck, goose, pigeon, turkey, and others.",
    "fg18": "Eggs (in grams): include hen egg, duck egg, and other eggs like ant egg, quail egg, turkey egg, and others.",
    "fg19": "Milk and Milk Products (in grams): consist of fresh whole milk, evaporated milk, recombined milk, powdered milk,
          condensed milk, and other milk products.",
    "fg20": "Whole Milk (in grams): consist of fresh whole milk, evaporated milk, recombined milk, powdered milk (infant
           formula, whole/full cream, filled, skimmed), and condensed milk.",
    "fg21": "Milk Products (in grams): refer to cheese and other milk products like ice cream, yogurt, cultured milk, and
          others.",
    "fg23": "Fats and Oils (in grams): refer to cooking oil, coconut meat, coconut cream, pork drippings/lard, butter,
          margarine, peanut butter, and others.",
    "fg24": "Miscellaneous (in grams): include beverages, condiments/spices, and other miscellaneous items.",
    "fg25": "Beverages (in grams): consist of coffee, tea, alcoholic beverages, cacao/chocolate-based beverages, fruit
          flavored drinks, and others.",
    "fg26": "Condiments and Spices (in grams): consist of salt, vinegar, catsup, and other seasonings.",
    "fg27": "Other Miscellaneous (in grams): include lemon grass, laurel leaves, oregano, turmeric, food coloring, and others.
          ",
    "Total_FoodIntake": "Total Food Intake (g): total intake across 27 food groups.",
    "Total_Energy": "Total Energy (kcal): total energy intake.",
    "Total_Protein": "Total Protein (g): total protein intake.",
    "Total_Calcium": "Total Calcium (mg): total calcium intake.",
    "Total_Iron": "Total Iron (mg): total iron intake.",
    "Total_VitaminA": "Total Vitamin A (mcg RE).",
    "Total_Thiamin": "Total Thiamin (mg).",
    "Total_Riboflavin": "Total Riboflavin (mg).",
    "Total_Niacin": "Total Niacin (mg).",
    "Total_VitaminC": "Total Vitamin C (mg).",
    "Total_CHO": "Total Carbohydrates (g): total carbohydrate intake.",
    "Total_Fat": "Total Fats (g): total fat intake.",
}


VALUE_LABEL_OVERRIDES = {
    "sex": {"1": "Male", "2": "Female"},
    "ethnicity": {
        "0": "No, Not an IP/Without Foreign Blood (default)",
        "1": "Yes, Indigenous People",
        "2": "Yes, 2/3 Filipino",
        "3": "Yes, with 1/2 Foreign Blood",
    },
    "current_smoking": {
        "0": "No, not at all",
        "1": "Yes, once a week",
        "2": "Yes, 2-6 times a week",
        "3": "Yes, every day, 7 times a week",
        "888888": "Not Applicable",
    },
```

```
    "ever_smk": {
        "0": "No, not at all",
        "1": "Yes, once a week",
        "2": "Yes, 2-6 times a week",
        "3": "Yes, every day",
        "4": "Yes, tried once",
        "5": "Yes, occasionally",
        "888888": "Not Applicable",
    },
    "alcohol": {
        "0": "No",
        "1": "Yes",
        "2": "Yes, occasionally, during socials",
        "888888": "Not Applicable",
    },
    "con_alcohol": {"0": "No", "1": "Yes", "999999": "Not Applicable"},
    "drnk_30days": {"0": "No", "1": "Yes", "999999": "Not Applicable"},
    "smoke_status": {"0": "Never", "1": "Current", "2": "Former", "9": "Not Applicable"},
    "alcohol_status": {"0": "Never", "1": "Current", "2": "Former", "9": "Not Applicable"},
    "binge_drink": {"0": "Non-binge drinker", "1": "Binge drinker", "99": "Not Applicable"},
}


DISPLAY_LABEL_OVERRIDES = {
    "age": "Age",
    "sex": "Sex",
    "waist": "Waist Circumference",
    "hip": "Hip Circumference",
    "weight": "Weight",
    "height": "Height",
    "epwt_fg1": "Cereal and Cereal Products",
    "fg14": "Fish, Meat and Poultry (in grams)",
    "epwt_fg14": "Fish, Meat and Poultry (in grams)",
    "smoke_status": "Smoking Status",
    "current_smoking": "Current Smoking Frequency",
    "ever_smk": "Smoking History",
    "alcohol_status": "Alcohol Use Status",
    "alcohol": "Alcohol Consumption",
    "con_alcohol": "Alcohol Use in Past 12 Months",
    "drnk_30days": "Alcohol Use in Past 30 Days",
    "binge_drink": "Binge Drinking Status",
}


AUTO_COMPUTED_TOTAL_FIELDS = {
    "Total_FoodIntake", "Total_Food_epwt", "Total_Energy", "Total_Ener", "Total_Protein", "Total_Prot"
}


FOOD_GROUP_COMPONENT_TOTALS = {
    1: [2, 3, 4],
    8: [9, 10],
    11: [12, 13],
    14: [15, 16, 17, 18],
    19: [20, 21],
    24: [25, 26, 27],
}


MISSING_INPUT_CODES = {9.0, 99.0, 888888.0, 999999.0}

CONDITIONALLY_ALLOWED_NA_CODES: dict[str, set[float]] = {
    "ever_smk": {888888.0},
    "current_smoking": {888888.0},
    "con_alcohol": {999999.0},
    "drnk_30days": {999999.0},
    "binge_drink": {99.0},
}


NO_HELP_FEATURES = {"Total_VitA", "Total_VitC", "Total_Thia", "Total_Ribo", "Total_Nia", "Total_VitaminA", "Total_VitaminC", "
      Total_Thiamin", "Total_Riboflavin", "Total_Niacin"}


FEATURE_UNITS = {
    "age": "years",
    "weight": "kg",
    "height": "cm",
    "waist": "cm",
    "hip": "cm",
    "BMI": "kg/m^2",
    "bmi": "kg/m^2",
    "whr": "ratio",
    "hemoglobin": "g/dL",
    "Total_FoodIntake": "g/day",
    "Total_Food_epwt": "g/day",
    "Total_Energy": "kcal/day",
    "Total_Ener": "kcal/day",
    "Total_Protein": "g/day",
    "Total_Prot": "g/day",
    "Total_CHO": "g/day",
    "Total_Fat": "g/day",
    "Total_Calcium": "mg",
    "Total_Calc": "mg",
    "Total_Iron": "mg",
    "Total_VitaminA": "mcg RE/day",
    "Total_VitA": "mcg RE/day",
    "Total_VitaminC": "mg",
    "Total_VitC": "mg",
    "Total_Thiamin": "mg",
    "Total_Thia": "mg",
    "Total_Riboflavin": "mg",
    "Total_Ribo": "mg",
    "Total_Niacin": "mg",
    "Total_Nia": "mg",
}
```

```
# Do not append a redundant unit to food-group labels because
# the dictionary names already include "(in grams)".
```

10.1.7 counterfactuals.py

```
from __future__ import annotations

from typing import Any

import numpy as np
import pandas as pd

from app_constants import DISPLAY_LABEL_OVERRIDES, FOOD_GROUP_COMPONENT_TOTALS
from backend import (
    RANGE_HINTS,
    build_input_values_from_widgets,
    feature_range,
    make_input_frame,
    predict_with_venn_abers,
    prepare_model_input,
)


NON_ACTIONABLE_FEATURES = {"age", "sex", "ethnicity", "ethnicity_group"}
REDUCE_ONLY_FEATURES = {"BMI", "bmi", "waist", "hip"}


def _carla_score(reduction_pct: float, relative_change_pct: float) -> float:
    """CARLA-style proximity-adjusted utility.

    Higher reduction with smaller relative change scores highest.
    Equivalent to CARLA's Wachter objective: maximise gain, minimise cost.
    """
    return reduction_pct / (1.0 + relative_change_pct / 100.0)


def _display_label(fname: str, dictionary_labels: dict[str, str]) -> str:
    """Return a clean display label - same priority order as the form UI."""
    if fname in DISPLAY_LABEL_OVERRIDES:
        return DISPLAY_LABEL_OVERRIDES[fname]
    raw = dictionary_labels.get(fname, "")
    if raw:
        return raw.split(":", 1)[0].strip().replace("  ", " ")
    return fname.replace("_", " ").strip().title()


def compute_counterfactuals(
    model: Any,
    preprocessor: Any,
    calibrator: Any,
    all_widget_values: dict,
    input_feature_names: list[str],
    dictionary_labels: dict[str, str],
    current_probability: float,
    top_n: int = 8,
) -> pd.DataFrame:
    from scipy.optimize import minimize_scalar  # type: ignore

    LAMBDA = 0.5
    Y_TARGET = 0.34
    DECISION_BOUNDARY = 0.35
    GRID_STEPS = 60  # fallback grid resolution when Wachter can't cross 0.35

    # Auto-sum parent food groups (e.g. fg14 = fg15+fg16+fg17+fg18) are not
    # directly actionable - skip them. Leaf food groups (fg15, fg16, ...) are
    # directly user-entered and should be scanned.
    _parent_fg_indices = frozenset(FOOD_GROUP_COMPONENT_TOTALS.keys())

    def _is_parent_food_group(name: str) -> bool:
        for prefix in ("epwt_fg", "fg"):
            if name.startswith(prefix):
                suffix = name[len(prefix):]
                return suffix.isdigit() and int(suffix) in _parent_fg_indices
        return False

    scan_features = [
        fname for fname in all_widget_values
        if fname in RANGE_HINTS
        and fname not in NON_ACTIONABLE_FEATURES
        and not _is_parent_food_group(fname)
    ]

    def _predict_for_value(fname: str, val: float) -> float:
        test_widget = dict(all_widget_values)
        test_widget[fname] = val
        try:
            rebuilt = build_input_values_from_widgets(input_feature_names, test_widget)
            frame = make_input_frame(input_feature_names, rebuilt)
            model_frame = prepare_model_input(frame, preprocessor)
            pred_result = predict_with_venn_abers(model, model_frame, calibrator)
            return pred_result.calibrated_probability
        except Exception:
            return current_probability

    rows = []
    for fname in scan_features:
        current_val = all_widget_values.get(fname)
        if current_val is None or (isinstance(current_val, float) and np.isnan(current_val)):
            continue
        current_val = float(current_val)
```

```
            minimum, maximum, _ = feature_range(fname)
            base_range_width = max(maximum - minimum, 1e-6)
            if fname in REDUCE_ONLY_FEATURES:
                maximum = current_val
            if minimum >= maximum:
                continue

            feature_range_width = base_range_width

            # -- Stage 1: Wachter minimisation - minimal change to cross 0.5 --
            def _wachter_loss(val: float, _fname=fname, _cur=current_val, _rng=feature_range_width) -> float:
                prob = _predict_for_value(_fname, val)
                pred_loss = (prob - Y_TARGET) ** 2
                proximity = ((val - _cur) / _rng) ** 2
                return LAMBDA * pred_loss + proximity

            try:
                opt = minimize_scalar(
                    _wachter_loss,
                    bounds=(minimum, maximum),
                    method="bounded",
                    options={"xatol": 1e-3, "maxiter": 200},
                )
                cf_val = float(np.clip(opt.x, minimum, maximum))
            except Exception:
                continue

            cf_prob = _predict_for_value(fname, cf_val)

            # -- Stage 2: if Wachter didn't cross 0.5, grid-search for the
            #    value that achieves the greatest absolute risk reduction --
            if cf_prob >= DECISION_BOUNDARY:
                grid = np.linspace(minimum, maximum, GRID_STEPS)
                grid_probs = np.array([_predict_for_value(fname, v) for v in grid])
                best_idx = int(np.argmin(grid_probs))
                grid_best_val = float(grid[best_idx])
                grid_best_prob = float(grid_probs[best_idx])
                if grid_best_prob < cf_prob:
                    cf_val = grid_best_val
                    cf_prob = grid_best_prob

            reduction = (current_probability - cf_prob) * 100.0
            abs_delta = abs(cf_val - current_val)
            relative_delta_pct = 0.0 if abs(current_val) < 1e-6 else (abs_delta / max(abs(current_val), 1e-6)) * 100.0

            score = _carla_score(reduction, relative_delta_pct)

            if reduction <= 0.05 or np.isclose(cf_val, current_val, atol=1e-3):
                continue

            rows.append({
                "Feature": _display_label(fname, dictionary_labels),
                "Current Value": round(current_val, 2),
                "Suggested Value": round(cf_val, 2),
                "Current Risk": f"{current_probability * 100:.1f}%",
                "Projected Risk": f"{cf_prob * 100:.1f}%",
                "Risk Reduction": f"{reduction:.1f}%",
                "CARLA Score": round(score, 3),
                "_delta": reduction,
            })

        if not rows:
            return pd.DataFrame()

        df = pd.DataFrame(rows)
        # Primary ranking: CARLA-style proximity-adjusted utility (higher = better tradeoff).
        df = (
            df.sort_values(["CARLA Score", "_delta"], ascending=[False, False])
            .drop(columns=["_delta"])
            .head(top_n)
            .reset_index(drop=True)
        )
        return df


    def compute_counterfactuals_safe(
        model: Any,
        preprocessor: Any,
        calibrator: Any,
        all_widget_values: dict,
        input_feature_names: list[str],
        dictionary_labels: dict[str, str],
        current_probability: float,
        top_n: int = 8,
        timeout: int = 180,
    ) -> pd.DataFrame:
        """Run compute_counterfactuals in an isolated subprocess.

        Prevents XGBoost's OpenMP thread pool from being initialised in the
        Streamlit main process, which would cause os._exit() on Windows when
        Streamlit's own threads try to run concurrently.

        If subprocess transport cannot be prepared, return an empty result instead
        of falling back to in-process computation.
        """
        import pathlib
        import pickle
        import subprocess
        import sys
        import tempfile
        import uuid

        _dir = pathlib.Path(__file__).parent
        worker = _dir / "_cf_worker.py"

        # Verify everything is picklable before spawning
```

```
    payload = {
        "model": model,
        "preprocessor": preprocessor,
        "calibrator": calibrator,
        "all_widget_values": all_widget_values,
        "input_feature_names": input_feature_names,
        "dictionary_labels": dictionary_labels,
        "current_probability": current_probability,
        "top_n": top_n,
    }
    try:
        _bytes = pickle.dumps(payload)
        del _bytes
    except Exception:
        # Not picklable - keep the app alive and degrade gracefully.
        return pd.DataFrame()

    uid = uuid.uuid4().hex
    tmp_dir = pathlib.Path(tempfile.gettempdir())
    tmp_in = tmp_dir / f"_cf_in_{uid}.pkl"
    tmp_out = tmp_dir / f"_cf_out_{uid}.pkl"

    try:
        with open(tmp_in, "wb") as fh:
            pickle.dump(payload, fh)

        try:
            proc = subprocess.Popen(
                [sys.executable, str(worker), str(tmp_in), str(tmp_out)],
                cwd=str(_dir),
                stdout=subprocess.DEVNULL,
                stderr=subprocess.PIPE,
            )
        except Exception:
            return pd.DataFrame()

        try:
            returncode = proc.wait(timeout=timeout)
        except subprocess.TimeoutExpired:
            proc.kill()
            proc.wait()
            return pd.DataFrame()

        if not tmp_out.exists():
            return pd.DataFrame()

        try:
            with open(tmp_out, "rb") as fh:
                result = pickle.load(fh)
        except Exception:
            return pd.DataFrame()

        return result if isinstance(result, pd.DataFrame) else pd.DataFrame()

    except Exception:
        return pd.DataFrame()
    finally:
        for f in (tmp_in, tmp_out):
            try:
                f.unlink(missing_ok=True)
            except Exception:
                pass
```

10.1.8 explainability.py

```
from __future__ import annotations

from typing import Any

import numpy as np
import pandas as pd

from backend import feature_default, feature_range


def _repeat_base_frame(base_input_frame: pd.DataFrame, rows: int) -> pd.DataFrame:
    return pd.concat([base_input_frame.iloc[[0]].copy()] * rows, ignore_index=True)


def _prediction_fn_for_subset_explainability(
    model: Any,
    explain_feature_names: list[str],
    base_input_frame: pd.DataFrame,
):
    def _predict(nd_array: np.ndarray) -> np.ndarray:
        subset_frame = pd.DataFrame(nd_array, columns=explain_feature_names).replace([np.inf, -np.inf], np.nan)
        full_frame = _repeat_base_frame(base_input_frame, len(subset_frame))

        for feature_name in explain_feature_names:
            full_frame[feature_name] = pd.to_numeric(subset_frame[feature_name], errors="coerce").fillna(float(
                  base_input_frame.iloc[0].get(feature_name, 0.0)))
        return np.asarray(model.predict_proba(full_frame))[:, 1]

    return _predict


def _build_subset_background_samples(
    feature_names: list[str],
    base_input_frame: pd.DataFrame,
    rows: int = 80,
) -> pd.DataFrame:
    rng = np.random.default_rng(42)
    records: list[dict[str, float]] = []
```

```
    base_row = base_input_frame.iloc[0]

    for _ in range(rows):
        row: dict[str, float] = {}
        for feature_name in feature_names:
            base_value = float(base_row.get(feature_name, 0.0))

            if "__" in feature_name:
                spread = max(abs(base_value) * 0.08, 0.03)
                row[feature_name] = float(rng.normal(loc=base_value, scale=spread))
                continue

            minimum, maximum, _ = feature_range(feature_name)
            default_value = feature_default(feature_name)

            if maximum <= minimum:
                row[feature_name] = float(default_value)
                continue

            spread = (maximum - minimum) * 0.08
            sampled = float(rng.normal(loc=default_value, scale=max(spread, 1e-6)))
            row[feature_name] = float(np.clip(sampled, minimum, maximum))

        records.append(row)

    return pd.DataFrame(records, columns=feature_names)


def _build_full_frame_background(base_input_frame: pd.DataFrame, rows: int = 20) -> pd.DataFrame:
    """Build a noisy background dataset in the full model-feature space."""
    rng = np.random.default_rng(42)
    base = base_input_frame.iloc[[0]].copy()
    records = []
    for _ in range(rows):
        row = base.copy()
        for col in base.columns:
            val = float(base.iloc[0][col])
            spread = max(abs(val) * 0.10, 0.05)
            row[col] = float(np.clip(rng.normal(val, spread), val - 3 * spread, val + 3 * spread))
        records.append(row)
    return pd.concat(records, ignore_index=True)


def try_compute_shap(
    model: Any,
    feature_names: list[str],
    input_frame: pd.DataFrame,
    base_input_frame: pd.DataFrame | None = None,
):
    try:
        import shap  # type: ignore
    except Exception:
        return None, None, "SHAP package is not installed. Install with: pip install shap"

    try:
        if base_input_frame is None:
            base_input_frame = input_frame

        # --- Try TreeExplainer first (XGBoost, LightGBM, CatBoost, RF, etc.) -----------
        # TreeExplainer reads tree structure directly - zero prediction calls,
        # no OpenMP thread-pool activation, no os._exit() risk, ~100x faster.
        _tree_ok = False
        try:
            _tree_exp = shap.TreeExplainer(model)
            _tree_ok = True
        except Exception:
            pass

        if _tree_ok:
            # Local: SHAP for the single test instance (use full model frame)
            _local_sv = np.asarray(_tree_exp.shap_values(base_input_frame))
            # Binary classifiers may return shape (2, n_samples, n_feat) or (n_samples, n_feat)
            if _local_sv.ndim == 3:
                _local_sv = _local_sv[1]          # class-1 slice
            _local_flat = _local_sv.reshape(-1)   # (n_all_features,)

            _all_cols = list(base_input_frame.columns)
            _col_idx = {c: i for i, c in enumerate(_all_cols)}
            _local_vals = np.array([
                _local_flat[_col_idx[f]] if f in _col_idx else 0.0
                for f in feature_names
            ])
            local_df = pd.DataFrame({
                "feature": feature_names,
                "shap_value": _local_vals,
                "abs_shap": np.abs(_local_vals),
            }).sort_values("abs_shap", ascending=False)

            # Global: mean |SHAP| over background rows
            _bg = _build_full_frame_background(base_input_frame, rows=20)
            _global_sv = np.asarray(_tree_exp.shap_values(_bg))
            if _global_sv.ndim == 3:
                _global_sv = _global_sv[1]
            _global_importance_full = np.mean(np.abs(_global_sv), axis=0)
            _global_vals = np.array([
                _global_importance_full[_col_idx[f]] if f in _col_idx else 0.0
                for f in feature_names
            ])
            global_df = pd.DataFrame({
                "feature": feature_names,
                "mean_abs_shap": _global_vals,
            }).sort_values("mean_abs_shap", ascending=False)

            return local_df, global_df, None

        # --- KernelExplainer fallback (non-tree models) --------------------------------
```

```
        background = _build_subset_background_samples(feature_names, base_input_frame, rows=20)
        predict_fn = _prediction_fn_for_subset_explainability(model, feature_names, base_input_frame)
        explainer = shap.KernelExplainer(predict_fn, background.values)

        local_shap_values = explainer.shap_values(input_frame.values, nsamples=50, silent=True)
        local_values = np.asarray(local_shap_values).reshape(-1)
        local_df = pd.DataFrame(
            {
                "feature": feature_names,
                "shap_value": local_values,
                "abs_shap": np.abs(local_values),
            }
        ).sort_values("abs_shap", ascending=False)

        global_shap_values = explainer.shap_values(background.values, nsamples=20, silent=True)
        global_array = np.asarray(global_shap_values)
        if global_array.ndim == 1:
            global_array = global_array.reshape(1, -1)
        global_importance = np.mean(np.abs(global_array), axis=0)
        global_df = pd.DataFrame(
            {
                "feature": feature_names,
                "mean_abs_shap": global_importance,
            }
        ).sort_values("mean_abs_shap", ascending=False)

        return local_df, global_df, None
    except BaseException as exc:
        if isinstance(exc, SystemExit):
            return None, None, f"SHAP computation failed: sys.exit({exc.code})"
        if not isinstance(exc, Exception):
            raise  # Re-raise StopException, RerunException, KeyboardInterrupt
        return None, None, f"SHAP computation failed: {exc}"


def try_compute_lime(
    model: Any,
    feature_names: list[str],
    input_frame: pd.DataFrame,
    base_input_frame: pd.DataFrame | None = None,
):
    try:
        from lime.lime_tabular import LimeTabularExplainer  # type: ignore
    except Exception:
        return None, "LIME package is not installed. Install with: pip install lime"

    if base_input_frame is None:
        base_input_frame = input_frame

    # Build background in model-feature space (i.e. the same standardised/OHE
    # scale that explain_input_frame lives in).  Using raw feature ranges here
    # is wrong because the model sees z-scored values; perturbing around the
    # actual test instance creates a valid local neighbourhood for LIME.
    rng_bg = np.random.default_rng(42)
    base_vals = input_frame.values  # shape (1, n_explain_features)
    noise = rng_bg.normal(0.0, 0.35, size=(50, len(feature_names)))
    bg_array = np.clip(base_vals + noise, -10.0, 10.0)
    background = pd.DataFrame(bg_array, columns=feature_names).replace([np.inf, -np.inf], np.nan)
    for name in feature_names:
        background[name] = pd.to_numeric(background[name], errors="coerce").fillna(float(input_frame.iloc[0].get(name, 0.0)))

    # Use the same full-frame predict approach as SHAP so the model always
    # receives all expected columns (non-explain columns filled from base_input_frame).
    _subset_predict_fn = _prediction_fn_for_subset_explainability(model, feature_names, base_input_frame)

    # Detect XGBoost booster so we can predict via DMatrix (single-threaded,
    # no OpenMP thread-pool activation, safe inside Streamlit on Windows).
    _xgb_booster = None
    _xgb_col_order: list[str] | None = None
    try:
        if hasattr(model, "get_booster"):
            import xgboost as _xgb  # type: ignore
            _xgb_booster = model.get_booster()
            _xgb_booster.set_param("nthread", 1)
            # Booster expects columns in the order it was trained on
            _xgb_col_order = list(base_input_frame.columns)
    except Exception:
        _xgb_booster = None

    def _lime_predict(nd_array: np.ndarray) -> np.ndarray:
        # Reconstruct full-feature frame from LIME-perturbed subset columns
        subset_frame = pd.DataFrame(nd_array, columns=feature_names).replace([np.inf, -np.inf], np.nan)
        full_frame = _repeat_base_frame(base_input_frame, len(subset_frame))
        for fname in feature_names:
            full_frame[fname] = pd.to_numeric(subset_frame[fname], errors="coerce").fillna(
                float(base_input_frame.iloc[0].get(fname, 0.0))
            )
        if _xgb_booster is not None:
            try:
                import xgboost as _xgb  # type: ignore
                if _xgb_col_order is not None:
                    full_frame = full_frame.reindex(columns=_xgb_col_order, fill_value=0.0)
                dm = _xgb.DMatrix(full_frame)
                proba_1 = _xgb_booster.predict(dm)
                return np.column_stack([1.0 - proba_1, proba_1])
            except Exception:
                pass
        proba_1 = np.asarray(model.predict_proba(full_frame))[:, 1]
        return np.column_stack([1.0 - proba_1, proba_1])

    lime_attempts = [
        {"discretize_continuous": True, "num_features": min(12, len(feature_names)), "num_samples": 500},
        {"discretize_continuous": False, "num_features": min(10, len(feature_names)), "num_samples": 300},
    ]

    last_error: Exception | None = None
```

```
    for attempt in lime_attempts:
        try:
            explainer = LimeTabularExplainer(
                training_data=background.values,
                feature_names=feature_names,
                class_names=["No HTN", "HTN"],
                mode="classification",
                discretize_continuous=attempt["discretize_continuous"],
                random_state=42,
            )

            explanation = explainer.explain_instance(
                data_row=input_frame.iloc[0].values,
                predict_fn=_lime_predict,
                num_features=int(attempt["num_features"]),
                num_samples=int(attempt["num_samples"]),
                top_labels=1,
            )

            available_labels = sorted(getattr(explanation, "local_exp", {}).keys())
            selected_label = 1 if 1 in available_labels else (available_labels[0] if available_labels else None)
            if selected_label is None:
                last_error = ValueError("LIME produced no label explanations")
                continue
            local_exp = getattr(explanation, "local_exp", {}).get(int(selected_label), [])
            if not local_exp:
                last_error = ValueError("LIME explanation returned zero feature contributions")
                continue

            rows = []
            for feat_idx, weight in local_exp:
                feat_idx_int = int(feat_idx)
                feat_name = feature_names[feat_idx_int] if 0 <= feat_idx_int < len(feature_names) else f"feature_{feat_idx_int
                    }"
                rule_text = ""
                try:
                    as_map = explanation.as_map()
                    label_map = as_map.get(int(selected_label), [])
                    # Use same index position from local_exp to fetch rule string.
                    pair_pos = next((i for i, p in enumerate(label_map) if int(p[0]) == feat_idx_int), None)
                    if pair_pos is not None:
                        pairs_txt = explanation.as_list(label=int(selected_label))
                        if pair_pos < len(pairs_txt):
                            rule_text = str(pairs_txt[pair_pos][0])
                except Exception:
                    rule_text = ""
                rows.append({"feature": feat_name, "rule": rule_text, "weight": float(weight)})

            if not rows:
                last_error = ValueError("LIME explanation returned zero feature contributions")
                continue

            return pd.DataFrame(rows, columns=["feature", "rule", "weight"]), None
        except BaseException as exc:
            if isinstance(exc, SystemExit):
                last_error = RuntimeError(f"sys.exit({exc.code})")
                continue
            if not isinstance(exc, Exception):
                raise  # Re-raise StopException, RerunException, KeyboardInterrupt
            last_error = exc

    return None, f"LIME computation failed: {last_error}"


def compute_explainability_safe(
    model: Any,
    feature_names: list[str],
    input_frame: pd.DataFrame,
    base_input_frame: pd.DataFrame,
    timeout: int = 120,
) -> tuple:
    """Run SHAP + LIME in an isolated subprocess.

    If the subprocess crashes for any reason (os._exit, GPU OOM, C-level
    fault, tqdm writing to a corrupted pipe), the calling Streamlit process
    is completely unaffected.  Results are exchanged via temp files so the
    Streamlit file-watcher never picks them up.

    Falls back to in-process computation if the model object cannot be
    pickled (required for subprocess transport).
    """
    import pickle
    import pathlib
    import subprocess
    import sys
    import tempfile
    import uuid

    _dir = pathlib.Path(__file__).parent
    worker = _dir / "_exp_worker.py"

    # -- 1. Verify model is picklable (required for subprocess transport) --
    try:
        _bytes = pickle.dumps(model)
        del _bytes
    except Exception:
        # Model not picklable - run in-process (no crash isolation)
        shap_local, shap_global, shap_err = try_compute_shap(
            model, feature_names, input_frame, base_input_frame
        )
        lime_df, lime_err = try_compute_lime(
            model, feature_names, input_frame, base_input_frame
        )
        return shap_local, shap_global, shap_err, lime_df, lime_err

    # -- 2. Write payload to system temp dir (NOT the webapp dir, so the
```

```
    #       Streamlit watchdog never picks up the temp files) --
    uid = uuid.uuid4().hex
    tmp_dir = pathlib.Path(tempfile.gettempdir())
    tmp_in = tmp_dir / f"_shap_in_{uid}.pkl"
    tmp_out = tmp_dir / f"_shap_out_{uid}.pkl"

    payload = {
        "model": model,
        "feature_names": list(feature_names),
        "input_frame": input_frame.to_dict("list"),
        "base_input_frame": base_input_frame.to_dict("list"),
    }

    try:
        with open(tmp_in, "wb") as fh:
            pickle.dump(payload, fh)

        # -- 3. Spawn worker; suppress all output so nothing corrupts pipes --
        proc = subprocess.Popen(
            [sys.executable, str(worker), str(tmp_in), str(tmp_out)],
            cwd=str(_dir),
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
        )

        try:
            returncode = proc.wait(timeout=timeout)
        except subprocess.TimeoutExpired:
            proc.kill()
            proc.wait()
            msg = f"Explainability timed out after {timeout}s"
            return None, None, msg, None, msg

        if not tmp_out.exists():
            msg = f"Explainability subprocess exited (code {returncode}) without writing results"
            return None, None, msg, None, msg

        # -- 4. Read results --
        with open(tmp_out, "rb") as fh:
            result = pickle.load(fh)

        if not result.get("ok"):
            err = result.get("error", "Unknown subprocess error")
            return None, None, err, None, err

        return (
            result["shap_local"],
            result["shap_global"],
            result["shap_error"],
            result["lime_df"],
            result["lime_error"],
        )

    except Exception as exc:
        msg = f"Explainability error: {exc}"
        return None, None, msg, None, msg

    finally:
        for _p in (tmp_in, tmp_out):
            try:
                _p.unlink(missing_ok=True)
            except Exception:
                pass
```

10.1.9 styles.py

```
from __future__ import annotations

import streamlit as st


APP_CSS = """
<style>
    .stApp {
        background:
            radial-gradient(circle at top left, rgba(220, 38, 38, 0.10), transparent 30%),
            radial-gradient(circle at top right, rgba(37, 99, 235, 0.08), transparent 26%),
            linear-gradient(180deg, #f8f5f2 0%, #f2ede8 45%, #fcfbfa 100%);
        color: #111827;
    }
    .stMarkdown,
    p,
    li,
    label,
    span,
    [data-testid="stCaptionContainer"],
    [data-testid="stText"],
    [data-testid="stMarkdownContainer"] p,
    [data-testid="stMarkdownContainer"] li,
    [data-testid="stMarkdownContainer"] span,
    [data-testid="stMetricLabel"],
    [data-testid="stMetricValue"],
    [data-testid="stDataFrame"] {
        color: #0b1220 !important;
    }
    .stCaption, [data-testid="stCaptionContainer"] p {
        color: #111827 !important;
        font-weight: 500;
    }
    [data-testid="stMarkdownContainer"] strong {
        color: #0b1220;
    }
    [data-testid="stTextInput"] input,
```

```
[data-testid="stNumberInput"] input,
[data-testid="stSelectbox"] div[role="combobox"] {
    color: #0b1220 !important;
}
[data-testid="stTextInput"] label,
[data-testid="stNumberInput"] label,
[data-testid="stSelectbox"] label {
    color: #0b1220 !important;
    font-weight: 600;
}
#MainMenu,
header,
footer,
.stAppHeader,
[data-testid="stHeader"],
[data-testid="stToolbar"],
[data-testid="stDecoration"] {
    visibility: hidden !important;
    display: none !important;
    height: 0 !important;
}
.block-container { padding-top: 0 !important; }
[data-testid="stAppViewContainer"] > .main {
    padding-top: 0 !important;
}
[data-testid="stAppViewContainer"] .main .block-container {
    padding-top: 0 !important;
    margin-top: 0 !important;
}
.landing-hero {
    background: linear-gradient(145deg, #0f172a 0%, #7f1d1d 55%, #1e3a5f 100%);
    color: #fff;
    padding: 2rem 2rem 5.75rem;
    text-align: center;
    position: relative;
    overflow: hidden;
    border-radius: 0 0 56px 56px;
    margin: -2.75rem -1rem 2.6rem;
    min-height: 29rem;
}
.landing-hero::before {
    content: "";
    position: absolute;
    inset: 0;
    background:
        radial-gradient(circle at 18% 45%, rgba(220,38,38,0.28) 0%, transparent 52%),
        radial-gradient(circle at 82% 18%, rgba(37,99,235,0.22) 0%, transparent 44%);
    pointer-events: none;
}
.landing-hero h1,
.landing-hero h1 *,
.landing-hero h1 span {
    font-size: 2.5rem; font-weight: 800; letter-spacing: -0.025em;
    margin: 0 0 0.9rem; position: relative;
    line-height: 1.2;
    color: #ffffff !important;
}
.landing-badge {
    display: inline-block;
    background: rgba(255,255,255,0.13);
    border: 1px solid rgba(255,255,255,0.24);
    border-radius: 999px;
    padding: 0.3rem 1.1rem;
    font-size: 0.82rem; letter-spacing: 0.12em; text-transform: uppercase;
    margin-bottom: 1.4rem; position: relative;
}
.landing-sub {
    font-size: 1.08rem; color: #ffffff !important;
    max-width: 760px; margin: 0 auto; line-height: 1.72; position: relative;
    text-align: center;
    display: block;
    width: 100%;
}
.landing-hero p {
    color: #ffffff !important;
}
.feature-cards {
    display: flex; gap: 1.2rem; justify-content: center;
    flex-wrap: wrap; padding: 2.4rem 1rem 0.4rem;
}
.feature-card {
    background: rgba(255,255,255,0.82);
    border: 1px solid rgba(24,33,47,0.09);
    border-radius: 20px; padding: 1.55rem 1.6rem; width: 210px;
    box-shadow: 0 8px 30px rgba(15,23,42,0.07); text-align: center;
}
.feature-card .fc-icon { font-size: 2rem; margin-bottom: 0.55rem; }
.feature-card .fc-title { font-weight: 700; font-size: 0.96rem; color: #0f172a; margin-bottom: 0.3rem; }
.feature-card .fc-desc { font-size: 0.83rem; color: #1f2937; line-height: 1.5; }
.landing-disclaimer {
    text-align: center; color: #0b1220 !important; font-size: 0.78rem;
    margin-top: 1.1rem; padding-bottom: 0.2rem;
}
.glass-card {
    padding: 1rem 1.1rem; border-radius: 22px;
    border: 1px solid rgba(24,33,47,0.08);
    background: rgba(255,255,255,0.78);
    box-shadow: 0 18px 40px rgba(15,23,42,0.07);
    margin-bottom: 1rem;
}
.section-header {
    padding: 1.6rem 0 0.4rem;
    border-bottom: 2px solid rgba(220,38,38,0.18);
    margin-bottom: 1.3rem;
}
.section-header h2 { font-size: 1.55rem; font-weight: 700; margin: 0; color: #0f172a; }
```

```
    .section-header p { color: #1e293b; margin: 0.25rem 0 0; font-size: 0.93rem; font-weight: 500; }
    .output-hero {
        background: linear-gradient(135deg, rgba(17,24,39,0.94), rgba(127,29,29,0.88));
        border-radius: 22px; padding: 1.4rem 1.8rem; color: #fff; margin-bottom: 0;
    }
    .output-hero .oh-label { font-size: 0.74rem; text-transform: uppercase; letter-spacing: 0.15em; color: rgba
          (255,255,255,0.93); }
    .output-hero .oh-value { font-size: 2.1rem; font-weight: 800; margin-top: 0.1rem; }
    .output-hero .oh-sub { font-size: 0.87rem; color: rgba(255,255,255,0.96); margin-top: 0.15rem; }
    .risk-badge { display: inline-block; border-radius: 999px; padding: 0.32rem 1rem; font-size: 0.95rem; font-weight: 700; }
    .risk-low        { background: #dcfce7; color: #15803d; }
    .risk-medium     { background: #fef9c3; color: #92400e; }
    .risk-high       { background: #fee2e2; color: #b91c1c; }
    .risk-at-risk    { background: #fee2e2; color: #b91c1c; }
    .risk-not-at-risk{ background: #dcfce7; color: #15803d; }
    .kpi-title { font-size: 0.78rem; text-transform: uppercase; letter-spacing: 0.18em; color: #1e293b; }
    .kpi-value { font-size: 1.8rem; font-weight: 700; margin-top: 0.15rem; color: #0f172a; }
    .kpi-subtitle { color: #1f2937; font-size: 0.92rem; margin-top: 0.18rem; }
    @media (max-width: 768px) {
        .output-hero {
            margin-bottom: 0.85rem;
        }
    }
    button[data-testid="stNumberInputStepDown"],
    button[data-testid="stNumberInputStepUp"] {
        display: none !important;
    }
    section[data-testid="stSidebar"] {
        background: rgba(255,255,255,0.82);
        border-right: 1px solid rgba(15,23,42,0.08);
    }
</style>
"""


def apply_global_styles() -> None:
    st.markdown(APP_CSS, unsafe_allow_html=True)
```

10.1.10 _cf_worker.py

```
"""Subprocess worker for counterfactual computation.

Invoked as:
    python _cf_worker.py <input_pickle> <output_pickle>

Reads a payload dict from <input_pickle>, runs compute_counterfactuals,
and writes the resulting DataFrame to <output_pickle>.
Any crash here is contained and does NOT affect the Streamlit main process.
"""
from __future__ import annotations

import os

# MUST be set before any ML library (XGBoost, LightGBM, sklearn) is imported.
# Without these, XGBoost's OpenMP thread-pool calls os._exit() on the first
# predict_proba inside a fresh subprocess on Windows, silently killing the worker.
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("KMP_DUPLICATE_LIB_OK", "TRUE")
os.environ.setdefault("MKL_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

import pickle
import sys

import pandas as pd


def main() -> None:
    if len(sys.argv) != 3:
        sys.exit(1)

    tmp_in = sys.argv[1]
    tmp_out = sys.argv[2]

    try:
        with open(tmp_in, "rb") as fh:
            payload = pickle.load(fh)

        from counterfactuals import compute_counterfactuals  # noqa: PLC0415

        cf_df = compute_counterfactuals(
            model=payload["model"],
            preprocessor=payload["preprocessor"],
            calibrator=payload["calibrator"],
            all_widget_values=payload["all_widget_values"],
            input_feature_names=payload["input_feature_names"],
            dictionary_labels=payload["dictionary_labels"],
            current_probability=payload["current_probability"],
        )
    except Exception:
        cf_df = pd.DataFrame()

    with open(tmp_out, "wb") as fh:
        pickle.dump(cf_df, fh)


if __name__ == "__main__":
    main()
```

10.1.11 _exp_worker.py

```
"""Standalone SHAP/LIME worker - always run as __main__ in a subprocess.

Any crash (os._exit, GPU OOM, C-level fault, tqdm stdout corruption) stays
contained here and never reaches the Streamlit server process.
"""
import sys
import pickle
import warnings

# Silence all warnings so nothing writes to stdout/stderr and corrupts pipes.
warnings.filterwarnings("ignore")

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit(1)

    in_path, out_path = sys.argv[1], sys.argv[2]
    result: dict = {}

    try:
        import pandas as pd  # noqa: PLC0415
        from explainability import try_compute_shap, try_compute_lime  # noqa: PLC0415

        with open(in_path, "rb") as fh:
            payload = pickle.load(fh)

        model = payload["model"]
        feature_names: list = payload["feature_names"]
        input_frame = pd.DataFrame(payload["input_frame"])
        base_input_frame = pd.DataFrame(payload["base_input_frame"])

        shap_local_df, shap_global_df, shap_error = try_compute_shap(
            model, feature_names, input_frame, base_input_frame
        )
        lime_df, lime_error = try_compute_lime(
            model, feature_names, input_frame, base_input_frame
        )

        result = {
            "ok": True,
            "shap_local": shap_local_df,
            "shap_global": shap_global_df,
            "shap_error": shap_error,
            "lime_df": lime_df,
            "lime_error": lime_error,
        }

    except Exception as exc:  # noqa: BLE001
        import traceback  # noqa: PLC0415
        result = {
            "ok": False,
            "error": f"{type(exc).__name__}: {exc}",
            "tb": traceback.format_exc(),
        }

    try:
        with open(out_path, "wb") as fh:
            pickle.dump(result, fh)
    except Exception:  # noqa: BLE001
        pass  # Nothing more we can do
```

10.1.12 _predict_worker.py

```
"""Subprocess worker for prediction.

Invoked as:
    python _predict_worker.py <input_pickle> <output_pickle>

Reads a payload dict, calls predict_with_venn_abers, writes PredictionResult.
Any crash here is contained and does NOT affect the Streamlit main process.
"""
from __future__ import annotations

import pickle
import sys


def main() -> None:
    if len(sys.argv) != 3:
        sys.exit(1)

    tmp_in = sys.argv[1]
    tmp_out = sys.argv[2]

    with open(tmp_in, "rb") as fh:
        payload = pickle.load(fh)

    from backend import predict_with_venn_abers  # noqa: PLC0415

    result = predict_with_venn_abers(
        model=payload["model"],
        input_frame=payload["input_frame"],
        calibrator=payload["calibrator"],
    )

    with open(tmp_out, "wb") as fh:
        pickle.dump(result, fh)


if __name__ == "__main__":
```

```
main()
```

10.2 Email Attachments - Data Collection

This section provides the documentation and approval emails regarding the acquisition of the 2015 National Nutrition Survey (NNS) datasets from the Department of Science and Technology - Food and Nutrition Research Institute (DOST-FNRI).

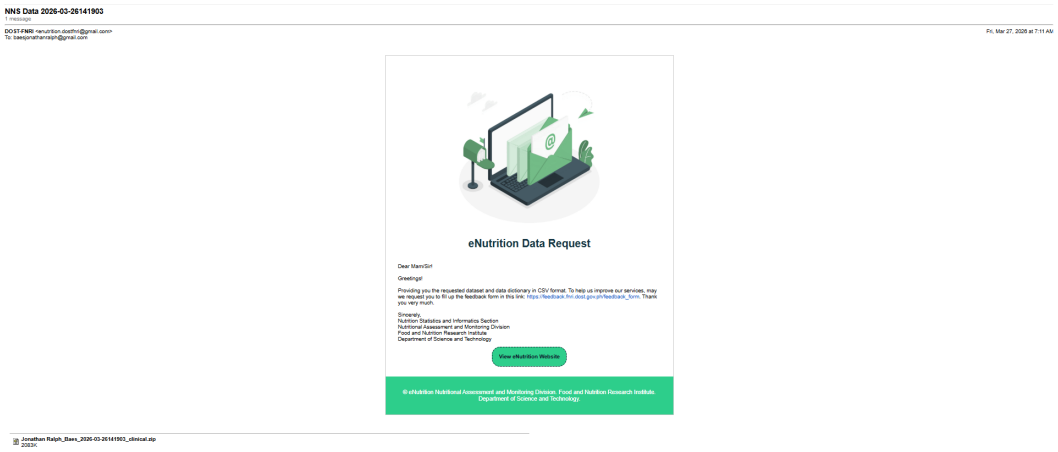

Figure 33: eNutrition Data Request Approval Email for Clinical and Health Component

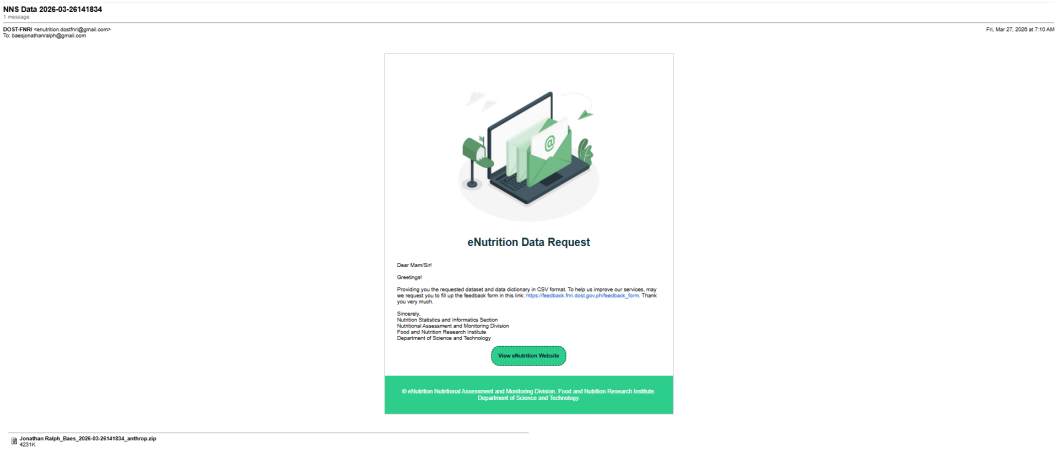

Figure 34: eNutrition Data Request Approval Email for Anthropometry Component

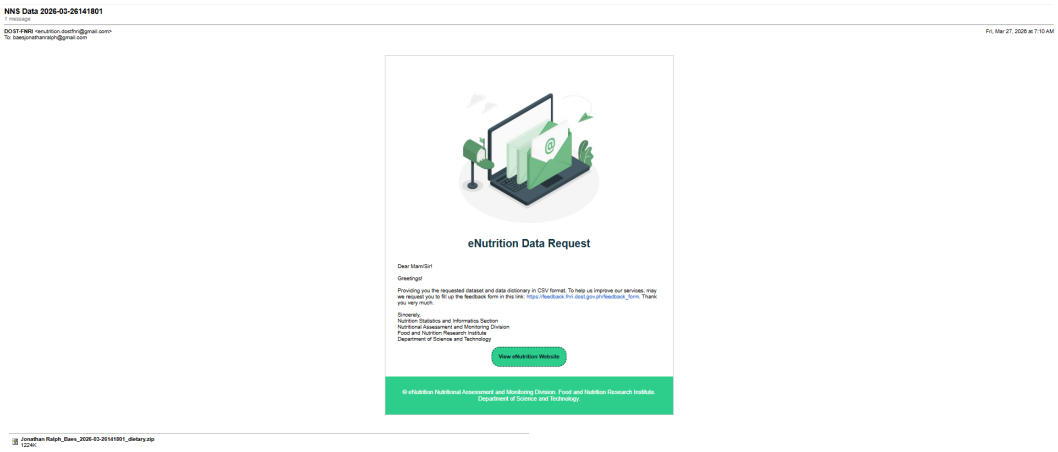

Figure 35: eNutrition Data Request Approval Email for Dietary Component

11 Acknowledgment

I would like to express my deepest gratitude to every person and institution that has helped me reach the finish line of this Special Problem and my college journey.

First and foremost, I would like to thank my adviser, Dr. Geoffrey A. Solano, for his invaluable guidance, patience, and expertise. His continuous support and insightful feedback not only shaped the direction of this research but also pushed me to grow as a computer scientist.

I am profoundly grateful to the Department of Science and Technology – Science Education Institute (DOST-SEI). The scholarship grant they provided was instrumental in funding the latter half of my college life and the specialized training necessary to see this heavy computational project through to completion.

To my family, thank you for being my constant anchor. A special thank you to my parents,for their unwavering encouragement and support.

To my friends and classmates, thank you for the shared struggles, the camaraderie, and the triumphs throughout our rigorous curriculum. Whether it was surviving grueling deadlines, or studying for our exams, your presence kept me sane.

Finally, a dedicated shoutout to caffeine for fueling my endless coding, debugging, and paper-writing sessions.

Thank you all for being a part of this journey.